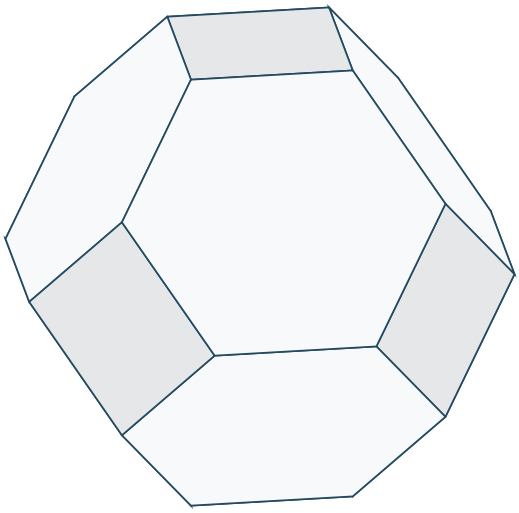
INSTRUKCJA APLIKACJI

LION Polyhedral Studio

Obsługa i architektura modelera 0!n

Wydanie R3.1 Pakiet 0.3.1 16 października 2026



24 wierzchołki 36 krawędzi 14 ścian

200 stron opisujących obsługę, model danych, strukturę kodu, katalog architektury, Git, symulację, protokół, API i utrzymanie.

Podstawa opracowania: przekazane archiwum LION_POLYHEDRAL_ARTIFACT_R1(3).zip. Stan analizy: 16 października 2026. Dokument opisuje implementację zawartą w tym archiwum.

Przeznaczenie: użytkownik studia, projektant architektury, opiekun repozytorium i programista rozwijający aplikację.

TEMAT 002

Zakres instrukcji i identyfikacja wydania

Instrukcja wyjaśnia, co można wykonać w dostarczonym programie oraz jak te operacje działają w kodzie. Opis obejmuje warstwę graficzną, lokalny serwer, dwie bazy danych, model dokumentu, katalog architektury i narzędzia pomocnicze. Nie jest potwierdzeniem działania usług federacji LION poza tym programem.

Wymiar wersji	Wartość i znaczenie
Aplikacja i API	3.1, identyfikator uruchomionego serwera
Pakiet Pythona	0.3.1 w pyproject.toml
Dokument projektu	lion.polyhedral.studio/v2
Katalog architektury	lion.architecture/v1
Szkic nawigacji	lion.navigation-draft/v1

Treść opracowano przez odczyt kodu, schematów, instrukcji i testów z archiwum. Identyfikatory źródeł w stopkach wskazują pliki opisane na stronach 195-198. Nazwa symbolu doprecyzowuje mechanizm. Odwołanie do testu wskazuje sprawdzany scenariusz, a nie gwarancję wszystkich możliwych zachowań programu.

Manifest pakietu obejmuje 134 pliki. Kod i konfiguracja są zgodne z jego zapisami. Pięć plików wynikowych w results ma inną zawartość: demo_evidence.jsonl, demo_result.json, impact_graph.json, message_candidate.json oraz mosaic_fixture.json. Są to generowane wyniki, chronione przed nadpisaniem przez instalator; ich zmiana sama nie dowodzi błędu kodu.

Historyczny raport R3.1 dokumentuje 246 testów oraz odbiór przeglądarkowy. Instrukcja nie przedstawia tej historii jako nowego badania aktualnego wdrożenia LION. Do ponownego sprawdzenia otrzymanej kopii służy run_checks.py.

W tej kopii nie ma UPGRADE_MANIFEST.json wymaganego przez install_upgrade.py. Sam plik instalatora nie stanowi kompletnej aktualizacji. CHECKPOINT.json zachowuje starsze oznaczenie R3 i 0.3.0; bieżącą wersję kodu określają server.VERSION i pyproject.toml. Te rozbieżności nie zostały w ramach instrukcji zmienione.

```
SHA-256 archiwum wejściowego
b712b003c587824457986a7cf772f663670
ce55ac181e400cdf1676b1779d46e
```

TEMAT 003

Jak korzystać z instrukcji

Pierwsze uruchomienie najlepiej wykonać na prostym szablonie. Pozwala to poznać wybieranie obiektu, wejście do wnętrza, edycję inspektora i zapis bez jednoczesnego analizowania tysięcy rekordów. Następnie można otworzyć katalog ai_platform i prześledzić ten sam zestaw operacji na rzeczywistej strukturze źródeł.

Potrzeba czytelnika	Punkt wejścia
Uruchomienie i praca w interfejsie	Strony 9-40
Budowa artefaktu i 14 ścian	Strony 41-72
Katalog architektury i Git	Strony 73-104
Przepływ wartości, czas i generacje	Strony 105-136
API, zapis, rewizje i granice dostępu	Strony 137-168
Protokół, rozwój i instalator	Strony 169-192
Słownik, źródła i szybka diagnostyka	Strony 193-200

Zakładki PDF zawierają wszystkie 200 tematów. Spis na kolejnych dwóch stronach jest mapą większych bloków i podaje strony otwierające kolejne grupy tematów. Wyszukiwanie w PDF pozwala przejść bezpośrednio do nazwy pola, pliku, funkcji, operacji API lub komunikatu. Kod i nazwy pól zachowują oryginalną pisownię.

Przykłady oznaczone jako deklaracja lub symulacja pokazują model, który można odtworzyć w Studio. Wynik takiego przykładu nie poświadcza uruchomienia zewnętrznej usługi. Z kolei przypięcie do commita identyfikuje określone źródło; nie ustala aktualnego stanu lokalnego pliku ani pracującego procesu.

Przy problemie warto zapisać identyfikator projektu, jego rewizję, identyfikator i rewizję katalogu oraz aktualny zakres. Te współrzędne umożliwiają odróżnienie błędu źródła od ustawienia widoku. Zapis projektu i jawne scalenie katalogu mają różne skutki, opisane w części użytkowej i technicznej.

Źródła: S07 Edycja w wielu wnętrzach i zapis; S42 navigateArchitecture

TEMAT 004

Spis tematów części użytkowej

Mapa podaje tematy otwierające kolejne czterostronicowe grupy. Pełna lista wszystkich tematów znajduje się w zakładkach PDF; każda strona ma własny tytuł i numer.

Temat otwierający grupę	Strona
Przygotowanie stanowiska Windows	9
Orientacja w głównym oknie	13
Kamera, wybór i orientacja w scenie	17
Wejście do wnętrza i powrót do rodzica	21
Kontekst zewnętrzny i przekraczanie granicy	25
Źródła artefaktu i interfejs plikowy	29
Pełny JSON i metarama założeń	33
Przywracanie zapisanych rewizji	37
Artefakt jako podstawowy rekord projektu	41
Pozycja, skala i stały profil bryły	45
Parametry: wartości, typy i jednostki	49
Ściana contract - jawność wymagań interfejsu	53
Ściana observability - co rzeczywiście można odczytać	57
Ściana knowledge - wiedza i materiał do sprawdzenia	61
Ściana evolution - zmiana jako jawny kandydat	65
Dataflow, wiązania i jawne przekształcenia	69
Pełny katalog a pojedyncza scena	73
Warstwy jako niezależne przekroje	77
ai_platform: zakres historycznego modelu	81
Dwie topologie: model projektu i drzewo Git	85
Stronicowanie i jawne ograniczenia widoku	89
Historia, przywracanie i eksport modelu	93
Rekurencyjne symbole Python	97
Skan dysku a indeks obiektów Git	101

Rozdziały techniczne można czytać niezależnie po ustaleniu modelu danych. Do interpretacji wyniku zawsze zachowaj nazwę formatu i identyfikator źródła. Słownik i wykaz plików zamykają instrukcję na stronach 193-200.

TEMAT 005

Spis tematów części technicznej

Mapa podaje tematy otwierające kolejne czterostronicowe grupy. Pełna lista wszystkich tematów znajduje się w zakładkach PDF; każda strona ma własny tytuł i numer.

Temat otwierający grupę	Strona
Trzy analizy: wartości, terminy i generacje	105
Exposes: komunikacja przez granicę artefaktu	109
Rewizje końców i świadome ponowne związanie	113
Odległość i siła związku: deklaracje mierzalne	117
SS: zależność startów i częściowa wiedza o czasie	121
Cykle czasu, terminy i diagnostyka arytmetyczna	125
Źródła portów procesu i kontrakty wartości	129
Deskryptory kandydatów i tożsamość wyniku	133
Backend jako lokalna usługa projektowa	137
Token sesji i uprawnienia żądań zapisujących	141
Zapis transakcyjny i konflikt dwóch edycji	145
Baza katalogów architecture.sqlite3	149
Projekcja katalogu do dokumentu sceny 3D	153
Odkrywanie, inspekcja i klonowanie repozytoriów	157
API zapisu projektu oraz kontroli struktury	161
Pakiet asystenta jako kontekst odtwarzalnej pracy	165
Dwie warstwy implementacji: mechanizm R1 i Studio R3.1	169
Rejestr połączeń i strukturalny handshake	173
Kontekst, ponowienia i rozwiązywanie konfliktów wiadomości	177
Układ repozytorium i punkty uruchomienia programu	181
Budowa katalogu atlasu i kontrola zakresu metadanych	185
Testy regresji i prawidłowe czytanie raportu R3.1	189

Rozdziały techniczne można czytać niezależnie po ustaleniu modelu danych. Do interpretacji wyniku zawsze zachowaj nazwę formatu i identyfikator źródła. Słownik i wykaz plików zamykają instrukcję na stronach 193-200.

TEMAT 006

Pierwsza kompletna sesja pracy

Celem pierwszej sesji jest utworzenie małego, zapisanego modelu i sprawdzenie, czy użytkownik rozumie różnicę między edycją, symulacją a utrwaleniem. Taki wynik stanowi lepszy punkt odniesienia niż samo zobaczenie brył w przeglądarce. Nie wymaga konta chmurowego, klucza AI ani połączenia z brokerami.

1. Otwórz katalog aplikacji w PyCharm, wybierz interpreter Python 3.11 lub nowszy i uruchom run_studio.py.
2. Przejdź pod adres wypisany przez terminal. Domyślnie jest to http://127.0.0.1:18765.
3. Wybierz Nowy projekt i szablon demonstratora przepływu. Obejrzyj parametry nadawcy, przetwarzania i odbiorcy.
4. Zaznacz relację. Odczytaj adresy końców, bramkę i wiązanie. Uruchom Symuluj i porównaj wynik z opisem demonstratora.
5. Zmień opis jednego artefaktu, zastosuj formularz i zapisz projekt. Sprawdź numer rewizji i dziennik.
6. Otwórz inny projekt, a potem wróć do zapisanego. Potwierdź zachowanie opisu.
7. Dopiero teraz otwórz Architektura / Git i wybierz fragment katalogu ai_platform do dalszego modelowania.

Oczekiwanym wynikiem jest zapisany dokument projektu z własnym identyfikatorem i rewizją oraz widoczny ślad operacji. Symulacja pozostaje osobnym wynikiem. Samo zastosowanie formularza zmienia kopię roboczą w przeglądarce; zamknięcie karty bez zapisu może utracić te zmiany.

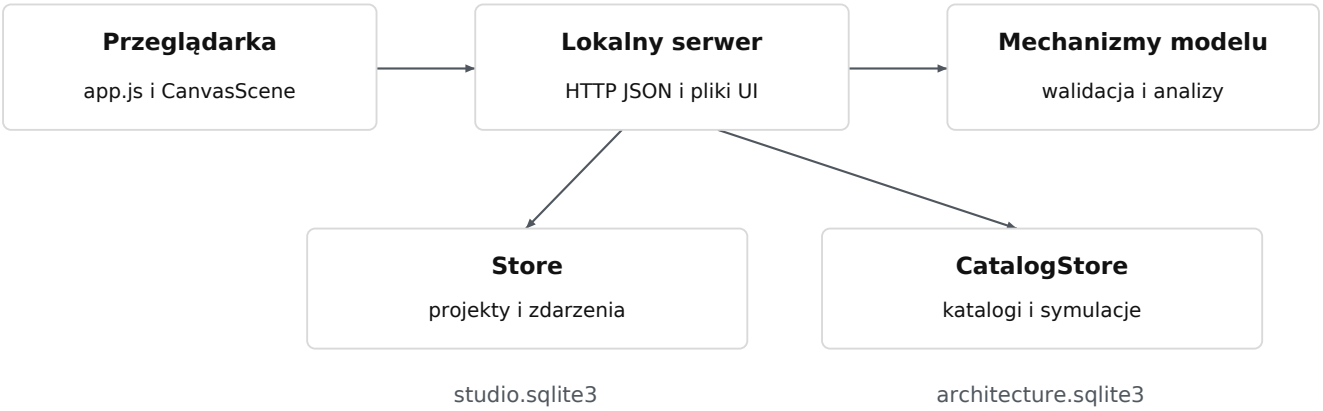
Jeżeli port jest zajęty, uruchom serwer z innym numerem, na przykład --port 18766, i korzystaj konsekwentnie z nowego adresu. Gdy zmienisz --workspace, zobaczysz inną bazę projektów. Pusty zestaw w nowym katalogu nie oznacza usunięcia poprzedniego zapisu.

Źródła: S16 main; S37 main; S36 new_project; S42 saveProject

TEMAT 007

Architektura aplikacji jako całość

Studio jest aplikacją lokalną z przeglądarkowym interfejsem. Serwer Pythona udostępnia pliki interfejsu i operacje JSON. Przeglądarka utrzymuje bieżącą edycję, wybór i zakres sceny. Serwer waliduje dokumenty, wykonuje analizy i utrwala zapis. Ten podział wyjaśnia, dlaczego zamknięcie karty i zatrzymanie serwera są różnymi zdarzeniami.



Główne granice implementacji. Strzałki oznaczają wywołania i zapis danych, nie połączenia działającej federacji.

Baza studio.sqlite3 przechowuje projekty, ich rewizje i zdarzenia. Baza architecture.sqlite3 przechowuje pełne katalogi architektury, historię tych katalogów oraz zarchiwizowane symulacje generacji. Scena 3D jest ograniczoną projekcją danych, dzięki czemu nie musi zawierać wszystkich rekordów naraz.

Warstwa dziedzinowa obejmuje walidację artefaktów, relacje i bramki, projekcję katalogu, topologię, analizę czasu oraz proces ewolucji. Wspólny interfejs graficzny nie zamienia tych mechanizmów w jeden silnik wykonawczy. Każda analiza ma własny kontrakt, ograniczenia i wynik.

Katalog źródeł może zostać uzupełniony przez indeksowanie Git albo skanowanie plików. Te mechanizmy odczytują dane i budują deskryptory. Nie przenoszą plików do nowej architektury i nie uruchamiają ich kodu. Materializacja tworzy kandydatów do dalszej pracy, a nie automatyczne wdrożenie.

Źródła: S37 StudioHTTPServer; S38 Store; S29 CatalogStore; S44 CanvasScene

TEMAT 008

Znaczenie modelu mozaikowego

Punktem wyjścia jest artefakt mający własną tożsamość, opis, wnętrze i interfejsy. Można opisać nim aplikację, komponent, plik, kontrakt, dowód albo projektowaną rolę. Wspólna rama ułatwia porównywanie obiektów, ale nie znosi różnic między ich źródłami i sposobem działania.

Ośmiościan ścięty porządkuje widok przez 14 nazwanych ścian. Ściana może zawierać wiele kafli, a kafel wiele portów. Położenie bryły nie ustala automatycznie odległości semantycznej. Wielkość obiektu nie daje mu uprawnień. Przynależność do katalogu nie czyni go właścicielem wszystkich procesów znajdujących się poniżej.

Oś opisu	Pytanie rozstrzygane przez model
Zawieranie	Co znajduje się we wnętrzu tego obiektu
Źródło	Który plik, commit lub zapis opisujemy
Poziom abstrakcji	W jakiej ramie porównujemy poziomy
Relacja	Jakie końce łączy dane twierdzenie lub kontrakt
Dowód i aktualność	Na jakiej podstawie i z jakiej chwili znamy opis
Geometria	Jak użytkownik ma zobaczyć i wybrać obiekt

W praktyce ta sama klasa może pozostawać w źródłowym pliku Pythona, a w projektowanym modelu zostać przypisana do innej kompozycji. Jest to nakładka projektowa. Zmiana rodzica w takim modelu nie wykonuje refaktoryzacji i nie zmienia ścieżki Git. Dopiero osobna, zweryfikowana zmiana kodu mogłaby zrealizować tę propozycję.

Wartość UNKNOWN jest użytecznym wynikiem modelowania. Informuje, że nie ustalono wartości lub nie spełniono warunków oceny. Nie powinna być zastępowana przez zero, fałsz ani domyślną zgodę. Tę zasadę realizują bramki, analizy i opisy źródeł, z różnymi szczegółami opisanymi w dalszych częściach.

TEMAT 009

Przygotowanie stanowiska Windows

Program działa lokalnie jako serwer Pythona i strona otwierana w przeglądarce. Do uruchomienia potrzebny jest Python w wersji co najmniej 3.11 oraz współczesna przeglądarka obsługująca moduły JavaScript i element canvas. Podstawowa praca nie wymaga instalowania bibliotek przez pip, środowiska Node ani klucza do modelu AI. Repozytorium może pozostawać w dotychczasowym katalogu projektu PyCharm.

Przed rozpoczęciem sprawdź, czy rozpakowany katalog zawiera run_studio.py, katalog src oraz README.md. Samo otwarcie pliku index.html z dysku nie uruchamia aplikacji: interfejs korzysta z lokalnych adresów API serwera. Właściwy punkt wejścia stanowi run_studio.py. Pliki źródłowe aplikacji i dane użytkownika to odrębne powierzchnie; dane powstają w wybranym katalogu workspace.

W istniejącej instalacji zachowaj interpreter .venv. Nie twórz nowego środowiska tylko dlatego, że otrzymałeś nowszy pakiet. Sprawdź wersję interpretera w ustawieniach projektu lub przez jego pełną ścieżkę. Dzięki temu późniejszy komunikat błędu odnosi się do konkretnego Pythona, a nie do przypadkowego polecenia python znalezionego przez systemowy PATH.

Domyślny adres programu to http://127.0.0.1:18765/. Adres 127.0.0.1 oznacza ten sam komputer, na którym działa serwer. Przeglądarka na innym komputerze nie uzyska pod tym adresem dostępu do tej instancji. Jeżeli używasz już innych usług LION, kieruj się adresem wypisanym przez Studio, zamiast rozpoznawać aplikację wyłącznie po podobnej nazwie zakładki.

Po pierwszym poprawnym uruchomieniu zachowaj widoczną konsolę. Pozwala ona odróżnić zamkniętą kartę przeglądarki od zatrzymanego procesu serwera. Warto również przygotować osobny katalog na eksporty projektu i pakiety asystenta. Ułatwia to przekazywanie dokumentów bez kopiowania aktywnej bazy danych i bez mieszania archiwalnych modeli z bieżącymi plikami aplikacji.

1. Otwórz katalog LION_POLYHEDRAL_ARTIFACT_R1 w PyCharm.

2. Sprawdź interpreter Python 3.11 lub nowszy.

3. Uruchom run_studio.py i odczytaj adres z konsoli.

4. Poczekaj na pojawienie się sceny, listy projektu i statusu zapisu.

TEMAT 010

Konfiguracja uruchomienia w PyCharm

W PyCharm otwórz run_studio.py z głównego katalogu projektu i uruchom ten plik jako skrypt Pythona. Launcher dodaje lokalny katalog src do ścieżki modułów. Dzięki temu uruchomienie nie zależy od wcześniejszego instalowania pakietu lion_studio w środowisku. Jest to ważne przy utrzymywaniu aplikacji bezpośrednio z kodu oraz przy aktualizowaniu jej w tym samym projekcie.

W konfiguracji Run/Debug sprawdź pole Script path. Powinno wskazywać run_studio.py, a nie run_demo.py ani run_checks.py. Demonstrator i zestaw testów mają inne zadania; ich poprawne zakończenie nie oznacza uruchomienia serwera graficznego. Pole interpretera powinno wskazywać środowisko używane przez ten projekt. Katalog roboczy najlepiej ustawić na główny katalog repozytorium, aby polecenia wykonywane ręcznie były czytelne.

Parametry wpisuje się w polu Parameters lub Script parameters, zależnie od wersji PyCharm. Podanie --port 18766 zastępuje port przygotowany przez launcher. Podanie --no-browser pozostawia serwer aktywny i wyłącza wyłącznie automatyczne otwarcie przeglądarki. Można wtedy samodzielnie otworzyć adres z konsoli, co pomaga porównać zachowanie różnych przeglądarek albo uniknąć otwarcia niewłaściwego profilu.

Jeżeli konfiguracja zawiera --workspace, zapisz jej pełną wartość. Dwie konfiguracje mogą korzystać z tej samej aplikacji i różnych danych. Objawem pomylenia katalogów bywa brak oczekiwanego projektu przy poprawnie działającym interfejsie. Samo ponowne rozpakowanie programu nie przenosi danych między workspace. Najpierw porównaj ścieżkę Projekty wypisaną podczas startu, a dopiero później szukaj uszkodzenia bazy.

Zakończ pracę po zapisaniu zmian. W konsoli uruchomienia możesz użyć Ctrl+C, jeżeli terminal przekazuje ten skrót do procesu; przycisk Stop w PyCharm również zatrzyma serwer. Po zmianie kodu uruchom proces ponownie. Po zmianie JavaScript lub stylów odśwież stronę, a przy podejrzeniu starej pamięci podręcznej użyj Ctrl+F5. Zatrzymany serwer pozostawia zapisane projekty w bazie.

1. Wybierz run_studio.py jako skrypt uruchomienia.

2. Ustaw interpreter istniejącego projektu i sprawdź parametry.

3. Uruchom konfigurację i porównaj port oraz ścieżkę Projekty.

4. Po zmianie kodu zatrzymaj proces, uruchom ponownie i odśwież kartę.

TEMAT 011

Uruchomienie z PowerShell i wybór portu

Uruchomienie z PowerShell daje tę samą aplikację co konfiguracja PyCharm. Najpierw przejdź do katalogu projektu albo podaj pełne ścieżki do interpretera i skryptu. Operator & jest potrzebny, kiedy ścieżka programu została zapisana w cudzysłowie. Przy katalogach zawierających spacje lub znak ! zawsze zachowuj cudzysłowy, aby powłoka nie rozdzieliła nazwy na kilka argumentów.

Launcher przekazuje domyślny port 18765, a argument użytkownika może go zastąpić. Gdy adres jest zajęty, serwer kończy start komunikatem zawierającym propozycję innego portu. W takiej sytuacji nie zamykaj przypadkowych procesów Pythona. Uruchom Studio z --port 18766 i otwórz dokładnie nowy adres. Dwa serwery na różnych portach mogą działać równolegle, ale najbezpieczniej przypisać im różne katalogi danych.

Parametr --workspace przyjmuje ścieżkę katalogu, w którym program przechowuje swoje bazy. Przy pierwszym użyciu katalog zostaje utworzony. Parametr --no-browser przydaje się w konsoli, do diagnostyki oraz podczas przygotowania zrzutów dokumentacyjnych. Nie zmienia trybu modelowania ani ochrony lokalnego adresu. Po starcie trzeba samodzielnie otworzyć wypisany URL.

Zwróć uwagę na różnicę pomiędzy numerem portu a rewizją projektu. Zmiana portu zmienia adres połączenia, natomiast nie tworzy nowej wersji modelu. Zmiana katalogu workspace może natomiast pokazać inne projekty. W zgłoszeniu problemu podaj oba parametry. Krótka informacja „otwieram ten sam program” nie wystarcza do rozpoznania dwóch instancji o różnych danych.

Zatrzymanie przez Ctrl+C powoduje zamknięcie serwera. Karta przeglądarki może nadal pokazywać ostatni obraz, lecz kolejne operacje wymagające API już nie odpowiedzą. Nie traktuj takiej karty jako działającej sesji. Po wznowieniu procesu odśwież stronę i sprawdź status projektu. Niezapisane zmiany przechowywane wyłącznie w pamięci karty nie są odzyskiwane z konsoli.

```
& '.\.venv\Scripts\python.exe' '.\run_studio.py' `
  --port 18766 `
  --workspace 'D:\LION-Studio-Projekty' `
  --no-browser
```

TEMAT 012

Katalog danych i bezpieczne kończenie pracy

W typowym uruchomieniu z repozytorium dane znajdują się w katalogu workspace obok run_studio.py. Projekty sceny i ich historia są przechowywane w studio.sqlite3. Pełne katalogi architektury, ich rewizje i archiwum symulacji ewolucji korzystają z architecture.sqlite3. Są to odrębne magazyny. Kopia jednego pliku nie zastępuje kopii całego kontekstu pracy z katalogiem.

Przed wykonaniem zwykłej kopii folderu zakończ proces Studio. Skopiowanie aktywnej bazy SQLite metodą przeciągnięcia w Eksploratorze nie jest dobrym sposobem tworzenia spójnego archiwum pracy. Alternatywę stanowi kopia transakcyjna SQLite, lecz wymaga świadomego zastosowania narzędzia do baz. Dla codziennego przekazania pojedynczego projektu łatwiejszy jest eksport JSON albo Pakiet asystenta.

Zamknięcie karty nie jest operacją zapisu. Interfejs pokazuje Niezapisane zmiany po modyfikacji modelu i ostrzega przed opuszczeniem strony. Potwierdzenie opuszczenia może utracić lokalne edycje. Przycisk Zapisz utrwala dokument projektu i usuwa lokalny stos cofania. Zapisane dane pozostają dostępne po zatrzymaniu i ponownym uruchomieniu serwera z tym samym workspace.

W modelu połączonym z katalogiem przycisk Zapisz obejmuje także skumulowany szkic nawigacji. Nie jest to jeszcze scalenie zmian z pełnym katalogiem architektury. Jeśli planujesz zakończyć sesję po edycjach w wielu wnętrzach, najpierw zapisz projekt, a następnie zdecyduj, czy potrzebna jest jawna operacja Zapisz edycje 3D do katalogu. Odrębne rewizje pozwalają zachować projekt bez od razu zmieniania katalogu.

Przy przenoszeniu danych na drugi komputer sprawdź zależności źródłowe. Lokalna ścieżka zapisana w deskryptorze może wskazywać katalog, którego tam nie ma. Zachowanie metadanych nie oznacza skopiowania repozytoriów. Dla projektu nawigującego po katalogu potrzebny jest również odpowiedni katalog i jego przypięta rewizja. Pełny eksport katalogu albo pakiet kontekstu pomaga zachować te współrzędne.

1. Sprawdź status Niezapisane zmiany i naciśnij Zapisz.

2. Poczekaj na komunikat Zapisano rewizję oraz Zapisano lokalnie.

3. Zatrzymaj serwer przed zwykłym kopiowaniem całego workspace.

4. Zachowaj razem bazy i eksporty potrzebne do dalszej nawigacji.

Źródła: S38 Store.__init__

TEMAT 013

Orientacja w głównym oknie

Górny pasek identyfikuje projekt. Lista rozwijana służy do przełączania zapisanych projektów, obok widoczny jest stan zapisu. Przyciski Nowy projekt, Zapisz i Pakiet asystenta odnoszą się do dokumentu Studio. Pod nimi znajduje się pasek poleceń: dodawanie artefaktów i relacji, lokalne cofanie, warsztat Architektura / Git, walidacja, symulacja, analiza czasu oraz import i eksport.

Lewy panel Struktura projektu przedstawia hierarchię artefaktów dostępnych w bieżącym dokumencie. Zawiera wyszukiwarkę i filtry. Dolny przycisk Źródła z katalogu uruchamia odczyt lokalnych plików do deskryptorów projektu. Metarama projektu otwiera nazwę ramy, jej wersję i założenia. W węższym oknie panel jest otwierany przyciskiem Struktura i można go zamknąć bez zmieniania modelu.

Środkowa przestrzeń zawiera scenę, ścieżkę wnętrza oraz przyciski Dopasuj, Obrót, Przesuń i Wyżej. W modelu katalogowym dochodzi pasek dwóch współrzędnych: Model projektu i Drzewo Git. Liczniki sceny informują o aktualnie widocznych obiektach, kontekstach i liniach. Nie są automatycznie licznikami całej federacji. Pełną liczbę rekordów należy odczytywać z katalogu architektury.

Prawy Inspektor pokazuje wybrany obiekt, ścianę albo relację. Wybór artefaktu udostępnia zakładki Obiekt, Ściany, Parametry, Czas, Relacje i Źródła. Wybór relacji zmienia zestaw pól na końce, przypięcia rewizji, bramkę, wiązania i topologię. Puste zaznaczenie pokazuje krótką ścieżkę rozpoczęcia pracy. Panel można schować, aby zwiększyć przestrzeń sceny.

Dolna konsola rozdziela Dziennik, Walidację, Symulację i Czas. Każda zakładka odpowiada innemu rodzajowi wyniku. Po edycji wcześniejsze analizy są czyszczone, aby nie sugerować zgodności starego wyniku z nowym modelem. Pasek statusu na dole pokazuje ostatni komunikat; jego treść warto skopiować do zgłoszenia błędu. Zmniejszenie konsoli zmienia układ widoku, a nie zawartość projektu.

Obszar	Podstawowe pytanie
Lista projektu	Na którym dokumencie pracuję?
Struktura	Jakie obiekty są dostępne?
Scena	W jakim wnętrzu jestem?
Inspektor	Co opisuje wybrany element?
Konsola	Jaki jest wynik operacji?



Mapa obszarów interfejsu. Układ jest schematem funkcjonalnym, a nie zrzutem ekranu.

Źródła: S43 main layout

TEMAT 014

Wybór właściwego szablonu projektu

Nowy projekt otwiera formularz z opcjonalną nazwą i polem Punkt wyjścia. Każdy szablon tworzy osobny projekt, dlatego eksperyment z demonstratorem nie musi zmieniać aktualnego modelu ai_platform. Jeżeli bieżący dokument zawiera niezapisane zmiany, interfejs pyta o ich odrzucenie. Przed zatwierdzeniem nowego projektu zapisz poprzednią pracę albo świadomie zrezygnuj z lokalnych edycji.

ai_platform - pełny katalog architektury i widok 3D służy do przeglądania dużego opisu źródeł z rekurencyjną nawigacją. Wbudowany model pochodzi z historycznego atlasu. Szablon Federacja LION - mapa źródłowa nadaje się do oglądania ról repozytoriów, jawnych braków i kontraktów na poziomie federacji. Nie należy oczekiwać identycznej szczegółowości każdej bryły w obu szablonach.

Studio jako artefakt - samopis aplikacji pokazuje narzędzie jako jeden z opisywanych systemów. Jest dobrym punktem wyjścia do rozpoznawania odpowiedzialności aplikacji i deskryptorów jej plików. Przepływ demo - parametry i bramki służy do nauki typowanych portów, wiązań oraz propagacji wartości. Wyniki tego przykładu są wynikami lokalnego modelu badawczego, a nie wykonaniem źródeł federacji.

Czas - równoległe gałęzie i oczekiwanie przygotowuje zadania oraz relacje potrzebne do analizy harmonogramu. Pusty projekt pozostawia swobodę zbudowania własnej hierarchii. Ten ostatni wybór jest najczytelniejszy przy nauce ręcznego modelowania, ponieważ każdy dodany element ma znane pochodzenie i nie trzeba odróżniać własnych deklaracji od obserwacji wyciągniętych z katalogu źródłowego.

Dobieraj szablon do pytania badawczego. Do rozpoznania metod wewnątrz pliku wybierz katalog ai_platform albo własny indeks Git; do sprawdzenia przepływu parametru wybierz demonstrator lub mały model ręczny. Duża mapa źródeł nie staje się gotową symulacją tylko dlatego, że ma wiele połączeń. Rodzaj relacji, zgodność portów i wiązania trzeba zdefiniować oddzielnie przed obliczaniem wartości.

1. Naciśnij Nowy projekt i wpisz rozpoznawalną nazwę.

2. Wybierz Punkt wyjścia zgodny z celem ćwiczenia.

3. Zatwierdź Utwórz i sprawdź nazwę na górnym pasku.

4. Odczytaj stan źródeł oraz przeznaczenie przykładu przed edycją.

Źródła: S42 actions['new-project']

TEMAT 015

Pierwsze ćwiczenie ręcznego modelowania

Rozpocznij od Pustego projektu o nazwie Warsztat kontraktu. Celem ćwiczenia jest utworzenie małej kompozycji i opisanie jej części, a nie od razu uruchomienie przepływu danych. Taki rozdział pozwala zobaczyć, że sama linia między obiektami może przenosić znaczenie, natomiast przekazywanie wartości wymaga dodatkowej konfiguracji portów oraz wiązań parametrów.

Naciśnij Artefakt. W formularzu Nowy artefakt wpisz nazwę Odbiorca, wybierz rodzaj Komponent i pozostaw poziom projektu jako rodzica. W polu Odpowiedzialność opisz jedno konkretne zadanie, na przykład przyjęcie zamówienia do analizy. Po zatwierdzeniu obiekt otrzyma własne ID, rewizję początkową i osie deklaracji. Program rozmieści go w przestrzeni bieżącego wnętrza.

Dodaj drugi artefakt, Weryfikator. Wybierz pierwszą bryłę przez nazwę w drzewie, aby uniknąć niepewności trafienia w ścianę. Otwórz zakładkę Relacje i wybierz Połącz. W formularzu określ Odbiorcę jako początek, Weryfikatora jako koniec oraz depends_on jako typ. Opisz, jaki warunek ma spełniać ta zależność. W takim połączeniu można pozostawić wybór portów pusty.

Przejdź do inspektora relacji i porównaj kierunek strzałki z własnym zdaniem. depends_on od Odbiorcy do Weryfikatora znaczy deklarację zależności Odbiorcy od wskazanego końca; sama nazwa obiektów nie odwraca kierunku. Jeśli zamierzasz opisać przesyłanie wyniku w przeciwnym kierunku, dodaj odrębną relację o odpowiedniej semantyce, zamiast oczekiwać automatycznego wnioskowania.

Sprawdź model, odczytaj wynik Walidacji i zapisz projekt. Poprawny wynik strukturalny potwierdza spójność wymaganą przez edytor, a nie prawdziwość opisu procesu biznesowego. Dopiero kolejne ćwiczenie powinno wprowadzić parametry, kafle i porty. Dzięki zachowaniu małego modelu masz punkt odniesienia do porównania tego, co wynika z deklaracji użytkownika, i tego, co faktycznie oblicza symulator.

1. Utwórz pusty projekt Warsztat kontraktu.

2. Dodaj Odbiorcę i Weryfikatora jako równorzędne komponenty.

3. Połącz je relacją depends_on i opisz warunek zależności.

4. Uruchom Sprawdź model, odczytaj komunikaty i wybierz Zapisz.

Źródła: S42 addArtifact; editRelation

TEMAT 016

Tożsamość projektu i trzy rodzaje rewizji

Rozpoznawalna nazwa ułatwia pracę człowiekowi, lecz dokument jest identyfikowany przez project_id. Dwa projekty mogą mieć podobne nazwy i różne identyfikatory. Edycja nazwy w Metaramie projektu nie przenosi danych do innego dokumentu. Przy przekazywaniu problemu zachowaj zarówno nazwę, jak i ID oraz numer rewizji. Pełny JSON udostępnia te pola w sposób jednoznaczny.

Rewizja projektu zwiększa się przy utrwaleniu dokumentu w magazynie. To inna wartość niż revision wewnątrz poszczególnych artefaktów. Zmiana semantycznych pól artefaktu zwiększa jego rewizję, a relacje zachowują dotychczasowe przypięcia końców. W efekcie po modyfikacji może pojawić się STALE. Zapis całego projektu nie zastępuje jawnego sprawdzenia i ponownego związania tych wersji.

Katalog architektury ma jeszcze odrębną rewizję. Wycinek 3D wskazuje konkretny katalog i jego wersję. Nawigacja pobiera zakresy z tego przypięcia, zachowując skumulowany szkic. Zapisz utrwala projekt Studio wraz ze szkicem. Przycisk Zapisz edycje 3D do katalogu wykonuje inne działanie: tworzy nową rewizję pełnego katalogu z nakładkami projektu. Należy świadomie wybrać moment takiego scalenia.

Przy zapisie program przesyła numer oczekiwanej rewizji. Jeśli inna sesja zapisała już nowszą treść, konflikt chroni przed cichym zastąpieniem zmian. Nie próbuj obchodzić go ręcznym zwiększeniem liczby w JSON. Zachowaj eksport lokalnych edycji, otwórz aktualny dokument i porównaj różnice. Sama zgodność numeru bez porównania zawartości nie zapewnia poprawnego połączenia dwóch prac.

Po udanym zapisie status zmienia się na Zapisano lokalnie, a lokalny stos Cofnij i Ponów zostaje wyczyszczony. Dziennik zapisanych rewizji pozostaje dostępny. Jeżeli dokument pochodzi z importu, pierwszy zapis tworzy nowy projekt; nie jest aktualizacją projektu o obcym ID. To celowa różnica między otwarciem zapisanej pozycji z listy a zatwierdzeniem dokumentu dostarczonego z zewnątrz.

Wersja	Co obejmuje
Rewizja projektu	Cały dokument Studio
Rewizja artefaktu	Znaczenie i kontrakt obiektu
Rewizja katalogu	Pełny model architektury

Źródła: S42 saveProject; editArtifact; navigateArchitecture

TEMAT 017

Kamera, wybór i orientacja w scenie

Scena przedstawia aktualne wnętrze jako zbiór brył i połączeń. W trybie Obrót przeciąganie tła zmienia kąt oglądania. Przytrzymanie Shift podczas przeciągania przesuwa punkt obserwacji. Kółko myszy zmienia przybliżenie względem wskazanego miejsca. Te czynności służą oglądaniu modelu; nie nadają artefaktom nowych rodziców i nie zmieniają semantycznych miar relacji.

Kliknięcie bryły, jej etykiety albo ściany ustawia zaznaczenie. Kliknięcie ściany otwiera odpowiadającą jej część inspektora. Trafienie w linię wybiera relację. Gdy scena jest gęsta, nazwy mogą być skracane lub pomijane, aby ograniczyć nakładanie napisów. Pełną nazwę i ID odczytaj w drzewie albo inspektorze. Brak etykiety na rysunku nie oznacza usunięcia artefaktu.

Przycisk Dopasuj obejmuje obiekty bieżącej sceny. Klawisz Home wykonuje podobne dopasowanie, gdy fokus znajduje się na canvas. Klawisz F centruje zaznaczony artefakt. Jeśli skrót nie działa, najpierw kliknij scenę; aktywne pole tekstowe lub otwarty dialog przejmuje klawiaturę. Wybór artefaktu w drzewie daje precyzyjniejsze zaznaczenie niż szukanie małej bryły po dalekim oddaleniu.

Obsługa klawiaturą obejmuje obrót strzałkami, przybliżenie klawiszami + i − oraz Enter do wejścia w wybrany artefakt lub ścianę. Escape usuwa zaznaczenie. Podczas korzystania ze skrótów nie myl klawisza Enter sceny z zatwierdzeniem otwartego formularza. W formularzu znaczenie ma jego własny przycisk Zastosuj lub Zastosuj zmiany obiektu.

Jeżeli po powrocie do wnętrza bryły wydają się zbyt małe, zacznij od Dopasuj, a następnie od ograniczenia kontekstu zewnętrznego. Duża liczba sąsiadów może poszerzać zakres kamery. Zmiana przybliżenia nie jest sposobem wyszukania elementu nieobecnego na bieżącej stronie. Do tego służą stronicowanie i katalog architektury. Najpierw ustal, czy problem dotyczy widoku, czy zakresu danych.

Sterowanie	Skutek
Przeciąganie	Obrót kamery
Shift i przeciąganie	Przesunięcie widoku
Kółko	Przybliżenie lub oddalenie
Home / Dopasuj	Objęcie bieżącego zakresu
F	Centrowanie zaznaczenia
Enter	Wejście do wybranego obiektu

Źródła: S44 CanvasScene.pointerDown; keyDown; wheel

TEMAT 018

Przesuwanie brył i znaczenie współrzędnych

Tryb Przesuń pozwala zmieniać położenie artefaktu w widoku. Włącz go przyciskiem nad sceną, a następnie przeciągnij bryłę. Ruch odbywa się w płaszczyźnie wynikającej z aktualnego ustawienia kamery. Jeśli potrzebujesz dokładnej wartości konkretnej osi, użyj pól X, Y i Z w zakładce Obiekt. Manipulacja myszą jest wygodna do kompozycji, formularz do precyzyjnego układu.

Po puszczeniu przycisku myszy nowa pozycja trafia do lokalnego modelu i staje się niezapisaną zmianą. Podgląd ruchu jest oddzielony od ostatecznej aktualizacji. Przerwane przeciąganie nie powinno być interpretowane jako zakończona edycja. Po przesunięciu sprawdź współrzędne w inspektorze i użyj Cofnij, jeśli bryła trafiła w niewłaściwe miejsce. Następnie zapisz projekt, aby utrwalić położenie.

Skala bryły jest dodatnią liczbą opisującą reprezentację. Nie oznacza automatycznie wielkości repozytorium, liczby wywołań ani priorytetu procesu. Podobnie bliskość dwóch brył nie wylicza siły związku. Jeżeli położenie ma pełnić określoną rolę interpretacyjną, zapisz tę konwencję w metaramie projektu i stosuj ją konsekwentnie. Formalne miary odległości i siły umieszcza się osobno w relacji.

Artefakty bez użytecznej, rozróżnialnej pozycji mogą otrzymać rozłożenie na siatce. Renderer rozdziela również nakładające się pozycje, aby kilka brył nie wyglądało jak jedna. Taki wygenerowany układ pomaga zobaczyć elementy, lecz nie jest dowodem przestrzennego porządku kodu. Ręczne pozycje, które zostały zapisane w modelu i nie wymagają zastępczego rozłożenia, są zachowywane przy wyświetlaniu.

Odnośniki kontekstu nie są zwykłymi edytowanymi bryłami. Ich przesunięcie nie służy projektowaniu właściwego obiektu źródłowego. Najpierw wejdź do wskazanego obiektu i tam zmień jego opis lub położenie. Po zakończeniu układania włącz Obrót, aby dalsze przeciąganie nie przestawiało przypadkowo artefaktów. Taki nawyk jest szczególnie przydatny przy analizie gęstych zależności i przygotowaniu materiału do przeglądu.

1. Wybierz Przesuń i przeciągnij właściwy artefakt.

2. Odczytaj X, Y, Z oraz Skalę bryły w Obiekcie.

3. W razie potrzeby wpisz dokładne liczby i zastosuj formularz.

4. Wróć do Obrót, sprawdź kompozycję i zapisz projekt.

Źródła: S44 CanvasScene.pointerMove; setData

TEMAT 019

Drzewo struktury i wybór zakresu

Panel Struktura projektu przedstawia zawieranie artefaktów. W ręcznym modelu rodzic jest zapisany przez parent_id, a element bez rodzica znajduje się na poziomie projektu. W modelu katalogowym drzewo obejmuje wczytany zakres, jego przodków i informacje o rzeczywistej liczbie dzieci. Licznik przy węźle może być większy od liczby obecnie rozwiniętych wierszy, ponieważ podobiekty są pobierane dopiero podczas nawigacji.

Kliknięcie nazwy wybiera artefakt i otwiera inspektor. Strzałka rozwijania porządkuje gałęzie już dostępne w dokumencie. W katalogu, gdy licznik zapowiada dzieci, lecz nie zostały one wczytane, użycie rozwinięcia może otworzyć właściwe wnętrze przez mechanizm nawigacji. Przycisk ze strzałką wejścia przy wierszu prowadzi bezpośrednio do zakresu danego obiektu. Wybór nazwy i wejście to różne działania.

Po wejściu do głębszego poziomu porównaj tytuł sceny ze ścieżką nad nią. Zaznaczony w drzewie rodzic nie musi być bryłą rysowaną we własnym wnętrzu; scena pokazuje przede wszystkim jego dzieci. Jest to naturalna różnica między aktualnym zakresem a wybranym obiektem. Pola inspektora mogą opisywać rodzica, podczas gdy bryły na środku reprezentują jego podobiekty.

Przy bardzo długiej ścieżce drzewo ogranicza widoczną część przodków i sygnalizuje ostatnie poziomy ścieżki. Pełniejsze przejście pozostaje nad sceną. Takie skrócenie zapobiega nadmiernemu wcięciu wąskiego panelu i nie zmienia topologii katalogu. Dla dokładnego pochodzenia elementu użyj pól źródła albo karty w Architektura / Git, zamiast odtwarzać ścieżkę wyłącznie z geometrii wcięć.

Drzewo pomaga również odróżnić obiekt od odnośnika kontekstu. Przodkowie i zewnętrzne końce relacji mają szczególne role widoku. Otwarcie ich inspektora pokaże objaśnienie nawigacji. Jeżeli szukasz pliku z innej gałęzi, pełny katalog będzie skuteczniejszy niż rozwijanie przypadkowych wierszy. Struktura w głównym oknie jest narzędziem pracy z aktualnym modelem, a nie nieograniczonym eksploratorem wszystkich plików dysku.

1. Kliknij nazwę, aby przeczytać opis bez zmiany wnętrza.

2. Użyj strzałki wejścia, aby zobaczyć bezpośrednie dzieci.

3. Porównaj tytuł sceny, licznik dzieci i ścieżkę nad sceną.

4. Przy brakującej gałęzi otwórz Katalog i wyszukaj ją globalnie.

Źródła: S42 renderTree; actions.expand

TEMAT 020

Wyszukiwanie i filtry lokalnego widoku

Pole Szukaj nazwy, roli, ID… w lewym panelu filtruje aktualnie załadowane artefakty. Implementacja porównuje zapytanie z nazwą, ID, rodzajem, właścicielem i opisem. Wielkość liter nie ma znaczenia. To wyszukiwanie pomaga ograniczyć scenę i rozpoznać wiersze w bieżącym dokumencie. Nie pobiera automatycznie wszystkich potomków ani rekordów z innych stron pełnego katalogu architektury.

Filtr Wszystkie rodzaje pozwala pozostawić jeden rodzaj artefaktu, a Wszystkie statusy odnosi się do klasy dowodu. Listy wynikają z aktualnego dokumentu, więc ich zawartość może zmieniać się przy przejściu do innego zakresu. Kilka filtrów działa łącznie. Puste pole nazwy nie usuwa ograniczenia rodzaju ani dowodu. Gdy widok niespodziewanie pustoszeje, sprawdź wszystkie trzy kontrolki.

W drzewie niedopasowane elementy są przygaszane, co zachowuje kontekst hierarchii. Scena wyświetla tylko dopasowane bryły z bieżącego wnętrza. To dlatego liczba wierszy, licznik całego zakresu i liczba narysowanych obiektów mogą się różnić. Komunikat Brak artefaktów dla wybranego filtra oznacza istnienie obiektów odrzuconych przez filtr, a nie dowód pustego katalogu.

Połączenia są rysowane wtedy, gdy ich końce mogą być przedstawione w scenie. Odfiltrowanie jednego artefaktu może ukryć linię albo zmienić licznik połączeń poza widokiem. Nie zmienia to rekordów relacji. Dlatego analizę braku zależności rozpocznij od usunięcia filtrów, wyboru Wszystkie połączenia i sprawdzenia listy Relacje w inspektorze. Dopiero potem oceniaj, czy relacja rzeczywiście nie została opisana.

Do wyszukiwania w całym atlasie użyj Architektura / Git. Tam zapytanie dotyczy katalogu i współpracuje z filtrem warstwy oraz stronicowaniem listy. Praktyczny schemat pracy to globalne znalezienie pliku, otwarcie jego wycinka w 3D, a następnie lokalne filtrowanie sąsiadów. Rozdzielenie tych dwóch wyszukiwarek pozwala zachować czytelną scenę bez wczytywania całego dużego modelu do jednego wnętrza.

1. Wpisz fragment nazwy i obserwuj scenę oraz przygaszone wiersze.

2. Sprawdź filtr rodzaju i klasy dowodu.

3. Wyczyść ograniczenia przed oceną kompletności połączeń.

4. Jeśli szukasz poza zakresem, przejdź do Architektura / Git.

Źródła: S42 matches; renderTree; renderScene

TEMAT 021

Wejście do wnętrza i powrót do rodzica

Wnętrze to zakres bezpośrednich podobiektów wskazanego artefaktu. Można je otworzyć dwuklikiem bryły, przyciskiem Wejdź do wnętrza w inspektorze, strzałką wejścia w drzewie albo klawiszem Enter przy aktywnej scenie i zaznaczeniu. R3.1 kieruje te drogi do wspólnego mechanizmu. Dzięki temu sposób kliknięcia nie powinien zmieniać tego, które podobiekty zostaną pobrane z katalogu.

W ręcznym projekcie przejście wykorzystuje artefakty zapisane w dokumencie. W projekcie związanym z katalogiem każde wejście pobiera bezpośrednie dzieci z pełnego modelu przypiętej rewizji. Głębokość początkowego wycinka nie zamyka dalszej eksploracji. Repozytorium może prowadzić do katalogu, katalog do pliku, plik do klasy, a klasa do metod, o ile takie rekordy rzeczywiście istnieją w opisie źródeł.

Po otwarciu zakresu sprawdź tytuł, liczbę bezpośrednich dzieci oraz współrzędne źródła. Jeśli widzisz komunikat Ten artefakt nie ma bezpośrednich podobiektów, osiągnąłeś liść danej topologii. Nie znaczy to, że kod jest pusty lub że nie ma zależności zewnętrznych. Podobiekty, relacje i treść źródła to trzy różne aspekty, dlatego każdy z nich należy odczytywać z właściwego panelu.

Przycisk Wyżej otwiera rodzica w aktualnie wybranej topologii. Okruszki ścieżki pozwalają przeskoczyć do wskazanego przodka. Przycisk Katalog na początku ścieżki wraca do korzenia modelu katalogowego; w zwykłym projekcie analogicznym punktem jest Projekt. Cofnięcie nawigacji nie oznacza cofnięcia edycji. Zmiany pozostają w tym samym dokumencie i jego skumulowanym szkicu.

Jeśli wejście kończy się komunikatem błędu, nie dodawaj od razu podobiektów mających zastąpić brakujące źródła. Najpierw sprawdź serwer, przypięcie katalogu, stronicowanie i tryb nawigacji. Wbudowany atlas jest historyczny i nie gwarantuje modelu każdego repozytorium na komputerze. Ręcznie dodany obiekt otrzyma deklarację projektową, a nie automatyczne potwierdzenie pochodzenia z Git.

1. Wybierz ai_platform, a następnie Źródła repozytorium.

2. Wejdź do cyber_lion i wybranego katalogu lub pliku.

3. Przejdź do deklaracji wewnątrz pliku, jeśli są opisane.

4. Wróć przez Wyżej lub nazwę przodka nad sceną.

Źródła: S42 enter; up; navigateArchitecture

TEMAT 022

Model projektu a przypięte drzewo Git

Pasek nad sceną katalogową udostępnia dwa tryby: Model projektu i Drzewo Git. Pierwszy używa projektowanego rodzica, jeśli został zadeklarowany w nakładce design, a w pozostałych przypadkach rodzica źródłowego. Drugi pokazuje źródłowe zawieranie zapisanej obserwacji. Przełączenie trybu zachowuje tożsamość obiektu, lecz ścieżka przodków może się zmienić, ponieważ odpowiada innemu porządkowi.

To rozdzielenie pozwala zaprojektować przyszłą strukturę bez udawania, że pliki zostały już przeniesione. Artefakt może otrzymać nową nazwę, odpowiedzialność albo projektowanego rodzica, a nadal zachować pierwotną ścieżkę i commit. Podczas przeglądu refaktoryzacji porównuj oba tryby. Model projektu odpowiada na pytanie o planowaną kompozycję, źródłowe drzewo na pytanie o strukturę, z której zbudowano obserwację.

Współrzędne źródła obejmują repozytorium, ref, commit i ścieżkę, gdy katalog je ustalił. Dłuższe informacje mogą być dostępne w podpowiedzi paska i w karcie elementu. Ten sam plik może mieć wiele symboli o różnych ID, które dzielą ścieżkę, lecz różnią się zakresem wierszy i rodzicem. Dlatego sama nazwa pliku nie identyfikuje wystarczająco klasy ani metody.

Nazwa Drzewo Git nie zmienia kwalifikacji rekordów. Historyczny atlas zawiera również lokalne snapshoty niezweryfikowane jako obiekty Git i deskryptory o niepełnym pokryciu. Po przełączeniu nadal zachowują te ograniczenia. Aktualny working tree, przypięty commit i model projektowanej kompozycji pozostają odrębnymi powierzchniami. Własną aktualną obserwację z innego commita należy utworzyć przez indeksowanie repozytorium.

Przy zmianie samej nazwy w trybie Git program nie powinien zastępować odmiennego projektowanego rodzica ani projektowanego kierunku relacji tylko dlatego, że pokazano źródłowe połączenie. Do katalogu trafiają faktycznie zmienione pola. Jeśli jednak zamierzasz przebudować topologię, zrób to świadomie w Modelu projektu i udokumentuj powód. Widok nie wykonuje przeniesienia plików ani operacji na gałęzi Git.

Tryb	Podstawa rodzica	Cel
Model projektu	design.parent_id lub źródło	Projektowana kompozycja
Drzewo Git	Źródłowy parent_id	Odczyt przypiętej struktury

TEMAT 023

Stronicowanie wnętrza i rzeczywiste liczniki

W dużym katalogu pojedynczy artefakt może zawierać więcej dzieci, niż warto rysować jednocześnie. R3.1 domyślnie pobiera stronę obejmującą 36 bezpośrednich podobiektów. Przyciski Następne i Poprzednie zmieniają zakres strony wewnątrz tego samego obiektu. Przedział, na przykład 37-72 z 128, oznacza fragment listy dzieci, a nie zmianę rodzica ani przeniesienie rekordów między katalogami.

Licznik całego zakresu pozostaje niezależny od liczby brył po zastosowaniu filtrów i od liczby odnośników kontekstu. Przykładowo wnętrze może mieć 53 dzieci, strona obejmować 36, scena po filtrze pokazywać 8, a dodatkowo zawierać kilka zewnętrznych sąsiadów. Każda liczba ma inne znaczenie. Przed wnioskiem o brakujących artefaktach porównaj opis zakresu, stronę i aktualne ograniczenia wyszukiwania.

Wejście w obiekt na drugiej stronie otwiera jego własne dzieci. Powrót do rodzica może ponownie rozpocząć widok od strony początkowej; nie należy używać numeru strony jako trwałego identyfikatora położenia. Trwałe odniesienie zapewniają ID węzła, katalog, rewizja i ścieżka źródła. W notatce z analizy zapisz te współrzędne zamiast samego zdania „trzeci obiekt na następnej stronie”.

Pusty zakres strony jest odróżniany od liścia. Komunikat o braku elementów na tej stronie sugeruje powrót przez Poprzednie albo rodzica. Komunikat o braku bezpośrednich podobiektów opisuje strukturę węzła. Z kolei brak po zastosowaniu filtra wymaga zmiany wyszukiwania. Takie rozróżnienie pozwala uniknąć ręcznego dodawania duplikatów w odpowiedzi na problem, który dotyczy wyłącznie prezentacji.

Przechodzenie stron zbiera edycje do szkicu nawigacji, więc ręczna zmiana nazwy albo utworzony podobiekt nie musi pozostać na ekranie, aby był zachowany w sesji. Nie jest to jednak automatyczny zapis na dysk. Po zakończeniu serii zmian wybierz Zapisz, a dla pełnego katalogu użyj jawnego scalenia. Stronicowanie ogranicza przesył i rysowanie danych; nie kasuje obiektów spoza bieżącego wycinka.

1. Odczytaj przedział strony oraz pełną liczbę dzieci.

2. Wybierz Następne, aby obejrzeć kolejną partię obiektów.

3. Przy pustym widoku rozróżnij filtr, pustą stronę i liść.

4. Zachowuj ID oraz ścieżkę, a nie pozycję obiektu na stronie.

TEMAT 024

Czytanie zależności i linii agregujących

W scenie katalogowej nie każda linia odpowiada pojedynczej relacji pomiędzy dwoma bezpośrednio widocznymi rekordami. Dokładna relacja zachowuje własny identyfikator, gdy jej końce można przedstawić bezpośrednio. Jeżeli połączenie leży głębiej w dwóch widocznych gałęziach, scena może pokazać agregację oznaczoną Σ. Podsumowanie pozwala dostrzec zależność bez jednoczesnego rozwijania całej struktury plików i symboli.

Przykładowo import pomiędzy plikami w dwóch podkatalogach może być widoczny poziom wyżej jako połączenie podkatalogów. Nie znaczy to, że katalog wykonuje import jako jednostka programu. Linia reprezentuje zbiór relacji potomków przypisanych do widocznych gałęzi. Po wejściu głębiej końce stają się dokładniejsze, a liczniki mogą rozłożyć się na kilka połączeń lub relacje zamknięte wewnątrz jednej gałęzi.

Kliknij agregację, aby otworzyć AGREGACJA RELACJI WIDOKU w inspektorze. Odczytaj liczbę relacji źródłowych i rozwiń Przypięcia i podsumowanie. Metadane zachowują identyfikatory, kwalifikacje oraz przykłady, w granicach zadeklarowanego zakresu widoku. Przycisk Otwórz katalog relacji kieruje do katalogu, w którym można badać właściwe rekordy. Samej projekcji nie edytuje się jak nowego kontraktu.

Pole Linie relacji pozwala wybrać Wszystkie połączenia, Połączenia zaznaczonego albo Ukryj linie. Gdy wczytany zakres ma ponad 120 relacji, początkowo wybierany jest tryb zaznaczenia. Kliknij wtedy artefakt, aby zobaczyć jego otoczenie. Brak linii przed wyborem nie jest równoznaczny z brakiem zależności. Licznik rozdziela liczbę dostępnych połączeń od liczby aktualnie narysowanych.

Krawędzie zawierania nie są przekształcane w przepływ danych. Import, odwołanie do ścieżki i zależność AST także nie stają się automatycznie wykonywalnym dataflow. Do symulacji wartości potrzebny jest jawny kontrakt portów i wiązań. Agregacja ma przede wszystkim znaczenie analityczne: pomaga wskazać granicę wymagającą dalszego odczytu i zaplanować, które wnętrza warto zbadać przed refaktoryzacją.

1. Usuń filtry obiektów i wybierz pożądany tryb linii.

2. Kliknij linię Σ i odczytaj liczbę relacji źródłowych.

3. Wejdź w jedną z połączonych gałęzi, aby uszczegółowić końce.

4. Do zmiany kontraktu wybierz właściwą relację źródłową.

Źródła: S42 navigationRelationInspector; renderNavigation

TEMAT 025

Kontekst zewnętrzny i przekraczanie granicy

Wnętrze artefaktu nie zamyka jego wszystkich zależności. Drugi koniec relacji może należeć do sąsiedniego katalogu, innego repozytorium albo obiektu na kolejnej stronie. Studio przedstawia takie połączenie przez odnośnik granicy, zamiast kopiować źródłowy obiekt do bieżącej kompozycji. Bryła kontekstu ma odmienny styl i prowadzi do właściwego rekordu. Jest elementem nawigacji, a nie nowo utworzonym podobiektem.

Kliknięcie takiego elementu otwiera inspektor KONTEKST NAWIGACJI. Przeczytaj wskazane źródło, opis i metadane pochodzenia. Przycisk Wejdź do wskazanego obiektu przechodzi do jego rzeczywistego zakresu. To właściwa droga, gdy chcesz zbadać klasę w innej gałęzi albo edytować właściwy artefakt. Edycja zastępczej bryły kontekstu byłaby niejednoznaczna, dlatego jest blokowana przez interfejs.

Pole Kontekst zewnętrzny udostępnia 12, 32 lub 64 sąsiadów. Domyślne 12 pozwala zachować czytelny kadr. Zwiększ limit przy badaniu szerszego sprzężenia, a zmniejsz, kiedy kamera odsuwa się zbyt daleko. Zmiana limitu pobiera ponownie aktualny zakres i zachowuje szkic edycji. Nie zwiększa automatycznie liczby bezpośrednich dzieci na stronie ani kompletności analizy dynamicznych wywołań.

Pasek nawigacji informuje o relacjach pominiętych z powodu limitu kontekstu i o grupach poza limitem widoku. Tych liczb nie należy pomijać przy ocenie wpływu zmiany. Jeśli potrzebujesz kompletnej listy, otwórz katalog i kartę elementu albo wyeksportuj pełny katalog. Zewnętrzny kontekst jest celowo ograniczoną projekcją, która ma ułatwiać wybór dalszej drogi badania.

Przejście do sąsiada może zmienić ścieżkę katalogową i repozytorium widoczne na pasku. Zachowaj tę informację w notatce, aby nie pomylić dwóch identycznie nazwanych plików. Wróć przez znanego przodka albo odszukaj poprzedni element w katalogu. Odnośniki granic i agregacje nie są scalane do katalogu jako ręczne kontrakty, więc samo ich oglądanie nie rozbudowuje sztucznie modelu federacji.

1. Wybierz bryłę oznaczoną jako kontekst poza zakresem.

2. Przeczytaj KONTEKST NAWIGACJI i współrzędne źródła.

3. Użyj Wejdź do wskazanego obiektu.

4. W razie potrzeby zwiększ liczbę sąsiadów i sprawdź pominięcia.

Źródła: S42 navigationContextInspector; renderNavigation

TEMAT 026

Inspektor obiektu i bezpieczna zmiana rodzica

Zakładka Obiekt jest podstawowym formularzem artefaktu. Obejmuje Nazwę, Rodzaj, Opis odpowiedzialności, Właściciela oraz Zawieranie - rodzic. Uzupełnij odpowiedzialność zdaniem opisującym czynność lub granicę. Rodzaj pomaga klasyfikować artefakt, lecz nie uruchamia automatycznie konkretnego zachowania programu. Broker i Fabryka są rodzajami opisu, a nie poleceniami utworzenia działającej usługi.

Pole rodzica określa wnętrze, w którym artefakt występuje jako bezpośrednie dziecko. Nie zastępuje relacji ownership, refines ani defines. Podczas zatwierdzenia interfejs sprawdza, czy wybór nie tworzy cyklu zawierania. Nie można uczynić artefaktu własnym przodkiem przez wskazanie jednego z jego potomków. W projekcie katalogowym wybieraj rodzica w Modelu projektu, pamiętając o odrębnej źródłowej strukturze Git.

Formularz zawiera współrzędne i Skalę bryły oraz niezależne osie opisu. Klasa dowodu, Materializacja, Aktualność, Płaszczyzna, Poziom szczegółu i Horyzont czasu odpowiadają różnym pytaniom. Zmiana jednej osi nie aktualizuje pozostałych automatycznie. Na przykład ustawienie SOURCE nie poświadcza TESTED_OFFLINE, a PINNED nie oznacza obserwacji uruchomionego procesu. Zachowaj źródło uzasadniające własną kwalifikację.

Poziom topologiczny wymaga ID poziomu, całkowitej Rangi i Ramy odniesienia. Wypełnienie tylko części tych pól jest błędem formularza. W celu usunięcia deklaracji poziomu wyczyść cały zestaw. Poziom semantyczny nie wynika ze współrzędnej Y ani z głębokości drzewa. Jeżeli dwa artefakty porównujesz jako równorzędne, określ wspólną ramę i odpowiednią relację, zamiast opierać wniosek na podobnej wysokości brył.

Przycisk Zastosuj zmiany obiektu aktualizuje model w pamięci przeglądarki. Dopiero Zapisz utrwala dokument. Duplikuj tworzy nowy artefakt i oznacza pochodzenie od poprzednika; nie kopiuje całego wnętrza jako osobnej hierarchii. Usuń obejmuje także wczytanych potomków i związane relacje po potwierdzeniu. Przed operacją sprawdź zakres i zapamiętaj, że usunięcia z wycinka nie propagują się do pełnego katalogu.

1. Wybierz artefakt i otwórz Obiekt.

2. Zmień opis oraz właściwe osie, zachowując uzasadnienia.

3. Sprawdź rodzica i komplet deklaracji poziomu.

4. Zastosuj formularz, sprawdź model i zapisz dokument.

Źródła: S42 objectInspector; object-form submit

TEMAT 027

Ściany, kafle ról i adresowane porty

Zakładka Ściany udostępnia czternaście funkcji profilu ośmiościanu ściętego. Pierwszych sześć pól odpowiada ścianom kwadratowym, pozostałe ośmiu sześciokątnym. Funkcje obejmują między innymi Tożsamość, Kontrakt, Granicę, Pochodzenie, Wejście, Wyjście, Kompozycję i Ewolucję. Kliknięcie pola wybiera ścianę i pokazuje przypisane do niej kafle. Wybranie ściany na bryle prowadzi do tej samej części opisu.

Kafel jest nazwaną rolą interfejsu, a nie kolejnym podobiektem zawierania. Jedna ściana może mieć wiele kafli. Przycisk Kafel otwiera etykietę, rolę, wybór ściany i opis. Etykieta służy czytelności, role takie jak producer, verifier lub router precyzują zadanie. Warto oddzielić różne obowiązki, jeżeli wymagają innych portów lub warunków, zamiast umieszczać wszystkie znaczenia w jednej ogólnej nazwie.

Przycisk Port dodaje adresowane wejście lub wyjście w obrębie kafla. Określ ID portu, nazwę, kierunek in, out lub inout, typ danych, jednostkę i pojemność. Zero pojemności oznacza brak limitu deklarowanego w tym polu. Port może wskazywać parametr artefaktu oraz schema_ref. Jego pełny adres składa się z identyfikatora artefaktu, kafla i portu, dlatego identyczne ID portu w różnych kaflach nie musi oznaczać konfliktu.

ID portu musi być unikatowe w obrębie wybranego kafla. Jeśli istniejąca relacja używa portu, interfejs blokuje prostą zmianę jego ID i wymaga wcześniejszej korekty końca relacji albo zachowania dotychczasowej nazwy technicznej. Usunięcie kafla lub portu pozostawia powiązane relacje do poprawienia. Po takim zabiegu uruchom walidację; samo zamknięcie okna edycji nie zapewnia spójności kontraktów.

Profil czternastu ścian jest reprezentacją projektową. Nie stanowi automatycznego adaptera do rodzimych portów S6/H8 i nie uruchamia brokera. Rzeczywisty kontrakt wymaga jawnego powiązania z właściwym protokołem. W małym ćwiczeniu zdefiniuj osobny kafel Wyjścia dla producenta i Wejścia dla odbiorcy, po jednym zgodnym porcie, a dopiero później połącz je relacją dataflow i wiązaniem parametrów.

1. Wybierz Ściany oraz funkcję, na przykład Wyjście.

2. Dodaj kafel z konkretną etykietą i rolą.

3. Dodaj port out lub in oraz typ i jednostkę.

4. Połącz zgodne adresy i uruchom Sprawdź model.

Źródła: S42 faceInspector; editTile; editPort

TEMAT 028

Parametry, jednostki i wartość UNKNOWN

Parametry przechowują wartości modelowane na artefakcie. W zakładce Parametry wybierz Parametr i podaj klucz, typ, jednostkę, wartość, znaczenie oraz pochodzenie. Dostępne typy to number, integer, boolean, string, object, array i any. Klucz powinien opisywać jedną stabilną wielkość, ponieważ porty i wiązania mogą odwoływać się do niego bezpośrednio. Sama zmiana etykiety portu nie zmienia klucza parametru.

Dla number wpisuj skończoną liczbę, dla integer liczbę całkowitą. Wartość boolean należy podać jako true lub false. Typ string zachowuje tekst. Dla object, array i any formularz oczekuje poprawnego JSON; obiekt i tablica mają dodatkowo właściwy kształt. Wpisanie tekstowego „false” w parametrze string nie daje wartości logicznej false. Typ jest częścią kontraktu i wpływa na zgodność propagacji.

Pole Wartość nieznana - UNKNOWN zapisuje null. Jest to jawna niewiedza, a nie liczba zero, pusty tekst lub wynik negatywny. Na przykład brak pomiaru opóźnienia nie uzasadnia wpisania 0 ms. Parametr z UNKNOWN może blokować bramkę albo propagację i właśnie taki wynik jest informacją analityczną. Doświadczenie z nieznaną wartością należy zachować jako stan badania, a nie maskować liczbą pomocniczą.

Jednostka opisuje znaczenie wartości. Zgodne liczby z różnymi jednostkami wymagają świadomej konwersji w wiązaniu. Edytor nie zgaduje mnożnika między sekundami i minutami ani między skalami umownymi. Pochodzenie wartości może pozostać DECLARED albo wskazywać konkretny materiał. Zmiana tego pola jest deklaracją autora; nie powoduje odczytu czujnika, pliku źródłowego ani działającej usługi federacji.

Istniejący klucz zachowuje się przy zwykłej edycji formularzem. Migracja klucza wymaga skoordynowanej zmiany przez pełny JSON, ponieważ dotyczy portów i wiązań. Usunięcie parametru również może pozostawić odwołania do poprawienia. Po edycji przejrzyj relacje i walidację. Wynik Symulacji pozostaje osobnym wynikiem obliczenia; nie zastępuje automatycznie bazowych parametrów dokumentu i nie potwierdza wartości w rzeczywistym procesie.

Zapis	Interpretacja
0	Znana liczba zero
false	Znana wartość logiczna
null / UNKNOWN	Brak ustalonej wartości
"false" jako string	Tekst, nie warunek logiczny

Źródła: S42 parameterInspector; editParameter; parseValue

TEMAT 029

Źródła artefaktu i interfejs plikowy

Zakładka Źródła zbiera odwołania do materiałów uzasadniających opis. Przycisk Źródło otwiera ścieżkę lub locator, repozytorium, Exact ref / commit, SHA-256, status oraz Pointer / strona / zakres. Jeden artefakt może mieć kilka takich wpisów. Przy analizie kodu warto wskazać konkretny commit i zakres wierszy; przy dokumentacji stronę lub sekcję. Sam ogólny adres repozytorium utrudnia późniejsze odtworzenie podstawy decyzji.

Formularz sprawdza zapis SHA-256: wypełniona wartość musi mieć 64 znaki szesnastkowe. Poprawny kształt hasha nie potwierdza jednak zgodności z plikiem. Wpisana ręcznie ścieżka również nie uruchamia odczytu zawartości. Źródła są odwołaniami o wskazanym statusie. W karcie pełnego katalogu można dodatkowo zobaczyć blob Git, drzewo, lokalną ścieżkę i dokładny odsyłacz internetowy, jeżeli zostały ustalone.

Jeśli artefakt posiada metadata.file_interface, zakładka pokazuje sekcję Interfejs plikowy i przycisk Edytuj. Zapis obejmuje ścieżkę, SHA-256, rozmiar w bajtach, media type, język, kodowanie oraz tablice eksportów, importów i luk. Wprowadzane tablice muszą być poprawnym JSON. Rozmiar może pozostawać nieznany; znana wartość musi być nieujemną liczbą całkowitą. Opis interfejsu nie zmienia pliku na dysku.

W sekcji Metadane / ograniczenia dostępny jest pełny zapis metadanych artefaktu. Zachowuj pola pochodzenia, kwalifikacji i powiązania z katalogiem, jeśli edytujesz dokument źródłowy. Zastąpienie całego obiektu metadanych przypadkowo skróconym fragmentem może usunąć informacje niepokazywane w prostym formularzu. Przed większą zmianą wyeksportuj model i porównaj wynik po walidacji oraz ponownym otwarciu elementu.

Praktyczna nota źródłowa powinna umożliwiać odpowiedź na trzy pytania: skąd pochodzi twierdzenie, do jakiej wersji odnosi się opis i czego nadal nie ustalono. Historyczny snapshot można wykorzystać do analizy, zachowując status HISTORICAL lub inną właściwą kwalifikację. Nie należy zmieniać go na OBSERVED tylko po otwarciu okna. Obserwacja działającego procesu wymaga osobnego dowodu i czasu odniesienia.

1. Dodaj odwołanie z repozytorium, ref i precyzyjnym pointerem.

2. Wpisz hash tylko wtedy, gdy masz podstawę jego wartości.

3. Przejrzyj luki i ograniczenia interfejsu plikowego.

4. Po zmianie zachowaj eksport oraz sprawdź kwalifikację źródła.

Źródła: S42 sourcesInspector; editSource; actions.file-interface

TEMAT 030

Tworzenie relacji między artefaktami

Przycisk Relacja albo Połącz w inspektorze otwiera formularz Połącz artefakty. Wymagane są co najmniej dwa edytowalne artefakty w bieżącym dokumencie. Nadaj relacji nazwę, wybierz typ, początek i koniec. Listy Od - kafel / port oraz Do - kafel / port pozwalają wskazać pełne adresy interfejsów. Kierunek i semantykę sprawdzaj przed zatwierdzeniem, ponieważ program nie wyprowadza ich z opisu naturalnego.

dataflow wymaga portów po obu stronach i służy modelowaniu przekazywania danych. exposes opisuje delegację portu między rodzicem i bezpośrednim dzieckiem. Relacje takie jak depends_on, refines, realizes, defines, verifies lub supports mogą łączyć same artefakty bez portów. Wybór typu nie dodaje automatycznie wiązania parametrów. Linia może więc poprawnie istnieć jako opis znaczenia i jeszcze nie propagować żadnej wartości.

Formularz obejmuje Znaczenie / warunki, Referencję kontraktu, Modalność, Polaryzację, Klasę dowodu i Aktualność. Relację krytyczną dla kompozycji oznacza się osobnym polem. Wpisz, co konkretnie zależy od poprawności połączenia. Ustawienie observed bez materiału obserwacyjnego nie staje się takim materiałem. Dobre modelowanie zachowuje różnicę między deklaracją, kandydatem i potwierdzonym faktem, nawet jeśli wszystkie mają podobną linię na scenie.

Po utworzeniu relacji inspektor udostępnia Bramkę, Wiązania parametrów, Zależność czasu oraz Topologię i dynamikę. Te części są odrębne. Przepływ wartości potrzebuje jawnego mapowania parametru źródłowego na docelowy. Zależność czasu potrzebuje własnej deklaracji FS, SS, FF albo SF. Miara siły lub odległości wymaga jednostki i uzasadnienia. Samo narysowanie strzałki nie wypełnia tych kontraktów.

Relacja zapisuje rewizje artefaktów na obu końcach. Po zmianie semantyki końca może być oznaczona STALE. Użyj Powiąż bieżące rewizje dopiero po sprawdzeniu zgodności portów, typów, jednostek i warunków. To działanie aktualizuje przypięcia, ale nie testuje rzeczywistego brokera. Po zmianach wybierz Sprawdź model, a dopiero przy poprawnym kontrakcie uruchamiaj symulację odpowiedniego rodzaju i zachowuj jej oddzielny wynik.

1. Wskaż właściwe artefakty i kierunek połączenia.

2. Dla dataflow wybierz zgodne porty na obu końcach.

3. Opisz znaczenie, źródło i modalność relacji.

4. Dodaj wymagane wiązania, sprawdź model i zapisz projekt.

Źródła: S42 editRelation; relationInspector

TEMAT 031

Cofanie, ponawianie i lokalny dziennik

Cofnij i Ponów służą do lokalnych zmian dokumentu przed zapisem. Dostępne są także skróty Ctrl+Z oraz Ctrl+Shift+Z, gdy użytkownik nie edytuje aktywnego pola tekstowego i nie ma otwartego dialogu. Stos obejmuje do 50 stanów cofania. Nie należy traktować go jako nieograniczonej historii ani jako zamiennika zapisanych rewizji. Dłuższą pracę dziel na nazwane, zachowane etapy.

Każda operacja modelowania umieszcza wcześniejszy projekt, zaznaczenie i zakres na stosie. Cofnięcie przywraca ten stan, a ponowienie przesuwa go w przeciwną stronę. Nowa edycja po cofnięciu czyści możliwość ponowienia poprzedniej gałęzi. Samo przechodzenie między wnętrzami nie jest zmianą semantyczną obiektu; zachowanie cofania w katalogu obejmuje jednak dokument ze szkicem i związanym zakresem nawigacji.

Dziennik łączy lokalne zdarzenia z zapisanymi zdarzeniami projektu. Wpis oznaczony lokalnie opisuje czynność wykonaną w bieżącej sesji. Nie jest jeszcze samodzielną trwałą rewizją bazy. Po wybraniu Zapisz powody lokalnych zmian są łączone w zapis zmiany, a lokalne listy i stosy są czyszczone. Zachowana historia dokumentu pozostaje dostępna i pozwala później odtworzyć wcześniejszą rewizję.

Wyniki walidacji, propagacji i czasu są resetowane przy modyfikacji. Jest to istotne także po Cofnij: przywrócenie wcześniejszego kształtu nie wczytuje automatycznie wcześniejszego wyniku obliczenia do konsoli. Ponownie uruchom właściwą analizę, aby mieć jednoznaczną podstawę. Używanie starego zrzutu wyników po zmianie modelu mogłoby sugerować zgodność, której aktualna sesja jeszcze nie sprawdziła.

Przed dużą operacją, na przykład usunięciem wnętrza lub zmianą pełnego JSON, zapisz punkt odniesienia i wyeksportuj dokument. Jeśli od razu zapiszesz pomyłkę, zwykłe Cofnij nie będzie już poprzednią drogą odzyskania. Wtedy korzysta się z przywrócenia zapisanej rewizji jako nowej wersji. Rozdzielenie lokalnej korekty i trwałego przywracania pozwala zachować historię decyzji bez udawania, że popełniona zmiana nigdy nie istniała.

Narzędzie	Zakres
Cofnij / Ponów	Lokalne stany przed zapisem
Dziennik lokalnie	Czynności bieżącej sesji
Zapisz	Nowa trwała rewizja projektu
Przywróć	Nowa rewizja z dawną treścią

Źródła: S42 mutate; actions.undo; actions.redo; renderConsole

TEMAT 032

Walidacja przed dalszym modelowaniem

Przycisk Sprawdź model przesyła bieżący dokument do walidatora. Badane są między innymi identyfikatory, końce relacji, porty, typy, granice, bramki i przypięcia rewizji. Wynik pojawia się w zakładce Walidacja. Każdy komunikat zawiera poziom ważności, kod, treść i ścieżkę w dokumencie. Ścieżka jest wskazówką do konkretnego miejsca, nie nazwą pliku źródłowego analizowanego repozytorium.

Zacznij od komunikatów ERROR. Brak portu może być skutkiem usunięcia kafla, a nie pomyłki samej relacji. Niezgodność typu może wymagać poprawy kontraktu obu końców, zamiast zmiany pojedynczej etykiety. Identyfikator nieistniejącego rodzica wskazuje problem zawierania. Przed korektą odczytaj powiązany artefakt, jego rewizję i opis źródła, aby nie naprawiać formatu kosztem znaczenia modelu.

Ostrzeżenia i kwalifikacje niewiedzy także wymagają interpretacji. Wartość UNKNOWN może być poprawnym, celowym zapisem braku danych, a nie błędem do zamiany na zero. STALE oznacza między innymi nieaktualne przypięcie końca relacji. Po sprawdzeniu zgodności można użyć Powiąż bieżące rewizje. Nie jest to jednak dowód przetestowania kontraktu w działającym środowisku; walidacja pozostaje badaniem lokalnego dokumentu.

Jeżeli model nie ma problemów strukturalnych, konsola pokazuje potwierdzenie poprawności. Ten wynik nie weryfikuje prawdziwości źródeł, aktualności deploymentu ani kompletności statycznej analizy kodu. Model może być formalnie spójny i jednocześnie zawierać hipotezę, która później zostanie odrzucona. Zachowuj zatem klasy dowodu i odrębne obserwacje. Celem walidatora jest wykrycie niespójności z zadeklarowanym standardem, a nie zastąpienie badań.

Po każdej większej zmianie uruchom kontrolę ponownie. Jest to szczególnie potrzebne po usunięciu parametru, portu, kafla lub po pełnej edycji JSON. Przy zgłoszeniu problemu zachowaj dokładny kod i ścieżkę, a najlepiej dołącz eksport modelu albo pakiet asystenta. Zrzut sceny bez diagnostyki może nie ujawniać, czy problem dotyczy typu, rewizji, bramki czy przekroczenia granicy zawierania.

1. Uruchom Sprawdź model i otwórz Walidację.

2. Czytaj najpierw ERROR wraz z kodem i ścieżką.

3. Popraw przyczynę w artefakcie lub relacji, zachowując znaczenie.

4. Ponów walidację i zapisz zaakceptowaną wersję dokumentu.

Źródła: S42 actions.validate; renderConsole

TEMAT 033

Pełny JSON i metarama założeń

Przycisk JSON otwiera Pełny model projektu. Jest to edytor całego dokumentu, obejmującego artefakty, relacje, grupy, ramę i rozszerzenia metadanych. Używaj go wtedy, gdy prosty formularz nie pokazuje potrzebnej struktury albo konieczna jest skoordynowana migracja kilku odwołań. Przed rozpoczęciem wyeksportuj bieżący JSON. Duży dokument łatwiej porównać z zachowaną wersją niż odtwarzać z pamięci.

W tym edytorze należy zachować project_id i revision. Zmiana tych pól jest blokowana komunikatem sugerującym Importuj dla nowego dokumentu. Po zatwierdzeniu uruchamiana jest walidacja, a błędy strukturalne uniemożliwiają zastosowanie treści. Poprawny JSON oznacza jedynie prawidłową składnię. Dokument może nadal mieć nieistniejące końce relacji, błędne typy albo cykl rodziców, dlatego potrzebna jest także walidacja modelu.

Zachowuj pełne obiekty, których nie zamierzasz zmieniać. Usunięcie nieznanego pola metadanych może pozbawić artefakt źródłowego przypięcia, informacji o lukach albo związku z katalogiem. W szczególności projekt nawigujący korzysta z catalog_view, navigation i navigation_draft. Przepisanie wyłącznie widocznych pól z formularza nie stanowi kompletnego dokumentu. Bezpieczniejsza jest minimalna edycja określonych kluczy oraz ponowna kontrola zakresu.

Metarama projektu jest osobnym formularzem. Pozwala zmienić nazwę dokumentu, ID i wersję ramy, opis założeń oraz tablicę assumptions. Założenia muszą być tablicą JSON. Wpisuj tam konwencje dotyczące znaczenia poziomów, odległości, pochodzenia i granic badania. Rama nadaje odniesienie interpretacji; nie zamienia opisanej reguły w wykonywany kod i nie wprowadza automatycznie walidatora dowolnego języka teorii.

Po zastosowaniu pełnego JSON zaznaczenie lub zakres mogą zostać wyczyszczone, jeśli wskazywany element już nie istnieje. Sprawdź wtedy korzeń projektu, liczbę artefaktów i relacji oraz nowy wynik Walidacji. Zastosowanie dokumentu pozostaje zmianą lokalną, możliwą do cofnięcia przed zapisem. Gdy wynik odpowiada zamierzonemu modelowi, zapisz rewizję i zachowaj krótki opis powodu transformacji w dzienniku pracy.

1. Wyeksportuj dokument przed rozległą edycją.

2. Otwórz JSON i zmień wyłącznie zamierzone struktury.

3. Zachowaj project_id, revision oraz wymagane przypięcia.

4. Zastosuj, sprawdź wynik i zapisz zaakceptowaną rewizję.

Źródła: S42 actions.advanced; actions.frame; jsonModal

TEMAT 034

Import projektu i import katalogu

Główne Importuj wczytuje dokument projektu Studio z pliku JSON. Po wybraniu pliku interfejs sprawdza limit 8 MB, analizuje składnię i przekazuje dokument do migracji oraz walidacji. Przed zastąpieniem aktualnego widoku sprawdza niezapisane zmiany. Sam import tworzy stan do przeglądu w przeglądarce. Dopiero Zapisz zachowuje go jako nowy, odrębny projekt w lokalnym magazynie.

Status Import - zapisz jako nowy przypomina o tym etapie. Obejrzyj nazwę, strukturę, źródła i zgłoszenia Walidacji przed zatwierdzeniem. Nie zakładaj, że ID z importowanego pliku ma nadpisać istniejący dokument. Serwer tworzy własną nową tożsamość zapisywanego projektu. Dzięki temu odbiorca może obejrzeć otrzymany model bez automatycznego zastąpienia własnej historii o podobnej nazwie.

Importuj model JSON w części Architektura / Git dotyczy innego formatu: pełnego katalogu lion.architecture/v1. Limit pliku wynosi tam 64 MB. Dokument zawiera węzły, relacje, repozytoria, warstwy i ewentualne procesy. Nie należy podawać go do głównego importu projektu, licząc na automatyczne zgadnięcie formatu. Widok 3D i pełny katalog mają inne struktury, zakresy oraz zasady rewizji.

Projekt zawierający navigation_draft odwołuje się do katalogu i jego konkretnej rewizji. Zachowanie szkicu w JSON nie oznacza automatycznego dostarczenia całego katalogu do nowego workspace. Przed dalszą nawigacją upewnij się, że odpowiadające przypięcie jest dostępne. Pakiet asystenta może zawierać pełną przypiętą wersję katalogu, ale jest archiwum kontekstu, a nie przyciskiem automatycznego odtworzenia wszystkich baz.

Błąd importu czytaj według warstwy: zbyt duży plik, niepoprawny JSON, niewłaściwy format lub problem walidacji modelu. Nie usuwaj losowych elementów tylko po to, by zmniejszyć dokument; możesz zerwać odwołania. Dla dużej architektury używaj pełnego katalogu oraz ograniczonych wycinków 3D. Zachowaj oryginalny plik wejściowy i opis migracji, aby móc odtworzyć źródło modelu po korektach.

Miejsce importu	Dokument	Limit
Główne Importuj	Projekt Studio JSON	8 MB
Architektura / Git	Katalog architektury JSON	64 MB

Źródła: S42 import-file change; actions.import

TEMAT 035

Eksport projektu i kandydatów plikowych

Eksportuj w głównym pasku otwiera wybór trzech formatów. Projekt JSON - pełny model zapisuje aktualny dokument do pliku lion-project.json. Obejmuje także niezapisane edycje. Jest użyteczny jako szybka kopia pracy, materiał do porównania lub wejście do importu. Samo pobranie takiego JSON nie tworzy nowej rewizji w bazie i nie czyści statusu niezapisanych zmian.

Pakiet asystenta - model, historia, źródła zapisuje bieżącą rewizję przed wygenerowaniem archiwum. Zawiera szerszy kontekst niż pojedynczy dokument. Kandydaci plikowi - deskryptory i szkielety także korzystają z zapisanej wersji i tworzą ZIP. Użytkownik otrzymuje kandydatów materializacji, nie zmianę źródeł projektu. Żaden z tych eksportów nie publikuje artefaktów do brokera ani nie wdraża ich do repozytoriów federacji.

Pełny katalog architektury eksportuje się osobno przez Eksportuj katalog w Architektura / Git. To ważne, gdy scena pokazuje tylko jeden zakres. Dokument sceny nie staje się pełnym atlasem przez nadanie mu nazwy ai_platform. Aby zachować wszystkie rekordy, ich warstwy i źródłowe relacje, wybierz operację katalogu. Bieżące ograniczenie strony lub linii relacji nie usuwa węzłów z tego pełnego eksportu.

Wyniki obliczeń mają własne przyciski eksportu. Symulacja głównego modelu udostępnia Eksport wyniku, a analiza czasu i procesy ewolucji mają oddzielne dokumenty. Zachowaj wynik obok definicji, na której został policzony. Po zmianie modelu wcześniejszy wynik jest materiałem historycznym; plik o tej samej nazwie może zostać zastąpiony przy kolejnym pobraniu, zależnie od ustawień przeglądarki.

Nazwę pobranego pliku warto uzupełnić datą, nazwą projektu i rewizją. Nie zmieniaj wewnętrznych identyfikatorów tylko dla porządku na dysku. Przed przekazaniem archiwum otwórz je i sprawdź, czy zawiera oczekiwany projekt oraz zakres źródeł. Jeśli przeglądarka nie pokazuje pobrania, sprawdź jej listę Pobrane i komunikat serwera. Pomyślne zamknięcie dialogu wyboru formatu nie jest samo w sobie dowodem zachowania pliku.

1. Wybierz format zgodny z celem przekazania.

2. Dla pełnej architektury użyj Eksportuj katalog.

3. Zachowaj wynik analizy razem z odpowiadającą mu definicją.

4. Sprawdź pobrany plik, projekt_id i rewizję przed wysłaniem.

Źródła: S42 actions.export; downloadPackage; download

TEMAT 036

Pakiet asystenta jako zapis kontekstu pracy

Pakiet asystenta służy do przekazania kontekstu dalszej analizy. Przycisk na górnym pasku najpierw zapisuje projekt, jeśli zawiera lokalne edycje lub pochodzi z importu. Następnie pobiera lion-assistant-context.zip. Program sam nie kontaktuje się z modelem AI i nie wysyła archiwum. Użytkownik wybiera, gdzie je przekazać, na przykład do następnej rozmowy po zakończeniu sesji modelowania.

W archiwum project.json zawiera zapisany dokument. events.jsonl opisuje zdarzenia, a revisions przechowuje pełne rewizje. validation.json zachowuje zgłoszenia walidacji, sources.json zbiera odwołania źródłowe, integrity.json przedstawia sprawdzenie łańcucha i jego głowę. assistant_context.md podaje tożsamość, rewizję, digest oraz wskazówki kontynuacji. Dzięki temu odbiorca może odczytać zarówno stan modelu, jak i powody wcześniejszych zmian.

impact.json pokazuje zmienione artefakty i konserwatywne sąsiedztwo grafu. Analiza przechodzi przez relacje w obu kierunkach, dlatego potencjalny wpływ nie jest dowodem rzeczywistej zależności wykonawczej. schedule.json zawiera analizę bazowego zapisanego projektu. Lokalne nadpisania scenariusza „co jeśli” z konsoli Czas nie zastępują w tym pliku bazowych deklaracji. Gdy chcesz przekazać scenariusz, zachowaj jego własny wynik oddzielnie.

Jeśli projekt jest związany z katalogiem, pakiet próbuje dołączyć przypięty architecture/catalog.json, historię katalogu i do pięciu ostatnich archiwalnych symulacji. Gdy wskazana rewizja nie jest dostępna, architecture/gap.json informuje o luce. Taki komunikat zachowaj w analizie. Brak katalogu nie powinien być maskowany przez zastąpienie go przypadkową najnowszą rewizją bez rozpoznania różnic źródłowych.

Dołącz do pakietu krótką instrukcję celu: który artefakt badać, jaki problem odtworzyć i jakiego wyniku oczekujesz. Hash dokumentu i łańcuch zdarzeń ułatwiają kontrolę spójności, ale nie są podpisem tożsamości autora. Wykrywanie obcięcia historii wymaga zachowania niezależnej głowy łańcucha. Ręczna wartość actor pozostaje deklaracją, więc audyt znaczenia i autorstwa należy prowadzić z odpowiednim źródłem dowodowym.

Plik	Cel odczytu
project.json	Stan projektu
events.jsonl	Powody zmian
revisions/	Pełne wcześniejsze dokumenty
validation.json	Problemy strukturalne
sources.json	Odwołania źródłowe
impact.json	Potencjalne sąsiedztwo wpływu
schedule.json	Bazowa analiza czasu

Źródła: S37 assistant_package

TEMAT 037

Przywracanie zapisanych rewizji

Dziennik w dolnej konsoli pokazuje zapisane zdarzenia projektu wraz z numerami rewizji i powodami. Przy starszym wpisie znajduje się Przywróć. Operacja nie usuwa późniejszej historii i nie cofa numeracji bazy. Treść wybranego dokumentu zostaje odtworzona jako nowa rewizja. Dzięki temu ścieżka decyzji pozostaje widoczna także wtedy, gdy zespół wraca do poprzedniej koncepcji architektury.

Przed zatwierdzeniem otwiera się formularz z powodem przywrócenia i ostrzeżeniem o zastąpieniu bieżących niezapisanych edycji. Zachowaj je przez eksport albo zapisz jako świadomy punkt historii, jeżeli są potrzebne do analizy. Nie używaj przywracania jako odpowiednika nawigacji do rodzica. Zakres sceny i treść całego zapisanego dokumentu to różne rzeczy; przywrócenie obejmuje model, nie sam kadr kamery.

Serwer sprawdza expected_revision również podczas tej operacji. Nowszy zapis innej sesji może spowodować konflikt. Najpierw porównaj bieżącą treść z własnym zamiarem powrotu. Ręczne ustawienie numeru rewizji w dokumencie nie rozwiązuje problemu współbieżnych zmian. Przywracana treść powinna być świadomie zaakceptowana jako całość, ze źródłami, relacjami i metadanymi właściwymi dla jej epoki.

Katalog architektury ma oddzielną Historię w Architektura / Git. Tam przywrócenie także tworzy nową rewizję katalogu. Przywrócenie projektu Studio nie przywraca automatycznie pełnego katalogu, a odtworzenie katalogu nie podmienia każdego istniejącego projektu sceny. Projekty mogą pozostawać przypięte do wcześniejszej rewizji. Ta niezależność jest użyteczna dla porównania wersji, lecz wymaga czytania ich współrzędnych.

Po odzyskaniu treści uruchom walidację i właściwą analizę. Zwróć uwagę na przypięcia końców relacji oraz na katalog wskazany przez navigation_draft. Sprawdź, czy przywrócenie odpowiada planowi, a nie tylko przywraca znajomą nazwę na ekranie. W opisie powodu wskaż konkretną przesłankę, na przykład odrzucenie hipotezy podziału odpowiedzialności. Taki zapis jest później znacznie użyteczniejszy niż ogólne „powrót, bo poprzednio działało”.

1. Znajdź właściwą rewizję w Dzienniku lub Historii katalogu.

2. Zachowaj potrzebne lokalne zmiany przed przywróceniem.

3. Wpisz konkretny powód i zatwierdź nową rewizję.

4. Sprawdź przypięcia, walidację i wynik ponowionej analizy.

Źródła: S42 actions.restore; renderConsole

TEMAT 038

Dołączanie źródeł z lokalnego katalogu

Przycisk Źródła z katalogu służy do ograniczonego odczytu plików z komputera, na którym działa serwer. W formularzu wskaż Lokalną ścieżkę katalogu i Maksymalną liczbę plików. Domyślna wartość to 150, a formularz dopuszcza od 1 do 500. Operacja buduje deskryptory, symbole i statyczne kandydackie relacje. Nie przenosi plików i nie uruchamia importowanego kodu Pythona.

Na dużym ai_platform zacznij od wąskiego podkatalogu. Projekt Studio ma limit 500 artefaktów oraz 2000 relacji, a każdy plik może dodać więcej niż jedną bryłę przez deklaracje symboli. Wybranie limitu 500 plików nie gwarantuje zmieszczenia ich wszystkich w projekcie. Po odczycie program pokazuje raport. Przeczytaj pominięcia, ograniczenia i nierozwiązane importy, zanim uznasz strukturę za kompletną.

Skaner pomija typowe katalogi techniczne, między innymi .git, .venv, node_modules, __pycache__ i .idea. Ogranicza także wielkość pojedynczego pliku do 10 MB, łączny odczyt do 50 MB, liczbę wpisów katalogowych oraz liczbę symboli Pythona na plik. Obsługuje wybrane rozszerzenia źródłowe i dokumentacyjne. Odczyt jest więc kontrolowaną obserwacją wybranego zakresu, a nie pełną kopią całego dysku.

Pliki Pythona są analizowane przez AST. Deskryptor może opisać deklaracje, sygnatury i importy, ale statyczna zależność nie potwierdza wykonania funkcji. Źródła otrzymują odniesienie UNVERIFIED_LOCAL_BYTES, ponieważ obserwacja lokalnych bajtów nie dowodzi ich zgodności z commitem. Jeśli potrzebujesz przypiętego modelu Git, użyj indeksowania repozytorium w warsztacie, które pracuje na wskazanych obiektach wersjonowania.

Po odczycie obejrzyj powstały projekt, jego źródła, interfejsy plikowe i walidację. Zapisz model, jeśli deskryptory odpowiadają zamierzonemu zakresowi. Przy ponownym skanowaniu sprawdzaj tożsamości i raport zamiast zakładać automatyczną synchronizację z poprzednią obserwacją. Zmiana pliku na dysku nie aktualizuje otwartego artefaktu sama. Oddziel czas odczytu, treść opisu i projektowaną semantykę odpowiedzialności modułu.

1. Wskaż mały katalog i pozostaw umiarkowany limit plików.

2. Uruchom Odczytaj źródła i przeczytaj raport.

3. Sprawdź deskryptory, statyczne relacje i luki.

4. Zapisz wynik z jasnym oznaczeniem lokalnych bajtów.

TEMAT 039

Praca nad zmianą obejmującą kilka wnętrz

Przykładowy przegląd refaktoryzacji rozpocznij od znalezienia modułu w Architektura / Git i otwarcia wycinka w 3D. Zapisz identyfikator katalogu oraz rewizję. W Modelu projektu zmień opis odpowiedzialności wybranego artefaktu. Następnie wejdź do jego wnętrza, dodaj ręczny podobiekt i nazwij rolę, którą chcesz wyodrębnić. Nowy element jest deklaracją projektu, nie obserwowanym plikiem źródłowym.

Przejdź do sąsiedniej gałęzi i doprecyzuj relację między odpowiedzialnościami. Każde przejście zbiera zmiany do lion.navigation-draft/v1. Szkic obejmuje odwiedzone wcześniej zakresy, więc ograniczona scena nie musi równocześnie zawierać wszystkich zmienionych artefaktów. Powrót do pierwszego wnętrza powinien odtworzyć edycję. To właściwy moment kontroli: porównaj nazwę, opis, rodzica i przypięcia z własną intencją.

Przełącz na Drzewo Git, aby porównać plan ze źródłowym porządkiem. Ścieżka pliku nie powinna zmienić się wyłącznie dlatego, że w projekcie przypisano inną kompozycję. Pamiętaj o źródłowej kwalifikacji modelu i o lokalnych snapshotach, które nie mają potwierdzonego bloba Git. Porównanie topologii jest narzędziem analizy, a nie wykonaniem przenoszenia plików pomiędzy katalogami lub repozytoriami.

Wybierz Zapisz, aby utrwalić projekt ze skumulowanym szkicem. Następnie otwórz Architektura / Git i użyj Zapisz edycje 3D do katalogu, gdy nakładki mają wejść do pełnego modelu. Operacja sprawdza oczekiwaną rewizję katalogu. Zmiana dokonana przez inną sesję powoduje konflikt, który wymaga porównania treści. Nie następuje ciche przełączenie szkicu na najnowsze źródło.

Po scaleniu sprawdź nową rewizję katalogu oraz wybrane elementy z kilku zakresów i ponownie zapisz projekt, jeśli jego przypięcie zostało zaktualizowane. Eksportuj katalog i Pakiet asystenta jako materiał do przeglądu. Usunięcie z ograniczonej sceny nie usuwa źródeł archiwalnych z pełnego katalogu. W planie refaktoryzacji opisz osobno przyszłe usunięcia, migracje kontraktów, testy i warunki przyjęcia implementacji; szkic sam ich nie wykonuje.

1. Zmień odpowiedzialność w pierwszym wnętrzu.

2. Dodaj podobiekt i doprecyzuj relację w kolejnym zakresie.

3. Wróć, sprawdź odtworzenie szkicu i zapisz projekt.

4. Scal edycje 3D do katalogu, zweryfikuj rewizję i eksportuj.

Źródła: S42 navigateArchitecture; S45 mergeView

TEMAT 040

Diagnostyka problemów podczas obsługi

Gdy interfejs nie działa, ustal najpierw, czy problem dotyczy startu serwera, pobrania danych czy interpretacji modelu. Komunikat Nie można uruchomić Studio z błędem portu wymaga wyboru wolnego adresu. Strona widoczna po zatrzymaniu procesu może być wyłącznie ostatnim obrazem. Otwórz adres health właściwej instancji i porównaj wersję. Ten odczyt jest prostszym testem niż przypadkowe zmienianie modelu.

Po ponownym uruchomieniu serwera stara karta może mieć nieaktualny token sesji. Komunikat Brak tokenu sesji. Odśwież Studio wskazuje właśnie tę drogę naprawy. Przed odświeżeniem zachowaj potrzebne lokalne edycje, jeśli jest to możliwe. Nie wpisuj tokenu ręcznie ani nie wyłączaj kontroli origin. Przy różnych portach sprawdź, czy karta i proces należą do tej samej instancji oraz tego samego workspace.

Puste wnętrze wymaga rozróżnienia co najmniej trzech sytuacji: brak bezpośrednich dzieci, strona poza zakresem oraz wynik filtra bez dopasowań. Sprawdź licznik dzieci, usuń filtry, użyj Poprzednie lub Wyżej i porównaj tryby Model projektu oraz Drzewo Git. Brak linii sprawdź osobno: wybierz artefakt, ustaw Wszystkie połączenia i odczytaj pominięcia kontekstu. Ukryty rysunek nie jest automatycznie utraconą relacją.

Przy konflikcie zapisu zachowaj eksport lokalnego dokumentu i porównaj aktualną rewizję. Przy błędzie walidacji skopiuj kod, treść oraz ścieżkę. Przy błędzie serwera przeczytaj lokalny terminal i zanotuj czynność, która go poprzedzała. Sam zrzut ekranu bywa niewystarczający, ponieważ nie pokazuje przypięć i danych strukturalnych. Pakiet asystenta może dostarczyć szerszy kontekst, jeśli zapis i eksport są dostępne.

Dobre zgłoszenie obejmuje wersję R3.1, adres i port, ścieżkę workspace, nazwę i ID projektu, rewizję katalogu, tryb nawigacji, ID obiektu oraz krótkie kroki odtworzenia. Dodaj oczekiwany wynik i faktyczny komunikat. Jeśli model pochodzi z historycznego atlasu, zaznacz to wprost. Pozwala to badać błąd oprogramowania bez mylenia go z luką danych, niezweryfikowaną semantyką lub różnicą między lokalnym checkoutem i commitem.

```
Invoke-RestMethod 'http://127.0.0.1:18765/api/health'
# Oczekuj: ok = True, version = 3.1, mode = LOCAL_DESIGN
```

Źródła: S37 main; Handler.do_GET; Handler.do_POST

TEMAT 041

Artefakt jako podstawowy rekord projektu

Artefakt w Studio jest opisanym obiektem modelu. Może reprezentować aplikację, repozytorium, dokument, moduł, kontrakt, hipotezę lub projektowany komponent. Bryła na scenie jest widokiem tego rekordu. Jej narysowanie nie tworzy procesu systemowego, nie uruchamia pliku i nie potwierdza, że przedstawiony element działa w federacji. Dzięki temu ten sam edytor obsługuje zarówno odwzorowanie istniejących źródeł, jak i projekt przyszłej architektury.

Dokument projektu używa schematu lion.polyhedral.studio/v2. Zawiera metaramę frame oraz kolekcje artifacts, relations, groups i relation_types. Każdy artefakt należy do konkretnego dokumentu przez obecność w artifacts. Stabilny project_id identyfikuje projekt, natomiast id identyfikuje obiekt w jego obrębie. Walidator wymaga niepustych identyfikatorów i zgłasza powtórzenie w tej samej kolekcji.

Podstawowe pola obiektu to nazwa, rodzaj, rewizja, rodzic, opis odpowiedzialności, owner_ref, pozycja i skala. Osie axes opisują kwalifikację, sources przechowuje odniesienia, parameters deklaruje wartości, a tiles grupuje interfejsy na ścianach. metadata jest miejscem rozszerzeń, w tym topologii, interfejsu plikowego i powiązania z katalogiem architektury.

Normalizacja dodaje brakujące wartości domyślne, ale nie naprawia błędnie podanych danych przez zastąpienie ich wymyślonymi wartościami. Przykładowo tekst w miejscu position pozostanie błędem. Ten rozdział używa krótkich fragmentów JSON jako przykładów pól; fragment bez otaczającego projektu nie jest samodzielnym plikiem do importu.

Przy projektowaniu zacznij od odpowiedzialności: co ten element obejmuje, czego potrzebuje i co udostępnia. Dopiero potem dobierz rodzaj, wnętrze i porty. Pusty artefakt z jasnym opisem jest czytelniejszy niż obiekt z czternastoma dekoracyjnymi interfejsami. Walidacja sprawdza spójność zapisu, natomiast trafność podziału pozostaje zadaniem analityka. Dodaj relację dopiero wtedy, gdy potrafisz jednoznacznie wskazać jej znaczenie, kierunek oraz źródło uzasadnienia.

```
{"id":"component:parser","name":"Parser kontraktów",
 "kind":"component","revision":1,"parent_id":null,
 "position":[0,0,0],"size":1,"parameters":{},"tiles":[]}
```

Źródła: S33 normalize_workspace; validate_workspace; S36 _base; _artifact

TEMAT 042

Tożsamość obiektu i trzy rodzaje wersjonowania

Nazwa artefaktu służy człowiekowi, a id służy referencjom. Dwa obiekty mogą mieć podobną nazwę, ale nie mogą używać tego samego identyfikatora w jednej kolekcji artifacts. Zmiana etykiety nie powinna być sposobem tworzenia nowej tożsamości. Relacje odwołują się do id, więc zwykłe przemianowanie zachowuje połączenia, chociaż zmiana semantyczna może wymagać ponownej oceny ich zgodności.

Pole revision artefaktu jest dodatnią liczbą całkowitą. Nowy obiekt utworzony w formularzu zaczyna od 1. Funkcja editArtifact porównuje semantyczny stan przed i po edycji; jeśli zmienił się opis, zwiększa rewizję. Porównanie pomija position, size i samo revision. Przesunięcie bryły nie oznacza zatem nowej wersji kontraktu. Projekt ma osobną rewizję dokumentu: stan wejściowy może mieć 0, natomiast pierwszy zapis w magazynie otrzymuje 1.

Relacja utrwala source_revision i target_revision. Po zmianie parametru albo portu obiektu połączenie może wciąż wskazywać wcześniejszą rewizję. Inspektor pokazuje STALE, a symulator blokuje propagację przez taką relację. Przycisk Powiąż bieżące rewizje aktualizuje numery, lecz nie sprawdza automatycznie poprawności biznesowej połączenia. Najpierw przejrzyj różnicę interfejsu, potem zwiąż wersje.

Katalog architektury ma dodatkowo własne rewizje przechowywane w oddzielnym magazynie. Są one czym innym niż revision obiektu oraz revision projektu sceny. W pracy z katalogiem zapis projektu i scalenie zmian do katalogu to rozdzielne czynności. Utrzymuj tę różnicę w opisach incydentów i notatkach, podając nazwę warstwy wersjonowania.

Referencyjny model R1 w pakiecie lion_polyhedral stosuje jeszcze parę artifact_id i tekstową revision oraz semantic_digest. Nie wolno mechanicznie kopiować jego pól do modelu Studio i oczekiwać identycznej semantyki. Import mozaiki R1 przechodzi przez migrację. Studio zachowuje dane pierwotne w metadanych, zamiast udawać, że oba formaty są jednym dokumentem.

Element	Wersja	Praktyczny skutek
Artefakt Studio	revision ≥ 1	Powiązania końców mogą się zestarzeć
Projekt Studio	revision ≥ 0	Kontrola równoczesnego zapisu
Katalog architektury	rewizja katalogu	Przypięta baza nawigacji i scalenia
Artefakt R1	revision + digest	Osobny format importowany migracją

Źródła: S42 semanticSnapshot; editArtifact; rebind; S33 migrate_workspace; S26 Artifact

TEMAT 043

Rodzaj artefaktu a jego rzeczywista odpowiedzialność

Pole kind określa kategorię obiektu, na przykład application, repository, broker, factory, document, contract, code lub hypothesis. Formularz udostępnia listę najczęstszych kategorii, lecz walidator podstawowego modelu wymaga przede wszystkim niepustego tekstu. Oznacza to możliwość zachowania rodzaju pochodzącego z importu, nawet jeśli nie ma polskiej etykiety w zestawie przycisków. W takiej sytuacji interfejs może wyświetlić jego oryginalny identyfikator.

Rodzaj jest klasyfikacją, nie uruchamianą klasą Pythona. Wybranie Broker nie włącza routingu wiadomości, a wybranie Fabryka nie uruchamia generatora kodu. Wpis Instancja też pozostaje elementem modelu, dopóki zewnętrzny materiał nie pokaże konkretnego uruchomienia. O tym, co faktycznie zostało opisane, informują odpowiedzialność, osie, źródła i relacje, rozpatrywane łącznie.

Dla istniejącego pliku można użyć code i materiału źródłowego, dla planowanego podziału component oraz materialization DESIGN, a dla proponowanego zastępcy candidate. Nie twórz dwóch obiektów tylko dlatego, że ten sam moduł pełni kilka funkcji. Jedną tożsamość można opisać wieloma kaflami i relacjami. Osobny artefakt jest uzasadniony wtedy, gdy potrzebujesz odrębnej granicy, wersji albo odpowiedzialności.

W projekcie architektury dobrze rozdzielić specyfikację od kandydata realizacji i od śladu wykonania. Połącz je relacjami opisującymi znaczenie, na przykład refines, realized_by lub evidence_for, zgodnie z przyjętym słownikiem projektu. Nie zmieniaj specyfikacji w dowód tylko przez przestawienie rodzaju. Zachowaj źródło każdego twierdzenia i jego zakres.

Praktyczne ćwiczenie: utwórz kontrakt wejścia, projektowany parser i dokument z wynikiem testu. W każdym opisie napisz jedno zdanie odpowiedzialności. Następnie sprawdź, czy odbiorca potrafi odróżnić wymaganie, realizację i materiał wspierający. Jeśli wszystkie trzy elementy nazywają się Parser, popraw nazwy przed dodawaniem następnych warstw. Czytelna klasyfikacja upraszcza późniejsze wyszukiwanie i analizę wpływu.

kind	Co opisuje	Czego samo nie potwierdza
contract	Wymagania lub opis interfejsu	Zgodności implementacji
code	Jednostkę kodu	Uruchomienia kodu
candidate	Proponowaną realizację	Dopuszczenia do wykonania
evidence	Materiał o konkretnym zakresie	Prawdziwości każdej tezy
broker	Roli pośrednika	Działającej usługi sieciowej

Źródła: S42 kinds; addArtifact; S33 validate_workspace; S36 RELATION_TYPES

TEMAT 044

Zawieranie, właściciel i zależność to różne relacje

parent_id określa pojedynczego rodzica geometrycznego w modelu projektu. Wartość null umieszcza artefakt na poziomie projektu. Wartość identyfikatora innego obiektu tworzy jego wnętrze: przy wejściu do rodzica użytkownik ogląda dzieci. To podstawowa oś nawigacji kompozycyjnej, ale nie należy traktować jej jako uniwersalnej odpowiedzi na pytania o własność, odpowiedzialność lub zależności wykonawcze.

owner_ref jest odrębnym tekstowym odniesieniem właściciela. Formularz nie zamienia go w parent_id, nie nadaje uprawnień i nie tworzy konta użytkownika. Można więc umieścić kontrakt we wnętrzu modułu, zachowując właściciela znaczenia w zewnętrznym obszarze architektury. Odwrotnie: wspólny właściciel wielu artefaktów nie wymaga przeniesienia ich do jednego katalogu ani do jednej bryły.

Zależność opisuje rekord relations. Biblioteka może być potrzebna komponentowi bez bycia jego dzieckiem. Kontrakt może definiować zachowanie modułów w różnych repozytoriach. Materiał dowodowy może wspierać wiele tez. Takie powiązania wyrażaj krawędziami, a nie kopiowaniem obiektu do kilku wnętrz. W modelu projektu jeden obiekt ma najwyżej jednego rodzica, lecz może uczestniczyć w wielu relacjach.

Zawieranie musi pozostać acykliczne. Nie wolno ustawić A wewnątrz B, jeżeli B znajduje się już wewnątrz A, również przez dłuższy łańcuch. Formularz przechodzi po rodzicach i odrzuca taki wybór; walidator sprawdza dokument ponownie. Semantyczny cykl zależności jest odmiennym problemem i nie jest automatycznie błędem samego drzewa zawierania.

W widoku katalogowym R3.1 dochodzi rozróżnienie drzewa projektu i drzewa Git. Zmiana modelowego rodzica nie przenosi pliku w repozytorium. Źródłowa ścieżka i commit pozostają faktami przypiętego indeksu. Przed scaleniem sprawdź oba konteksty: gdzie element rzeczywiście leży oraz gdzie umieszczasz go w projektowanej kompozycji. Taki zapis pozwala modelować refaktoryzację bez fałszowania stanu źródeł.

```
{"id":"contract:input","parent_id":"component:parser",
 "owner_ref":"owner:architecture",
 "description":"Kontrakt danych przyjmowanych przez parser"}
```

Źródła: S42 objectInspector; object-form; S33 CONTAINMENT_CYCLE; PARENT_REFERENCE; S34 scope_view

TEMAT 045

Pozycja, skala i stały profil bryły

Każdy artefakt ma position w postaci trzech skończonych liczb oraz dodatnie size. Są to współrzędne i skala prezentacji. Domyślnie nowy obiekt otrzymuje miejsce w siatce zależnej od liczby rodzeństwa. Przeciąganie w trybie przesuwania zmienia położenie, a formularz Obiekt pozwala wpisać wartości X, Y i Z bezpośrednio. NaN, nieskończoność i niedodatnia skala są niedozwolone.

Profil truncated-octahedron-14 przedstawia ośmiościan ścięty: sześć kwadratów i osiem sześciokątów. Generator referencyjnej geometrii buduje 24 wierzchołki z permutacji współrzędnych 0, ±1 i ±2. Weryfikuje 36 krawędzi, dwie ściany przy każdej krawędzi, jednakową długość krawędzi oraz charakterystykę Eulera równą 2. Te sprawdzenia dotyczą poprawności bryły, nie kompletności modelu aplikacji.

Rola ściany ma stabilny identyfikator. Obrót kamery nie zamienia wejścia w wyjście i nie zmienia adresu portu. Znaczenie połączenia wynika z JSON, nie z tego, czy dwa wielościany akurat stykają się na ekranie. Możesz rozsunąć ciasny widok bez naruszania kontraktów. Samo zwiększenie size nie zwiększa pojemności portów, budżetu fabryki ani ważności dowodu.

Odległość semantyczną zapisuje się osobno w metadata.topology relacji. Jeżeli bliskość brył ma w projekcie oznaczać wspólny obszar odpowiedzialności, opisz tę konwencję w metaramie. Nie wyciągaj liczbowej siły zależności z odległości w pikselach. Studio nie stosuje fizyki, która automatycznie obliczałaby prawa oddziaływań na podstawie położenia.

Praktycznie najpierw ustaw czytelny układ dla aktualnego wnętrza, potem użyj dopasowania kamery. Przy dużej liczbie obiektów ogranicz widok nawigacją lub filtrem. Nie próbuj odtworzyć całej federacji jednocześnie przez ściskanie brył. Widok ma pomagać zauważać granice, a pełny katalog przechowuje szczegóły, których bieżąca projekcja nie musi zawierać. Rozróżnienie danych i obrazu jest podstawą skalowalnego modelowania.

Pole lub czynność	Warstwa	Wpływ
position / size	Widok	Rozmieszczenie i skala
Obrót kamery	Widok	Perspektywa odbiorcy
parent_id	Topologia kompozycji	Wnętrze i nawigacja
Port i kontrakt	Semantyka interfejsu	Zgodność połączenia

Źródła: S33 POSITION; SIZE; S51 build; S18 Co wnosi geometria

TEMAT 046

Osie kwalifikacji i ich poprawne czytanie

axes przechowuje niezależne etykiety opisu. Klasa dowodu evidence, materialization i currentness odpowiadają na różne pytania: na jakiej podstawie coś zapisano, w jakiej postaci to istnieje i jak odnosi się do czasu lub przypiętej wersji. Nie należy sprowadzać tych pól do jednego wskaźnika jakości. Źródłowy snapshot może być historyczny, a projekt przyszłej funkcji może mieć precyzyjnie udokumentowane założenia.

Formularz oferuje dla evidence między innymi DECLARED, SOURCE_SNAPSHOT, STATIC_CANDIDATE, TESTED_OFFLINE, OBSERVED i UNKNOWN. Wybór jest deklaracją autora modelu. Studio nie wykonuje pomiaru tylko dlatego, że użytkownik zaznaczył OBSERVED. Podobnie TESTED_OFFLINE powinno wskazywać konkretny raport, zakres testu i wersję źródeł; bez nich etykieta pozostaje słabo uzasadniona.

materialization rozróżnia DESIGN, CANDIDATE, SOURCE, INSTANCE, SIMULATED i UNKNOWN. currentness udostępnia UNVERIFIED, PINNED, STALE, HISTORICAL i UNKNOWN. Przypięcie oznacza odniesienie do określonej wersji; nie jest twierdzeniem o aktualnym stanie usługi. Nie zamieniaj SIMULATED w INSTANCE po uzyskaniu poprawnego wyniku symulacji. Modelowany przebieg i rzeczywiste wykonanie mają różne źródła wiedzy.

Pozostałe osie to plane, detail, temporal i domain. Formularz zawiera płaszczyzny SEM, INF, MAND; szczegółowość L0-L6; horyzont FUTURE, PRESENT, HISTORY oraz dowolny tekst domeny. Są to współrzędne opisu przyjęte w ramie projektu. W szczególności detail nie ustala rangi abstrakcji topologicznej, a MAND samo nie nadaje władzy wykonania.

Ćwiczenie kontrolne: wybierz importowany moduł z przypiętego commitu i planowany adapter. Pierwszy może otrzymać SOURCE_SNAPSHOT, SOURCE, PINNED, drugi DECLARED, DESIGN, UNVERIFIED oraz FUTURE. Dodaj źródła i opis zakresu. Następnie zadaj pytanie, czego nadal nie wiesz: działania runtime, kompletności importów, zgodności kontraktu? Zachowaj te braki jawnie, zamiast podnosić wszystkie osie do najbardziej optymistycznej wartości.

Oś	Przykładowa wartość	Pytanie kontrolne
evidence	STATIC_CANDIDATE	Z czego wynika opis?
materialization	DESIGN	Co rzeczywiście powstało?
currentness	PINNED	Do jakiej wersji przypięto dane?
temporal	FUTURE	Jakiego horyzontu dotyczy model?

Źródła: S42 axes; axisNames; S33 AXES; S36 _base

TEMAT 047

Poziomy abstrakcji i topologia deklaratywna

Poziom abstrakcji zapisuje rozszerzenie metadata.topology.level. Wymaga ono id, całkowitej rangi rank i niepustego frame_ref. Większa ranga oznacza wyższą abstrakcję wyłącznie w obrębie tej samej ramy. Liczba 4 w jednej ramie nie jest automatycznie porównywalna z liczbą 2 w innej. Bez zgodnego odniesienia porównanie otrzymuje stan incomparable. W raporcie zawsze podawaj identyfikator tej ramy.

Formularz Obiekt udostępnia trzy pola poziomu. Uzupełnij wszystkie, jeśli chcesz go zadeklarować. Wyczyszczenie całego zestawu usuwa poziom. Zapis częściowy jest odrzucany przez formularz. Wartość rank może być ujemna; istotny jest porządek w przyjętej ramie. Nie należy utożsamiać jej z głębokością katalogu, liczbą rodziców, rozmiarem bryły ani etykietą axes.detail.

Relacja może zawierać level_relation: peer, defines, realizes lub refines. Walidator ostrzega, gdy peer łączy różne rangi w tej samej ramie albo defines prowadzi od niższej rangi do wyższej lub równorzędnej. Realizes i refines są rozpoznawanymi deklaracjami; sam walidator nie dowodzi, że realizacja faktycznie spełnia specyfikację. Spójny kierunek nie jest dowodem semantycznym.

Topologia relacji mieści również distance, strength i dynamics. Odległość wymaga interpretacji metryki, jednostki, metody oraz źródła. Siła potrzebuje jednostki, znaczenia i źródła; normalized i probability ograniczają wartość do przedziału od 0 do 1. Bez wymaganej interpretacji liczba pozostaje niepełną deklaracją. Nie traktuj zwykłej wagi jako prawdopodobieństwa poprawności.

Dynamics opisuje tryb static, event lub continuous. Prawo law jest tekstem albo obiektem danych i nie jest wykonywane. Dla trybów zdarzeniowego i ciągłego brak prawa lub jednostki czasu powoduje ostrzeżenie oraz UNKNOWN. Użyj tych pól do uzgodnienia modelu i założeń, a następnie osobno wybierz mechanizm symulacyjny, który rzeczywiście potrafi dane założenia interpretować.

```
{"topology":{"level":{"id":"contracts",
 "rank":3,"frame_ref":"frame:ai-platform:v1"}}}
```

Źródła: S40 validate_topology; topology_summary; S22 $defs/level; S42 object-form; relation-topology-form

TEMAT 048

Metadane, źródła i granica między opisem a faktem

metadata pozwala dołączać rozszerzenia bez przebudowy podstawowej struktury artefaktu. Studio używa tej przestrzeni między innymi dla topology, file_interface i identyfikatorów katalogowych. Edytor Metadane JSON wymaga obiektu. Nie usuwa automatycznie nieznanych kluczy, co pomaga zachować dane specjalistycznego modelu. Swoboda formatu nie oznacza jednak, że każde własne pole zostanie zinterpretowane przez walidator albo interfejs.

sources jest oddzielną tablicą odnośników. Formularz źródła zawiera path, repo, ref, sha256, status i pointer. path wskazuje lokalizator materiału, repo przestrzeń pochodzenia, ref konkretny punkt odniesienia, a pointer fragment: linię, stronę lub zakres. SHA-256 musi mieć 64 małe cyfry szesnastkowe, jeśli jest podany. Formularz normalizuje wielkość liter hasha, ale nie oblicza go przez odczyt wskazanego pliku.

Dodanie źródła nie potwierdza jego dostępności, prawdziwości ani przynależności do bieżącego commitu. Ręcznie wpisany commit pozostaje deklaracją. Indeksator Git wiąże dane z własnym odczytem obiektów repozytorium; deskryptor pliku pochodzi z bajtów przekazanych analizatorowi. Zachowaj informację, którą drogą uzyskano opis. Dzięki temu późniejszy audyt odróżni wynik narzędzia od uzupełnienia użytkownika.

W widokach architektury nie zmieniaj przypadkowo pól catalog_node_id, architecture_catalog_id i danych nawigacyjnych. Są one technicznym powiązaniem projekcji z pełnym katalogiem. Przy duplikacji aplikacja świadomie usuwa tożsamość źródłowego węzła i nadaje kopii nowy kontekst projektu. Ręczne skopiowanie tych identyfikatorów może spowodować próbę nadpisania innego elementu podczas scalenia.

Dobra praktyka to zapisać przy własnym rozszerzeniu jego wersję, definicję i zakres, na przykład metadata.review z identyfikatorem procedury i wynikiem przeglądu. Gdy czegoś nie ustalono, użyj jawnego opisu luki. Nie zapisuj tokenów dostępu ani haseł w materiale eksportowanym dla asystenta. Metadane są częścią modelu i mogą znaleźć się w przenośnym pakiecie wraz z historią oraz kontekstem źródłowym.

```
{"path":"src/parser.py","repo":"example/project",
 "ref":"commit-ustalony-w-indeksie","sha256":"",
 "status":"USER_DECLARED","pointer":"L12-L48"}
```

Źródła: S42 editSource; metadata; duplicate; S33 source_checks; normalize_workspace

TEMAT 049

Parametry: wartości, typy i jednostki

parameters jest obiektem, którego klucze są nazwami parametrów. Każda wartość pod kluczem jest osobnym rekordem zawierającym co najmniej type i zwykle value oraz unit. Może też przechowywać description i origin. Parametr należy do artefaktu, a nie bezpośrednio do ściany. Port może wskazać go swoim polem parameter, dzięki czemu ten sam stan obiektu otrzymuje jawny interfejs komunikacji.

Studio rozpoznaje number, integer, boolean, string, object, array i any. Typ number obejmuje skończone liczby; integer wymaga liczby całkowitej. Wartości logiczne nie są liczbami tego kontraktu, mimo że Python ma techniczne pokrewieństwo bool i int. object oznacza obiekt JSON, array tablicę, a string tekst. any pozwala zachować nieustaloną strukturę, lecz osłabia możliwość rozstrzygania porównań w bramkach.

Wartość null oznacza brak znanej wartości i jest dopuszczalna dla każdego typu parametru. Nie jest równoważna zeru, pustemu tekstowi, false ani pustej tablicy. Przykładowo brak zmierzonej latencji nie powinien być zapisany jako 0 ms. Taki zapis udawałby idealny pomiar. W symulacji propagacja nieznanej wartości jest blokowana, a porównanie może otrzymać UNKNOWN.

unit jest tekstem i uczestniczy w sprawdzaniu zgodności. Pusty tekst oznacza jawnie zadeklarowaną jednostkę bez nazwy, odpowiednią na przykład dla wartości bezwymiarowej. Brak samego pola unit powoduje ostrzeżenie i może uniemożliwić propagację. Studio nie posiada automatycznego słownika jednostek; ms i s nie są zamienne bez jawnej transformacji.

Zgodność typów między końcami dopuszcza integer do number, ten sam typ oraz any. Nie oznacza to dowolnej konwersji danych. Tekst "12" nie staje się liczbą 12, a liczba 2.5 nie staje się integer. Przy projektowaniu kontraktu najpierw ustal domenę wartości, dopiero potem typ techniczny. Opis parametru powinien wyjaśniać także zakres, źródło i interpretację wartości, których sam typ JSON nie obejmuje.

```
{"latency":{"type":"number","value":null,"unit":"ms",
 "description":"Czas odpowiedzi; pomiar jeszcze nieznany",
 "origin":"DECLARED"}}
```

Źródła: S33 PARAMETER_TYPES; _typed; _compatible; S42 parseValue; editParameter

TEMAT 050

Dodawanie i edycja parametrów w inspektorze

Wybierz artefakt i otwórz kartę Parametry. Dodanie parametru rozpoczyna formularz z kluczem, typem, jednostką, wartością, opisem znaczenia i pochodzeniem. Klucz jest adresem używanym przez porty, bramki oraz wiązania relacji. Wybierz nazwę stabilną i jednoznaczną, na przykład accepted_count, zamiast nazwy odnoszącej się wyłącznie do bieżącego ekranu lub przypadkowej wartości.

Dla number i integer wpisujesz liczbę; dla boolean dokładnie true albo false. Tekst zapisuje się bez wymagania cudzysłowów JSON, ponieważ formularz zna typ string. Dla object, array i any pole wartości musi zawierać poprawny JSON. Checkbox Wartość nieznana - UNKNOWN zapisuje null niezależnie od zawartości pola. Jest to zalecana droga oznaczania brakującego pomiaru lub przyszłego wyniku.

Formularz odrzuca klucz już istniejącego parametru. Przy edycji istniejącego rekordu nie pozwala po prostu przemianować klucza: migracja wymaga aktualizacji wszystkich powiązań w pełnym JSON. Ta blokada chroni przed pozostawieniem portów i bramek wskazujących dawną nazwę. Jeżeli rzeczywiście zmieniasz kontrakt, zaplanuj zmianę parametru wraz z jego konsumentami, a nie jako kosmetyczne przemianowanie.

Po zastosowaniu zmiany semantycznej zwiększa się rewizja artefaktu. Powiązane relacje mogą stać się STALE. Sprawdź parametry po obu stronach, przypisania portów i dopiero wtedy użyj ponownego związania rewizji. Zapis projektu utrwala dokument; samo zamknięcie formularza nie oznacza jeszcze zapisu do magazynu. Po edycji poprzedni wynik symulacji jest kasowany z bieżącego stanu analizy, ponieważ dotyczył innego modelu.

Przećwicz bezpiecznie dwie wartości: źródło ma count typu integer z wartością 3, odbiorca count typu integer z null. Zapisz jednostkę pustą po obu stronach i dodaj krótkie opisy. Na tym etapie istnieje tylko stan dwóch artefaktów. Dopiero porty, relacja dataflow oraz wiązanie parametrów tworzą kompletną konfigurację propagacji. Dzięki temu możesz osobno sprawdzić dane wejściowe i sposób ich przekazywania.

1. Otwórz Parametry i wybierz dodanie nowego rekordu.

2. Ustaw klucz count, typ integer, wartość 3 i pustą jednostkę.

3. Dodaj opis znaczenia oraz origin wskazujący założenie.

4. Zastosuj zmianę, sprawdź rewizję artefaktu i zapisz projekt.

Źródła: S42 parameterInspector; editParameter; resetAnalyses; S33 validate_workspace

TEMAT 051

Ściana, kafel i port: trzy poziomy interfejsu

Profil bryły ma czternaście nazwanych ścian. Na każdej można umieścić wiele kafli tiles. Kafel opisuje konkretną rolę na tej ścianie, ma własne id, etykietę label, tekst role, opis i listę ports. Port jest punktem odniesienia dla końca relacji. Dzięki temu czternaście ścian nie ogranicza artefaktu do czternastu połączeń ani czternastu funkcji technicznych.

Adres portu w modelu Studio składa się z artifact_id, tile_id i port_id. Identyfikator kafla musi być unikatowy w artefakcie, a identyfikator portu w kaflu. Etykieta może się zmieniać bez zmiany adresu. Pole face wskazuje jedną z nazw profilu, natomiast role jest tekstem znaczenia konkretnego kafla. Rola producer na ścianie output nie jest tym samym polem co rodzaj kind całego artefaktu.

Port określa direction: in, out albo inout, typ danych, unit, powiązany parameter, capacity i schema_ref. Wartość capacity równa 0 oznacza brak limitu deklarowanego w modelu. Dodatnia liczba ogranicza budżet połączeń dataflow; aliasy exposes dzielą wspólny limit. Nie jest to jednak aktywna rezerwacja przepustowości w zewnętrznym brokerze ani pomiar ruchu sieciowego.

Żeby dodać interfejs, wybierz kartę Ściany, wskaż rolę profilu i dodaj kafel. W kaflu dodaj port, najlepiej po uprzednim zdefiniowaniu parametru. Formularz blokuje powtórzony ID portu. Nie pozwoli też zmienić identyfikatora używanego przez relację bez wcześniejszej migracji końca połączenia. Usunięcie portu zachowuje relacje do korekty, więc konieczna jest walidacja.

Przypisanie portu do parametru sprawdzane jest pod względem istnienia, typu i jednostki. schema_ref pozostaje tekstową referencją schematu; dwa różne, niepuste odniesienia na połączonych portach powodują błąd zgodności. Sam tekst referencji nie uruchamia walidacji dowolnego zewnętrznego schematu. Dla realnego adaptera opisz osobny artefakt i jawne przekształcenie, zamiast ignorować rozbieżność.

```
{"artifact_id":"component:parser",
 "tile_id":"parser:output","port_id":"result"}
```

Źródła: S33 FACES; DIRECTIONS; validate_workspace; S42 faceInspector; editTile; editPort

TEMAT 052

Ściana identity - tożsamość i ciągłość obiektu

identity jest pierwszą kwadratową ścianą profilu. Jej odpowiedzialnością pojęciową jest przedstawienie tożsamości, rodzaju i rewizji. W Studio rzeczywiste pola identyfikujące znajdują się jednak na poziomie artefaktu: id, kind i revision. Dodanie kafla na identity nie przenosi tam automatycznie tych pól i nie zastępuje tożsamości obiektu. Ściana porządkuje interfejsy dotyczące identyfikacji.

Praktycznym zastosowaniem jest kafel Karta komponentu z rolą descriptor. Może opisywać port udostępniający obiektową reprezentację nazwy, wersji i zadeklarowanych możliwości. Najpierw zdefiniuj parametr descriptor typu object, następnie port out na kaflu identity wskazujący ten parametr. Będzie to model wymiany danych identyfikacyjnych; nie działający endpoint HTTP ani automatyczne odzwierciedlenie wszystkich pól artefaktu.

W opisie kafla ustal, jaki podmiot jest identyfikowany. Logiczny komponent, konkretna rewizja źródła i instancja wykonania potrzebują różnych odniesień. Nie zapisuj jednego przypadkowego identyfikatora procesu jako trwałej tożsamości modułu. Gdy modelujesz uruchomienie, utwórz osobny obiekt albo jawny rekord instancji i połącz go ze specyfikacją zgodnie ze słownikiem projektu.

Ta ściana szczególnie pomaga przy porównywaniu kopii i zmian. Duplikacja w Studio nadaje nowe id, resetuje rewizję do 1 i zapisuje derived_from. Kopia zachowuje część opisów źródłowych, ale nie jest tym samym węzłem katalogu. W przeglądzie projektu sprawdź, czy etykieta sugerująca tożsamość nie przeczy faktycznemu nowemu identyfikatorowi. Czytelne nazwy powinny ujawniać, że powstał wariant.

Błąd typowy to próba zapewnienia integralności przez wpisanie hasha do dowolnego pola identity. Hash jest użyteczny dopiero z określeniem algorytmu, zakresu bajtów i procedury obliczenia. Studio nie przelicza automatycznie digestu całego obiektu po dodaniu kafla. Referencyjny model R1 ma osobny semantic_digest, ale należy go czytać w kontekście jego własnego formatu. Dokumentuj, czego dokładnie dotyczy identyfikator i jak zachowuje się przy zmianie.

Pytanie projektowe	Pole lub opis
Który logiczny element?	id artefaktu
Który stan opisu?	revision
Jaka kategoria?	kind
Skąd powstał wariant?	metadata.derived_from

Źródła: S18 Profil faset 6 + 8; Tożsamość i mikroświaty; S42 duplicate; faceInspector; S26 Artifact

TEMAT 053

Ściana contract - jawność wymagań interfejsu

contract jest kwadratową ścianą odpowiedzialną pojęciowo za kontrakty, schematy i zgodność. W modelowaniu oznacza miejsce zebrania wymagań: jakie dane są przyjmowane, jakie wyniki powstają i jakie warunki muszą spełniać połączenia. Nie należy jednak zakładać, że samo dodanie kafla contract tworzy wykonywalny walidator kontraktów. Zachowanie wynika z istniejących funkcji sprawdzających model, nie z nazwy ściany.

Studio oferuje kilka konkretnych narzędzi opisu. Porty mają typ, jednostkę i schema_ref. Relacja ma contract_ref oraz bramkę i wiązania. Artefakt rodzaju contract może reprezentować osobną specyfikację, do której prowadzą źródła i relacje. Kafel na ścianie contract może grupować porty służące wymianie takiego opisu, ale nie zastępuje dokumentu wymagań ani dowodu zgodności implementacji.

Praktyczny wariant: kontrakt parse-result/v1 opisuje obiekt wyniku z polami status i records. W obu współpracujących portach użyj identycznego schema_ref, a przy relacji wpisz odniesienie do dokumentu wymagań. Jeśli konsument oczekuje parse-result/v2, nie maskuj różnicy pustym schematem. Zaprojektuj adapter albo zmień zgodnie cały łańcuch. Walidator wykrywa różne niepuste schema_ref; nie rozstrzyga automatycznie zgodności strukturalnej dwóch niezależnych schematów.

Wersja kontraktu powinna być rozpatrywana przy każdej zmianie parametrów lub portów. Zwiększona rewizja artefaktu powoduje zestarzenie związanych końców relacji. Sprawdzenie STALE pomaga zauważyć tę zmianę, lecz przycisk ponownego związania nie analizuje dokumentu kontraktu. Potrzebny jest świadomy przegląd: zgodność typu, jednostki, kierunku, znaczenia i warunków błędu.

W źródłach kontraktu zapisuj dokładne miejsce dokumentu lub kodu wraz z wersją. Kontrakt proponowany oznacz jako projekt lub kandydat. Nie przedstawiaj statycznie odczytanej sygnatury funkcji jako pełnego publicznego API, zwłaszcza przy dynamicznym Pythonie. Ściana contract jest najbardziej użyteczna wtedy, gdy oddziela to, co wymagane, od tego, co już sprawdzono, oraz pokazuje brakujące dowody.

1. Wskaż dokument kontraktu i jego wersję w źródłach.

2. Zdefiniuj typy, jednostki i schema_ref obu portów.

3. Dodaj contract_ref relacji i warunki bramki.

4. Po zmianie interfejsu przejrzyj zgodność przed rebind.

Źródła: S18 Profil faset 6 + 8; S33 PORT_SCHEMA_MISMATCH; STALE_REVISION; S42 editRelation; editPort

TEMAT 054

Ściana boundary - granica kompozycji

boundary jest kwadratową ścianą profilu, pojęciowo grupującą eksport wnętrza, ograniczenia i wymagania dostępu. W Studio najważniejszą wykonywalną regułą granicy jest rozdzielenie zwykłego przepływu między rodzeństwem od jawnej delegacji portu rodzic-dziecko. Nie jest to mechanizm kontroli dostępu systemu operacyjnego. Bryła ani jej ściana nie nadają uprawnień do pliku, sieci czy narzędzia.

Jeżeli A zawiera B, zewnętrzny konsument nie powinien w modelu omijać publicznej granicy A. Relacja exposes wskazuje port rodzica i port jego bezpośredniego dziecka. Kierunek in prowadzi od rodzica do dziecka, a out od dziecka do rodzica. Typ i jednostka muszą być zgodne. Granica zachowuje wartość przez identity; transformacja scale wymaga osobnego adaptera poza samą delegacją.

Zwykłe dataflow między obiektami o różnych parent_id jest ostrzegane jako przekroczenie granicy i blokowane w symulacji. To celowa reguła modelu kompozycji. Aby przesłać wartość do wnętrza innego komponentu, opisz publiczne porty obu kompozycji i delegacje na kolejnych szczeblach. W ten sposób można prześledzić, gdzie dokładnie zmienia się zakres odpowiedzialności.

Kafel boundary może opisywać rolę public-api lub access-requirements. Umieść w jego opisie oczekiwane ograniczenia i źródło polityki. Jeżeli używasz parametru z deklaracją warunku, pamiętaj, że tekst polityki sam nie staje się bramką. Jawna bramka relacji może sprawdzać wartości w symulacji, lecz nawet wynik TRUE nie jest dopuszczeniem działania w rzeczywistym runtime.

Podczas przeglądu granicy sprawdź cztery rzeczy: czy port publiczny ma rzeczywisty odpowiednik we wnętrzu, czy nie zmieniono jednostki po drodze, czy wspólny limit pojemności nie został ominięty aliasami oraz czy zewnętrzne relacje używają właściwego poziomu. Taki przegląd zamienia geometryczne wnętrze w czytelną strukturę kontraktową. Wymagania uwierzytelnienia i wykonania trzeba następnie powiązać z właściwymi istniejącymi ownerami LION.

Mechanizm	Reguła granicy
dataflow	Rodzeństwo o tym samym parent_id
exposes in	Rodzic → bezpośrednie dziecko
exposes out	Dziecko → bezpośredni rodzic
capacity	Wspólny budżet portu i aliasów

Źródła: S18 Profil faset 6 + 8; Tożsamość i mikroświaty; S33 EXPOSES_BOUNDARY; EXPOSES_FLOW; BOUNDARY_CROSSING

TEMAT 055

Ściana provenance - pochodzenie bez utraty zakresu

provenance jest kwadratową ścianą służącą organizacji pochodzenia: materiałów źródłowych, historii wyprowadzenia i odniesień do dowodów. W Studio podstawowa tablica sources należy do całego artefaktu. Kafel na provenance może uporządkować interfejsy dostępu do takich opisów albo wskazać odpowiedzialność za ich utrzymywanie. Nie jest jednak jedynym miejscem, w którym zapisuje się źródła.

Dla odwzorowanego modułu potrzebujesz nie tylko nazwy repozytorium, ale także konkretnej ścieżki i wersji. Dla dokumentu potrzebny jest fragment odpowiadający twierdzeniu, na przykład strona lub nagłówek. Dla raportu testu ważny jest zakres badanej konfiguracji. Źródło mówiące o jednym kontrakcie nie potwierdza automatycznie całej federacji. Każdy odnośnik powinien odpowiadać na pytanie, co dokładnie uzasadnia.

Kafel może nosić rolę evidence-index. Jeżeli chcesz modelować przepływ opisu dowodu, dodaj parametr object i port out oraz połącz go ze współpracującym artefaktem. Otrzymasz model przekazania rekordu. Nie otrzymasz automatycznie kryptograficznego podpisu, niezależności dowodu ani trwałego archiwum. Tekst sha256 wskazuje tożsamość bajtów tylko wtedy, gdy wiadomo, jak i z czego został obliczony.

Przy duplikacji artefaktu zachowane źródła mogą nadal wyjaśniać genezę wariantu, ale nie muszą dowodzić jego nowego zachowania. Dodaj opis różnicy i wykorzystaj derived_from. W widokach katalogowych zmiany projektowe przechowuje się jako nakładkę, aby nie przepisać obserwowanych faktów Git na planowaną strukturę. Pochodzenie powinno pozwalać odtworzyć tę granicę w przeglądzie.

W referencyjnym fundamencie opisano łańcuch hashy i archiwizację, lecz sam łańcuch nie dowodzi autorstwa ani ochrony przed przeliczeniem całości. W instrukcji operacyjnej nie traktuj go jako podpisu. Dla codziennego modelowania najważniejsze są jednoznaczne odnośniki, zachowane wersje oraz jawne luki. Jeśli brak aktualnego odczytu runtime, oznacz źródło historyczne zamiast ukrywać ten brak pod ogólną etykietą obserwacji.

1. Wybierz tezę lub odpowiedzialność, którą uzasadnia materiał.

2. Dodaj repo, ref, path i pointer z właściwego źródła.

3. Zapisz status oraz ograniczenie zakresu w opisie.

4. Po zmianie wariantu przejrzyj, które źródła nadal pasują.

Źródła: S18 Profil faset 6 + 8; Archiwizacja i ewolucja; S42 editSource; sourcesInspector; duplicate

TEMAT 056

Ściana lifecycle - faza istnienia a stan uruchomienia

lifecycle jest kwadratową ścianą profilu odpowiedzialną pojęciowo za fazę istnienia i dopuszczalne przejścia. Pozwala rozmawiać o projekcie, kandydacie, specyfikacji, instancji i elemencie wycofanym bez mieszania tych stanów. W Studio nie ma jednak uniwersalnego automatu, który sam wykona każde przejście zapisane w opisie kafla. Nazwa roli porządkuje model, a rzeczywiste operacje wynikają z dostępnych modułów.

Zacznij od ustalenia, jaki cykl opisujesz. Cykl dokumentu może obejmować szkic, przegląd i zatwierdzenie; cykl komponentu projekt, kandydaturę i zastąpienie; cykl zadania gotowość i zakończenie. Są to różne porządki. Nie wkładaj stanu procesu wykonania do pola revision, ponieważ rewizja identyfikuje zmianę opisu, a nie etap uruchomienia. Użyj opisanych parametrów albo wersjonowanego rozszerzenia metadata.

Praktyczny kafel review-state może zawierać parametr status typu string oraz port out udostępniający tę wartość. Relacja semantyczna może wskazywać, że raport wspiera dopuszczenie kandydata. Jeżeli bramka porównuje status z reviewed, otrzymasz sprawdzalny warunek modelu. Wpisanie takiego tekstu przez autora nie jest dowodem wykonania przeglądu; dodaj źródło i osobny zapis wyniku.

axes.materialization oraz axes.temporal pomagają opisać postać i horyzont obiektu. Nie są zamiennikiem pełnego lifecycle. SOURCE i PRESENT nie dowodzą aktywnego działania, a HISTORY może dotyczyć wciąż potrzebnej specyfikacji dawnej wersji. Oceniaj te osie razem z opisem i aktualnością, zamiast budować jedną domyślną ścieżkę dla wszystkich rodzajów artefaktów.

Przy wycofaniu elementu nie zaczynaj od usunięcia go z modelu. Najpierw sprawdź relacje, źródła i potrzebę zachowania historii, a następnie zdecyduj, czy oznaczasz wariant jako historyczny, tworzysz następcę czy usuwasz błędny wpis. Takie podejście zachowuje możliwość analizy zmian. Rzeczywiste zatrzymywanie usług albo usuwanie plików pozostaje poza edycją deklaratywnego cyklu życia w Studio.

```
{"status":{"type":"string","value":"reviewed",
 "unit":"","origin":"USER_DECLARED",
 "description":"Stan przeglądu; raport wskazany w sources"}}
```

Źródła: S18 Profil faset 6 + 8; S42 axes; editParameter; S33 evaluate_gate

TEMAT 057

Ściana observability - co rzeczywiście można odczytać

observability jest kwadratową ścianą profilu, przeznaczoną do opisu zdarzeń, pomiarów, śladów i odczytu skutków. W architekturze pozwala wskazać, skąd wiadomo, że proces zrobił to, co deklaruje. W Studio należy odróżnić obserwowalność projektowanej aplikacji od logu samego modelera. Historia edycji projektu dokumentuje zmiany modelu, a nie wykonanie modułów przedstawionych na scenie.

Praktyczny kafel telemetry może opisywać port wyjściowy z rekordem metryki. Parametr metrics typu object może zawierać licznik i jednostkę w uzgodnionym schemacie. Jeżeli wartość pochodzi tylko z założenia, zapisz origin i evidence zgodnie z tą kwalifikacją. Aby twierdzić, że jest obserwacją, wskaż konkretny pomiar, czas, wersję badanego elementu oraz źródło. Samo zaznaczenie OBSERVED nie wykona odczytu.

Symulator wartości zwraca trace i blocked. Są to ślady deterministycznej propagacji projektu, które pomagają wyjaśnić wynik bramki, konflikt zapisu lub zatrzymanie na granicy. Nie są telemetrią produkcyjną. Tak samo symulacja generacji tworzy przebieg modelowy; nie dowodzi wykonania badań, testów ani materializacji plików w przedstawionym systemie. Zachowaj te rozróżnienia przy eksportowaniu dowodów.

Zaprojektuj obserwację wraz z kontraktem. Ustal, czy licznik jest sumą, próbką czy wartością chwilową; co oznacza brak wpisu; jaki zegar i identyfikator przebiegu stosujesz. Pusty zbiór zdarzeń może oznaczać brak aktywności, błąd zbierania lub ograniczenie zakresu. Zamiast zgadywać, opisz lukę. Parametr null pozwala zachować nieznaną wartość bez udawania zera.

Podczas przeglądu architektury połącz element wykonujący rolę z artefaktem opisującym raport albo sensor relacją observes, verifies lub inną jawnie zdefiniowaną kategorią. Wskaż kierunek znaczenia w opisie relacji. Dzięki temu użytkownik może odróżnić ścieżkę danych od ścieżki uzasadnienia. Obserwowalność jest kompletna dopiero wtedy, gdy wiadomo również, czego dany odczyt nie obejmuje i jak wykrywa się jego nieaktualność.

Materiał	Co potwierdza w swoim zakresie
Historia projektu	Zapisane zmiany dokumentu modelu
trace symulacji	Przebieg propagacji deklarowanych wartości
Raport źródłowy	Treść konkretnego materiału
Odczyt runtime	Tylko zmierzony stan wskazanej instancji

Źródła: S18 Profil faset 6 + 8; S33 simulate_workspace; S42 localEvent; axes

TEMAT 058

Ściana input - przyjmowanie danych przez jawny port

input jest sześciokątną ścianą profilu służącą modelowaniu danych, żądań i obserwacji przyjmowanych przez artefakt. Najprostszy przypadek to kafel request z portem in. Port opisuje typ, jednostkę i ewentualny schemat danych, a jego parameter wskazuje miejsce przechowania modelowanej wartości. Dzięki temu wejście nie pozostaje jedynie strzałką na bryle, lecz ma konkretny adres i kontrakt.

Wybierz artefakt odbiorcy, utwórz parametr payload typu object z nieznaną wartością, następnie dodaj kafel input i port request. Ustaw direction na in, typ object, tę samą jednostkę oraz parameter payload. Jeśli znasz schemat, wpisz schema_ref. Pojemność portu może ograniczać liczbę powiązań dataflow w projekcie, ale nie mierzy liczby żądań na sekundę ani rozmiaru kolejki rzeczywistej usługi.

Relacja dataflow musi prowadzić od portu out lub inout nadawcy do in lub inout odbiorcy. Potrzebuje też wiązania parametrów. Samo wybranie portów tworzy strukturalne połączenie, ale bez bindings nie przepisuje żadnej wartości. Jeśli nadawca i odbiorca mają różnych rodziców, potrzebna jest jawna ścieżka przez granice kompozycji; zwykły przepływ przez taką granicę jest blokowany.

Nie utożsamiaj przyjęcia danych z ich dopuszczeniem jako instrukcji albo z autoryzacją efektu. Model może zawierać bramkę semantyczną lub zgodnościową, która rozstrzyga TRUE, FALSE lub UNKNOWN na podstawie parametrów. Wynik TRUE oznacza spełnienie warunku w symulacji. Nie daje dostępu do wykonania kodu i nie potwierdza wiarygodności materiału wejściowego.

Projektując wejście, zapisz także znaczenie błędów: co jest brakiem danych, co wadliwym typem, a co poprawną pustą wartością. Parametr null zachowuje niewiedzę; pusty object jest znaną, konkretną strukturą bez kluczy. W dokumentacji kafla wyjaśnij różnicę, aby odbiorca nie zastąpił nieznanej wartości przypadkowym pustym rekordem tylko po to, żeby uzyskać zielony wynik walidacji.

```
{"id":"request","direction":"in","type":"object",
 "unit":"","parameter":"payload","capacity":1,
 "schema_ref":"schema:request:v1"}
```

Źródła: S18 Profil faset 6 + 8; S33 DATAFLOW_DIRECTION; NO_BINDINGS; S42 editPort; editBinding

TEMAT 059

Ściana output - wynik i jego kwalifikacja

output jest sześciokątną ścianą modelującą wyniki, odpowiedzi i publikowane zdarzenia. Standardowy kafel producer ma port out powiązany z parametrem wyniku. Ściana pomaga odróżnić to, co artefakt wystawia otoczeniu, od danych przechowywanych wyłącznie we wnętrzu. W samym Studio utworzenie portu wyjściowego nie powoduje automatycznego wywołania funkcji ani publikacji komunikatu na zewnętrznym brokerze.

Praktyczne ćwiczenie rozpoczyna się od dwóch artefaktów na tym samym poziomie zawierania. W źródle dodaj parametr count typu integer z wartością 3 i pustą jednostką. Na output utwórz port out wskazujący count. W odbiorcy przygotuj odpowiedni parametr oraz port in. Dopiero relacja dataflow z identity pomiędzy tymi parametrami umożliwia symulację przekazania wartości.

Wynik powinien mieć opis zakresu i pochodzenia. Ten sam obiekt JSON może być propozycją, wynikiem algorytmu lub odczytem rzeczywistego stanu; jego kształt techniczny nie ustala klasy epistemicznej. Zachowaj axes, sources oraz opis odpowiedzialności. Jeżeli output reprezentuje prognozę modelu, nie zmieniaj go w obserwację tylko dlatego, że walidator zaakceptował format.

Nie zakładaj, że porty output muszą zawsze zawierać liczby. Możesz użyć string dla identyfikatora, object dla koperty opisu, array dla listy kandydatów i boolean dla konkretnego warunku. Typ any nadaje się do importu nieustalonego interfejsu, lecz utrudnia rozstrzyganie bramek. Dla trwałego kontraktu wybierz możliwie precyzyjny typ i jawny schemat.

Przeglądając błędy, sprawdź szczególnie direction, jednostkę, parameter i rewizję końca. Po zmianie wyniku obiekt może otrzymać nową rewizję, a relacja zachować stare wiązanie wersji. Wtedy symulacja blokuje przepływ jako STALE_REVISION. Nie poprawiaj wyłącznie numeru, jeśli zmieniło się znaczenie danych. Sprawdź, czy każdy konsument nadal rozumie format, zakres i założenia nowego wyniku.

1. Zdefiniuj parametr wyniku wraz z jednostką i opisem.

2. Dodaj kafel output oraz port out wskazujący ten parametr.

3. Połącz zgodny port odbiorcy i dodaj binding.

4. Po walidacji sprawdź trace, a nie tylko końcową wartość.

Źródła: S18 Profil faset 6 + 8; S33 simulate_workspace; STALE_REVISION; S42 editTile; editPort; editBinding

TEMAT 060

Ściana context - zakres zadania i punkt widzenia

context jest sześciokątną ścianą odpowiedzialną pojęciowo za kontekst zadania, zakres oraz projekcję odbiorcy. W holomozaikowym modelu ten sam materiał może być istotny dla różnych działań w innym zakresie. Kafel kontekstowy pozwala jawnie pokazać, jaka misja, założenie lub zestaw ograniczeń wpływa na interpretację danych. Nie oznacza automatycznego pobierania pamięci ani uruchomienia RAG.

Praktyczny kafel task-context może udostępniać parametr object z identyfikatorem zadania, zbiorem założeń i odniesieniem do źródeł. Nazwy kluczy wewnątrz tego obiektu są umową konkretnego projektu. Studio nie narzuca uniwersalnego formatu payloadu każdej ściany. Jeżeli potrzebujesz interoperacyjności między repozytoriami, opisz schemat, jego wersję i właściciela, a następnie umieść jego referencję w porcie.

Kontekst może być wejściem do bramki relacji. Przykładowo warunek porównuje parametr mode z tekstem review. Jest to deterministyczne sprawdzenie zadeklarowanej wartości, a nie semantyczna interpretacja całego opisu misji. Brak parametru, null albo niezgodna jednostka powodują UNKNOWN. Nie zastępuj brakującego kontekstu domyślnym przyzwoleniem tylko po to, żeby przepuścić propagację.

W R3.1 słowo kontekst występuje też przy odnośnikach nawigacyjnych do zewnętrznych końców zależności. To techniczna rola projekcji, odrębna od ściany context. Artefakt oznaczony jako context_boundary jest punktem wejścia do właściwego węzła katalogu i nie powinien być edytowany jak pełny rekord źródłowy. Najpierw przejdź do wskazanego wnętrza, a dopiero potem zmieniaj właściwy obiekt.

Dobry opis kontekstu odpowiada na pytania: dla kogo, w jakiej misji, względem której wersji i pod jakimi ograniczeniami obowiązuje dany model. Wpisz również, co zostało pominięte. Dzięki temu późniejsze scalenie albo analiza wpływu nie rozszerzy bezwiednie wniosku z lokalnego scenariusza na całą federację. Ściana pomaga utrzymać jawność ramy, ale sama nie ocenia jej kompletności.

Znaczenie słowa kontekst	Miejsce w danych
Kontekst funkcjonalny artefaktu	Kafel na face context
Rama założeń projektu	frame.assumptions
Odnośnik zewnętrzny w nawigacji	metadata.navigation_role
Warunek modelowanej relacji	gate oraz parametry

Źródła: S18 Profil faset 6 + 8; S42 navigationContext; navigationContextInspector; S33 evaluate_gate

TEMAT 061

Ściana knowledge - wiedza i materiał do sprawdzenia

knowledge jest sześciokątną ścianą pojęciowo grupującą odczyt pamięci, twierdzeń i materiału dowodowego. W Studio może reprezentować interfejs dokumentacji, korpusu badań lub zbioru kontraktów. Nie jest wbudowaną bazą wiedzy, która automatycznie rozumie wszystkie pliki. Użytkownik modeluje odpowiedzialność i powiązania, a skaner oraz indeksator dostarczają ograniczony opis źródeł.

Zamiast kopiować całą dokumentację do każdego artefaktu, utwórz osobny obiekt dokumentu albo kolekcji i połącz go relacjami. Przykładowo komponent depends_on kontrakt, a raport evidence_for wspiera określoną tezę. Kierunek i znaczenie zapisz w słowniku relation_types oraz w opisie konkretnej relacji. Sama bliskość brył nie mówi, czy dokument jest wymaganiem, źródłem wiedzy czy wynikiem pracy.

Kafel corpus-access może zawierać port out z tablicą odnośników albo port in dla zapytania modelowanego procesu. Każdy port potrzebuje zwykłych właściwości: kierunku, typu, jednostki, schematu i ewentualnego parametru. Wybranie ściany knowledge nie nadaje portom innych reguł technicznych. Typowane połączenia zachowują te same ograniczenia co na input lub output.

Przy wiedzy szczególnie ważna jest kwalifikacja źródła. Historyczny opis architektury może być wartościowy dla odtworzenia decyzji, ale nie potwierdza bieżącego stanu kodu. Snapshot pliku mówi o odczytanych bajtach, a nie o wszystkich zachowaniach programu. Nie promuj wielokrotnego przekazania jednego materiału do rangi wielu niezależnych dowodów. Zachowaj odniesienie do pierwotnego materiału.

Praktyczny przegląd polega na wybraniu jednego kontraktu i znalezieniu jego specyfikacji, implementacji oraz raportu walidacji. Model powinien pokazać trzy różne role i łączące je relacje. Jeśli znany jest tylko dokument, zaznacz brak implementacji i testu. Brakujący element można zaprojektować jako kandydat, lecz nie wolno przedstawiać go jako istniejącego zasobu wiedzy. To rozróżnienie pomaga planować dalsze badania i refaktoryzację.

1. Utwórz osobny artefakt dokumentu lub korpusu.

2. Przypisz źródło, wersję i zakres twierdzeń.

3. Połącz elementy relacjami o jawnym znaczeniu.

4. Oznacz luki implementacji oraz walidacji zamiast je domyślać.

TEMAT 062

Ściana cognition - hipotezy, kandydatury i niepewność

cognition jest sześciokątną ścianą służącą porządkowaniu hipotez, propozycji i ocen niepewności. Pozwala zaznaczyć, że artefakt uczestniczy w tworzeniu lub ocenie możliwych rozwiązań. W dostarczonej aplikacji samo dodanie tej roli nie wywołuje modelu językowego i nie uruchamia procesu badawczego. Studio opisuje oraz symuluje zadeklarowane struktury; rzeczywiste działania wymagają odrębnego wykonawcy.

Hipotezę najlepiej reprezentować jako osobny artefakt rodzaju hypothesis z opisem twierdzenia, zakresem oraz kryterium falsyfikacji. Połącz ją ze źródłami i przewidywaną realizacją. Kafel cognition może grupować interfejs przekazywania kandydatur albo ocen. Jeśli chcesz modelować liczbę score, zdefiniuj jej typ, jednostkę, metodę i interpretację. Sama wartość od 0 do 1 nie oznacza skalibrowanego prawdopodobieństwa.

Rozdziel tryb algorytmu od klasy wypowiedzi. Deterministyczny algorytm może operować na niepewnych danych, a wynik modelu stochastycznego może zawierać fragmenty sprawdzalne deterministycznie. W modelu zapisz osobno źródło, materializację, aktualność i znaczenie oceny. Nie istnieje jedna flaga, która zamienia propozycję w prawdę po przejściu przez bramkę.

Praktyczny scenariusz to hipoteza, że rozdzielenie parsera i walidatora zmniejszy sprzężenie. Dodaj dwa planowane komponenty, wskaż zależności kontraktowe i zapisz warunek oceny. Wynik badania albo testu umieść później w oddzielnym artefakcie. Do tego czasu relacja supports opisuje deklarowane uzasadnienie, a nie uzyskaną poprawę. Pozwala to śledzić ewolucję założeń bez nadpisywania historii.

W symulacji generacji warunki i etapy są deklarowane w formacie procesu, ale ślad nadal pozostaje SIMULATION_ONLY. Nie zapisuj na jego podstawie OBSERVED dla realnej aplikacji. Jeśli brakuje danych do rozstrzygnięcia, zachowaj UNKNOWN i zaplanuj zdobycie kontekstu. W ten sposób cognition staje się miejscem jawnego zarządzania niewiedzą, a nie dekoracyjnym zapewnieniem, że system samodzielnie rozumie projekt.

Obiekt	Potrzebny opis
Hipoteza	Twierdzenie, zakres, falsyfikacja
Kandydat	Proponowana zmiana i baza
Ocena	Metoda, skala i źródło
Wynik weryfikacji	Konfiguracja i konkretny raport

Źródła: S18 Profil faset 6 + 8; Determinizm i probabilistyka; S31 simulate_process; S42 kinds; axes

TEMAT 063

Ściana composition - wnętrze i publiczna całość

composition jest sześciokątną ścianą grupującą składanie, podział, dzieci i eksport portów. Jej sens najlepiej widać w obiekcie aplikacji: na zewnątrz ma własną tożsamość i interfejsy, a we wnętrzu zawiera komponenty oraz relacje. Nie musi być geometryczną sumą brył dzieci. Nowa powłoka oznacza nową granicę logiczną, dzięki której otoczenie nie musi znać wszystkich szczegółów implementacji.

W Studio faktyczne zawieranie określa parent_id dzieci. Kafel composition sam nie tworzy drzewa. Możesz opisać w nim rolę orchestration lub decomposition, lecz potomków dodajesz przyciskiem Dziecko albo wyborem rodzica w formularzu. Wejście do wnętrza zmienia zakres oglądanego modelu. W katalogu R3.1 pobiera kolejną porcję elementów z pełnego indeksu, zamiast ograniczać się do pierwotnie załadowanej projekcji.

Praktyczny przykład to kompozycja ingestion zawierająca reader i parser. Zewnętrzny port wejściowy rodzica deleguje dane do reader przez exposes, reader przekazuje wynik do parser przez dataflow, a wynik parsera jest wystawiony na porcie rodzica przez kolejne exposes. Każdy etap ma jawne parametry, kierunki, jednostki i rewizje. Tak powstaje czytelna ścieżka od publicznego interfejsu do wnętrza.

Nie każda relacja oznacza zawieranie. Współdzielona biblioteka powinna pozostać jednym obiektem z odnośnikami, jeśli model nie opisuje osobnych kopii. Grupa artefaktów również nie zastępuje rodzica: służy zebraniu wybranych obiektów i relacji, bez zmiany ich położenia w drzewie. Ustal, czy dana operacja tworzy granicę, zbiór analityczny czy zwykłą zależność.

Podział istniejącego komponentu wymaga kontroli kontraktów i konsumentów. Samo rozdzielenie bryły nie dowodzi zachowania równoważności kodu. W Studio przygotujesz strukturę kandydacką, źródła oraz plan połączeń, a później sprawdzisz implementację i testy właściwymi narzędziami. Szczególnie zachowaj rozróżnienie modelowego przeniesienia węzła od fizycznego przeniesienia pliku w Git; oba kroki mogą być potrzebne, ale są różnymi zmianami.

1. Utwórz rodzica typu composition lub application.

2. Dodaj dzieci i ich relacje wewnętrzne.

3. Zaprojektuj publiczne porty oraz delegacje exposes.

4. Sprawdź granice, rewizje i wynik propagacji.

Źródła: S18 Profil faset 6 + 8; Tożsamość i mikroświaty; S42 addArtifact; add-child; enter; S33 validate_workspace

TEMAT 064

Ściana routing - pośrednictwo i odnajdywanie końców

routing jest sześciokątną ścianą opisującą odkrywanie endpointów i pośrednictwo. W modelu LION broker także może być artefaktem z własnym wnętrzem, kontraktami i źródłami. Dzięki temu da się przedstawić jego odpowiedzialność tym samym standardem co odpowiedzialność komponentu lub dokumentu. Kategoria broker oraz kafel routing nie uruchamiają jednak samodzielnie sieci brokerów w aplikacji Studio.

Zacznij od ustalenia, co pośrednik robi z relacją. Czy tylko przekazuje komunikat, wybiera odbiorcę, publikuje katalog adresów, czy transformuje format? Te funkcje potrzebują osobnych opisów. Jeśli broker zmienia dane, pokaż adapter i kontrakt transformacji. Jeżeli tylko przekazuje odnośnik, zachowaj pierwotne źródło i kwalifikację materiału. Kolejny przeskok nie daje automatycznie nowego niezależnego dowodu.

Praktyczny model ma trzy artefakty na wspólnym poziomie: producent, pośrednik i konsument. Pośrednik otrzymuje osobne parametry wejścia i wyjścia oraz odpowiednie porty. Dwa dataflow mogą przekazywać wartości według wiązań. Sam wewnętrzny związek między parametrami pośrednika również trzeba opisać; Studio nie wykona domyślnie dowolnej funkcji routingu tylko dlatego, że obiekt nazwano Router.

Dla relacji semantycznej routes_to można połączyć same artefakty bez portów. Taki zapis mówi o modelowanej trasie, lecz nie jest wykonywalną propagacją wartości. Jeżeli chcesz badać przepływ, użyj dataflow i konkretnych bindings. Jeżeli chcesz opisać zależność konfiguracji, pozostań przy relacji semantycznej i jasno nazwij jej znaczenie w relation_types.

Pośrednictwo nie może ukrywać granic wersji. Relacja wskazuje konkretne końce i ich rewizje; po zmianie kontraktu trzeba ponownie sprawdzić zgodność. W referencyjnym fundamencie broker zachowuje origin, correlation, causation i tożsamość payloadu, ale integracja produkcyjnego dostarczania pozostaje poza samym modelerem. Używaj routing do projektowania jawnych dróg, a nie do zakładania gwarancji dostarczenia, uwierzytelnienia czy exactly-once, których model nie wykonuje.

Cel	Odpowiedni zapis
Opis kierunku pośrednictwa	Relacja semantyczna routes_to
Propagacja wartości	dataflow + porty + binding
Eksport portu dziecka	exposes
Zmiana formatu	Osobny adapter i kontrakt

Źródła: S18 Profil faset 6 + 8; Protokół; S42 editRelation; S33 simulate_workspace

TEMAT 065

Ściana evolution - zmiana jako jawny kandydat

evolution jest ostatnią sześciokątną ścianą profilu. Jej odpowiedzialnością pojęciową są propozycje zmian, migracje i analiza wpływu. W modelerze pozwala przedstawić nie tylko docelowy komponent, ale też powód jego powstania, bazę porównania i kryteria oceny. Nie oznacza samoczynnego przepisywania źródeł aplikacji po dodaniu kafla. Planowana zmiana i zrealizowany efekt pozostają odrębnymi obiektami wiedzy.

Zacznij od artefaktu kandydata, który opisuje jedną konkretną zmianę, na przykład wydzielenie adaptera kontraktu. Zapisz odniesienie do bazowego modułu i jego wersji, założenia oraz przewidywane skutki. Relacja derived_from może wskazać genezę, a refines uszczegółowienie. Zdefiniuj znaczenie tych relacji w projekcie. Samo podobieństwo nazw nie daje narzędziu podstawy do wnioskowania o pochodzeniu.

Kafel evolution może grupować porty przyjmujące propozycję albo udostępniające wynik oceny. Typy i schematy portów działają tak samo jak na innych ścianach. Jeżeli zapisujesz score, wyjaśnij metodę i zakres. Jeżeli wynik nie został jeszcze uzyskany, użyj null lub jawnego opisu luki zamiast pozytywnej wartości domyślnej. Kandydat nie powinien sam potwierdzać warunków swojej akceptacji.

Studio posiada osobny model procesu generacji i jego deterministyczną symulację. Jest to narzędzie badania zadeklarowanych etapów, zależności i warunków, a nie automatyczne wykonanie badań, testów lub wdrożeń. Powstałe deskryptory i trace zachowują kwalifikację symulacji. Ściana evolution może odsyłać do tego procesu, ale nie zamienia wszystkich opisanych etapów w działające narzędzia.

Przy zmianie reguł oceny zachowaj osobną rewizję i uzasadnienie. W fundamentach LION fabryka oraz jej walidatory także są artefaktami; zmiana kryteriów jest odrębnym problemem od wygenerowania kandydata. W praktyce najpierw zapisz projekt bazowy, potem wariant i różnice, następnie przeprowadź właściwą weryfikację. Taki porządek pozwala rozumieć, dlaczego architektura ewoluowała, oraz co nadal pozostaje hipotezą.

1. Wskaż wersję bazową oraz przyczynę zmiany.

2. Utwórz kandydata z jawnymi różnicami i zależnościami.

3. Ustal kryteria oceny oraz potrzebne dowody.

4. Oddziel wynik symulacji od rzeczywistej materializacji.

Źródła: S18 Profil faset 6 + 8; Archiwizacja i ewolucja; S31 default_process; simulate_process; S42 editRelation

TEMAT 066

Tworzenie i poprawianie kompletnego artefaktu

Nowy artefakt możesz utworzyć na aktualnym poziomie projektu albo jako dziecko wybranego obiektu. Formularz prosi o nazwę, rodzaj, rodzica i opis odpowiedzialności. Po utworzeniu obiekt otrzymuje nowe id, rewizję 1, pusty zestaw źródeł, parametrów i kafli oraz pozycję w siatce rodzeństwa. W osiach ma DECLARED, DESIGN i UNVERIFIED; dla ręcznie tworzonego obiektu formularz ustawia również horyzont FUTURE.

Wybierz nową bryłę i otwórz Obiekt. Uzupełnij odpowiedzialność i owner_ref, sprawdź rodzica oraz osie. Jeżeli projekt wymaga poziomów abstrakcji, wpisz komplet id, rank i frame_ref. Nie ustawiaj wyższej rangi tylko dlatego, że bryła jest rodzicem. Ranga opisuje umówioną relację abstrakcji, natomiast parent_id określa wnętrze. Obie współrzędne mogą się różnić, jeśli uzasadnia to metarama.

Następnie dodaj źródła, parametry i potrzebne kafle. Nie ma obowiązku zapełnienia czternastu ścian. Zdefiniuj wyłącznie interfejsy rzeczywiście potrzebne w bieżącym modelu. Parametr przygotuj przed portem, żeby móc wybrać go z listy. Połączenie z innym obiektem twórz dopiero wtedy, gdy znasz znaczenie obu końców. Dla samej zależności semantycznej porty mogą pozostać nieokreślone.

Zastosowanie formularza zmienia lokalny model i oznacza go jako niezapisany. Edycja semantyczna zwiększa rewizję obiektu; przesuwanie i skala są z tego porównania wyłączone. Operacje trafiają do lokalnego logu przyczyn i stosu cofania. Nowa edycja kasuje bieżące analizy symulacyjne, ponieważ były wyliczone dla wcześniejszych danych. Po serii spójnych zmian uruchom walidację i zapisz projekt.

W projekcie powiązanym z katalogiem nowy ręczny element jest oznaczany jako manual_design i otrzymuje tożsamość dla scalenia. Zmiana nie tworzy pliku w repozytorium. Po nawigacji szkic zmian zachowuje opis, a zapis projektu i scalenie do katalogu nadal pełnią różne funkcje. Używaj opisu i źródeł, żeby kolejna osoba wiedziała, czy ogląda odwzorowanie istniejącego modułu, czy projekt jego przyszłego zastępcy.

1. Utwórz obiekt i określ odpowiedzialność oraz rodzica.

2. Uzupełnij osie, źródła i ewentualny poziom abstrakcji.

3. Dodaj parametry, kafle i porty wymagane przez kontrakt.

4. Połącz elementy, zwaliduj dokument i zapisz rewizję.

Źródła: S42 addArtifact; object-form; editArtifact; mutate; S41 merge_view; collect_draft

TEMAT 067

Duplikowanie, usuwanie i bezpieczne granice zmian

Duplikuj tworzy kopię pojedynczego artefaktu. Aplikacja nadaje jej nowe id, dopisuje do nazwy informację o kopii, ustawia rewizję 1 i przesuwa pozycję w osi X. Zachowuje parametry, źródła i kafle, przy czym kafle otrzymują nowe identyfikatory. Dodaje metadata.derived_from wskazujące źródłowy obiekt i jego rewizję. Osie kopii zmieniają się na DECLARED, CANDIDATE i UNVERIFIED.

Operacja nie kopiuje rekurencyjnie dzieci ani relacji. Dlatego kopia kompozycji nie zawiera automatycznie całego jej wnętrza. Porty w kaflach zachowują własne identyfikatory, ale nowe tile_id oznaczają inny adres końca. Istniejące relacje nadal wskazują oryginał. Po duplikacji sprawdź nazwę, znaczenie i źródła, a potem jawnie utwórz połączenia wariantu. Zachowane źródło może wyjaśniać genezę, lecz nie potwierdza nowego zachowania.

Usuń dla artefaktu zbiera jego załadowanych potomków, usuwa powiązane relacje i porządkuje odwołania grup. Przed operacją pokazuje liczbę obiektów i relacji. Zmianę można cofnąć w bieżącej sesji przed utrwaleniem. W zwykłym, kompletnym projekcie ta operacja obejmuje dostępne wnętrze dokumentu. W projekcji katalogowej nie zakładaj jednak, że załadowany widok zawiera wszystkich potomków pełnego katalogu.

R3.1 przechowuje zmiany katalogowe jako nakładki; mechanizm merge_view i collect_draft nie wprowadza uniwersalnego tombstone usunięcia źródłowego węzła. Zniknięcie obiektu ze sceny nie jest więc dowodem trwałego skasowania go z katalogu architektury. Nigdy nie traktuj tej czynności jako usunięcia pliku z Git. Do zarządzania trwałym źródłem potrzebny jest właściwy proces zmiany repozytorium.

Usunięcie kafla, portu lub parametru ma inne skutki niż usunięcie całego artefaktu. Zachowane relacje mogą wskazywać nieistniejący endpoint albo parametr i wymagają korekty. Po takiej edycji uruchom walidację, przejrzyj błędy i dopiero zapisz spójny model. Przy większym przeprojektowaniu utwórz wariant lub eksport bazowy, aby porównanie nie zależało wyłącznie od ograniczonego stosu cofania.

Operacja	Co trzeba sprawdzić
Duplikuj	Nowa tożsamość; brak kopii dzieci i relacji
Usuń artefakt	Zakres załadowanego wnętrza i połączeń
Usuń port / kafel	Pozostawione końce relacji
Usuń parametr	Bindings, porty oraz bramki

Źródła: S42 duplicate; delete-artifact; delete-port; delete-tile; delete-parameter; S41 merge_view; collect_draft

TEMAT 068

Relacje semantyczne i słownik projektu

Relacja jest pełnoprawnym rekordem, a nie tylko linią na scenie. Ma id, name, kind, końce source i target, związane rewizje oraz opis. Przechowuje contract_ref, group_id, modality, polarity, critical, axes, sources, metadata, bindings i gate. Dzięki temu możesz rozróżnić zależność kontraktową, uszczegółowienie, pochodzenie, wsparcie twierdzenia i przepływ wartości, nawet gdy wyglądają podobnie w rzucie.

Dla relacji semantycznej endpoint może wskazywać tylko artifact_id. Nie jest wtedy wymagany port. Kategorie takie jak depends_on, supports, refines, defines, observes lub contradicts wyrażają zadeklarowane znaczenie. Symulator podstawowy nie wykonuje ich jako przepływu wartości. Użycie nazwy dataflow albo exposes uruchamia natomiast szczególne sprawdzenia portów i zachowanie propagacji, dlatego nie zamieniaj nazw swobodnie.

Słownik relation_types opisuje identyfikatory, etykiety i znaczenie kategorii. Walidator ostrzega o niezarejestrowanym typie semantycznym, ale nie uznaje samej nieobecności opisu za błąd endpointu. Wielkość liter ma znaczenie: DEPENDS_ON i depends_on są różnymi identyfikatorami tekstowymi. Jeśli importujesz kilka źródeł, zachowaj ich kwalifikację albo przygotuj jawną mapę słownika zamiast cicho scalać różne interpretacje.

Modality może opisywać relację deklarowaną, kandydacką, obserwowaną, wymaganą, możliwą lub warunkową. Polarity rozróżnia positive, negative i neutral. To etykiety interpretacji, nie dodatkowy wykonywalny silnik logiki. critical pozwala zaznaczyć istotność dla kompozycji; nie zatrzymuje automatycznie usług. Znaczenie tych pól powinno wynikać z metaramy i procedury przeglądu.

Praktyczne ćwiczenie: połącz hipotezę z raportem relacją supports i opisz, którą tezę raport wspiera oraz w jakim zakresie. Dodaj źródło do relacji, a nie tylko do obiektów. Następnie utwórz depends_on między implementacją i kontraktem. Odbiorca powinien rozumieć, że pierwsza krawędź opisuje uzasadnienie, druga wymaganie, a żadna sama nie propaguje wartości parametru. Dopiero ten poziom precyzji pozwala sensownie analizować wpływ zmian.

```
{"id":"relation:parser-contract","kind":"depends_on",
 "source":{"artifact_id":"component:parser"},
 "target":{"artifact_id":"contract:input"},
 "source_revision":2,"target_revision":1,
 "description":"Parser wymaga formatu input/v1"}
```

Źródła: S33 UNREGISTERED_RELATION_TYPE; validate_workspace; S42 editRelation; relationInspector; S36 RELATION_TYPES

TEMAT 069

Dataflow, wiązania i jawne przekształcenia

dataflow opisuje przepływ między zgodnymi portami artefaktów będących rodzeństwem, czyli mających to samo parent_id. Źródło wymaga out albo inout, cel in albo inout. Endpointy zawierają artifact_id, tile_id i port_id, a relacja wiąże rewizje obu obiektów. Te warunki kontrolują strukturę, lecz do przekazania wartości potrzebne są jeszcze bindings wskazujące parametry.

Wiązanie source_parameter → target_parameter musi odwoływać się do istniejących parametrów końców relacji. Jeżeli port deklaruje konkretny parameter, binding nie może go omijać. Typ parametru i portu oraz jednostki muszą odpowiadać kontraktowi. Jedna relacja nie może dwa razy zapisywać tego samego parametru docelowego. Konkurencyjne zapisy z różnych relacji są wykrywane podczas symulacji i powodują konflikt kroku.

Transformacja identity kopiuje wartość. scale liczy wartość × factor + offset i jest dozwolona dla typów liczbowych. Gdy jednostki portów się różnią, potrzebny jest scale oraz unit_conversion z dokładnym from i to. Edytor nie dobiera współczynnika automatycznie. Przykład ms do s wymaga mnożnika 0.001; nazwy jednostek bez takiego zapisu nie wystarczą do ustalenia konwersji.

Symulacja działa synchronicznie: relacje w kroku czytają wartości poprzedniego stanu, a zapis jest stosowany po zebraniu propozycji. Łańcuch A → B → C może więc potrzebować kilku kroków do ustabilizowania. Nie jest to wykonywanie funkcji Pythona ani fizyczny czas transmisji. Trace pokazuje APPLIED, UNCHANGED lub BLOCKED i pozwala odtworzyć, dlaczego wynik się zmienił albo pozostał nieznany.

Przed uruchomieniem sprawdź bramkę, wartości null i rewizje. TRUE przepuszcza poprawne bindings, FALSE oraz UNKNOWN blokują. Zestarzała wersja, przekroczenie granicy, brak wiązań albo nieznana wartość mają osobne powody blokady. Limit kroków nie dowodzi zbieżności. Jeśli wynik to STEP_LIMIT, przeanalizuj cykle i transformacje zamiast zwiększać limit bez zrozumienia dynamiki modelu.

```
{"source_parameter":"duration_ms",
 "target_parameter":"duration_s","transform":"scale",
 "factor":0.001,"offset":0,
 "unit_conversion":{"from":"ms","to":"s"}}
```

Źródła: S33 validate_workspace; simulate_workspace; S42 editBinding

TEMAT 070

Exposes - publiczny port i wnętrze artefaktu

exposes modeluje delegację portu między rodzicem i jego bezpośrednim dzieckiem. Jest sposobem przedstawienia publicznego interfejsu kompozycji. Nie łączy dowolnych odległych potomków; jeżeli ścieżka ma kilka poziomów, potrzebujesz kolejnych jawnych delegacji. Dzięki temu można odczytać, gdzie przebiega każda granica i które wewnętrzne porty są dostępne dla otoczenia.

Dla wejścia oba porty mają direction in, a relacja prowadzi rodzic → dziecko. Dla wyjścia oba porty mają direction out, a relacja prowadzi dziecko → rodzic. inout jest rozpoznawany jako ten sam kierunek po obu stronach, lecz nadal wymaga bezpośredniego zawierania. Typy muszą być zgodne, jednostki identyczne, a dwa niepuste schema_ref nie mogą się różnić.

Delegacja zachowuje wartość przez identity. Nie wolno używać scale, ponieważ alias granicy nie powinien ukrywać transformacji. Jeżeli potrzebujesz przeliczenia jednostek albo zmiany formatu, zaprojektuj adapter z osobnymi portami i zwykłym dataflow w odpowiednim zakresie. Taki układ jest dłuższy graficznie, ale ujawnia odpowiedzialność, której nie należy chować w eksporcie portu.

Port i jego aliasy exposes dzielą budżet capacity. Walidator łączy je w jedną klasę ograniczeń i liczy użycie przez dataflow. Publiczne wystawienie kilku aliasów nie zwiększa zatem automatycznie limitu wewnętrznego portu. Wartość 0 nadal oznacza brak zadeklarowanego limitu, a nie zerową pojemność. To ograniczenie modelu strukturalnego, nie rozproszona rezerwacja zasobów runtime.

Przykład kontrolny: kompozycja parent ma parametr request oraz port in; dziecko parser ma odpowiadający parametr i port in. Ustaw parent_id dziecka na identyfikator parent, utwórz exposes i binding identity. Zwiąż bieżące rewizje oraz zwaliduj projekt. Zewnętrzne dataflow do parent i wewnętrzna delegacja do parser tworzą przejrzysty model. Jeśli zamiast tego połączysz zewnętrzny obiekt bezpośrednio z parser, symulacja pokaże przekroczenie granicy zwykłego przepływu.

Cecha	dataflow	exposes
Położenie końców	To samo parent_id	Bezpośredni rodzic i dziecko
Kierunek	out/inout → in/inout	Taki sam po obu stronach
Transformacja	identity albo scale	identity
Jednostki	Jawna konwersja możliwa	Muszą być identyczne

Źródła: S33 EXPOSES_BOUNDARY; EXPOSES_DIRECTION; EXPOSES_FLOW; EXPOSES_TRANSFORM; S42 editRelation; editBinding

TEMAT 071

Interfejs pliku: opis bajtów i kandydatów symboli

Artefakt reprezentujący plik może zawierać metadata.file_interface zgodne z lion.file-interface/v1. Jest to deskryptor odczytanych bajtów i wyników statycznej analizy. Wymaga względnej ścieżki POSIX, SHA-256, rozmiaru bytes, pól media_type, encoding, language, tablic exports, imports i contract_refs oraz stałych effect READ_ONLY_SOURCE i resolution STATIC_CANDIDATE. Całe rozszerzenie jest opcjonalne; niepełny obiekt nie zastępuje poprawnego deskryptora.

path odnosi się do korzenia skanu. Nie może zawierać ścieżki bezwzględnej, dysku Windows, backslash, segmentów . lub .. ani pustych segmentów. SHA dotyczy oryginalnych bajtów, wraz z kodowaniem, BOM i końcami linii. Dwa tekstowo podobne pliki z różnymi zakończeniami linii mogą mieć różne hashe. media_type dobrane po rozszerzeniu jest kandydaturą klasyfikacji, nie dowodem zawartości.

Dla Pythona analizator statyczny zapisuje kandydatów klas oraz funkcji, zakresy linii, nazwy kwalifikowane i sygnatury. Anotacje, dekoratory oraz wartości domyślne pozostają tekstem źródłowym. Nie są wykonywane przez eval ani import. Pusta tablica exports nie dowodzi braku API: parser mógł napotkać błąd albo limit, dlatego czytaj analysis, gaps i pozostałe kwalifikacje.

Importy zachowują resolution UNRESOLVED_RUNTIME również wtedy, gdy wskazano lokalnego kandydata. Python może rozwiązywać moduły dynamicznie, więc statyczne podobieństwo ścieżek nie jest potwierdzonym wywołaniem. symbol_ref jest lokalizatorem opartym na ścieżce, hashu i pointer albo jawnym powiązaniem w skanie; nie jest automatycznie stabilną tożsamością logicznego interfejsu po przeniesieniu pliku.

Edytor interfejsu plikowego zmienia deklarację modelu i oznacza ją USER_DECLARED_UPDATE. Nie zapisuje źródłowego pliku i nie potwierdza aktualnych bajtów. Walidator sprawdza kształt deskryptora oraz zgodność zapisanych odniesień, ale nie otwiera ponownie pliku i nie dowodzi przynależności do commitu. Po zmianie kodu wykonaj właściwy ponowny skan lub indeksowanie, zamiast ręcznie aktualizować sam hash i udawać nowy odczyt.

Pole lub wynik	Interpretacja
READ_ONLY_SOURCE	Zakres pracy indeksatora
STATIC_CANDIDATE	Kandydatura opisu interfejsu
UNRESOLVED_RUNTIME	Import niepotwierdzony w wykonaniu
USER_DECLARED_UPDATE	Ręczna zmiana opisu

TEMAT 072

Bramki, UNKNOWN i końcowa kontrola modelu

Bramka relacji jest deklaracją logiczną, ocenianą w logice trójwartościowej. Wyniki to TRUE, FALSE i UNKNOWN. all, any i not składają warunki; literal przechowuje true, false albo null; porównania eq, ne, lt, lte, gt, gte i in korzystają z wartości parametrów lub jawnych operandów. Bramka nie jest kodem Pythona ani funkcją JavaScript wpisywaną przez użytkownika.

Operand odwołania wskazuje artifact_id i parameter. Operand stały wymaga value, type i unit. Brak wartości, null, typ any, niezgodność typów albo różne jednostki mogą uniemożliwić rozstrzygnięcie. UNKNOWN nie jest tym samym co FALSE: zachowuje informację, że warunku nie ustalono. W symulacji oba stany blokują relację, lecz powód blokady pozostaje odmienny i jest widoczny w trace.

Łączenie warunków respektuje informację częściową. all z jednym FALSE daje FALSE nawet wtedy, gdy inny warunek jest nieznany; any z jednym TRUE daje TRUE. Jeśli brak rozstrzygającego warunku, niewiedza przechodzi do wyniku. Puste all jest TRUE, a puste any FALSE. Te reguły są deterministyczne; nie wyliczają prawdopodobieństwa ani semantycznej wiarygodności twierdzenia.

Edytor pozwala zbudować proste porównanie lub wpisać złożoną formułę JSON. Po zmianie kontrolek użyj przycisku budowania formuły, ponieważ to zawartość JSON jest zapisywana do modelu. Walidator ogranicza bramkę do 16 poziomów i 256 węzłów. Dodatkowe gate_kind może kwalifikować warunek jako semantic, compatibility albo admission_reference, ale żadna z tych etykiet nie nadaje uprawnień runtime.

Przed oddaniem modelu przejrzyj tożsamości, rodziców, poziomy, źródła, porty i bindings. Uruchom walidację, usuń błędy oraz wyjaśnij ostrzeżenia STALE, UNKNOWN i przekroczenia granic. Następnie wykonaj właściwą symulację i przeanalizuj ślad. Zapisz projekt wraz z uzasadnieniem zmian. Poprawny model gwarantuje jedynie sprawdzone własności opisu; zgodność architektury z rzeczywistym kodem i zachowaniem wymaga nadal odpowiednich źródeł, indeksowania oraz testów implementacji.

```
{"op":"gte",
 "left":{"artifact_id":"component:parser",
         "parameter":"accepted_count"},
 "right":{"value":1,"type":"integer","unit":""}}
```

Źródła: S33 _gate_issues; evaluate_gate; _evaluate_valid_gate; simulate_workspace; S42 gateEditor; S18 Granice wniosku

TEMAT 073

Pełny katalog a pojedyncza scena

Katalog architektury jest trwałym opisem całej analizowanej struktury. Scena jest ograniczonym wycinkiem tego opisu, przekształconym w artefakty edytora 3D. Rozdzielenie tych dokumentów jest warunkiem poprawnej pracy z ai_platform: kilkadziesiąt brył widocznych na ekranie nie zastępuje ponad dwudziestu tysięcy rekordów katalogu. Otwierając następne wnętrze, Studio odwołuje się do identyfikatorów katalogowych, zamiast uznawać aktualnie załadowaną scenę za kompletny świat.

Format lion.architecture/v1 zawiera repozytoria, warstwy, węzły, relacje, procesy, źródła oraz opis pokrycia. Format lion.polyhedral.studio/v2 zawiera artefakty i interfejsy potrzebne do pracy edytora. Powiązanie sceny z katalogiem zapisuje metadata.catalog_view. Na poziomie artefaktu odpowiada mu catalog_node_id; na poziomie dokładnej relacji - catalog_relation_id. Tożsamość źródłowa pozwala przenieść zmianę nazwy lub kontraktu z ograniczonego widoku do właściwego rekordu bez ponownego wyszukiwania go po etykiecie.

Praktyczna konsekwencja jest istotna przy zapisie. Główny przycisk Zapisz utrwala projekt Studio. Operacja Zapisz edycje 3D do katalogu scala nakładki do katalogu architektury i tworzy jego kolejną rewizję. Te operacje mogą dotyczyć tego samego zadania projektowego, ale mają odrębne dokumenty oraz liczniki rewizji. Eksport samej sceny nie jest automatycznie eksportem wszystkich źródeł, poddrzew i relacji federacji.

Przykładowy błąd interpretacji polega na wyeksportowaniu trzydziestu sześciu widocznych dzieci i nazwaniu takiego pliku pełnym modelem ai_platform. Weryfikacja jest prosta: odczytaj catalog_nodes i catalog_relations, porównaj je z view_nodes i view_relations, a następnie sprawdź projection_is_not_full_catalog. Wartości różne są oczekiwane. Scena służy pracy człowieka, natomiast katalog zachowuje ciągłość dużego modelu. W dokumentacji przeglądu podawaj oba rozmiary, aby odbiorca wiedział, jaką część struktury rzeczywiście oglądał. Przy pracy zespołowej uzgodnij, który dokument jest przedmiotem przeglądu: pełny katalog, ograniczony projekt czy sama propozycja nakładek.

1. Otwórz Architektura / Git i wybierz katalog.

2. Sprawdź liczniki pełnego katalogu przed utworzeniem sceny.

3. Otwórz wycinek w 3D, wykonaj zmianę i zapisz projekt.

4. Scal jawnie edycje 3D, jeżeli mają wejść do modelu katalogowego.

Źródła: S28 project_view; S45 mergeView

TEMAT 074

Dokument lion.architecture/v1

Dokument katalogu ma własny identyfikator catalog_id i nazwę. Nie należy wykorzystywać nazwy jako klucza technicznego: dwa modele mogą mieć podobne etykiety, a nadal przedstawiać różne snapshoty lub odmienne projekty. Wymagane kolekcje są tablicami, także wtedy, gdy pozostają puste. Pozwala to odróżnić poprawny model bez repozytorium od dokumentu o nieprawidłowej strukturze. Coverage jest obiektem opisującym zakres twierdzeń i braki pokrycia.

Walidator architecture.validate_catalog sprawdza kształt podstawowych pól, unikalność identyfikatorów, istnienie rodziców, warstw i repozytoriów oraz poprawność końców relacji. Kontroluje dwa grafy zawierania: źródłowy i wynikający z nakładek design. Nie dopuszcza cyklu ukrytego poza aktualną sceną. Przy źródłach sprawdza typy lokalizatorów i dodatnie zakresy linii. To kontrola spójności dokumentu; poprawna walidacja nie dowodzi, że opisana rola odpowiada rzeczywistemu zachowaniu programu.

Poniższy szkielet jest punktem wyjścia do importu nowego, ręcznego katalogu. Nie zawiera węzłów, więc nie przedstawia jeszcze aplikacji. W praktyce wygodniej utworzyć korzeń przez formularz Nowy model ręczny, a JSON traktować jako świadomą edycję pełnej struktury. Przy imporcie należy nadać nowy catalog_id. Powtórzenie istniejącego identyfikatora nie jest poleceniem cichego nadpisania poprzedniej historii. Nazwy dodatkowych pól powinny jasno wskazywać, czy opisują obserwację, deklarację autora, czy wynik analizy.

Przy walidacji zwracaj uwagę na kod i ścieżkę błędu, a nie tylko pierwszy komunikat. CATALOG_PARENT dotyczy odwołania do rodzica, CATALOG_ENDPOINT wskazuje koniec relacji, a CATALOG_LAYER - przypisanie do warstwy. Poprawiając dokument, zachowaj niezmienne ID istniejących rekordów. Zmiana ID bez aktualizacji odwołań może pozostawić pozornie podobny model z zerwanymi relacjami. Sprawdzenie schematu JSON nie zastępuje kontroli pełnego grafu przez walidator aplikacji. Lista błędów jest wynikiem kontroli dokumentu, nie automatycznie wygenerowanym planem naprawy zależności w kodzie produktu.

```
{
  "schema": "lion.architecture/v1",
  "catalog_id": "architecture:przyklad",
  "name": "Model projektowy 0!n",
  "repositories": [], "layers": [],
  "nodes": [], "relations": [], "processes": [],
  "sources": [],
  "coverage": {"status": "DECLARED", "runtime_observed": false}
}
```

Źródła: S19 top-level schema; S28 validate_catalog

TEMAT 075

Węzeł: tożsamość, zawieranie i znaczenie

Węzeł katalogu nie musi być plikiem. W dostarczonym modelu występują repozytoria, katalogi, pliki, klasy, funkcje, metody, sekcje dokumentów, nazwy pól i koncepty właścicieli. Każdy rekord ma id, name i kind. Parent_id wskazuje pojedynczego rodzica w grafie zawierania albo null dla korzenia. Ten sam węzeł może należeć do kilku warstw przez layer_ids. Węzeł pliku pozostaje jednym obiektem nawet wtedy, gdy analizujemy go jednocześnie jako implementację kontraktu oraz nośnik wiedzy.

Pole source wiąże obiekt z miejscem pochodzenia. Semantic.role opisuje interpretowaną odpowiedzialność, natomiast semantic.qualification określa status tego opisu. Metadata przechowuje dodatkowe przesłanki i wyniki analizy. Te trzy przestrzenie warto czytać osobno. Przykładowo obecność klasy w drzewie AST jest innym ustaleniem niż teza, że klasa jest brokerem całej federacji. Nazwa Broker sama nie potwierdza prawa do wykonywania efektów ani tego, że instancja działa w środowisku użytkownika.

Ręcznie utworzony węzeł może nie mieć źródła Git. Jest wtedy pełnoprawnym elementem projektu, lecz nie należy dopisywać mu wymyślonego bloba ani commita. Projektowane położenie, etykieta i rodzic istniejącego źródła powinny znaleźć się w design. Identyfikator zachowuje ciągłość między obserwacją a propozycją zmiany. Przed scaleniem sprawdź, czy kopia artefaktu otrzymała nową tożsamość; dwa artefakty wskazujące ten sam catalog_node_id mogłyby oznaczać dwie sprzeczne edycje jednego rekordu. Mechanizm scalania odrzuca takie niejednoznaczne powiązanie.

Dobrym wzorcem nazewnictwa jest oddzielenie etykiety czytelnej dla człowieka od identyfikatora używanego w relacjach. Przemianowanie obiektu Kompilator na Kompilator architektury nie powinno wymagać poprawiania wszystkich strzałek. Przy własnych polach metadata dopisz, skąd pochodzi wartość i czy została zweryfikowana. Samo zapisanie strength:0.9 byłoby niejasne bez określenia skali, metody i podstawy. Model pozwala przechować opis, ale nie nadaje liczbie uniwersalnej interpretacji.

Pole	Pytanie użytkownika
id	Który dokładnie obiekt opisuję?
parent_id	W czym obiekt jest zawarty?
layer_ids	Do jakich przekrojów należy?
semantic.role	Za co według opisu odpowiada?
source	Na jakim źródle opieram opis?
design	Jak proponuję go przebudować?

Źródła: S28 validate_catalog; S41 merge_view

TEMAT 076

Relacja jest twierdzeniem o dwóch końcach

Relacja katalogowa ma własne id, rodzaj kind oraz skierowane końce source i target. Końce wskazują istniejące węzły, a nie swobodne napisy z nazwami plików. Kwalifikacja oraz evidence lub source_ref wyjaśniają podstawę połączenia. Strzałka między dwoma obiektami może oznaczać import, odwołanie do ścieżki, dziedziczenie, deklarowany kontrakt lub inny rodzaj zależności. Sam kierunek linii nie określa jeszcze przekazania danych w działającym procesie.

Wbudowany atlas zawiera między innymi REFERENCES_PATH, NATIVE_IMPORTS, NATIVE_CALLS, TEST_IMPORTS_SOURCE, IMPORTS_SOURCE, MARKDOWN_LINK oraz NATIVE_INHERITS. Są to nazwy pochodzące z konkretnej analizy. NATIVE_CALLS nie należy interpretować jako śladu wykonania tylko dlatego, że występuje słowo CALLS. O statusie rozstrzygają kwalifikacje i przesłanki rekordu. Nowy indeks Git zapisuje imports i references jako statycznych kandydatów, z nieustalonym rozstrzygnięciem wykonawczym.

Projekcja do Studio rozdziela obserwowane relacje od połączeń projektowych. Dokładne źródłowe krawędzie otrzymują domyślnie prefiks source:, aby przypadkowo nie uruchamiały logiki przepływu parametrów. Nakładka design może przechować projektowany rodzaj, bramkę, bindings oraz adres kafla i portu. Wtedy istnieją obok siebie pierwotne końce źródłowe i proponowane końce interfejsowe. Zmiana w modelu nie przepisuje starej przesłanki. Przy analizie awarii najpierw ustal, czy oglądasz dokładny rekord, agregację wielu rekordów, czy ręcznie zadeklarowany kontrakt. Dopiero potem oceniaj jego sens i kompletność.

W analizie kontraktów warto zapisywać oddzielnie oczekiwanie producenta i wymaganie konsumenta. Przykładowo moduł może deklarować przygotowanie manifestu, a kolejny obiekt wymagać manifestu sprawdzonego względem określonego schematu. Połączenie nazw plików nie wyraża tego warunku. Uzupełnij opis projektowanej relacji, gate i ewentualne bindings dopiero po ustaleniu znaczenia danych. Nie kopiuj bezrefleksyjnie wszystkich źródłowych krawędzi jako przepływów parametrów. Kontrakt relacji powinien być czytelny także bez obrazu sceny.

1. Otwórz kartę węzła i sprawdź relacje przychodzące oraz wychodzące.

2. Rozwiń Kontrakt relacji, aby zobaczyć pełny rekord.

3. Zapisz rodzaj, kwalifikację i lokalizator przesłanki.

4. Przejdź do drugiego końca; porównaj jego rolę z kierunkiem relacji.

Źródła: S28 _design_endpoint; S45 renderRelations

TEMAT 077

Warstwy jako niezależne przekroje

Warstwa katalogu jest zbiorem logicznym, do którego przypisuje się węzły przez layer_ids. Nie jest dodatkowym rodzicem i nie wymusza przeniesienia pliku w drzewie. Ten sam plik może realizować implementację, deklarować schemat oraz być przedmiotem testu. Wielowartościowa przynależność pozwala opisać takie przekroje bez kopiowania obiektu. Warstwa nie jest również automatycznie poziomem abstrakcji ani fazą wykonania procesu.

Dostarczony katalog wyróżnia implementację, kontrakty i schematy, wiedzę i dokumentację, dowody i historię, wdrożenie i operacje, testy i falsyfikację oraz metaramę i reguły. Wszystkie te warstwy mają kwalifikację ANALYTICAL_SOURCE_ROLE_NOT_RUNTIME_LAYER. Jest to przydatny podział analityczny źródeł, a nie zapewnienie, że środowisko LION uruchamia siedem dokładnie takich warstw usług. Rozróżnienie jest szczególnie ważne przy zestawianiu modelu Studio z własnymi native layers i flows ai_platform.

Przycisk Dodaj warstwę zapisuje id, nazwę i wymiar. Następnie jej identyfikator można przypisać wybranym elementom w edycji JSON. Walidator odrzuci odwołanie do nieistniejącej warstwy. Warto najpierw określić pytanie, na które nowy przekrój ma odpowiadać: właściciel semantyki, obszar kontraktu, plan refaktoryzacji albo poziom weryfikacji. Nie mieszaj tych znaczeń w jednej nazwie. Filtr Warstwa w workbenchu ułatwia porównywanie obiektów należących do wspólnego przekroju mimo różnych rodziców źródłowych. Liczba elementów warstwy nie musi sumować się do liczby wszystkich węzłów, ponieważ przynależności mogą się nakładać.

Przed dodaniem nowej warstwy porównaj jej wymiar z istniejącymi. Jeśli chcesz wyróżnić elementy planowanej migracji, możesz utworzyć przekrój opisujący tę migrację, zamiast przemianowywać źródłową warstwę implementation na przyszłość. Dzięki temu zachowasz informację o funkcji źródła oraz o planie przebudowy. Warstwa jest użyteczna wtedy, gdy użytkownik potrafi wyjaśnić regułę członkostwa; sama efektowna nazwa bez kryterium utrudnia późniejsze filtrowanie.

Oś	Właściwe pole	Przykład
Zawieranie	parent_id	plik w katalogu
Przekrój logiczny	layer_ids	kontrakty
Propozycja kompozycji	design.parent_id	plik w fabryce
Znaczenie	semantic.role	weryfikator
Geometria	design.position	współrzędne X/Y/Z

Źródła: S45 addLayer; S30 layers

TEMAT 078

Obserwacja źródła i nakładka design

Najważniejszą zasadą modelowania istniejącego kodu jest zachowanie rozdziału między tym, co odczytano, a tym, co chcemy zbudować. Źródłowy węzeł przechowuje nazwę, rodzica, lokalizator i kwalifikacje obserwacji. Design jest nakładką projektową: może zmieniać nazwę widoczną, opis, rodzaj artefaktu, projektowanego rodzica, położenie, parametry, kafle i inne właściwości edytora. Odczytując efektywny model, Studio bierze odpowiednie pole z design, jeśli zostało zadeklarowane.

Przeniesienie pliku do projektowanego obiektu Fabryka nie jest operacją git mv. Źródło pozostaje przypięte do starej ścieżki, a drzewo Git nadal pokazuje pierwotne zawieranie. Dzięki temu można modelować przyszłą strukturę, równocześnie wskazując istniejący kod odpowiedzialny za zachowanie. Porównanie obu współrzędnych jest podstawą planowania refaktoryzacji: inna organizacja projektowa nie wymaga utraty dowodu, z którego wyprowadzono propozycję.

Scalanie widoku zachowuje surowe source, parent_id, layer_ids oraz semantic istniejącego rekordu. Zmienione pola interfejsu zapisuje do design z kwalifikacją USER_DECLARED_DESIGN. Metadane wygenerowane na potrzeby nawigacji są usuwane z nakładki użytkownika. Nie należy więc oczekiwać, że ręczna edycja etykiety zaktualizuje pliki źródłowe albo udowodni wykonanie kontraktu. Jeżeli potrzebujesz zmienić deklarację źródłową w pełnym JSON katalogu, traktuj to jako odrębną, świadomą korektę modelu, ze wskazaniem przyczyny w historii. Standardowy szkic nawigacyjny celowo nie dopuszcza podmiany źródłowych faktów istniejącego rekordu.

Przykład: plik pozostaje w cyber_lion/architecture_projection, ale projektowany rodzic reprezentuje grupę Kompilacja i formalizacja. W Drzewie Git odnajdziesz go po pierwotnej ścieżce, a w Modelu projektu przez nową grupę. To zamierzona różnica. Późniejsza rzeczywista refaktoryzacja wymaga osobnego działania w repozytorium i ponownego odczytu źródeł. Dopiero nowy commit może potwierdzić, że zaprojektowana reorganizacja stała się faktem kodowym. Dotyczy to również relacji: nowy kierunek w design oznacza propozycję projektanta, podczas gdy pierwotny kierunek wraz z przesłanką pozostaje zapisany w źródłowym rekordzie katalogowym.

```
{
  "id": "source:file-id",
  "design": {
    "name": "Materializator",
    "description": "Projektowana odpowiedzialność modułu",
    "parent_id": "source:factory-id",
    "qualification": "USER_DECLARED_DESIGN"
  }
}
```

Źródła: S41 merge_view; S34 apply_draft

TEMAT 079

Lokalizator źródła: plik, linie i pochodzenie

Lokalizator source odpowiada na pytanie, gdzie znajduje się podstawa opisu. Repo_id wskazuje repozytorium, path jest ścieżką względem jego korzenia, a line_start i line_end mogą zawęzić zakres do deklaracji. Linie są liczone od jednego. Dla katalogu lub korzenia zakres może nie istnieć. W nowym indeksie Git pusta ścieżka wskazuje korzeń; w dostarczonym atlasie karta Źródła repozytorium używa ścieżki kropki. Obie postacie należy interpretować zgodnie z pochodzeniem rekordu.

Commit, tree i blob przypinają obiekty Git, natomiast sha256 odnosi się do odczytanych bajtów. Hash zawartości i identyfikator bloba nie są zamiennymi polami: Git identyfikuje własny obiekt, a SHA-256 dokumentuje bajty użyte przez analizę. Local_path wskazuje lokalizację na komputerze, lecz nie dowodzi istnienia pliku w chwili oglądania ani zgodności z przypiętym blobem. Symbol i jego plik mogą mieć tę samą ścieżkę oraz różne zakresy linii; to poprawny opis dwóch poziomów zawierania.

Pasek źródła w nawigacji składa informacje z węzła i deskryptora repozytorium. Jeżeli oblicza ścieżkę lokalną z zadeklarowanego katalogu bazowego, nadaje jej kwalifikację DECLARED_PATH_REFERENCE_NOT_FILE_EXISTENCE. Nie odczytuje przy tym pliku. Link URL powinien prowadzić do konkretnego commita, a nie do ruchomej nazwy gałęzi. W przypadku obiektu local-only bez ustalonego powiązania Git link może być null; taki brak jest bardziej użyteczny niż przekierowanie do podobnie nazwanego, ale innego źródła.

Zakres linii powinien być odczytywany wraz z commitem i hashem. Po dopisaniu importu na początku pliku ta sama metoda może przesunąć się o kilka wierszy w bieżącym edytorze. Link do przypiętego źródła pozostaje wtedy poprawny historycznie, choć nie wskazuje identycznych numerów w lokalnym working tree. Przy zgłoszeniu błędu podaj pełny lokalizator zamiast samego numeru linii lub zrzutu nazwy zakładki PyCharm.

Pole	Znaczenie
repo_id / path	Repozytorium i miejsce względne
commit / tree / blob	Przypięte obiekty Git
sha256	Hash odczytanej zawartości
line_start / line_end	Zakres w przypiętym źródle
local_path	Odwołanie do lokalizacji dyskowej
status / identity	Kwalifikacja powierzchni źródła

Źródła: S34 _location; S35 _source

TEMAT 080

Git: ref, commit, tree i blob

Gałąź jest nazwą prowadzącą do commita i może z czasem wskazywać inny obiekt. Ref wybrany w Studio zostaje rozwiązany do konkretnego commita przed indeksowaniem. Commit wskazuje drzewo projektu, a drzewo opisuje powiązania ścieżek z obiektami. Blob zawiera bajty pliku. Takie przypięcie umożliwia późniejszą rekonstrukcję podstaw analizy niezależnie od dalszych zmian gałęzi. W rekordzie repozytorium Studio zapisuje także selected_ref, dzięki czemu wiadomo, którą nazwą użytkownik wybrał snapshot.

Bieżący HEAD i stan working tree są oddzielnymi obserwacjami. Można indeksować inną lokalnie dostępną gałąź lub ref bez przełączania katalogu roboczego. Nie oznacza to pobrania najnowszego stanu z sieci. Jeżeli lokalny ref jest starszy, katalog opisze właśnie tę dostępną wersję. Studio nie zastępuje bajtów commita zmodyfikowanym plikiem dyskowym. Flaga local_path_is_snapshot pozostaje false, aby lokalna ścieżka nie została pomylona z gwarancją zgodności zawartości.

Przy porównywaniu wyników najpierw sprawdź commit, dopiero potem etykietę gałęzi. Dwa modele nazwane master mogą dotyczyć różnych chwil i mieć różne catalog_id. Dla indeksu Git identyfikator katalogu zawiera tożsamość repozytorium oraz commit. Inny commit tworzy odrębny snapshot, a ponowne indeksowanie tego samego może rozszerzyć już istniejący opis. Poniższe polecenia są odczytem diagnostycznym, przydatnym w terminalu PyCharm; nie wykonują checkoutu. Gdy Git zgłasza błąd własności, zachowaj tę informację jako nierozstrzygniętą obserwację, zamiast uznawać brak wyniku za brak repozytorium.

Gdy użytkownik wybierze konkretny hash zamiast nazwy gałęzi, model nadal może mieć sensowną etykietę, ale nie potrzebuje nazwy branch do ustalenia źródła. Istotne jest, aby obiekt był dostępny w lokalnej bazie Git. Brak ref nie dowodzi usunięcia całego projektu; może oznaczać niepobraną historię albo inną lokalną nazwę. Sprawdź diagnostykę przed zastępowaniem oczekiwanego snapshotu bieżącym HEAD.

```
$repo = 'C:\Users\d2j3\PycharmProjects\ai_platform'
git -C $repo rev-parse --verify 'HEAD^{commit}'
git -C $repo rev-parse --verify 'HEAD^{tree}'
git -C $repo status --short
```

Źródła: S35 index_repository; S35 inspect_repository

TEMAT 081

ai_platform: zakres historycznego modelu

Wbudowany katalog ai_platform został utworzony z dostarczonego atlasu źródłowego, a nie z bieżącego odczytu całej federacji w chwili uruchomienia Studio. Coverage oznacza go jako SOURCE_SNAPSHOT_WITH_EXPLICIT_GAPS, z live_observation i runtime_observed ustawionymi na false. Podstawą historyczną jest commit d367e47cf4f72e450cc212908b2cdfed14ceb805 oraz tree b92133d258b6fbf2c7f1d16d992e08d2a3609a0c. Zapisana chwila obserwacji to 15 października 2026 roku, 10:59:19 UTC.

Model zawiera 20 760 węzłów i 15 470 relacji. Węzły obejmują 2238 plików, w tym 2203 ai_platform i 35 prototypu Studio, 177 katalogów, 15 082 symbole, 2495 sekcji, 710 nazw pól, 47 konceptów właścicieli oraz 11 deskryptorów repozytoriów. Są to różne kategorie rekordów, dlatego liczby kart plików nie należy porównywać bezpośrednio z ogólną liczbą artefaktów. Wszystkie końce zaimportowanych relacji istnieją w tym katalogu.

Atlas deklaruje także szersze liczniki, między innymi 39 664 krawędzie i 25 945 kandydatów portów sygnatur. Nie są to liczby aktywnych połączeń ani portów utworzonych w Studio. Import wykorzystuje pełne rekordy dostępne w dostarczonym zbiorze, a nie odtwarza brakujących projekcji z samych podsumowań. Nawigacja R3.1 umożliwia rekurencyjne otwieranie tych rekordów; nie aktualizuje automatycznie ich pochodzenia. Aby analizować inny commit, należy jawnie zaindeksować repozytorium i utworzyć model tego snapshotu. Przed rozpoczęciem refaktoryzacji zanotuj więc, czy pracujesz na atlasie historycznym, czy na nowym indeksie źródeł.

Wbudowany model stanowi również opis samego prototypu Studio, lecz trzydzieści pięć kart plików prototypu odnosi się do chwili atlasu. Nie należy z tej liczby wnioskować, że dokumentuje ona wszystkie późniejsze pliki R3.1. Aktualną architekturę wykonawczą aplikacji ustalamy z dołączonego kodu tej wersji. Rozróżnienie pozwala używać samomodelu jako punktu wyjścia bez nadawania historycznej reprezentacji statusu automatycznie aktualizującego się odbicia programu.

Kategoria	Liczba
Pliki	2238
Katalogi	177
Symbole Python	15 082
Sekcje / nazwy pól	2495 / 710
Koncepty / repozytoria	47 / 11
Wszystkie węzły / relacje	20 760 / 15 470

Źródła: S11 coverage; S30 coverage

TEMAT 082

Kwalifikacje i luki atlasu

Pokrycie źródłowe nie jest równoznaczne z pełnym rozpoznaniem semantyki. W dostarczonym katalogu 15 082 symbole mają kwalifikację STATIC_AST_DECLARATION, a 2495 sekcji - SOURCE_HEADING. Są to twierdzenia o deklaracjach lub strukturze tekstu. Znaczna część plików, dokładnie 2232 rekordy, ma status SOURCE_DERIVED_DRAFT_REQUIRES_REVIEW. Oznacza to opis wymagający przeglądu, nawet jeśli nazwa roli brzmi bardzo konkretnie. Sześć plików pozostaje PROTECTED_METADATA_ONLY.

Katalog zawiera również 47 konceptów z kwalifikacją SOURCE_OWNER_DECLARATION_NOT_AUTHORITY. Właściciel wskazany w źródłach nie staje się przez ten zapis organem uprawnionym do zatwierdzania operacji w Studio lub federacji. Podobnie strukturalna przynależność katalogu nie nadaje nadrzędnej roli procesowej. W analizie należy oddzielić właściciela znaczenia, rodzica geometrycznego, lokalizację dyskową oraz podmiot dopuszczający wykonanie działania. To różne relacje, choć mogą dotyczyć jednego obiektu.

Coverage dokumentuje 286 pozycji występujących na dysku względem badanego drzewa GitHub. Nie oznacza to dowodu statusu untracked: wcześniejszy odczyt Git nie rozstrzygnął tego z powodu własności repozytorium. Wartości takie jak UNKNOWN, NOT_IN_SUPPLIED_ATLAS czy complete_dynamic_dependency_closure:false powinny pozostać widoczne przy przenoszeniu modelu do raportów. Brak rozstrzygnięcia nie jest ani potwierdzeniem braku zależności, ani przyzwoleniem na jej wymyślenie. Dobry opis ręczny dodaje przesłankę, zakres i kwalifikację, a nie tylko zmienia etykietę na bardziej stanowczą. W szczególności linia między plikami nie potwierdza aktywnej komunikacji brokerów.

Warto osobno odnotować source_capture: niektóre karty powstały przez AST, inne przez parsowanie JSON, skan tekstu albo wyłącznie hash i metadane. Zakres nabytej wiedzy jest różny dla każdej z tych metod. Porównując dwa moduły, sprawdź metodę analizy przed porównaniem pozornej szczegółowości opisu. Bogatsza karta może oznaczać lepsze pokrycie badawcze, a nie większą złożoność lub większe znaczenie architektoniczne samego modułu.

1. Otwórz Pokrycie i dostępność repozytoriów.

2. Sprawdź live_observation, runtime_observed i listę gaps.

3. W karcie obiektu porównaj semantic.qualification z source.status.

4. Przy własnej interpretacji dopisz uzasadnienie oraz status deklaracji autora.

Źródła: S30 nodes[].semantic.qualification; S11 coverage.source_comparison

TEMAT 083

Wyszukiwanie katalogu i karta elementu

Workbench Architektura / Git udostępnia przeglądanie pełnego katalogu niezależnie od sceny. Lista przyjmuje tekst wyszukiwania, rodzaj, warstwę oraz opcjonalny zakres rodzica. Zapytanie dzieli tekst na słowa i wymaga wystąpienia każdego z nich w badanych polach, takich jak identyfikator, nazwa, ścieżka, rodzaj, repozytorium lub rola semantyczna. Nie jest to wyszukiwarka całej treści kodu ani odpytywanie modelu językowego. Szukanie fragmentu ciała funkcji może więc nie przynieść wyniku.

Wybór elementu otwiera jego kartę, przodków, podstawowy lokalizator oraz relacje w obu kierunkach. Przycisk Pokaż wnętrze zawęża listę do podobiektów. Otwórz wycinek w 3D tworzy projekcję dla edytora. Głębokość początkowego wycinka oraz limit brył określają rozmiar tej pierwszej projekcji; dalsze wejścia w R3.1 korzystają z pełnego katalogu. Wyświetlane nazwy mogą uwzględniać design, ale wyszukiwarka i karta źródłowego zawierania nie są pełnym widokiem wszystkich nakładek projektowych.

Relacje na karcie mają osobne liczniki przychodzących i wychodzących rekordów. Interfejs pokazuje ograniczony zbiór, do stu rekordów danego kierunku; pełne dane pozostają w eksporcie katalogu. Brak linii w scenie i brak pozycji na pierwszej stronie nie oznaczają zatem pustego grafu. Przy diagnostyce zapisuj identyfikator elementu i licznik pełnego zbioru, a nie wyłącznie zrzut aktualnej listy. Jeśli wyszukiwanie jest puste, sprawdź pozostawiony filtr warstwy, rodzaju i rodzica. Zmiana jednego filtra nie musi oznaczać automatycznego powrotu do wszystkich korzeni.

Użyteczny test wyszukiwania polega na rozpoczęciu od dokładnej ścieżki, a następnie stopniowym poszerzaniu do nazwy pliku i rodzaju obiektu. Jeśli identyczna nazwa występuje w kilku repozytoriach, rozstrzyga repository_id i pełny lokalizator. Okruszki karty workbencha pokazują źródłowych przodków; projektowaną kompozycję oceniaj w nawigacji 3D. Tak unikniesz błędnego uznania, że ręczne przeniesienie artefaktu nie zostało zapisane tylko dlatego, że karta nadal dokumentuje pierwotne źródło.

1. Wyszukaj architecture_compiler.py w katalogu ai_platform.

2. Sprawdź ścieżkę cyber_lion/architecture_projection.

3. Otwórz kartę i odczytaj commit oraz kwalifikację roli.

4. Rozwiń Kontrakt relacji lub otwórz wycinek w 3D.

Źródła: S28 query_catalog; S45 renderNodeDetail

TEMAT 084

Ręczny model i kontrolowane rozszerzanie katalogu

Nowy model ręczny tworzy odrębny katalog, który może opisywać aplikację jeszcze nieistniejącą w kodzie. Formularz zapisuje korzeń typu concept, warstwę design oraz kwalifikacje deklaracji. Source.status otrzymuje UNMAPPED, ponieważ samo narysowanie obiektu nie wiąże go z plikiem. Taki model nadaje się do projektowania przyszłej fabryki, podziału odpowiedzialności lub hipotezy architektonicznej przed jej materializacją.

Przycisk Dodaj podobiekt działa na wybranym rodzicu. Użytkownik podaje nazwę, rodzaj, rolę, opis oraz identyfikatory warstw. Element otrzymuje własne id i zawieranie w katalogu. Dodaj relację wymaga pełnych identyfikatorów obu końców; oba muszą już istnieć. Relacja zostaje zapisana jako deklarowana, z opisem w metadanych. To świadomy etap budowy modelu, który nie tworzy automatycznie kontraktów wykonawczych w repozytorium. Warto zaczynać od kilku obiektów o odrębnych odpowiedzialnościach, zamiast od razu kopiować całe drzewo katalogów.

Dla istniejącego ai_platform wygodniejsze bywa dodanie ręcznego podobiektu do rzeczywistej gałęzi niż budowa konkurencyjnego modelu od zera. Zachowaj wtedy źródłowe obiekty jako punkt odniesienia, a nowy komponent oznacz jako projektowany. Nazwa Materializator 0!n może reprezentować przyszłą jednostkę złożoną z kilku plików. Jej związki z tymi plikami wymagają jawnego opisu rodzaju relacji. Samo zawarcie pliku wewnątrz bryły nie wyjaśnia jeszcze, czy plik jest implementacją, kontraktem, dowodem czy narzędziem testowym. Każdy taki związek powinien odpowiadać na inne pytanie analityczne i zachować własną kwalifikację.

Na początek przyjmij jednoznaczną zasadę: każdy nowy podobiekt ma mieć odpowiedź na pytanie, co wnosi do modelu. Może porządkować odpowiedzialność, stanowić hipotezę komponentu albo grupować kontrakty, lecz nie powinien być pustą kopią folderu o innej nazwie. Po zapisaniu otwórz jego JSON i sprawdź rodzica oraz warstwy. Taka krótka kontrola wykrywa pomyłki wcześniej niż rozbudowanie kilkudziesięciu relacji wokół źle umieszczonego obiektu.

1. Utwórz katalog ręczny albo wybierz istniejący model.

2. Dodaj najpierw warstwy potrzebne do przekrojów.

3. Dodaj podobiekty z rolami i opisami, bez fikcyjnych źródeł.

4. Połącz istniejące ID relacjami i sprawdź pełny JSON.

5. Zapisz rewizję katalogu, a następnie otwórz widok 3D.

Źródła: S45 newManualCatalog/addNode/addRelation

TEMAT 085

Dwie topologie: model projektu i drzewo Git

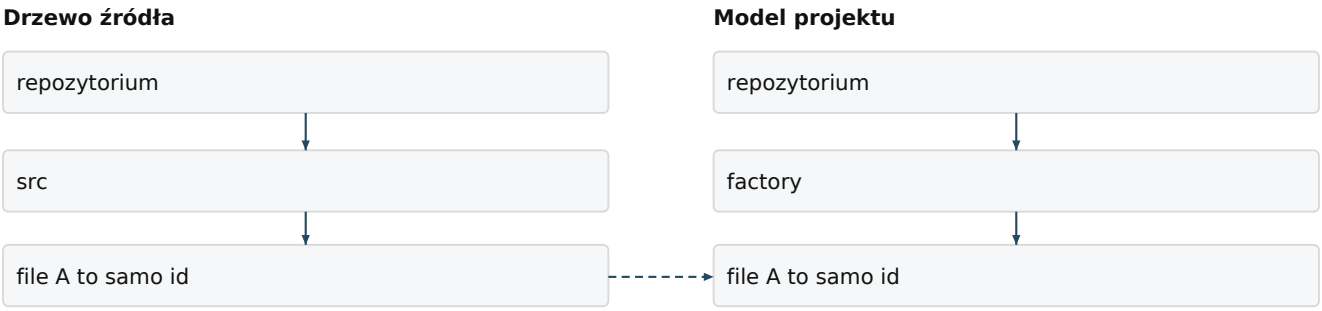
Tryb Model projektu wybiera rodzica z design.parent_id, jeżeli to pole istnieje; w przeciwnym razie dziedziczy source parent_id węzła. Tryb Drzewo Git używa wyłącznie źródłowego parent_id. Obie topologie działają na tych samych tożsamościach, dlatego przełączenie trybu może zachować wybrany obiekt, ale pokazać inną drogę dojścia i inne rodzeństwo. To podstawowy mechanizm porównywania obecnej organizacji źródeł z projektowaną kompozycją.

Źródłowe zawieranie jest szersze niż same foldery. Plik zawiera deklaracje, klasa może zawierać metody, a dokument - sekcje. Nawigacja wewnątrz symbolu nie zmienia ścieżki pliku, lecz zmienia zakres i rodzica w modelu. Dlatego pasek ścieżki katalogowej oraz okruszki topologii nie powinny być utożsamiane. W historycznym atlasie Drzewo Git może również pokazać rekord local-only lub deskryptor repozytorium o nieznanym pokryciu. Nazwa trybu nie podnosi ich kwalifikacji do obserwowanych obiektów Git.

Podczas scalania edycji wykonanej w trybie Git zmiana nazwy lub parametrów nie staje się poleceniem zastąpienia projektowanego rodzica rodzicem źródłowym. Mechanizm porównuje zmienione pola z właściwą projekcją bazową. Dzięki temu można poprawić etykietę we współrzędnych źródła i zachować wcześniejszą propozycję kompozycji. Przed przenoszeniem obiektu upewnij się, że pracujesz w Modelu projektu. Próba wprowadzenia cyklu, również przez obiekty niewidoczne w scenie, zostanie odrzucona podczas walidacji pełnej topologii. Zawieranie pozostaje acykliczne, choć inne rodzaje relacji mogą modelować sprzężenia zwrotne.

Poziom abstrakcji i geometryczna odległość także pozostają niezależne od rodzica. Przesunięcie bryły bliżej innej nie ustanawia automatycznie silniejszej zależności, wspólnej warstwy ani wyższego prawdopodobieństwa współpracy. Jeśli takie znaczenie ma być częścią modelu, musi zostać jawnie zapisane w odpowiednim opisie i kontrakcie analitycznym. Układ przestrzenny pomaga orientacji, ale nie zastępuje formalnej definicji osi i relacji. Wybór trybu określa współrzędne nawigacji, a nie poziom zaufania.

Tryb	Rodzic	Zastosowanie
Model projektu	design.parent_id lub parent_id	Planowana kompozycja
Drzewo Git	parent_id	Źródłowe zawieranie
Przełączenie	Ta sama tożsamość	Porównanie dróg dojścia



Przykład: ten sam plik ma różne miejsce w źródle i propozycji projektu. Źródłowa ścieżka pozostaje zachowana.

Źródła: S34 _topology; S41 merge_view

TEMAT 086

Wejście do wnętrza i pobieranie zakresu

W R3.1 wejście do katalogowego artefaktu pobiera jego bezpośrednie dzieci z pełnego katalogu. Nie polega już wyłącznie na zmianie lokalnego identyfikatora scope w załadowanym fragmencie sceny. Dwuklik bryły, przycisk wejścia w inspektorze, strzałka drzewa, okruszki ścieżki oraz przejście poziom wyżej korzystają z tego samego mechanizmu. Dzięki temu można przejść repozytorium → katalog → plik → klasa → metoda, nawet gdy początkowy wycinek miał małą głębokość.

Żądanie navigate wskazuje catalog_id, przypiętą revision, node_id, tryb i parametry stronicowania. Node_id:null oznacza korzeń katalogu, który może obejmować kilka repozytoriów. Wynik zawiera nowy projekt zakresu oraz skumulowany szkic zmian. Nawigacja nie tworzy rewizji katalogu. Jeżeli bieżący projekt ma ręczne edycje, są zbierane przed przejściem i odtwarzane w kolejnych projekcjach. Projekty ręczne bez powiązania z katalogiem nadal korzystają z lokalnego zawierania.

Liść pozostaje własnym zakresem i otrzymuje status EMPTY_LEAF. Studio nie przenosi wtedy użytkownika niejawnie do rodzica. PAGE_OUT_OF_RANGE oznacza natomiast, że zakres ma dzieci, ale wskazana strona leży poza aktualną listą. Te stany wymagają innych interpretacji. Pusty liść mówi jedynie o braku bezpośrednich podobiektów w dostępnym katalogu, a nie o braku działania lub zależności. Jeżeli oczekiwana klasa nie występuje w pliku, sprawdź zakres źródeł, wynik parsowania i pochodzenie katalogu. Samo ponawianie wejścia nie uzupełni danych, których analiza nie dostarczyła.

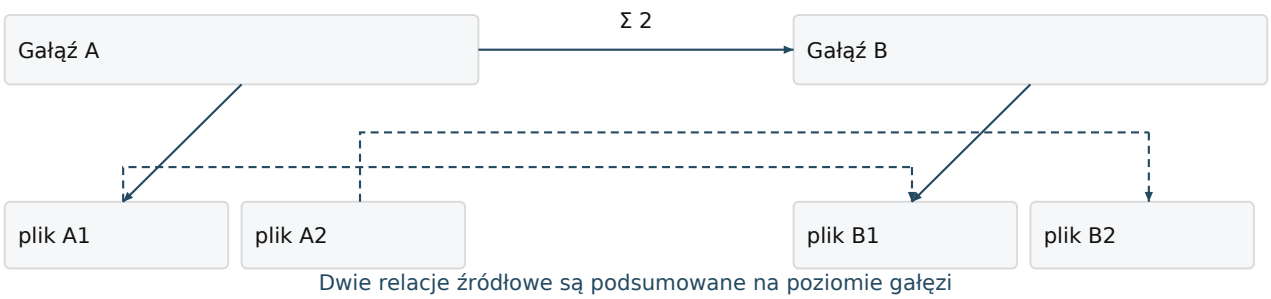
Przejście może chwilowo wymagać odczytu dużego katalogu i zbudowania projekcji. Poczekaj na zakończenie operacji przed kolejnym działaniem. Jeśli serwer zwróci błąd, zachowaj komunikat oraz bieżący projekt; nie twórz w pośpiechu ręcznych kopii spodziewanych dzieci. Takie kopie miałyby inne tożsamości niż brakujące rekordy źródłowe i mogłyby zamaskować właściwą przyczynę: błędny zakres, brak analizy symboli albo niezgodną rewizję szkicu.

1. Otwórz wnętrze repozytorium i sprawdź licznik dzieci.

2. Przejdź przez Źródła repozytorium do cyber_lion.

3. Wejdź do pliku Python, następnie do klasy.

4. Wróć przez okruszki i porównaj Drzewo Git z Modelem projektu.



Przykład agregacji między gałęziami. Licznik nie tworzy nowej relacji wykonawczej.

Źródła: S34 scope_view; S07 Przejście przez rzeczywisty model

TEMAT 087

Agregowanie zależności pomiędzy gałęziami

Na poziomie katalogów wiele relacji łączy obiekty schowane kilka poziomów niżej. Rysowanie wyłącznie krawędzi o bezpośrednio widocznych końcach ukrywałoby te zależności. Nawigacja wyznacza więc dla każdego potomka najbliższe bezpośrednie dziecko bieżącego zakresu. Funkcja _owners oblicza takie przyporządkowanie iteracyjnie. Następnie relacje źródłowe mogą zostać przedstawione między reprezentującymi je gałęziami. Nie oznacza to zmiany właściwych końców w katalogu.

Jeżeli oba końce są bezpośrednio widoczne, scena zachowuje dokładną relację oraz jej katalogowy identyfikator. Jeżeli przynajmniej jeden koniec jest reprezentowany przez gałąź, tworzona jest agregacja grupująca zgodny kierunek i rodzaj. Etykieta z mnożnikiem wskazuje liczbę podsumowanych rekordów. Metadane zawierają source_relation_ids, source_relation_count oraz do pięciu przykładów z ich kwalifikacjami. Taki opis pozwala przejść od obrazu całości do konkretnych przesłanek bez mylenia linii podsumowania z nowym kontraktem.

Relacje mieszczące się całkowicie wewnątrz jednego widocznego dziecka zasilają licznik internal_relation_count. Stają się czytelniejsze po wejściu do tej gałęzi. Krawędzie zawierania, takie jak contains lub lexical_contains, są wyłączane z linii zależności, ponieważ rodzicielstwo już przedstawia topologia. Zależność zagregowana ma kwalifikację AGGREGATED_SOURCE_RELATIONS i nie jest zapisywana jako nowa ręczna relacja przy scalaniu. Jeżeli dwie gałęzie łączy sto statycznych odwołań, jedna linia z licznikiem nie potwierdza stu aktywnych kanałów komunikacyjnych. Kierunek i wielokrotność są informacją o wejściowych rekordach, a sens wykonawczy wymaga odrębnej weryfikacji.

Agregacja może zmienić wygląd po przełączeniu topologii, ponieważ ten sam plik może należeć do innej projektowanej gałęzi. Liczba źródłowych rekordów pozostaje podstawą, lecz reprezentujące je bryły są inne. To przydatne narzędzie oceny proponowanej modularności: pozwala zobaczyć, jakie zależności przecinają nową granicę. Nie jest jednak automatyczną miarą jakości architektury. Waga semantyczna pojedynczej relacji może być większa niż liczba prostych odwołań tekstowych.

Sytuacja	Prezentacja
Dwa widoczne końce	Dokładna relacja
Końce w różnych podgałęziach	Agregacja między gałęziami
Oba końce w jednej gałęzi	Licznik wewnętrzny
Relacja zawierania	Osobna topologia, bez linii przepływu

Źródła: S34 _owners/scope_view

TEMAT 088

Granica zakresu i odnośniki zewnętrzne

Bieżące wnętrze nie obejmuje całego grafu. Zależność może wychodzić do innego katalogu, repozytorium, rodzica albo dziecka znajdującego się na następnej stronie. Studio przedstawia taki koniec jako odnośnik granicy z symbolem ↗. Jest to artefakt kontekstowy w projekcji, a nie nowy podobiekt źródłowy. Jego metadata.navigation_target_id wskazuje właściwy rekord, do którego prowadzi wejście. Dzięki temu użytkownik może podążać za zależnością bez ręcznego szukania identyfikatora.

Kontekst zewnętrzny ma limit, domyślnie dwanaście celów. Interfejs pozwala wybrać również trzydzieści dwa lub sześćdziesiąt cztery. Ograniczenie zapobiega otoczeniu małego wnętrza tak dużą liczbą brył, że kamera musi oddalić się i utracić czytelność. Po zmianie limitu pobierany jest ten sam zakres, z zachowaniem szkicu. Metadane podają liczbę relacji pominiętych przez ograniczenie kontekstu; pełny katalog nie jest przycinany.

Nie należy edytować odnośnika granicy tak, jakby był prawdziwym komponentem katalogu. Otrzymuje kwalifikację BOUNDARY_REFERENCE_NOT_CATALOG_CHILD i jest pomijany podczas scalania. Analogicznie przodkowie dodani wyłącznie dla orientacji są kontekstem nawigacyjnym. Jeśli chcesz zmienić właściwy obiekt, przejdź do celu i pracuj na jego rekordzie. Odróżnienie reprezentacji od tożsamości źródłowej chroni przed powielaniem repozytoriów lub kontraktów przy każdej zmianie sceny. Brak odnośnika na ekranie może wynikać z limitu, filtra linii lub aktualnego zakresu; weryfikuj go przez kartę relacji oraz eksport, zanim uznasz zależność za nieobecną.

Przypadek graniczny występuje, gdy cel leży w tym samym zakresie, ale poza aktualnie załadowaną stroną. Odnośnik nadal ma sens: informuje, że zależność przekracza granicę bieżącej projekcji, choć nie przekracza granicy rodzica katalogowego. Dlatego zewnętrzny kontekst należy czytać jako poza aktualnym widokiem, a nie zawsze poza repozytorium. Informacja o docelowej tożsamości pozwala rozstrzygnąć tę różnicę bez zgadywania z położenia bryły.

1. Zaznacz bryłę i odczytaj jej połączenia.

2. Sprawdź, czy drugi koniec jest oznaczony ↗.

3. Wejdź do odnośnika, aby otworzyć rzeczywisty cel.

4. Przy dużym pominięciu zwiększ Kontekst zewnętrzny i porównaj licznik.

Źródła: S34 scope_view: context_boundary; S41 _context

TEMAT 089

Stronicowanie i jawne ograniczenia widoku

Nawigacja ładuje stronę bezpośrednich dzieci wybranego zakresu. Domyślna strona interfejsu mieści trzydzieści sześć podobiektów, natomiast funkcja backendu ma domyślne limit=120 i dopuszcza maksymalnie trzysta. Są to różne wartości domyślne dwóch warstw programu. Licznik total_children opisuje pełny zakres, loaded_children - bieżącą stronę, a has_more i has_previous wskazują możliwość dalszego przejścia. Przesunięcie offset nie jest poziomem abstrakcji ani głębokością drzewa.

Osobne limity dotyczą przodków, okruszków, kontekstów i grup relacji. Scope_view zachowuje do sześćdziesięciu czterech przodków, do dwustu pięćdziesięciu sześciu pozycji ścieżki i do sześćdziesięciu czterech kontekstów. Domyślnie przygotowuje najwyżej pięćset grup relacji; parametr funkcji może przyjąć od zera do dwóch tysięcy. Pominięcia są opisane w metadata.navigation, w tym omitted_ancestors, omitted_breadcrumbs oraz relation_summaries. Flaga truncated odnosi się do ograniczonej projekcji, nie do skasowania źródłowego katalogu.

Widoczność linii jest dodatkowym filtrem renderowania. Można pokazywać wszystkie, wyłącznie związane z zaznaczeniem albo ukryć linie. Przy ponad stu dwudziestu połączeniach scena początkowo wybiera połączenia zaznaczenia. Liczba narysowanych linii może więc być mniejsza niż liczba relacji dostarczonych do sceny. Nie zmienia to ich zapisanych rekordów. Podczas porównywania zrzutów zawsze zanotuj zakres, stronę, tryb topologii, limit kontekstu oraz filtr linii. Bez tych współrzędnych dwa różne obrazy tego samego katalogu mogą wyglądać jak sprzeczne wyniki analizy, choć przedstawiają inne wycinki.

Jeśli widzisz pustą stronę po zmianie zakresu danych, wróć do pierwszej lub poprzedniej strony i sprawdź licznik. W API offset może być większy od liczby dzieci i daje wtedy PAGE_OUT_OF_RANGE. Duży limit nie zawsze poprawia użyteczność: więcej brył zwiększa nakładanie etykiet i utrudnia pracę z relacjami. Najpierw wybierz właściwy poziom analizy, potem dopasuj rozmiar strony do zadania, zamiast używać maksymalnych wartości wszędzie.

Mechanizm	Domyślnie	Maksimum
Dzieci: interfejs / funkcja	36 / 120	300
Konteksty	12	64
Grupy relacji funkcji	500	2000
Przodkowie / okruszki	64 / 256	64 / 256

Źródła: S34 scope_view/MAX_CHILDREN; S07 Przejście przez rzeczywisty model

TEMAT 090

Szkic edycji z wielu wnętrz

Szkic lion.navigation-draft/v1 zbiera niezapisane nakładki z odwiedzanych zakresów. Zawiera catalog_id, revision oraz tablice nodes i relations. Dla istniejącego rekordu wystarcza id i design. Dla nowego obiektu potrzebna jest pełna deklaracja węzła lub relacji. Szkic jest przechowywany w metadata.navigation_draft projektu Studio, dlatego zapis projektu i eksport jego JSON mogą zachować pracę wykonaną na kilku poziomach, mimo że ostatnia scena pokazuje tylko jeden z nich.

Przed zmianą zakresu collect_draft odtwarza projekcję bazową z tej samej przypiętej rewizji i dotychczasowego szkicu. Porównuje pola edytowalne oraz metadane użytkownika. Dzięki temu samo wejście do obiektu nie zamienia automatycznego układu, kafli źródłowych lub linii importu w deklaracje autora. Zmiana jednego pola nie musi nadpisywać innych, wcześniej zaprojektowanych właściwości. To szczególnie istotne podczas przełączania pomiędzy Modelem projektu a Drzewem Git.

Apply_draft odrzuca szkic innego katalogu albo innej rewizji. Dla istniejących rekordów nie pozwala podmienić source lub surowego rodzica. Waliduje także cały wynik, włącznie z obiektami poza aktualną sceną. Szkic nie jest przenośną bazą pełnego modelu: do dalszej nawigacji potrzeba odpowiadającej mu rewizji katalogu. W praktyce warto zachować zarówno projekt ze szkicem, jak i eksport katalogu, jeżeli pracę przenosi się na inny komputer. Ręczne dopisanie fałszywego numeru revision nie rozwiązuje konfliktu, ponieważ podstawą porównania pozostaje inny zestaw rekordów.

Własny szkic można przeglądać w eksporcie projektu, ale nie należy ręcznie kopiować do niego wygenerowanych odnośników granicy. Tablice przechowują zmiany właściwych rekordów, a nie stan kamery ani wszystkie obiekty orientacyjne. Zasada ta upraszcza późniejsze scalanie: powrót do poprzedniego wnętrza odtwarza ten sam projektowany obiekt, zamiast dokładać kolejną reprezentację. Przy migracji między komputerami sprawdź zarówno catalog_id, jak i numer przypiętej rewizji. Sam szkic nie ustanawia nowej obserwacji repozytorium.

```
{
  "schema": "lion.navigation-draft/v1",
  "catalog_id": "architecture:przyklad", "revision": 3,
  "nodes": [
    {"id": "file:compiler", "design": {"name": "Kompilator"}}
  ],
  "relations": []
}
```

Źródła: S41 collect_draft; S34 apply_draft

TEMAT 091

Scalanie sceny do pełnego katalogu

Operacja Zapisz edycje 3D do katalogu jest jawnym momentem przeniesienia projektu do katalogu architektury. Serwer pobiera rewizję wskazaną przez expected_revision, zbiera bieżące zmiany oraz wcześniejszy navigation_draft, stosuje nakładki, a następnie zapisuje nową rewizję katalogu. Wynik zawiera liczniki zmian i rozmiar skumulowanego szkicu. W ten sposób edycja wykonana trzy wnętrza wcześniej nie ginie tylko dlatego, że użytkownik zakończył pracę na innym poziomie.

Merge_view mapuje artefakty sceny na katalogowe tożsamości. Istniejący catalog_node_id prowadzi do właściwego węzła; nowy artefakt otrzymuje identyfikator w przestrzeni ręcznych elementów katalogu. Nowe obiekty zachowują status USER_DECLARED_NO_GIT_BINDING oraz kwalifikację deklaracji. Relacje przechowują projektowane końce wraz z opcjonalnymi tile_id i port_id. Taki adres jest dokładniejszy niż sama para brył, ale nadal wymaga zgodności interfejsów i osobnej oceny kontraktu.

Scalanie jest operacją dopisania lub aktualizacji nakładek, bez propagowania usunięć. Brak elementu w ograniczonej scenie nie może usunąć go z całego katalogu. Dotyczy to również ręcznego skasowania bryły w widoku: nie jest to polecenie kasowania rekordu źródłowego. Konteksty graniczne, przodkowie orientacyjne i agregacje relacji są pomijane. Jeżeli próba scalenia kończy się błędem, sprawdź tożsamość katalogu, przypiętą rewizję, zduplikowane powiązania i poprawność rodziców. Nie próbuj naprawiać konfliktu przez kasowanie metadanych źródła. Najpierw zachowaj projekt ze szkicem, a następnie porównaj go z najnowszą rewizją.

Szczególny przypadek stanowi zmiana przypięcia source_revision lub target_revision relacji. Collect_draft traktuje takie pole jako edycję, nawet jeśli nazwa i geometria pozostały bez zmian. Po powrocie do zakresu wybrane numery powinny się odtworzyć. Samo przypięcie nie potwierdza zgodności kontraktu; oznacza jedynie, do których rewizji artefaktów odnosi się projektowane połączenie. Warto sprawdzić je ponownie po większej zmianie portów.

1. Zapisz projekt Studio wraz ze szkicem.

2. W Architektura / Git wybierz Zapisz edycje 3D do katalogu.

3. Sprawdź komunikat o nowej rewizji i licznikach zmian.

4. Otwórz ponownie zmieniony zakres i zweryfikuj nakładki.

Źródła: S37 workbench_post: /api/architecture/merge-view; S41 merge_view

TEMAT 092

Magazyn katalogów i kontrola rewizji

CatalogStore zapisuje modele w pliku architecture.sqlite3 wybranego workspace. Jest to magazyn odrębny od zapisanych projektów Studio. Tabela catalogs wskazuje najnowszą rewizję i podsumowanie, natomiast catalog_revisions przechowuje pełne dokumenty jako skompresowane dane, hash, poprzedni hash, czas oraz powód zapisu. Rewizje nie są różnicami wymagającymi odtworzenia całego łańcucha, lecz pełnymi snapshotami dokumentu.

Przy odczycie magazyn dekompresuje JSON i sprawdza jego digest. Niezgodność zgłasza jako błąd integralności. Nie jest to podpis cyfrowy ani dowód zewnętrznego pochodzenia; kontrola pozwala wykryć niespójność między zapisanym dokumentem i jego hashem. Gzip jest tworzony deterministycznie z mtime=0. Operacje zapisu korzystają z transakcji SQLite, BEGIN IMMEDIATE i blokady w procesie, aby uniknąć częściowego zapisu w ramach obsługiwanej operacji.

Każdy zapis wymaga expected_revision zgodnego z aktualnym numerem. Dla nowego katalogu wartością oczekiwaną jest null. Jeśli inna edycja utworzyła nowszą rewizję, zapis kończy się konfliktem, zamiast cicho nadpisać cudzy wynik. Nawigacja odczytuje natomiast dokładnie przypiętą rewizję, dzięki czemu zakres nie zmienia się pod użytkownikiem podczas pracy. Odróżniaj numer rewizji katalogu od rewizji projektu i poszczególnych artefaktów. Wszystkie mogą być równe liczbowo, ale opisują inne dokumenty. Przed przeniesieniem workspace lub archiwizacją zachowaj spójny komplet danych, a do wymiany modelu używaj pełnego eksportu katalogu, zamiast kopiować przypadkową aktywną bazę podczas zapisu.

Przy pierwszym użyciu magazyn może zaszczepić dostarczony atlas jako własny katalog. Nie modyfikuje przy tym skompresowanego pliku danych w pakiecie aplikacji. Bundled_catalog zwraca odizolowaną kopię, dlatego późniejsze ręczne edycje nie przepisują instalacyjnego źródła. Jeśli użytkownik ma kilka workspace, każdy może posiadać własny rozwój katalogu. Sama identyczna nazwa atlasu na dwóch komputerach nie gwarantuje równej rewizji i równego hasha dokumentu.

Zapis	Zakres	Kontrola
catalogs	Bieżący wskaźnik	ID katalogu
catalog_revisions	Pełny model	Rewizja i SHA-256
simulation_runs	Ślad symulacji	Hash treści wyniku
expected_revision	Warunek zapisu	Wykrycie konfliktu

Źródła: S29 CatalogStore.__init__/get/save

TEMAT 093

Historia, przywracanie i eksport modelu

Historia katalogu pokazuje rewizję, SHA-256 dokumentu, poprzedni hash, czas i powód zapisu. Pozwala prześledzić rozwój modelu bez utożsamiania go z historią commitów Git. Jeden commit źródłowy może mieć kilka rewizji katalogu, ponieważ użytkownik dopracowuje role, warstwy, kontrakty oraz projekty. Z kolei inny commit może być osobnym katalogiem. W raporcie trzeba zatem podawać oba odniesienia: źródłowy commit oraz rewizję dokumentu modelowego.

Przywrócenie wcześniejszej treści nie usuwa późniejszych zapisów. CatalogStore.restore odczytuje dawną rewizję i zapisuje ją jako nową, z jawnym powodem kompensacji. Wymaga przy tym oczekiwanej bieżącej rewizji, więc równoległa edycja nadal może spowodować konflikt. Jest to przywrócenie modelu, a nie reset repozytorium ani odwrócenie refaktoryzacji w plikach. Po operacji sprawdź, czy otwarta scena nadal jest przypięta do odpowiedniej wersji, i świadomie otwórz aktualny widok.

Eksportuj katalog zapisuje pełny dokument JSON. Eksport obejmuje rekordy, nakładki, procesy i pochodzenie, lecz nie jest kopią wszystkich plików źródłowych. Źródła mogą być reprezentowane lokalizatorem, hashem albo kartą metadanych. Import nowego katalogu zachowuje jego tożsamość i waliduje strukturę; kolizja istniejącego catalog_id nie jest mechanizmem zastępowania historii. Przy wymianie modelu z drugim komputerem zachowaj również powiązanie ze szkicem projektu. Jeżeli eksportujesz wyłącznie projekt 3D z navigation_draft, odbiorca potrzebuje pasującej rewizji katalogu. Sam JSON sceny nie odtworzy niezaładowanych poddrzew z pustej bazy.

Powód zapisu jest praktycznym miejscem odnotowania zakresu zmiany: na przykład korekta roli kompilatora albo dodanie relacji test-kontrakt. Czytelna historia jest użyteczniejsza niż seria opisów edycja. Przy pełnym imporcie zachowaj oryginalny plik wymiany i jego datę, aby można było odtworzyć pochodzenie. Model może być integralny technicznie, a nadal zawierać błędną interpretację; historia pomaga wskazać, kiedy i dlaczego taka interpretacja została wprowadzona.

1. Otwórz Historia w workbenchu.

2. Porównaj powody zapisów i numery rewizji.

3. Wyeksportuj katalog przed większą zmianą struktury.

4. Przywracając, sprawdź nową rewizję zamiast oczekiwać cofnięcia licznika.

Źródła: S29 history/restore; S45 showHistory/exportCatalog

TEMAT 094

Odkrywanie lokalnej federacji

Odkrywanie repozytoriów zaczyna się od wskazanego katalogu bazowego. Dla tej instalacji naturalnym punktem jest C:\Users\d2j3\PycharmProjects. Discover_repositories wykonuje ograniczony spacer po katalogach i rozpoznaje miejsca zawierające .git. Może to być katalog metadanych albo plik charakterystyczny dla worktree. Znaleziona lokalizacja trafia do inspect_repository, które osobno próbuje odczytać stan Git. Wykrycie ścieżki i poprawny odczyt repozytorium są więc dwoma różnymi ustaleniami.

Spacer jest rekurencyjny, ale ograniczony do głębokości pięciu, dwudziestu tysięcy odwiedzonych katalogów, stu repozytoriów i trzydziestu sekund. Pomija łącza symboliczne i junctions oraz katalogi generowane, takie jak .venv, node_modules czy .git. Gdy limit zostanie osiągnięty, wynik otrzymuje PARTIAL i diagnostykę, zamiast deklarować pełne rozpoznanie dysku. Brak repozytorium na liście może wynikać z głębszego położenia, ograniczenia czasu lub niedostępności ścieżki, a nie tylko z braku lokalnej kopii.

Wbudowany atlas ma jedenaście deskryptorów repozytoriów, lecz dziewięć z nich nie zawiera źródeł w dostarczonym zbiorze. Lokalna obecność pozostaje UNKNOWN. Dotyczy to między innymi swarm, chunk-chunk, glitchlab, HA2D i writeups. Zadeklarowana lokalna ścieżka jest wskazówką, nie wynikiem świeżego skanowania. Aby rozliczyć federację, porównaj listę deskryptorów z faktycznie wykrytymi repozytoriami i ich diagnostyką. Dla brakujących elementów zdecyduj osobno, czy potrzebujesz klonu, ręcznej deklaracji zależności, czy pozostawienia jawnej luki. Odkrywanie nie pobiera niczego automatycznie i nie scala repozytoriów w jeden produkt.

Wynik worktree_only:true oznacza, że spacer jest ukierunkowany na lokalne drzewa robocze, a nie ogólny audyt wszystkich możliwych bare repositories. Nie traktuj go jako inwentaryzacji każdego magazynu obiektów Git na dysku. Dla modelowania federacji ważniejsza jest świadoma lista korzeni i dostępnych snapshotów. Repozytoria zagnieżdżone mogą zostać wykryte osobno, co nie przesądza, czy są odrębnym produktem, dostawcą biblioteki czy narzędziem operacyjnym.

1. Otwórz zakładkę Repozytoria Git.

2. Wpisz korzeń PycharmProjects i rozpocznij wykrywanie.

3. Sprawdź status COMPLETE albo PARTIAL oraz diagnostykę.

4. Wybierz konkretny korzeń repozytorium przed indeksowaniem.

Źródła: S35 discover_repositories; S30 repositories

TEMAT 095

Inspekcja repozytorium i rozpoznanie zmian

Inspect_repository odczytuje lokalny korzeń, HEAD, tree, dostępne lokalne i zdalne refs oraz publiczną postać adresów remote. Sprawdza także status working tree przez porcelain Git, zachowując kody zmian i ścieżki. Dla rename lub copy może zapisać original_path. Wynik worktree ma własny status, dirty oraz tożsamość LOCAL_WORKTREE. Snapshot_substitution:false wyraźnie wskazuje, że zmiany dyskowe nie zastępują później źródeł przypiętego commita.

Wybierz rzeczywisty korzeń repozytorium. Jeśli wskażesz jego podkatalog, kontrola --show-toplevel wykryje różnicę i zwróci diagnostykę NOT_REPOSITORY_ROOT. Lista branches obejmuje refs/heads i refs/remotes dostępne lokalnie, z oznaczeniem bieżącej gałęzi. Nie jest potwierdzeniem świeżości serwera zdalnego ani listą wszystkich repozytoriów federacji. W stanie detached HEAD pole branch może pozostać puste mimo poprawnie ustalonego commita. To normalna różnica między nazwą gałęzi a tożsamością snapshotu.

Gdy odczyt zostaje zablokowany przez dubious ownership, Studio zwraca GIT_OWNERSHIP_UNKNOWN i nie zmienia ustawień Git. Brak narzędzia daje GIT_UNAVAILABLE, a przekroczenie czasu lub rozmiaru wyjścia ma własną diagnostykę. Nie interpretuj takich stanów jako pustego drzewa. Interfejs pokazuje UNKNOWN, ponieważ nie uzyskano wystarczającej obserwacji. Przy zgłoszeniu problemu zapisz dokładny komunikat i wybraną ścieżkę. Usuń z materiału udostępnianego na zewnątrz lokalne dane, których nie chcesz publikować. Adresy remote są w Studio pozbawiane credential i query, ale sam model nadal może zawierać lokalną strukturę katalogów użytkownika.

Dane remote są porządkowane i oczyszczane z jawnych elementów uwierzytelniających. Dla znanego adresu GitHub Studio może zbudować publiczny URL źródła. Brak takiego URL nie unieważnia lokalnego indeksu: repozytorium może istnieć wyłącznie lokalnie albo używać innego hostingu. Podstawą tożsamości jest wówczas lokalizacja repozytorium, a nie wymyślony link GitHub. Odbiorca eksportu powinien znać tę różnicę, zanim spróbuje otworzyć źródła na własnym komputerze.

Wynik	Interpretacja
OBSERVED	Inspekcja uzyskała dane
dirty:true	Istnieją lokalne zmiany
branch:null	Nazwa nie została ustalona
GIT_OWNERSHIP_UNKNOWN	Git odrzucił odczyt
NOT_REPOSITORY_ROOT	Wybrano niewłaściwy katalog

Źródła: S35 inspect_repository/_public_remote/_git

TEMAT 096

Indeksowanie wskazanego ref

Przycisk Indeksuj przypięty stan Git uruchamia index_repository dla wybranego korzenia i ref. Backend rozwiązuje ref do commita, ustala tree i pobiera rekurencyjną listę obiektów przez ls-tree. Treści plików odczytuje z bazy obiektów przez cat-file --batch. Nie wykonuje checkoutu, nie importuje analizowanych modułów i nie uruchamia ich kodu. Lokalnie zmodyfikowany plik o tej samej ścieżce nie wchodzi do tego snapshotu jako zamiennik bloba.

Każdy węzeł otrzymuje identyfikator oparty na repozytorium, commicie i ścieżce, a symbole dodatkowo na deklaracji. Lokalizator zachowuje commit, tree, blob, ścieżkę oraz hash odczytanych bajtów. Katalog ma warstwę implementation. Coverage opisuje liczbę plików drzewa, liczbę zindeksowanych pozycji, budżety treści, błędy parsowania, pominięcia i nierozstrzygnięte importy. Ustawienie include_symbols:false ogranicza analizę deklaracji, ale nie zmienia znaczenia przypiętego źródła.

Domyślny limit to dziesięć tysięcy plików, z możliwością wyboru od jednego do dwudziestu tysięcy. Osiągnięcie limitu daje PARTIAL i FILE_LIMIT. Częściowy katalog nadal może być użyteczny, jeśli użytkownik rozumie jego zakres. Nie należy przedstawiać go jako pełnego modelu semantyki, zwłaszcza gdy treści chronione lub duże bloby pozostały nieodczytane. Indeksowanie innego commita tworzy odrębny catalog_id. Jeżeli celem jest analiza najnowszej pracy w PyCharm, najpierw ustal, czy potrzebujesz modelu zapisanego commita, czy odrębnego odczytu aktualnych plików. To dwie różne powierzchnie i dwa różne rodzaje dowodu.

Etykieta formularza wspomina również sekcje dokumentów, ale aktualny indeksator Git tworzy deklaracje Python, nie ogólny parser sekcji wszystkich języków. Sekcje obecne w wbudowanym atlasie pochodzą z jego wcześniejszej analizy. To ograniczenie warto uwzględnić przy porównaniu modeli: świeży indeks może mieć mniej kategorii metadanych niż bogatszy atlas, a mimo to być lepiej przypięty do wybranego commita. Liczby należy porównywać wraz z metodą ich uzyskania.

1. Wybierz repozytorium po poprawnej inspekcji.

2. Wskaż lokalnie dostępny ref, np. master albo konkretny commit.

3. Ustaw limit plików i analizę symboli.

4. Po indeksowaniu sprawdź coverage oraz source_basis.

5. Otwórz kartę wybranego pliku i porównaj commit z oczekiwanym.

Źródła: S35 index_repository; S45 indexRepository

TEMAT 097

Rekurencyjne symbole Python

Indeksator Git analizuje pliki o rozszerzeniu .py, jeżeli ich treść została odczytana i użytkownik włączył symbole. Wykrywa deklarowane kodowanie, dekoduje bajty i buduje AST. Z tego drzewa tworzy klasy, funkcje i metody. Zagnieżdżone deklaracje zachowują właściwego rodzica, qualified_name oraz zakres linii. Dla funkcji zapisuje nazwy parametrów, a dla funkcji asynchronicznej flagę async. Nie oznacza to wykonania dekoratorów ani rozpoznania wszystkich obiektów tworzonych dynamicznie.

Przejście uwzględnia deklaracje ukryte wewnątrz konstrukcji takich jak try/except lub match/case, zamiast ograniczać się do pierwszego poziomu modułu. Funkcja wewnętrzna może więc być potomkiem funkcji, a metoda - potomkiem klasy. W nawigacji otwarcie takiej bryły pokazuje deklaracje obecne w katalogu. Nie należy oczekiwać pełnego grafu dziedziczenia, rozstrzygnięcia monkey patchingu lub instancji utworzonych przez fabryki. Aktualny indeks jest statyczną obserwacją struktury kodu.

Błędy składni, kodowania albo nadmiernej rekurencji nie są zastępowane pustym sukcesem. Węzeł pliku otrzymuje UNKNOWN_PARSE_ERROR, a coverage zapisuje rodzaj błędu i ewentualną linię. Zbiór staje się częściowy. Jeżeli wnętrze pliku wygląda pusto, sprawdź symbol_analysis: NOT_REQUESTED oznacza inną sytuację niż NOT_PYTHON lub UNKNOWN_PARSE_ERROR. Wbudowany atlas i świeży indeks używają też innych zasobów metadanych; nie każdy nowy symbol ma kompletny opis sygnatury z typami. Kandydaci portów powstają wtedy, gdy źródło rzeczywiście dostarczy callable signature_contract, a nie wyłącznie z samej listy nazw parametrów.

Nie wszystkie symbole utworzone przez analizę mają identyczny zakres stabilności tożsamości. Identyfikator deklaracji z nowego indeksu zawiera między innymi linię i nazwę kwalifikowaną. Po zmianie pliku w nowym commicie może powstać nowa tożsamość źródłowa. Ciągłość logicznego interfejsu pomiędzy wersjami wymaga więc osobnego opisu migracji, a nie założenia, że podobna metoda zawsze pozostaje tym samym artefaktem modelu.

Metadana	Co wyjaśnia
qualified_name	Pozycję deklaracji w zagnieżdżeniu
async	Rodzaj deklaracji funkcji
symbol_analysis	Czy i jak analizowano plik
line_start / line_end	Zakres deklaracji w źródle
OBSERVED_PYTHON_AST	Pochodzenie strukturalne symbolu

Źródła: S35 index_repository: visit; S28 project_view: signature_contract

TEMAT 098

Importy i kandydaci odwołań

Statyczny indeks Git tworzy relacje imports wtedy, gdy zapis importu daje się dopasować do jednego lokalnego kandydata modułu. Mapa uwzględnia pliki Python, __init__.py oraz alternatywną postać ścieżek zaczynających się od src/. Analiza obsługuje zapis import i from oraz próbuje odtworzyć importy względne. Nie uruchamia systemu importowania Pythona i nie ustala środowiska instalacji pakietów. Jednoznaczne dopasowanie nazwy w drzewie jest kandydatem zależności, a nie dowodem rozstrzygnięcia podczas wykonania.

Jeżeli nie znaleziono jednoznacznego celu, rekord trafia do coverage.unresolved_imports ze ścieżką, linią, nazwami modułów i statusem UNRESOLVED_OR_EXTERNAL. Przyczyna może być prosta: moduł pochodzi z biblioteki zewnętrznej. Może jednak oznaczać alternatywny układ pakietów, dynamiczne importowanie albo brakujący fragment drzewa. Nie należy automatycznie budować fikcyjnego węzła o takiej nazwie i ogłaszać problemu rozwiązanym. Weryfikacja wymaga odpowiedniej konfiguracji projektu oraz dalszych przesłanek.

Indeksator tworzy także references dla jednoznacznych dopasowań nazw deklaracji wewnątrz pliku. Są to LEXICAL_NAME_CANDIDATE, z runtime_resolution:UNKNOWN. Użycie identyfikatora nie musi być wywołaniem funkcji, a podobna nazwa może mieć inne znaczenie w runtime. W katalogu powiązanie zachowuje source_ref z linią, co pozwala sprawdzić konkretny zapis. Dla własnego modelu 0!n można na tej podstawie zaproponować kontrakt, lecz należy oddzielić statyczną przesłankę od projektu kanału komunikacji. Również nierozstrzygnięte wyrażenia z atlasu nie powinny otrzymywać domyślnego statusu sukcesu.

Kolejna pułapka dotyczy nazw zduplikowanych w jednym pliku. Jeżeli kilka deklaracji ma tę samą nazwę, indeksator nie wybiera arbitralnie jednego celu odwołania. Pozostawienie niejednoznaczności jest zamierzone. Przy projektowaniu kanału możesz wybrać właściwy interfejs na podstawie dodatkowej wiedzy, ale kwalifikacja takiej relacji powinna wskazywać decyzję autora. Automatyczne dopasowanie leksykalne i ręczne rozstrzygnięcie semantyczne mają odmienną podstawę i warto je zachować osobno.

1. Otwórz coverage.unresolved_imports świeżego indeksu.

2. Sprawdź, czy moduł jest zewnętrzną zależnością czy brakującym źródłem.

3. Przejdź do podanej linii w przypiętym commicie.

4. Zapisz własne rozstrzygnięcie jako odrębną przesłankę lub deklarację.

Źródła: S35 index_repository: module_map/relation

TEMAT 099

Źródła chronione, budżety i częściowe analizy

Indeks Git ma osobny budżet dla listy plików i dla odczytu zawartości. Pojedynczy blob może mieć najwyżej jeden MiB, a suma kwalifikowanych treści trzydzieści dwa MiB. Katalog indeksowania jest ograniczony do sześćdziesięciu tysięcy węzłów. Przekroczenia oraz pominięcia trafiają do coverage. Karta pliku może zatem istnieć, choć jego treści nie odczytano. Obecność ścieżki i obiektu Git nie jest dowodem pełnej analizy semantycznej.

Funkcja _protected rozpoznaje katalogi generowane i vendor oraz nazwy wskazujące na poufne źródła. Pomijane są między innymi .env, .ssh, katalogi credentials, secrets, tokens, keys, pliki kluczy i certyfikatów prywatnych. Dla nich pozostają metadane z kwalifikacją PROTECTED_SOURCE albo GENERATED_OR_VENDOR. Jest to reguła oparta na ścieżkach i nazwach, a nie uniwersalny detektor sekretów w dowolnej treści. Nie należy więc traktować modelu jako automatycznie bezpiecznego do publicznego opublikowania.

Komendy Git działają z wyłączonym interaktywnym pytaniem o credential, hookami i monitorowaniem systemu plików. Mają limity czasu oraz łącznej wielkości wyjścia. Błąd odczytu obiektów może pozostawić częściową strukturę z jawną diagnostyką. Do oceny wyniku używaj tree_complete, content_complete i analysis_complete, ponieważ odnoszą się do innych etapów. Pełna lista ścieżek nie oznacza pełnych treści, a pełne treści nie oznaczają pełnego grafu dynamicznych zależności. Przy ograniczeniu budżetu najlepiej wykonać świadomy, opisany zakres pracy i zapisać jego luki, zamiast usuwać diagnostykę tylko po to, by uzyskać etykietę COMPLETE.

Dla symlinków Git o trybie 120000 indeksator nie odczytuje celu jako zwykłej treści źródła. Submodule również pozostaje wskazaniem do odrębnego obiektu i nie jest rekurencyjnie analizowany przez sam indeks rodzica. Jeżeli kontrakt zależy od takiego składnika, zapisz lukę i zaindeksuj dostępne repozytorium oddzielnie. W przeciwnym razie model może poprawnie opisać granicę, lecz nie będzie przedstawiał implementacji po drugiej stronie.

Ograniczenie	Wartość	Skutek
Pliki Git	1-20 000	Częściowe drzewo po limicie
Pojedynczy blob	1 MiB	Metadane bez treści
Treści łącznie	32 MiB	Jawne pominięcia
Węzły indeksu	60 000	Ograniczenie deklaracji

Źródła: S35 _protected/index_repository/_git

TEMAT 100

Brakujące repozytorium i pobranie bez checkoutu

Formularz Brakuje repozytorium? Pobierz je lokalnie obsługuje jawne pobranie publicznego repozytorium GitHub. Wymaga adresu HTTPS w postaci owner/repository i nowego katalogu docelowego. Nie przyjmuje credential w URL, query, fragmentów ani dowolnych transportów Git. Rodzic lokalizacji musi już istnieć, a sam katalog docelowy nie może istnieć. Dzięki temu operacja nie staje się niezamierzonym nadpisaniem lokalnego projektu.

Clone_repository wykonuje git clone --no-checkout --no-recurse-submodules, z wyłączonymi hookami. Opcjonalna nazwa gałęzi jest sprawdzana przez check-ref-format. Operacja ma limit stu dwudziestu sekund. Nie instaluje zależności, nie uruchamia programu i nie pobiera rekurencyjnie submodule. Repozytorium po pobraniu może być od razu indeksowane z bazy obiektów, mimo braku zwykłych plików working tree. Pusty widok katalogu plików produktu po takim klonie nie jest sam w sobie błędem pobrania.

Nieudany klon zwraca UNKNOWN oraz diagnostykę i nie usuwa automatycznie częściowego katalogu. Może on zawierać dane potrzebne do ustalenia przyczyny albo pliki utworzone równolegle. Nie ponawiaj więc operacji do tej samej ścieżki bez jej oceny. Dla prywatnych repozytoriów aktualny formularz nie oferuje przepływu logowania; do modelowania potrzebna jest dostępna lokalnie kopia uzyskana odpowiednim narzędziem użytkownika. W federacji opis brakującego repozytorium może pozostać jako deskryptor UNKNOWN, dopóki nie uzyskamy zawartości. Taki stan lepiej zachowuje granicę wiedzy niż automatyczne tworzenie pustych komponentów udających przeanalizowane źródła.

Pobranie repozytorium jest operacją inną niż odkrywanie i indeksowanie. Dopiero po udanym pobraniu należy sprawdzić, jakie refs są dostępne, a następnie wskazać snapshot do analizy. Nie zakładaj, że opcjonalny branch oznacza aktualny stan całej federacji. Klon jednego repozytorium nie pobiera automatycznie brakujących projektów wymienionych w dokumentacji, a nieobecność submodule nie jest uzupełniana przez utworzenie fikcyjnych węzłów z zawartością.

1. Wpisz publiczny adres https://github.com/OWNER/REPO.

2. Wybierz nowy katalog pod PycharmProjects.

3. Opcjonalnie podaj prawidłową nazwę gałęzi.

4. Po pobraniu sprawdź clone_status i diagnostykę.

5. Zaindeksuj właściwy ref, aby utworzyć katalog architektury.

Źródła: S35 clone_repository; S45 cloneForm/cloneRepository

TEMAT 101

Skan dysku a indeks obiektów Git

Studio ma dwa odmienne sposoby pozyskiwania opisu źródeł. Index_repository odczytuje obiekty zapisanego commita i tworzy katalog architektury. Scan_sources odczytuje aktualne pliki w wybranym katalogu lokalnym i dodaje kandydatów do projektu Studio. Druga operacja może zobaczyć niezacommitowane zmiany, lecz nie ustala ich zgodności z Git. Jej raport ma status PARTIAL_LOCAL_READBACK_NOT_COMMIT_VERIFIED, a source_ref_verified i snapshot_atomic pozostają false.

Skan lokalny przyjmuje od jednego do pięciuset plików, domyślnie sto pięćdziesiąt. Limit dotyczy plików źródłowych, nie liczby wygenerowanych symboli. Obowiązują dodatkowo dziesięć MiB na plik, pięćdziesiąt MiB łącznie, pięćset artefaktów projektu, dwa tysiące relacji, dwadzieścia tysięcy wpisów katalogowych i sześćdziesiąt cztery symbole na plik. Korzeń musi być rzeczywistym katalogiem, a łącza i junctions są odrzucane lub omijane. Analizowane rozszerzenia obejmują między innymi Python, Markdown, JSON, JavaScript, TypeScript, PowerShell i pliki konfiguracyjne.

Żaden z tych odczytów nie wykonuje kodu analizowanego produktu. Importy lokalnego skanu zachowują UNRESOLVED_RUNTIME nawet wtedy, gdy istnieje kandydat celu w tym samym zakresie. Skan nie tworzy też automatycznie nowej stabilnej tożsamości logicznego interfejsu. Wybór metody powinien wynikać z pytania: model konkretnego commita wymaga indeksu Git; ogląd bieżącej, nieutrwalonej pracy wymaga odczytu dysku z zachowaniem jego kwalifikacji. Porównując oba wyniki, zestawiaj ścieżki, hashe i zakresy, a nie wyłącznie podobne nazwy artefaktów. Różnica bajtów może być zamierzonym etapem refaktoryzacji.

Skanowanie dysku nie jest też operacją śledzącą zmiany w czasie rzeczywistym. Raport opisuje konkretny odczyt, którego pliki mogły zmieniać się podczas pracy edytora. Pole snapshot_atomic:false ujawnia tę własność. Przy analizie poważnej regresji utrwal źródłowy stan odpowiednim mechanizmem repozytorium albo wykonaj kontrolowane porównanie hashy. Model z odczytu lokalnego pozostaje wartościowym kandydatem, ale jego kwalifikacja musi towarzyszyć wnioskom.

Cecha	Indeks Git	Skan lokalny
Podstawa	Przypięty commit	Bieżące bajty dysku
Dokument	Katalog architektury	Projekt i raport
Lokalne zmiany	Osobna obserwacja	Mogą wejść do odczytu
Atomowy snapshot	Obiekty commita	Nie jest zapewniony

Źródła: S35 index_repository; S32 scan_sources

TEMAT 102

Interfejs pliku i kandydaci sygnatur

Format lion.file-interface/v1 opisuje konkretny plik jako powierzchnię źródłową. Zawiera ścieżkę, hash bajtów, rozmiar, proponowany typ mediów, kodowanie, język, eksporty, importy i odwołania do kontraktów. Effect:READ_ONLY_SOURCE określa charakter opisu, a resolution:STATIC_CANDIDATE ogranicza siłę wniosku. Typ mediów jest kandydatem wyprowadzonym z rozszerzenia. Obecność exports oznacza deklaracje znalezione przez parser, nie pełną publiczną API ani gotowy kontrakt uruchomieniowy.

Describe_file_interface działa na przekazanych bajtach, nie otwiera samodzielnie pliku i nie wykonuje modułu. Dla Python buduje AST i zapisuje do sześćdziesięciu czterech kandydatów deklaracji. Symbol_ref najpierw identyfikuje lokalizator: ścieżkę, hash i pointer obejmujący nazwę kwalifikowaną oraz linię. Powiązanie go z artefaktem projektu wymaga jawnego przypisania. Błędy parsowania i nadmiar symboli trafiają do gaps. Importy mają UNRESOLVED_RUNTIME i początkowo pustą listę lokalnych kandydatów.

Katalogowa projekcja 3D może tworzyć kafle parametrów i wyniku dla callable signature_contract dostarczonego w metadanych. Typy str, int, float, bool, list i dict są mapowane na prymitywne typy Studio; inne adnotacje pozostają any. Self i cls są pomijane jako argumenty wejściowe. Wartości domyślne nie są kopiowane do aktywnych parametrów. Powstały port jest kandydatem źródłowej sygnatury, a nie dowodem, że broker obsługuje dany kanał. Budując rzeczywisty kontrakt, uzupełnij nazwę danych, typ, warunek użycia i sposób potwierdzenia zgodności. Dopiero wtedy można świadomie modelować połączenie między kaflami, zamiast utożsamiać deklarację funkcji z gotowym protokołem federacji.

Walidacja interfejsów plików sprawdza między innymi wymagane pola, dozwolone effect i resolution oraz limit eksportów. Nie rozstrzyga znaczenia dowolnej adnotacji typów ani polityki uruchamiania. Dla pliku innego niż Python opis może pozostać BYTES_ONLY, z pustymi eksportami i importami. Jest to poprawny stan, jeśli parser nie ma wiedzy o danym języku. Nie należy go poprawiać przez ręczne oznaczenie AST_PARSED bez odpowiadającego odczytu.

```
{
  "schema": "lion.file-interface/v1",
  "path": "src/example.py",
  "effect": "READ_ONLY_SOURCE",
  "resolution": "STATIC_CANDIDATE",
  "export_semantics": "STATIC_DECLARATIONS_NOT_PUBLIC_API"
}
```

Źródła: S32 describe_file_interface/_file_interface; S28 project_view

TEMAT 103

Ponowny indeks i zachowanie własnego modelu

Ponowne indeksowanie tego samego commita nie powinno usuwać ręcznych opisów. Serwer sprawdza, czy istnieje katalog o tym samym catalog_id, i wywołuje preserve_reindex. Funkcja rozpoczyna od świeżego wyniku, następnie łączy nodes, relations i layers według identyfikatorów. Istniejące rekordy wygrywają przy kolizji, a procesy są kopiowane z aktualnego modelu. Coverage otrzymuje politykę SAME_COMMIT_ENRICHMENT_PRESERVES_EXISTING_MODEL. Wynik jest zapisywany jako kolejna rewizja.

Takie zachowanie pozwala wzbogacić wcześniej ograniczony indeks, na przykład przez większy limit plików lub włączenie symboli, bez wymazania już opracowanych ról. Nie jest to jednak ogólny mechanizm scalania różnych commitów. Preserve_reindex odrzuca różne catalog_id, ponieważ zmiana źródłowego snapshotu może oznaczać inne pliki, linie i deklaracje. Przeniesienie nakładek między wersjami wymaga odrębnej analizy tożsamości i zmian. Sama podobna nazwa pliku nie rozstrzyga, czy reprezentuje ten sam interfejs logiczny.

Istniejące rekordy mają pierwszeństwo także wtedy, gdy świeży odczyt zawiera bogatsze metadane dla tego samego identyfikatora. Polityka chroni wcześniejszy model, lecz nie powinna być mylona z automatyczną aktualizacją każdego pola analitycznego. Po rozszerzeniu indeksu sprawdź wybrane karty, liczniki oraz opis pokrycia i zachowaj eksport przed dalszą pracą. Jeśli celem jest nowy stan gałęzi, wykonaj indeks właściwego commita jako oddzielny katalog. W raporcie migracji nazwij relację między modelem historycznym, modelem aktualnych źródeł i projektowaną architekturą, zamiast nadawać im wszystkim jedną etykietę bieżący LION.

Przy eksperymencie z większym limitem warto sprawdzić nie tylko wzrost liczby rekordów, lecz również te obiekty, które wcześniej były dopracowane ręcznie. Zachowanie ich opisów jest głównym celem tej polityki. Jeśli nowy commit zmienia interfejs, utrzymanie starej nakładki bez przeglądu mogłoby wprowadzić fałszywą zgodność. Rozdzielenie snapshotów zmusza do nazwania tego przejścia i przygotowania świadomego mapowania między wersjami.

1. Wyeksportuj model z ręcznymi nakładkami.

2. Zaindeksuj ten sam ref i sprawdź rozwiązany commit.

3. Porównaj nowe liczby węzłów i relacji.

4. Zweryfikuj zachowanie opisów, warstw i procesów.

5. Dla innego commita pracuj na odrębnym katalogu źródłowym.

Źródła: S41 preserve_reindex; S37 workbench_post: /api/repositories/index

TEMAT 104

Przekrój praktyczny: kompilator architektury 0!n

Dobrym pierwszym przekrojem ai_platform jest łańcuch reguła formalizacji → manifest → kompilator → test → wynik → impact. Rozpocznij od wyszukania cyber_lion/architecture_projection/architecture_compiler.py. W historycznym modelu karta opisuje ArchitectureCompiler jako element sprawdzający CandidateDesign, źródłowy baseline, warstwy, flows i semantic owners, a następnie tworzący EvolutionDelta, podgląd AFM i RequiredFormalizationSet. Jest to opis źródłowo wyprowadzony, który nadal wymaga przeglądu semantyki i nie potwierdza bieżącego wykonania.

Otwórz kartę pliku i sprawdź przypięty commit, zakres linii oraz status. Następnie przejdź do deklaracji i relacji. W scenie porównaj agregacje na poziomie katalogu z dokładniejszymi połączeniami po wejściu do pliku. Ręcznie dodaj projektowany obiekt, który ma reprezentować kompilator jako artefakt mozaikowy. Nadaj mu rolę, warstwy i interfejsy, a istniejący plik pozostaw jako źródłowy punkt odniesienia. Związek implementuje lub weryfikuje musi być zadeklarowany świadomie; zwykłe zawieranie nie wyjaśni całego procesu.

W drugim kroku odszukaj manifest i plik testu. Powiąż obiekty z konkretnymi źródłami oraz określ, jaki dowód ma potwierdzać zgodność. Świeży wynik testu jest osobnym artefaktem dowodowym; istniejąca karta testu nie jest takim wynikiem. Zapisz projekt ze szkicem, przejdź do kolejnych wnętrz i scal nakładki do katalogu. Po ponownym otwarciu sprawdź tożsamości, kwalifikacje i zachowane źródła. Ostateczny model powinien pozwalać odpowiedzieć: co istnieje w kodzie, co proponujemy, na czym opieramy znaczenie i jaki sprawdzian jest jeszcze potrzebny. Tak przygotowany przekrój można następnie wykorzystać w deklaracji procesu ewolucji, zachowując granicę między symulacją a rzeczywistą materializacją.

W tym ćwiczeniu skala sukcesu nie polega na liczbie narysowanych brył. Rezultatem jest zamknięty, czytelny przekrój: użytkownik może przejść od tezy architektonicznej do właściwego pliku i symbolu, zobaczyć projektowany kontrakt oraz wskazać brakujący dowód. Jeżeli któregoś odnośnika nie udało się ustalić, pozostaw go jawnie jako lukę, zamiast domykać łańcuch domyślną relacją.

1. Źródło: znajdź plik, symbol i przypięty lokalizator.

2. Model: opisz odpowiedzialność oraz adresy interfejsów.

3. Relacje: rozdziel implementację, kontrakt i dowód.

4. Weryfikacja: wskaż test oraz osobny świeży wynik.

5. Zapis: utrwal projekt, scal nakładki i sprawdź historię.

Źródła: S05 Co wymaga dalszej pracy; S30 architecture_compiler.py source card

TEMAT 105

Trzy analizy: wartości, terminy i generacje

Studio udostępnia trzy różne mechanizmy obliczeniowe. Symulacja przepływu oblicza wartości parametrów między typowanymi portami. Analiza czasu wyznacza najwcześniejsze terminy zadań wynikające z jawnych ograniczeń. Symulacja ewolucji przechodzi przez zadeklarowane etapy kolejnych generacji i wytwarza opisy kandydatów. Wspólna reprezentacja JSON ułatwia porównywanie wyników, jednak każdy mechanizm czyta inne pola i ma własne statusy. Wynik jednego nie staje się automatycznie wejściem następnego.

Przed rozpoczęciem pracy określ pytanie. Pytanie „jaką wartość otrzyma odbiorca po przeskalowaniu danych?” kieruje do propagacji. Pytanie „kiedy walidacja może zacząć się po dwóch gałęziach?” kieruje do czasu. Pytanie „jaka mutacja zostanie wybrana po falsyfikacji hipotezy?” wymaga procesu ewolucyjnego. Nazwa artefaktu, jego kolor, umiejscowienie wewnątrz innej bryły ani semantyczna relacja depends_on nie wybierają silnika obliczeń.

W praktyce warto przechowywać trzy osobne raporty dla jednej decyzji projektowej. Pierwszy dokumentuje zgodność i przepływ danych, drugi deklarowane terminy, a trzeci reguły zmiany projektu. Dopiero autor analizy zestawia ich przesłanki. Na przykład poprawna propagacja nie dowodzi, że dostępne są zasoby potrzebne do wykonania zadań. Zakończona generacja nie dowodzi, że wygenerowany kod istnieje na dysku. Analizowany model może opisywać przyszłe rozwiązanie.

Wszystkie trzy funkcje obliczeniowe traktują wejście jako dokument projektu lub procesu. Tworzą wynik bez wykonywania kodu repozytorium. Propagacja kopiuje parametry, analiza czasu zachowuje dokument bazowy, a ewolucja tworzy deterministyczne deskryptory. Interfejs może następnie wyświetlić lub wyeksportować wynik. Archiwizacja wywołania ewolucji przez serwer jest oddzielnym efektem zapisu, a nie wykonaniem etapów procesu.

Dobrym punktem wejścia są szablony Przepływ demo i Czas oraz domyślny proces dziewięciu etapów. Ułatwiają odróżnienie problemu deklaracji od usterki samej analizy. Najpierw odtwórz spodziewany wynik szablonu, potem zmieniaj jedną przesłankę naraz i porównuj ślad.

Pytanie	Mechanizm	Wynik główny
Wartości portów	simulate_workspace	values, trace, blocked
Terminy zadań	analyze_schedule	tasks, relations, makespan
Zmiana pokoleń	simulate_process	events, artifacts, generations

Źródła: S33 simulate_workspace

TEMAT 106

Parametry: wartość, typ i jednostka

Parametr jest nazwanym rekordem w obiekcie parameters artefaktu. Jego znaczenie wynika łącznie z nazwy, typu, wartości oraz jednostki. Typy podstawowe to number, integer, boolean, string, object, array i any. Wartość null reprezentuje brak znanej wartości. Nie należy wpisywać napisu „UNKNOWN” do parametru liczbowego: byłby tekstem, który nie spełnia kontraktu number. Interfejs prezentuje nieznaną wartość jako UNKNOWN, zachowując w dokumencie null.

Typowanie jest celowo ścisłe w miejscach, gdzie język Python dopuszcza uproszczenia. Wartość true nie jest liczbą jeden, mimo że niektóre operacje języka porównują je jako równe. Integer opisuje liczbę całkowitą, a number obejmuje także wartości zmiennoprzecinkowe. Liczby muszą być skończone. NaN i nieskończoność nie są poprawnym sposobem zapisania niepewności i powodują diagnostykę dokumentu JSON.

Jednostka jest tekstem. Pusta jednostka oznacza jawnie zadeklarowaną wielkość bezwymiarową i różni się od całkowicie nieobecnego pola unit. Brak jednostki generuje ostrzeżenie UNIT_UNKNOWN. Takie dane mogą uniemożliwić propagację lub porównanie. Studio nie ma słownika, który automatycznie uznawałby sekundy i milisekundy za równoważne w modelu portów. Różnica nazw jednostek wymaga jawnej konwersji w relacji.

Parametr pozostaje stanem modelu, a nie odczytem funkcji z pliku źródłowego. Nadanie mu nazwy duration nie uruchamia analizy czasu. Analizator czasu czyta metadata.timing.duration. Analogicznie parametr completed o wartości true jest przesłanką bramki, a nie potwierdzeniem zakończenia procesu w systemie operacyjnym. Opis i pochodzenie wartości powinny wyjaśniać, czy autor użył pomiaru, założenia czy przykładu syntetycznego.

Przy przygotowaniu scenariusza nazwij parametry według roli kontraktu, podaj jednostki i pozostaw null tam, gdzie brak danych. Następnie dodaj porty związane z odpowiednimi parametrami. Po zmianie typu sprawdź wszystkie relacje dochodzące i wychodzące: zgodność portu, parametru i wiązania jest badana oddzielnie. Pozornie kosmetyczna zmiana jednostki może więc zmienić model poprawny w niepoprawny.

```
"parameters": {
  "latency": {"type": "number", "value": 0.25, "unit": "s"},
  "approved": {"type": "boolean", "value": null, "unit": ""}
}
```

TEMAT 107

Porty i kontrakt wykonawczy przepływu

Relacja dataflow wymaga pełnego wskazania obu końców: artefaktu, kafla i portu. Samo narysowanie połączenia między bryłami nie tworzy wystarczającego kontraktu. Port źródłowy musi mieć kierunek out albo inout, a docelowy in albo inout. Kafel musi istnieć na danym artefakcie, a identyfikator portu musi należeć do tego kafla. Walidator odróżnia brak artefaktu, brak kafla i brak portu, aby wskazać właściwe miejsce naprawy.

Każdy port deklaruje typ, jednostkę, pojemność capacity i opcjonalnie nazwę parametru. Jeśli parameter jest niepusty, port wskazuje konkretny parametr swego artefaktu. Wiązanie relacji nie może omijać tego przypisania i zapisywać innego parametru. Typ oraz jednostka parametru muszą odpowiadać kontraktowi portu. Dzięki temu obiekt może mieć kilka niezależnych interfejsów i nie uzależnia komunikacji od przypadkowej kolejności pól.

Typ integer może zasilać kontrakt number. Odwrotna deklaracja nie oznacza automatycznego zaokrąglenia. Typ any poszerza zgodność kontraktu, ale nie zwalnia z kontroli konkretnego wyniku transformacji ani nie daje podstawy do rozstrzygania bramek porównania. Gdy oba porty mają niepuste schema_ref, odmienne odwołania do schematów wymagają osobnego adaptera. Sama zgodność nazw typów nie scala różnych kontraktów obiektowych.

Capacity jest nieujemną liczbą całkowitą. Zero oznacza brak zadeklarowanego limitu, a dodatnia liczba ogranicza liczbę połączeń dataflow korzystających z portu. Nie jest przepustowością w bajtach na sekundę ani czasem wykonania. Eksponowane aliasy dzielą budżet pojemności, więc utworzenie publicznego portu rodzica nie zwiększa dozwolonej liczby połączeń ukrytego portu dziecka.

Przed symulacją sprawdź model i usuń błędy kontraktów. Ostrzeżenie o nieaktualnej rewizji ma inne znaczenie niż błąd typu: model da się przeanalizować, lecz konkretna relacja będzie zablokowana. Rzetelny opis interfejsu powinien wymieniać odpowiedzialność portu, jednostkę, parametr oraz wersję końca, z którym utrzymywany jest kontrakt. Tak powstaje przejście od metafory „trzymania się za ręce” do jednoznacznego połączenia danych.

Źródła: S33 validate_workspace

TEMAT 108

Wiązania i jawne przekształcenia wartości

Tablica bindings opisuje, co dokładnie relacja przenosi między parametrami. Każdy rekord wymaga source_parameter i target_parameter wskazujących istniejące nazwy po obu stronach połączenia. Relacja może posiadać kilka wiązań, ale nie może dwukrotnie zapisywać tego samego parametru docelowego. Identyfikatory kafli i portów określają interfejs, natomiast wiązania określają wartości podlegające propagacji. Te dwie warstwy kontraktu muszą być ze sobą zgodne.

Dostępne transformacje to identity i scale. Identity kopiuje wartość, zachowując jej strukturę JSON. Nie wykonuje mnożenia mimo przypadkowo wpisanych współczynników: factor różny od jedności lub offset różny od zera przy identity stanowią błąd deklaracji. Scale realizuje zależność y = x × factor + offset. Oba parametry muszą być liczbowe, a współczynniki skończone. Nie ma interpretowania dowolnych wyrażeń, skryptów ani wywołań funkcji.

Gdy porty mają różne jednostki, konwersja wymaga transformacji scale oraz obiektu unit_conversion ze zgodnymi polami from i to. Studio nie wyprowadza współczynnika z nazw jednostek. Autor odpowiada za sens wartości factor. Dla 1500 ms przekazywanych do portu w sekundach poprawny przykład używa factor równego 0.001 i offset równego zero. Sama deklaracja jednostek bez poprawnego mnożnika byłaby formalnie czytelna, ale merytorycznie błędna.

Transformacja musi wytworzyć wartość zgodną z typem docelowego parametru. Mnożenie dające liczbę zmiennoprzecinkową nie zostanie automatycznie zaokrąglone do integer. Przekroczenie zakresu prowadzące do nieskończoności daje blokadę NONFINITE_RESULT. Błąd obliczenia daje TRANSFORM_ERROR. Jeśli w jednej relacji nie powiedzie się którekolwiek wiązanie, żaden jej przygotowany zapis nie trafia do zestawu zmian tego kroku.

W edytorze zaczynaj od jednego wiązania i identity, potwierdź typy oraz jednostki, a następnie wprowadź konwersję. W opisie relacji zapisz pochodzenie współczynników i oczekiwany przykład liczbowy. Dzięki temu przyszły przegląd może oddzielić poprawność strukturalną od poprawności założeń fizycznych lub biznesowych. Poprawna walidacja nie jest dowodem, że zadeklarowany przelicznik odpowiada rzeczywistości.

```
{
  "source_parameter": "milliseconds",
  "target_parameter": "seconds",
  "transform": "scale", "factor": 0.001, "offset": 0,
  "unit_conversion": {"from": "ms", "to": "s"}
}
```



Przykład przepływu z transformacją scale. Taki opis wymaga prawidłowych portów, pinów i bramek.

TEMAT 109

Exposes: komunikacja przez granicę artefaktu

Zagnieżdżenie artefaktów wyznacza granice kompozycji. Zwykły przepływ dataflow jest wykonywany wyłącznie między artefaktami posiadającymi tego samego bezpośredniego rodzica. Bezpośrednie połączenie elementu wewnątrz modułu z elementem poza modułem może pozostać w dokumencie jako ostrzeżenie, ale symulacja zablokuje je kodem BOUNDARY_CROSSING. Publiczny interfejs kompozytu buduje się przez relacje exposes łączące rodzica z jego bezpośrednim dzieckiem.

Porty expose muszą mieć ten sam kierunek, zgodny typ i tę samą jednostkę. Dla wejścia kierunek przepływu wiedzie od portu rodzica do portu dziecka. Dla wyjścia wiedzie od dziecka do rodzica. Relacja musi rzeczywiście łączyć bezpośrednią parę rodzic-dziecko; nie można pominąć kilku poziomów zagnieżdżenia. Wartość jest delegowana przez identity. Zmiana jednostki lub znaczenia wymaga osobnego adaptera, a nie ukrycia obliczenia w ekspozycji portu.

Przykładowa ścieżka wejściowa prowadzi od nadawcy A do publicznego wejścia kompozytu B przez dataflow. Następnie exposes przekazuje wartość z B do wewnętrznego wykonawcy C. Ponieważ symulacja jest synchroniczna, wartość przechodzi kolejne krawędzie w kolejnych krokach, a nie natychmiast podczas rysowania połączenia. Analogiczna ścieżka wyjściowa zaczyna się w C, prowadzi przez publiczny port B i kończy na zewnętrznym odbiorcy.

Eksponowanie nie nadaje uprawnień do uruchomienia kodu ani nie tworzy połączenia sieciowego. Opisuje dopuszczalną drogę wartości w modelu. Publiczny port i jego aliasy mają wspólny budżet capacity. Podłączenie wielu zewnętrznych odbiorców do różnych aliasów tego samego ukrytego portu może przekroczyć limit, nawet gdy każdy alias osobno wygląda na nieprzeciążony.

Przy refaktoryzacji najpierw utwórz publiczne porty kompozytu, potem ekspozycje i dopiero po nich przepływy między kompozytami. Sprawdź szczególnie kierunek wejście-wyjście, jednostki oraz aktualne rewizje. W dokumentacji wskaż, które porty są kontraktem stabilnym dla otoczenia. Pozwala to zmieniać wnętrze modułu bez ręcznego przepinania wszystkich relacji zewnętrznych i zachowuje czytelną odpowiedzialność poszczególnych warstw.

TEMAT 110

Bramki przepływu: składnia i operandy

Bramka gate jest obiektem JSON przypisanym do relacji. Silnik propaguje wartości tylko wtedy, gdy jej wynik wynosi TRUE. FALSE i UNKNOWN powodują blokadę tej relacji, lecz mają inne znaczenie diagnostyczne. Bramka nie jest fragmentem Pythona ani JavaScriptu. Dostępne operatory tworzą ograniczone drzewo deklaracji, które można bezpiecznie analizować bez wykonywania kodu źródłowego projektu.

Literal przyjmuje true, false lub null. Operatory all i any łączą tablicę podbramek, a not neguje pojedynczy argument arg. Porównania obejmują eq, ne, lt, lte, gt, gte oraz in. Operandy mogą wskazywać parametr przez parę artifact_id i parameter albo zawierać bezpośrednio rekord wartości z polami value, type i unit. Brak któregokolwiek z tych trzech pól pozbawia porównanie podstawy do jednoznacznego rozstrzygnięcia.

Porównanie wymaga zgodnych jednostek. Liczba pięć w sekundach i liczba pięć w milisekundach nie tworzą automatycznie równości ani nierówności; wynik to UNKNOWN z UNIT_MISMATCH. Typy number i integer mogą być porównywane liczbowo. Boolean nie jest liczbą. Typ any nie dostarcza rozstrzygającego kontraktu operandu. Porządkowanie jest dostępne dla liczb i tekstu, natomiast nie dla dowolnych obiektów lub tablic.

Operator in sprawdza przynależność wartości do tablicy za pomocą typowanej równości albo obecność fragmentu tekstu w tekście. Dzięki kontroli typów true nie zostaje uznane za to samo co liczba jeden. Porównanie obiektów i tablic przez eq uwzględnia strukturę, typy oraz wartości zagnieżdżone. Nie dokonuje dopasowania semantycznego tekstów ani oceny podobieństwa znaczeń.

W interfejsie relacji edytuj bramkę jako JSON i sprawdź model przed uruchomieniem symulacji. W pierwszej próbie użyj literal true, aby oddzielić problem samego przepływu od problemu warunku. Następnie dodawaj porównania pojedynczo. Maksymalna głębokość bramki wynosi szesnaście, a liczba jej węzłów dwieście pięćdziesiąt sześć. Przekroczenie limitu jest błędem deklaracji, a nie wynikiem FALSE.

```
{"op":"gte",
 "left":{"artifact_id":"plan","parameter":"duration"},
 "right":{"value":6,"type":"number","unit":"s"}}
```

Źródła: S33 evaluate_gate

TEMAT 111

TRUE, FALSE i UNKNOWN: poprawne wnioskowanie

UNKNOWN oznacza brak wystarczających podstaw do rozstrzygnięcia bramki. Nie jest równoznaczne z odmową, zerem, pustym tekstem ani błędem aplikacji. Typowy powód stanowi null w parametrze wejściowym, ale także brak jednostki, niezgodność typów albo niepoprawny operand. Pole reason w wyniku evaluate_gate pozwala odróżnić te przypadki. Dopiero analiza przyczyny wskazuje, czy trzeba uzupełnić obserwację, poprawić kontrakt czy naprawić składnię.

Studio stosuje trójwartościową logikę Kleene’a. W połączeniu all pojedynczy warunek FALSE wystarcza, aby całe wyrażenie było FALSE, nawet jeśli inny warunek pozostaje UNKNOWN. Jeśli nie występuje FALSE, ale istnieje UNKNOWN, wynik całości pozostaje UNKNOWN. Dla any wystarczy jeden TRUE. Jeśli TRUE nie występuje, lecz istnieje UNKNOWN, nie wolno uznać całego wyrażenia za FALSE.

Negacja nie rozstrzyga niewiedzy. not UNKNOWN nadal daje UNKNOWN. Puste all jest spełnione, a puste any niespełnione; model wykorzystuje puste all jako domyślną bramkę relacji. Jest to świadoma konwencja logiczna, nie wykrycie warunków w repozytorium. W procesach ewolucyjnych odpowiednikami tych operatorów są and i or, a ich tablice muszą zawierać co najmniej jeden argument. Nie należy kopiować składni między silnikami bez sprawdzenia schematu.

Rozważ warunek dopuszczenia: zatwierdzenie jest znane i fałszywe, a pomiar jakości nieznany. Całe all jest FALSE, ponieważ znana odmowa już przesądza wynik. Jeśli zatwierdzenie jest prawdziwe, brak pomiaru pozostawia UNKNOWN. Taki wynik powinien skierować autora do danych o jakości, a nie do sztucznego wpisania zera. Nieznana wartość zachowuje informację o granicy obserwowalności modelu.

Przy audycie zapisuj zarówno stan bramki, jak i jego przesłanki. Sam napis BLOCKED w śladzie przepływu nie wyjaśnia, czy nastąpiła świadoma odmowa GATE_FALSE, czy brak danych GATE_UNKNOWN. Rozdzielenie tych sytuacji jest szczególnie ważne przy odtwarzaniu decyzji o refaktoryzacji, weryfikacji hipotez oraz późniejszym wiązaniu projektu z rzeczywistym systemem wykonawczym.

A	B	all(A,B)	any(A,B)
TRUE	UNKNOWN	UNKNOWN	TRUE
FALSE	UNKNOWN	FALSE	UNKNOWN
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN
TRUE	FALSE	FALSE	TRUE

Źródła: S33 _evaluate_valid_gate

TEMAT 112

Synchroniczne kroki propagacji i zbieżność

Symulacja zaczyna od kopii parametrów wszystkich artefaktów i wybiera wyłącznie relacje dataflow oraz exposes. Pozostałe typy mogą opisywać strukturę i znaczenie, lecz nie stają się wykonywanymi przepływami. Relacje są porządkowane według identyfikatora. Domyślny limit wynosi dwadzieścia kroków, a żądanie może wskazać liczbę całkowitą od jednego do stu. Dokument wejściowy pozostaje niezmieniony.

W każdym kroku wszystkie bramki i źródła czytają stan z początku tego kroku. Silnik zbiera przygotowane zapisy, kontroluje konflikty, a następnie stosuje zmiany razem. W łańcuchu A → B → C odbiorca C nie zobaczy nowej wartości B w tym samym kroku, w którym B ją otrzymał. Jest to istotne przy interpretowaniu śladu: kolejność w tablicy relacji ani kolejność narysowania linii nie przyspieszają przejścia wartości.

Dla początkowych wartości A = 4, B = null i C = null oraz wiązań B = 2A, C = B pierwszy krok zmienia tylko B na osiem. Drugi krok przekazuje osiem do C. Trzeci stwierdza, że żaden zapis nie zmienia wartości. Wtedy wynik jest zbieżny. Jeżeli inne relacje pozostają zablokowane, status przyjmuje CONVERGED_WITH_BLOCKS zamiast CONVERGED. Zbieżność dotyczy stabilnego stanu wartości, a nie spełnienia wszystkich intencji projektu.

Ślad rozróżnia APPLIED i UNCHANGED. Ponowne zapisanie identycznej wartości nie przedłuża analizy w nieskończoność. Zablokowane relacje są odnotowywane przy każdym kroku, a pole blocked wyniku opisuje blokady ostatniego kroku. Dlatego wczesna blokada spowodowana brakującą wartością może zniknąć po jej dotarciu. Pełną historię należy czytać w trace, nie tylko w końcowej liście blokad.

Jeśli limit kończy się przed krokiem potwierdzającym stabilność, status to STEP_LIMIT. Oznacza brak potwierdzenia zbieżności w zadanym budżecie. Nie przesądza o niemożności osiągnięcia stanu stałego. Przykładowo limit dwóch kroków w opisanym łańcuchu daje już C = 8, ale nadal nie zawiera trzeciego kroku, który potwierdziłby brak dalszych zmian.

Stan	A	B	C
Początek	4	null	null
Po kroku 1	4	8	null
Po kroku 2	4	8	8
Krok 3	bez zmian	bez zmian	bez zmian

TEMAT 113

Rewizje końców i świadome ponowne związanie

Każdy artefakt ma dodatnią rewizję, a relacja przechowuje source_revision i target_revision. Te liczby określają, z którymi wersjami końców sprawdzono połączenie. Symulacja wymaga zgodności obu zapisanych rewizji z aktualnymi artefaktami. Nieaktualne lub nieobecne przypięcie daje blokadę STALE_REVISION. Walidator zgłasza ten stan jako ostrzeżenie, ponieważ dokument zachowuje wartość historyczną i może nadawać się do przeglądu mimo wstrzymanej propagacji.

Edycja semantyczna artefaktu w interfejsie podnosi jego rewizję, jeśli rzeczywiście zmieniła się treść semantyczna. Dotyczy to między innymi parametrów, portów, źródeł i metadanych. Zmiana samego położenia lub rozmiaru jest wyłączona z porównania semantycznego. Przesunięcie bryły nie powinno więc wymuszać ponownego uzgadniania wszystkich kontraktów. Inaczej jest po zmianie typu parametru lub roli interfejsu.

Inspektor relacji pokazuje związane wersje końców i oznaczenie STALE, kiedy dostrzega rozbieżność. Przycisk Powiąż bieżące rewizje aktualizuje przypięcia. Użyj go dopiero po sprawdzeniu, czy obecne typy, jednostki, parametry i bramka nadal realizują zamierzone znaczenie. Mechaniczne kliknięcie wszystkich ostrzeżeń usuwa informację o potrzebnym przeglądzie, lecz samo nie dostarcza dowodu zgodności kontraktu.

Przykład kontrolny: źródło w rewizji jeden przekazuje cztery sekundy do odbiorcy, a relacja jest związana z wersją jeden. Zmiana źródła na sześć sekund przez edycję parametru podnosi jego rewizję. Następna symulacja nie przekaże sześciu sekund przez stare przypięcie. Po świadomym przeglądzie i ponownym związaniu wynik może zostać obliczony z nowej wartości. Poprzedni raport nadal opisuje starszy wariant modelu.

Nie mieszaj rewizji projektu, artefaktu i katalogu architektury. Rewizja projektu dotyczy zapisu dokumentu w lokalnym magazynie. Rewizja artefaktu wspiera zgodność końców relacji. Rewizja katalogu przypina szerszy model źródłowy i nakładki projektowe. Aktualizacja jednej z tych liczb nie jest automatycznym potwierdzeniem pozostałych. W raportach porównawczych zachowuj cały dokument wejściowy i wynik, aby dało się odtworzyć właściwy zestaw wersji.

Źródła: S42 editArtifact; relationInspector; rebind

TEMAT 114

Konflikty zapisu, cykle i budżet obliczeń

Dwie relacje mogą próbować zapisać ten sam parametr docelowy w jednym kroku. Studio traktuje taki układ jako konkurencyjne zapisy, nawet jeśli obie proponowane wartości są identyczne. Nie wybiera zwycięzcy według kolejności plików, identyfikatora ani kolejności dodania relacji. Zwraca status CONFLICT oraz diagnostykę COMPETING_WRITES. W trace znajduje się lista proponowanych zapisów, która wskazuje oba źródła problemu.

Wykrycie konfliktu zatrzymuje zastosowanie całego kroku, a nie tylko spornego parametru. Inne przygotowane zmiany tego samego kroku również nie są zatwierdzane. Wynik values zachowuje stan sprzed tego kroku, w tym ewentualne zmiany wcześniejszych kroków. Ta atomowość chroni przed raportem sugerującym spójny wynik, gdy część zależności została rozstrzygnięta arbitralnie. Naprawa wymaga jawnego rozdzielenia warunków albo utworzenia osobnego artefaktu agregującego.

Cykle relacji semantycznych są dopuszczalne. Cykle zagnieżdżenia geometrycznego są błędem. Cykle wykonywalnych przepływów mogą być analizowane w ograniczonej liczbie kroków, ale ich wynik zależy od wartości i transformacji. Dwukierunkowe identity przy równych wartościach jest stabilne, a przy różnych może oscylować aż do STEP_LIMIT. Konflikt wymaga konkurencyjnych zapisów do tego samego parametru. Cykl z mnożnikiem większym od jedności może wciąż zmieniać wartości aż do STEP_LIMIT lub blokady wyniku liczbowego. Sam kształt pierścienia nie dowodzi stabilności.

W żądaniu max_steps można wybrać od jednego do stu kroków, domyślnie dwadzieścia. Ślad mieści najwyżej dwadzieścia tysięcy wpisów. Po osiągnięciu jego limitu pojawia się ostrzeżenie TRACE_LIMIT; dalsze obliczenia mogą być kontynuowane bez dopisywania pełnego śladu. Limit projektu wynosi pięćset artefaktów i dwa tysiące relacji. Dotyczy lokalnego dokumentu symulacji, a nie pełnego katalogu architektury.

Jeśli pracujesz w stronicowanym wnętrzu katalogu, pamiętaj o zakresie wejścia. Grupy relacji i odnośniki kontekstu są projekcją nawigacyjną, nie kompletnym obliczeniowym grafem całej federacji. Do eksperymentu przygotuj jawny, samowystarczalny projekt z rzeczywistymi portami i wiązaniami. Zwiększenie limitu kroków nie uzupełni brakujących podobiektów ani nie zamieni statycznych importów kodu w wykonywalne zależności danych.

TEMAT 115

Ćwiczenie: producent, plan i dwie blokady

Utwórz nowy projekt z szablonu Przepływ demo. Model zawiera producenta, plan skalowany dwukrotnie, odbiorcę dopuszczonego warunkiem oraz dwie gałęzie kontrolne. Producent ma duration równe cztery sekundy, completed ustawione na true i observed ustawione na null. W pozostałych artefaktach duration jest początkowo nieznane. Wszystkie przepływy działają we wspólnej granicy rodzica demo i są związane z początkowymi rewizjami.

Uruchom Sprawdź model, a następnie symulację. Relacja demo:scale ustawia plan na osiem sekund. Relacja demo:ready wymaga jednocześnie planu nie krótszego niż sześć sekund oraz ukończenia producenta. Odbiorca otrzyma więc osiem. Gałąź FALSE oczekuje completed równego false i pozostanie pusta. Gałąź UNKNOWN oczekuje dodatniej obserwacji observed, której brak nie pozwala rozstrzygnąć warunku. Wynik końcowy to CONVERGED_WITH_BLOCKS po trzech krokach.

Otwórz szczegóły przebiegu. W pierwszym kroku demo:ready jest jeszcze zablokowane, bo plan nie otrzymał wartości. W drugim warunek staje się TRUE i odbiorca dostaje osiem sekund. Trzeci potwierdza stabilność. Końcowe blocked zawiera gałęzie demo:blocked i demo:missing z przyczynami odpowiednio GATE_FALSE oraz GATE_UNKNOWN. To wzorcowy przykład sytuacji, w której lokalna zbieżność współistnieje z jawnie nierozstrzygniętą obserwacją.

Teraz zmień duration producenta na dwie sekundy. Najpierw uwzględnij zmianę rewizji źródła: przejrzyj relacje i świadomie powiąż bieżące końce. Plan otrzyma cztery sekundy, ale próg sześciu nie będzie spełniony. Odbiorca dopuszczony wcześniej pozostanie w wynikowym stanie wynikającym z aktualnego wejścia dokumentu; symulacja nie zapisuje automatycznie wcześniejszych wyników jako nowych parametrów. Przy niezmienionych bazowych null nie otrzyma więc wartości.

Na końcu ustaw observed na true i ponownie uzgodnij dotknięte kontrakty. W ten sposób ćwiczysz oddzielnie wartość liczbową, znany warunek logiczny i uzupełnienie obserwacji. Eksportuj każdy raport i opisz jedną zmienioną przesłankę. Nie dopisuj do raportu, że producent faktycznie wykonał zadanie: cały przykład jest syntetycznym modelem propagacji.

Parametr po analizie	Wartość	Znaczenie
demo:plan.duration	8 s	4 × 2
demo:accepted.duration	8 s	TRUE
demo:false.duration	null	GATE_FALSE
demo:unknown.duration	null	GATE_UNKNOWN

```
from lion_studio.seeds import new_project
from lion_studio.model import simulate_workspace

r = simulate_workspace(new_project("demo"))
assert r["status"] == "CONVERGED_WITH_BLOCKS"
assert r["steps"] == 3
assert r["values"]["demo:accepted"]["duration"]["value"] == 8
```

TEMAT 116

Ramy i rangi poziomów abstrakcji

Poziom abstrakcji artefaktu zapisuje się w metadata.topology.level. Rekord wymaga id, całkowitego rank oraz niepustego frame_ref. Wyższa ranga oznacza wyższą abstrakcję, ale tylko we wspólnej, jawnie nazwanej ramie. Liczba sześć nie ma uniwersalnego znaczenia bez wskazania ramy. Dwa poziomy mogą mieć różne identyfikatory i tę samą rangę, jeśli autor uznaje je za równoległe w danym porządku.

Nie wyprowadzaj rangi z rodzica geometrycznego, axes.detail ani rodzaju pliku. Parent_id opisuje zawieranie, axes.detail poziom szczegółowości, a level.rank abstrakcję. Metarama może być rodzeństwem zadań we wnętrzu jednej aplikacji i jednocześnie mieć wyższą rangę od nich. Dokładnie tak skonstruowany jest szablon Czas: metarama posiada rangę dwa, a zadania rangę zero w tej samej ramie odniesienia.

Topologia relacji może deklarować peer, defines, realizes lub refines. Peer zakłada równe rangi. Defines jest skierowane od wyższej rangi do niższej. Walidator zgłasza ostrzeżenie, gdy deklaracja peer łączy odmienne rangi albo defines prowadzi od rangi równej lub niższej. Nie odwraca automatycznie połączenia. Dla realizes i refines sama nazwa nie narzuca w tej wersji dodatkowej nierówności rang; ewentualną regułę należy opisać w kontrakcie.

Podsumowanie porównuje źródło względem celu. Same oznacza równe rangi, upper wyższą rangę źródła, a lower niższą. Incomparable pojawia się przy brakujących danych, niepoprawnym poziomie lub różnych ramach. Jednakowa liczba w dwóch różnych frame_ref nie wystarcza do porównania. Jeśli zadeklarowano relację poziomów, której nie da się zweryfikować, walidator zachowuje ostrzeżenie o nieporównywalności.

Przy budowie nowej metaramy najpierw określ znaczenie rang i stabilny identyfikator ramy. Potem przypisz poziomy artefaktom i dopiero na końcu opisz relacje między nimi. Nie koryguj rang wyłącznie po to, aby uciszyć ostrzeżenie: mogłoby to zniszczyć znaczenie modelu. Sprawdzenie formalne potwierdza spójność deklaracji, lecz nie orzeka, czy wybrana hierarchia trafnie reprezentuje rzeczywistą architekturę LION.

```
"topology": {"level": {
  "id":"architecture-contract",
  "rank":4, "frame_ref":"lion.frame.design"
}}
```

Źródła: S40 _comparison; validate_topology

TEMAT 117

Odległość i siła związku: deklaracje mierzalne

Odległość topologiczna jest oddzielną deklaracją w relacji, a nie wynikiem odczytania współrzędnych brył. Pole distance obejmuje metric, value, unit, method i source_ref. Aby podsumowanie uznało pomiar za zadeklarowany, potrzebna jest nieujemna skończona liczba oraz komplet kontekstu tekstowego. Brak wartości lub niepełna metoda pozostawiają status UNKNOWN. Wartość zero oznacza zadeklarowane zero w konkretnej metryce; nie zastępuje braku wiedzy.

Przykład stanowi odległość liczona liczbą przejść w określonym grafie zależności. Autor może wpisać metric równą declared-dependency-hops, value równe dwa i unit równą hop, wskazując w method sposób liczenia oraz w source_ref odpowiednią ścieżkę lub raport. Walidator nie uruchamia algorytmu najkrótszej drogi i nie sprawdza, czy wpisane dwa odpowiadają rzeczywistemu grafowi. To zadeklarowany wynik wraz z podstawą do przeglądu.

Siła związku znajduje się w strength i obejmuje value, unit, meaning oraz source_ref. Dla jednostek normalized i probability zakres musi należeć do przedziału od zera do jedności. Pozostałe jednostki nie mają automatycznie tego ograniczenia, lecz wartość nadal musi być skończona i nieujemna. Wpisanie 0.8 jako normalized nie ustanawia osiemdziesięcioprocentowego prawdopodobieństwa. Meaning powinno wyraźnie określać, czy chodzi o wagę sprzężenia, ocenę ekspercką czy inną wielkość.

Pochodzenie i metoda mają znaczenie równie duże jak sama liczba. Dwa zespoły mogą przydzielić identyczne wagi na podstawie różnych skal i uzyskać wartości nieporównywalne. Studio zachowuje takie deklaracje bez automatycznej normalizacji. Nazwa źródła nie jest dowodem istnienia pomiaru, a poprawny kształt source_ref nie potwierdza jego treści. Dokumentacja musi umożliwiać dotarcie do faktycznej przesłanki.

W inspektorze relacji uzupełnij sekcję Topologia i dynamika, następnie sprawdź model. Przesuń bryły i ponownie odczytaj pomiar: deklarowana odległość pozostanie ta sama. To prosty test odróżniający metrykę semantyczną od geometrii prezentacji. Jeśli zmiana położenia ma wpływać na model domenowy, potrzebny jest osobno zdefiniowany i zaimplementowany mechanizm obliczeniowy; aktualna topologia go nie zapewnia.

Źródła: S40 _measurement_issues; _measurement_summary

TEMAT 118

Dynamika topologii i granice interpretacji

Dynamika relacji opisuje oczekiwany sposób zmiany związku w czasie. Rekord dynamics może wskazywać mode równy static, event lub continuous, pole law oraz time_unit. Jest to metadana projektu, a nie program wykonywalny. Law może być tekstem albo obiektem JSON, lecz aplikacja nie interpretuje go jako skryptu, równania różniczkowego ani subskrypcji zdarzeń. Tryb continuous nie uruchamia automatycznie ciągłej symulacji.

Tryby event i continuous potrzebują niepustego prawa oraz jednostki czasu, aby deklaracja była kompletna. Brak któregoś z tych elementów daje ostrzeżenie i UNKNOWN w podsumowaniu. Static może zostać zadeklarowane bez prawa i jednostki czasu. Nieznany tryb lub niewłaściwy typ pola są błędami. Null zachowuje brak deklaracji; nie jest zamieniany na static i nie ustanawia domyślnej stałości relacji.

Przykładowa reguła zdarzeniowa brzmi: „ponownie oceń wagę po zmianie rewizji źródła”. Taka deklaracja pomaga projektantowi wskazać zależność między wersjonowaniem i analizą, ale nie tworzy obserwatora systemu plików. Aby reguła miała skutek automatyczny, potrzebna byłaby osobna implementacja, kontrakt zdarzenia, uprawnienia i potwierdzony przebieg. W bieżącym Studio zapis prawa jest materiałem architektonicznym przeznaczonym do przeglądu.

Podsumowanie topology_summary zwraca schemat lion.topology-summary/v1, konwencję wyższej rangi jako wyższej abstrakcji i perspektywę źródła względem celu. Pole display_coordinates_used ma wartość false. Dla poprawnie zadeklarowanej dynamiki powód brzmi DECLARATIVE_LAW_NOT_EXECUTED. Ten zapis jest istotny podczas przekazywania modelu innemu agentowi: bogaty opis dynamiki nadal nie stanowi dowodu wykonania prawa ani powstania procesu w systemie operacyjnym.

Podczas przeglądu całości rozdziel trzy pytania. Czy poprawnie zapisano poziomy i pomiary? Czy ich znaczenie jest uzasadnione źródłami? Czy istnieje komponent wykonujący opisaną zmianę? Topologia pomaga odpowiedzieć na pierwsze i uporządkować drugie. Trzecie wymaga odrębnego dowodu. Takie rozdzielenie pozwala rozwijać elastyczny metamodel bez przypisywania aplikacji zdolności, których nie zaimplementowano w jej silnikach obliczeniowych.

Źródła: S40 _dynamics_issues; topology_summary

TEMAT 119

Czas zadań: deklaracje, jednostki i zero scenariusza

Artefakt uczestniczy w analizie czasu, gdy ma obiekt metadata.timing. W interfejsie wybierz artefakt i zakładkę Czas. Zadeklaruj duration, unit, release oraz opcjonalnie deadline, opis i źródło założenia. Duration określa czas trwania, release najwcześniejszą dostępność, a deadline termin oceny zakończenia. Wszystkie wartości są odnoszone do wspólnego zera scenariusza. To nie są daty kalendarzowe ani wskazania zegara systemowego.

Dozwolone jednostki to ms, s, min i h. Wynik jest przeliczany na sekundy. Czas trwania, dostępność i termin muszą być nieujemnymi skończonymi liczbami albo null w dozwolonych miejscach. Wartość logiczna nie jest liczbą. Brak duration oznacza czas nieznany. Pominięte release domyślnie daje zero, ale jawne release: null oznacza UNKNOWN. Brak lub null w deadline oznacza brak terminu do porównania.

Sam parametr artefaktu o nazwie duration nie czyni go zadaniem. Analogicznie relacja o nazwie „poprzedza” nie tworzy ograniczenia czasu, dopóki nie ma metadata.timing z poprawnym rodzajem zależności. Wyraźne rozdzielenie pozwala opisywać model semantyczny bez niezamierzonego narzucania kolejności. Rodzic, grupa, ranga abstrakcji i położenie 3D również nie dodają krawędzi czasowych.

Analiza używa najwcześniejszych nieujemnych startów wynikających z dostępności oraz wszystkich zadeklarowanych zależności. Nie przypisuje maszyn, pracowników, limitów brokerów ani kalendarzy zmianowych. Dwa zadania mogą więc otrzymać ten sam start, choć w rzeczywistym środowisku rywalizowałyby o jeden zasób. Status COMPLETE oznacza kompletne obliczenie tego modelu ograniczeń, a nie potwierdzoną wykonalność harmonogramu w organizacji.

Dodaj do source_ref identyfikator przesłanki czasu: pomiaru, deklaracji lub hipotezy. Następnie uzupełnij opis rozróżniający czas pracy, opóźnienie i oczekiwanie. Nie wpisuj czasu oczekiwania jako duration tylko dlatego, że występuje w poprzednim raporcie; może to podwójnie policzyć ograniczenie relacji. Oczekiwanie jest wyliczane oddzielnie jako start minus release i pojawia się w szczegółach zadania.

```
"timing": {"duration": 2, "unit": "s", "release": 0,
  "deadline": 9, "source_ref": "scenario:baseline",
  "description": "Syntetyczne założenie projektu"}
```

Źródła: S39 validate_timing; analyze_schedule

TEMAT 120

FS: następca czeka na koniec poprzednika

Zależność FS łączy zakończenie zadania źródłowego ze startem zadania docelowego. Jeśli A kończy się po dziesięciu sekundach, a lag wynosi trzy sekundy, B nie może wystartować wcześniej niż w sekundzie trzynastej. Analizator nie sumuje czasów wszystkich poprzedników: bierze maksimum dolnych ograniczeń startu. Wspólna dostępność zadania może narzucić jeszcze późniejszy termin. Każda zależność jest wymagana, o ile pozostaje w modelu.

Wzór FS ma postać start_B ≥ finish_A + lag. Czas trwania B nie zmienia tego dolnego ograniczenia startu, ale wpływa na jego koniec i makespan. Lag ma własną jawną jednostkę i może być dodatni, zerowy albo ujemny. Dodatni opisuje opóźnienie, ujemny wyprzedzenie. Jawne null oznacza brak znanego opóźnienia, a nie zero; wówczas zależność może uniemożliwić określenie terminu następcy.

Sprawdź przykład stosowany w testach: A ma release równy dwie minuty i duration równe dwie minuty, więc zaczyna w sekundzie sto dwudziestej i kończy w dwieście czterdziestej. B trwa dziewięćdziesiąt tysięcy milisekund, czyli dziewięćdziesiąt sekund. FS z lag równym pół minuty daje start B w sekundzie dwieście siedemdziesiątej i koniec w trzysta sześćdziesiątej. Mieszanie jednostek jest tu jawne i przeliczane przez analizator czasu.

Jeśli B ma release równe trzysta sekund, wystartuje w trzysta, a nie dwieście siedemdziesiąt. Jeżeli lag jest bardzo ujemny i daje dolną granicę poniżej zera, start nadal nie stanie się ujemny. Maksimum obejmuje zero oraz dostępność. Ujemne ograniczenie nie wymusza uruchomienia przed początkiem scenariusza i nie tworzy samoistnie błędu arytmetycznego.

W inspektorze relacji dodaj Zależność czasu, wybierz FS, jednostkę i opóźnienie. Następnie uruchom Analizuj czas i sprawdź start, finish oraz drivers. Drivers zawiera identyfikatory relacji, których dolna granica dokładnie wyznaczyła start. Pusta lista może oznaczać, że termin ustaliła dostępność lub zero scenariusza. Nie interpretuje się jej automatycznie jako braku poprzedników.

```
A: release = 120 s, duration = 120 s, finish = 240 s
B: duration = 90 s, lag(FS) = 30 s
start_B = max(0, release_B, 240 + 30) = 270 s
finish_B = 270 + 90 = 360 s
```

Źródła: S39 analyze_schedule; S49 TemporalScheduleTests.test_all_four_precedence_equations_with_explicit_units

TEMAT 121

SS: zależność startów i częściowa wiedza o czasie

Zależność SS wiąże start odbiorcy ze startem źródła. Wzór ma postać start_B ≥ start_A + lag. Odbiorca nie musi czekać na zakończenie A. Jest to właściwy model wtedy, gdy B wymaga rozpoczęcia określonego procesu lub zachowania przesunięcia między startami, a nie gotowego wyniku. Użycie SS powinno wynikać z opisu procesu; bliskie położenie brył lub wspólna grupa nie ustanawiają takiej zależności.

Dla A zaczynającego w sekundzie sto dwudziestej i lag równego trzydzieści sekund najwcześniejszy start B to sto pięćdziesiąt sekund, o ile jego dostępność nie wymusza późniejszej chwili. Gdy B trwa dziewięćdziesiąt sekund, kończy się w sekundzie dwieście czterdziestej. Zmiana czasu trwania A z dwóch do dziesięciu minut nie zmienia ograniczenia SS, jeśli start A pozostaje ten sam.

Ta własność pozwala uzyskać częściowy plan przy nieznanym końcu poprzednika. A może mieć duration: null i release równe dwie sekundy. Jego start będzie znany i równy dwa, ale koniec pozostanie UNKNOWN. Relacja SS z lag cztery sekundy do zadania B o duration trzy sekundy pozwoli obliczyć B od szóstej do dziewiątej sekundy. Cały wynik nadal będzie PARTIAL, ponieważ nie znamy zakończenia A ani pełnego makespan.

W podobnym układzie FS nie wystarczy znajomość startu A: potrzebuje jego końca i otrzyma UNKNOWN_REQUIRED_TIME. Dzięki temu analizator nie zaraża automatycznie całego grafu niewiedzą, jeśli konkretny rodzaj relacji potrzebuje wyłącznie znanego zdarzenia. Ważne jest więc rozpoznanie, czy realny kontrakt dotyczy rozpoczęcia, zakończenia czy obu granic zadania. Dobór skrótu FS lub SS jest decyzją semantyczną.

Przy przeglądzie raportu patrz na start i finish oddzielnie. Zadanie o statusie UNKNOWN może mieć ustalony start, co interfejs pokazuje jako znacznik bez pełnego paska czasu. Nie zastępuj nieznanego końca czasem startu i nie zakładaj zerowego trwania. Jeśli proces wymaga obu ograniczeń, dodaj dwie jawne relacje o właściwych rodzajach, a następnie sprawdź ich przypięcia rewizji i warunki bramek.

Zdarzenie	Wartość	Podstawa
start A	2 s	release
finish A	UNKNOWN	duration = null
start B	6 s	SS + 4 s
finish B	9 s	duration B = 3 s

Źródła: S39 analyze_schedule

TEMAT 122

FF i SF: ograniczenia końca odbiorcy

Zależność FF wiąże koniec odbiorcy z końcem źródła: finish_B ≥ finish_A + lag. Aby wyznaczyć najwcześniejszy start B, analizator odejmuje duration_B. Otrzymuje więc dolną granicę start_B ≥ finish_A + lag − duration_B. Taki układ może pozwalać B rozpocząć się przed zakończeniem A, jeśli obie prace mają jedynie zakończyć się w określonym porządku. Wymaga znanego końca źródła i znanego czasu trwania odbiorcy.

SF łączy start źródła z końcem odbiorcy: finish_B ≥ start_A + lag. Dolna granica startu wynosi start_A + lag − duration_B. Jest to inny kontrakt niż FS; zamiana liter odwraca, które zdarzenia są potrzebne. Sama nazwa etapu nie wystarcza, aby uznać SF za właściwe. W opisie relacji warto zapisać pełne zdanie, na przykład jaki moment rozpoczęcia A ogranicza dopuszczalne zakończenie B.

Użyj tego samego przykładu dla porównania czterech rodzajów: A startuje w sekundzie sto dwudziestej i kończy w dwieście czterdziestej, B trwa dziewięćdziesiąt sekund, a lag wynosi trzydzieści sekund. FS daje start B równy dwieście siedemdziesiąt, SS sto pięćdziesiąt, FF sto osiemdziesiąt, SF sześćdziesiąt. Różnice nie wynikają z optymalizacji zasobów, tylko z czterech odmiennych nierówności.

Nieznane duration_B uniemożliwia wyznaczenie jego startu z FF lub SF. Nawet jeśli znany jest wymagany koniec, analizator nie zgaduje, kiedy rozpocząć zadanie bez wiedzy o jego długości. Natomiast nieznane duration_A nie musi blokować SF, jeśli znany jest start A. Te zasady wyjaśniają, dlaczego sąsiednie zadania mogą mieć odmienne statusy UNKNOWN w jednym raporcie.

Wszystkie starty są ograniczane zerem i release. Jeżeli obliczona dolna granica jest ujemna, nie oznacza pracy w czasie ujemnym; staje się niewiążąca względem zera. Deadline pozostaje osobną oceną końca. Analizator nie przesuwa zadania wcześniej kosztem złamania zależności tylko po to, aby spełnić termin. Przekroczenie deadline jest raportowane, a nie automatycznie naprawiane.

Rodzaj	Dolna granica startu B	Wynik
FS	240 + 30	270 s
SS	120 + 30	150 s
FF	240 + 30 − 90	180 s
SF	120 + 30 − 90	60 s

Źródła: S39 analyze_schedule

TEMAT 123

Dwie równoległe gałęzie i wspólna walidacja

Szablon Czas zawiera cztery zadania: Badanie trwa dwie sekundy, Prototyp pięć, Projekt testów trzy, a Walidacja dwie. Badanie otwiera dwie gałęzie przez FS. Walidacja ma dwa wymagane poprzedniki i może rozpocząć się dopiero po obu. Cztery relacje defines od metaramy opisują poziomy, ale nie należą do grafu czasu. Grupa Równoległe gałęzie pomaga nawigować i również nie tworzy ograniczeń czasowych.

W bazowej analizie Badanie zajmuje przedział od zera do dwóch sekund. Prototyp działa od dwóch do siedmiu, a Projekt testów od dwóch do pięciu. Walidacja startuje w siódmej sekundzie i kończy w dziewiątej. Jej wiążącym ograniczeniem jest koniec Prototypu. Projekt testów kończy się wcześniej, lecz jego relacja pozostaje wymagana: gdyby została zablokowana bramką lub nieznaną rewizją, nie wolno jej po prostu pominąć.

Pole waiting oznacza start minus release. Przy release równym zero Walidacja ma waiting równe siedem sekund. Nie oznacza to, że sama czekała siedem sekund po ukończeniu projektu testów. Jest to całkowite odroczenie startu względem zadeklarowanej dostępności, a nie lokalny przestój po wybranym poprzedniku. Interpretowanie waiting wymaga więc zawsze znajomości release i wszystkich ograniczeń zadania.

Termin Walidacji wynosi dziewięć sekund. Ponieważ koniec jest dokładnie równy terminowi, deadline_missed ma wartość false. Naruszenie zachodzi dopiero dla finish większego od deadline. Wynik COMPLETE i makespan równy dziewięć potwierdzają kompletność obliczenia deklarowanego grafu. Nie potwierdzają, że dwie gałęzie mają niezależne zasoby ani że rzeczywiście wystartowały w środowisku wykonawczym.

To ćwiczenie pozwala sprawdzić działanie równoległości bez edytowania kodu aplikacji. Otwórz szczegóły zadań i odczytaj drivers Walidacji. Następnie porównaj scenariusze z kolejnej strony. Wyraźna zamiana gałęzi wyznaczającej start lepiej wyjaśnia wpływ zależności niż samo przesuwanie brył. Geometria sceny służy czytelności, a terminy wynikają z zadeklarowanych relacji i liczb.

Zadanie	Start	Koniec	Oczekiwanie
Badanie	0	2	0
Prototyp	2	7	2
Projekt testów	2	5	2
Walidacja	7	9	7

TEMAT 124

Nieznane czasy i warunki blokujące plan

Analiza czasu zachowuje różnicę między brakiem zadania, nieznaną deklaracją i zablokowanym ograniczeniem. Artefakt bez timing, który nie jest końcem żadnej relacji czasu, zostaje pominięty. Jeśli jednak uczestniczy w takiej relacji, trafia do zbioru zadań z powodem MISSING_TIMING. Obiekt timing bez duration tworzy zadanie o nieznanym czasie trwania. Brak danych nie jest automatycznie zastępowany zerem.

Nieznane release uniemożliwia wyznaczenie pewnego startu. Nieznane lag pozostawia UNKNOWN_LAG na relacji. Nieznany koniec źródła blokuje obliczenie FS i FF, natomiast SS lub SF mogą skorzystać ze znanego startu. Gdy ograniczenie potrzebuje brakującego czasu, otrzymuje UNKNOWN_REQUIRED_TIME. Raport może więc zawierać obok siebie poprawnie zaplanowane zadania, zadania z samym startem i zadania bez żadnego ustalonego terminu.

Bramki są sprawdzane na zapisanych parametrach dokumentu. FALSE daje relację BLOCKED, a UNKNOWN daje UNKNOWN. Wymagana zależność nie znika z grafu po niespełnieniu warunku. Odbiorca nie dostaje wcześniejszego terminu tylko dlatego, że jego poprzednik został odrzucony. Przyczynę GATE_FALSE lub GATE_UNKNOWN widać w relacji i powodach zadania. To zachowuje logikę „należy poczekać na ten warunek”, zamiast traktować brak zgody jako usunięcie zobowiązania.

Nieaktualne source_revision lub target_revision daje STALE_REVISION i blokadę ograniczenia. Samo wykonanie propagacji parametrów wcześniej w interfejsie nie podmienia zapisanych parametrów używanych przez analizator czasu. Podobnie nadpisanie czasu trwania w scenariuszu nie zmienia parametru artefaktu, do którego odwołuje się bramka. Dzięki temu raport opisuje dokładnie zadeklarowane wejście, lecz autor musi świadomie połączyć przesłanki analiz.

Wynik całości jest PARTIAL, gdy choć jedno zadanie nie ma pełnego statusu SCHEDULED. Wtedy makespan pozostaje null; nie jest maksimum samych znanych końców. Ten brak ma znaczenie: najpóźniejszy znany koniec nie jest dowodem zakończenia całego systemu. Zachowaj szczegóły powodów i uzupełniaj brakującą przesłankę, zamiast przedstawiać częściowy plan jako pełną prognozę terminu.

TEMAT 125

Cykle czasu, terminy i diagnostyka arytmetyczna

Analizator czasu wymaga acyklicznego grafu ograniczeń. Wyszukuje silnie spójne składowe oraz pętle własne i oznacza zadania rzeczywiście należące do cyklu statusem CYCLE z powodem TEMPORAL_CYCLE. Nie próbuje rozwiązywać cyklicznych układów nierówności, nawet jeśli matematycznie mogłyby mieć rozwiązanie. Przykładami są wzajemne SS z zerowym lag albo pętla przekazywania gotowości. Do opisu iteracji lepiej służy jawna granica generacji procesu ewolucyjnego.

Potomkowie cyklu nie są automatycznie oznaczani jako jego członkowie. Otrzymują blokadę wynikającą z poprzednika CYCLE, a dalsza propagacja przyczyn pozwala wskazać granicę problemu. To ważna różnica diagnostyczna: zadanie czekające na cykliczny układ nie musi samo tworzyć cyklu. Aby usunąć przyczynę, trzeba naprawić właściwą składową, a nie kasować wszystkie zadania pojawiające się później w grafie.

Cykle relacji semantycznych bez timing nie wpływają na analizę czasu. Relacje o znaczeniu depends_on, supports lub refines mogą tworzyć obieg wiedzy bez ustanawiania cyklu temporalnego. Również zawieranie nie dodaje krawędzi planu. Sprawdzając problem, filtruj relacje po obecności metadata.timing i analizuj ich rzeczywiste końce oraz rodzaj FS, SS, FF albo SF.

Deadline służy ocenie zakończenia, a nie zmianie kolejności. Finish większy od deadline daje deadline_missed: true, równość nie jest spóźnieniem. Brak znanego końca pozostawia ocenę terminu nieznaną. W interfejsie termin widać jako osobny znacznik, a zadanie po terminie otrzymuje stosowne oznaczenie. To ostrzeżenie o wyniku scenariusza, nie automatyczne przeplanowanie i nie alarm z pomiaru rzeczywistego wykonania.

Bardzo duże liczby są kontrolowane przy konwersji jednostek i sumowaniu. Nie wolno traktować nieskończonego wyniku jako niezwykle odległego terminu. Błąd konwersji może unieważnić wejście, a przepełnienie wyliczenia dać blokadę i diagnostykę TIMING_OVERFLOW. Szczególnie ujemna dolna granica, która nie wiąże nieujemnego startu, nie powinna być mylona z takim przepełnieniem. Raport zachowuje skończone liczby lub jawne null.

Źródła: S39 _cycles; analyze_schedule

TEMAT 126

Scenariusze „co jeśli” i porównywanie wariantów

Scenariusz czasu zmienia wyłącznie dane analizy. W konsoli czasu wybierz Zmień czas scenariusza, wskaż zadanie, wpisz duration oraz jednostkę i przelicz. Nadpisanie jest przechowywane w duration_overrides jako para value i unit pod identyfikatorem artefaktu. Musi wskazywać istniejący artefakt z jawnym timing. Nie nadaje nowego czasu dowolnemu plikowi ani nie tworzy automatycznie brakującego zadania.

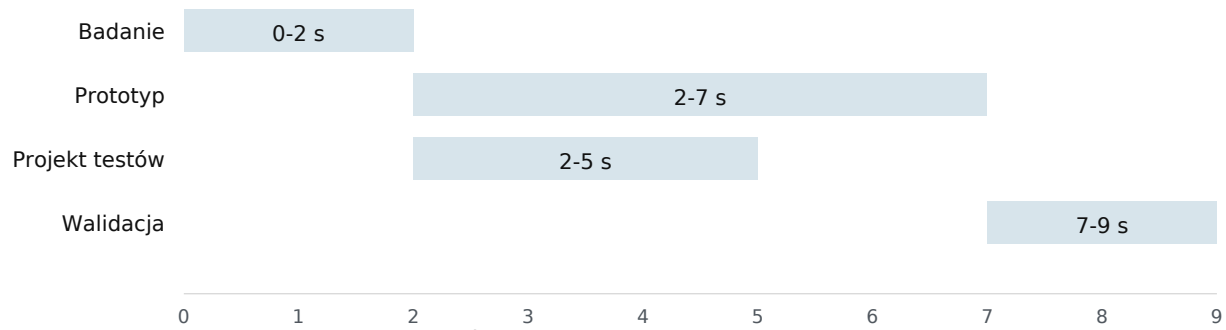
W szablonie Czas bazowy makespan wynosi dziewięć sekund. Nadpisz czas Projektu testów z trzech na siedem sekund. Po Badaniu gałąź ta zakończy się w sekundzie dziewiątej, Walidacja ruszy wtedy i zakończy się w jedenastej. To Projekt testów stanie się wiążącym poprzednikiem, a termin dziewięciu sekund zostanie przekroczony. Wariant pokazuje zmianę struktury ograniczenia, nie tylko dodanie czterech sekund do dowolnego końcowego wyniku.

Usuń poprzednie nadpisanie albo wybierz Czasy bazowe, a następnie skróć Prototyp z pięciu do trzech sekund. Obie gałęzie zakończą się w piątej sekundzie, więc Walidacja potrwa od pięciu do siedmiu. Obie relacje do Walidacji mogą znaleźć się w drivers, ponieważ jednakowe dolne granice wspólnie wyznaczają start. To poprawny remis, a nie losowy wybór jednej gałęzi.

Nadpisanie null służy badaniu utraty wiedzy o czasie trwania. Nie oznacza wyzerowania zadania. Wynik może stać się PARTIAL, zwłaszcza gdy następcy wymagają nieznanego końca przez FS. Zmiana scenariusza nie modyfikuje bazowych metadanych, rewizji artefaktu ani zapisanych parametrów bramek. Eksport wyniku zawiera użyte nadpisania, dzięki czemu odbiorca raportu widzi różnicę między projektem bazowym i wariantem.

Dla porównania zachowaj osobne eksporty bazowy, wolny projekt testów i szybki prototyp. Opisz tylko zmienioną przesłankę, makespan, drivers i ocenę terminu. Nie sumuj korzyści z dwóch wariantów bez ich wspólnego przeliczenia: maksimum gałęzi jest zależnością nieliniową względem wybranej ścieżki. Pakiet asystenta zawiera analizę bazowego projektu; świadomy eksport scenariusza jest właściwym sposobem przekazania badanego wariantu.

```
{"duration_overrides": {
  "temporal:test-design": {"value": 7, "unit": "s"}
}}
```



Harmonogram bazowego demonstratora czasu. Równoległe gałęzie spotykają się przed walidacją.

Źródła: S42 editDurationOverride; analyzeSchedule; scheduleConsole

TEMAT 127

Dokument procesu ewolucyjnego

Proces ewolucyjny jest osobnym dokumentem o schemacie lion.evolution-process/v1. Określa stabilne id, etapy stages, założenia assumptions, wiązania feedback oraz reguły mutations. Może wskazywać architecture_catalog_id, aby zachować odniesienie do katalogu architektury. Nie jest tym samym co dokument sceny lion.polyhedral.studio/v2. Artefakty sceny opisują strukturę, natomiast proces wyraża deklarowane przejścia między rolami w kolejnych generacjach.

Etap wymaga własnego identyfikatora i roli kind. Może deklarować name, duration w sekundach, depends_on, wejścia, wyjścia, bramkę oraz architecture_refs. Czas jest nieujemną liczbą albo null. Brak czasu pozostaje UNKNOWN; nie blokuje automatycznie logicznego statusu etapu, ale ogranicza wiedzę o terminach. W przeciwieństwie do analizy czasu workspace ten silnik nie używa czterech rodzajów FS, SS, FF i SF.

Założenia są nazwanymi wartościami z jawnym type i value. Pole by_generation pozwala wskazać wyjątki dla konkretnych numerów generacji, zapisanych jako tekstowe klucze od jeden do pięćdziesięciu. Na przykład verification_passes może być domyślnie true, a w pierwszej generacji false. Taki zapis modeluje scenariusz naprawy po niepowodzeniu; nie pobiera rzeczywistych wyników testów z repozytorium.

Typy wbudowane obejmują boolean, string, number, integer, object, array i any. Możliwe są również nazwane typy kontraktowe, których wartości mają postać obiektów. Silnik sprawdza zgodność źródeł portów oraz ich konkretnych wartości. Wartość null ma stan UNKNOWN i nie jest wymyślana na potrzeby zakończenia procesu. Brak wymaganego wejścia może więc zatrzymać sensowną interpretację etapu mimo poprawności formatu dokumentu.

Proces mieści od jednego do dwustu etapów, a łączna liczba portów i wiązań feedback nie może przekroczyć dwóch tysięcy. Cały proces i nadpisania scenariusza podlegają wspólnemu limitowi dwustu pięćdziesięciu sześciu KiB JSON w UTF-8. Zanim uruchomisz generacje, sprawdź identyfikatory, odwołania i typy. Błąd walidacji zwraca INVALID bez częściowego udawania wykonanego procesu.



Sprzężenie zwrotne przechodzi przez granicę kolejnej generacji

Kolejność dziewięciu etapów procesu domyślnego. Diagram przedstawia symulację procesu.

Źródła: S31 validate_process

TEMAT 128

Dziewięć etapów domyślnej ewolucji

Domyślny proces przedstawia dziewięć kolejnych ról: intencję, badanie, hipotezę, kontrakt, plan, opis kandydata materializacji, weryfikację scenariusza, ocenę i wybór mutacji oraz następną generację. Każdy etap trwa deklarowaną jedną sekundę. Jest to edytowalny projekt rozumowania o zmianie architektury, a nie sztywny wymóg każdej aplikacji LION. Użytkownik może stworzyć własny graf etapów i własne typowane kontrakty.

Intencja pobiera tekst celu oraz opcjonalny stan poprzedniej generacji. Badanie odwołuje się do research_available, hipoteza do hypothesis_viable, a weryfikacja do verification_passes. Pozostałe zwykłe etapy przekazują deskryptor wyniku poprzednika. Wartości tych trzech założeń są wejściem scenariusza. Ich ustawienie na true nie jest dowodem przeprowadzenia badania, obrony hipotezy lub pomyślnego uruchomienia testów.

Ocena otrzymuje szczególny rodzaj danych: obserwacje symulowanych statusów hipotezy i weryfikacji. Dzięki temu potrafi ocenić niepowodzenie wcześniejszego etapu zamiast wymagać wyłącznie jego poprawnego produktu. Może następnie wybrać repair, refine albo investigate. Etap następnej generacji przenosi wynik decyzji do jawnego feedback. Intencja kolejnej generacji ma dostęp do tego stanu, co tworzy kontrolowaną pętlę bez cyklu wewnątrz pojedynczego pokolenia.

Przy wszystkich domyślnych założeniach równych true trzy generacje dają dwadzieścia siedem zdarzeń i tyle samo opisów kandydatów. Makespan wynosi dwadzieścia siedem zadeklarowanych sekund, a w każdej generacji wybrana zostaje mutacja refine. Liczba rzeczywistych wykonań, uruchomień testów i zmaterializowanych plików wynosi zero. Te metryki są częścią wyniku i powinny pozostać widoczne podczas opisywania eksperymentu.

Aby sprawdzić falsyfikację, zmień jedną przesłankę w pierwszej generacji i porównaj statusy wszystkich następników. Nie zastępuj kryteriów analizy samą liczbą generacji. Duża liczba pokoleń z niezmiennymi założeniami powtarza reguły scenariusza; nie zwiększa automatycznie wiarygodności opisu. Wartość eksperymentu wynika z sensownych warunków, jawnych kontraktów oraz możliwości wyjaśnienia każdego przejścia.

Fragment procesu	Wejście kontrolne	Znaczenie
Badanie	research_available	Dostępność przesłanek
Hipoteza	hypothesis_viable	Warunek przyjęcia hipotezy
Weryfikacja	verification_passes	Założony wynik scenariusza
Ocena	obserwacje statusów	Wybór mutacji

Źródła: S31 default_process

TEMAT 129

Źródła portów procesu i kontrakty wartości

Port procesu ma name, type i jawne źródło source; wejście może dodatkowo deklarować required, domyślnie true. Źródła assumption, stage, observation, feedback, literal oraz input mają różne znaczenia. Assumption odwołuje się do nazwanej przesłanki scenariusza. Stage pobiera nazwane wyjście wcześniejszego etapu. Observation udostępnia opis jego symulowanego statusu. Feedback sięga do stanu poprzedniej generacji. Literal przenosi wpisaną wartość. Input służy wyłącznie do wyjść przekazujących wejście bieżącego etapu.

Każda rozpoznana wartość ma typ, stan KNOWN albo UNKNOWN, value i ref. Null pozostaje UNKNOWN. Jeśli źródłowy etap nie ma statusu COMPLETED, odczyt jego zwykłego wyjścia staje się nieznany z odpowiednią przyczyną UPSTREAM. Nie dotyczy to obserwacji statusu: można znać fakt symulowanego niepowodzenia i wykorzystać go do wyboru działania naprawczego. To mechanizm domyślnej oceny po odrzuconej hipotezie lub niespełnionej weryfikacji.

Zgodność kontraktu dopuszcza identyczne nazwy typów, any po którejkolwiek stronie oraz przejście integer do number. Wartości nadal muszą pasować do konkretnego kontraktu docelowego. Nazwane typy spoza zbioru wbudowanego reprezentuje obiekt JSON. Brak zadeklarowanego założenia może pozostać poprawnym strukturalnie źródłem UNKNOWN, natomiast odwołanie do nieistniejącego wyjścia etapu lub nieistniejącego feedback jest błędem walidacji procesu.

Wyjście etapu bez source ma szczególną regułę. Dla object, any i nazwanego typu kontraktowego silnik potrafi utworzyć obiekt opisujący symulowanego kandydata. Dla prymitywnego boolean, string, number, integer lub array nie wymyśla wartości i zwraca UNKNOWN z UNDECLARED_OUTPUT_VALUE. Jeżeli potrzebny jest rzeczywisty literał lub przekazanie wejścia, trzeba wskazać jawne source. Ukończony etap może więc mieć nieznane wyjście i skutkować częściową generacją.

Architecture_refs łączą etapy z identyfikatorami katalogu, zachowując ślad do pliku i miejsca w Git. Domyślne mapowanie wybiera kandydatów po nazwie, ścieżce i roli semantycznej, maksymalnie dwanaście na etap. Nie analizuje ich kodu wykonawczego i nie dowodzi, że wskazany plik realizuje kontrakt etapu. Każde mapowanie wymaga przeglądu w kontekście właściwego snapshotu katalogu.

TEMAT 130

DAG w generacji i feedback między generacjami

Wewnątrz pojedynczej generacji etapy tworzą DAG, czyli skierowany graf bez cykli. Zależności wynikają z depends_on, odczytów stage i observation w portach oraz odwołań do statusów etapów w bramkach. Dla etapu evaluate dochodzą również odwołania do statusów użyte w regułach mutacji. To zabezpiecza przyczynowość: decyzja nie powinna czytać statusu etapu, który nie został jeszcze rozpatrzony.

Silnik wyznacza stabilną kolejność topologiczną. Gdy kilka etapów jest gotowych jednocześnie, rozstrzyga według identyfikatora. Nie jest to priorytet biznesowy ani przydział zasobów, lecz sposób zapewnienia powtarzalnego wyniku. Deklarowany start etapu jest maksimum końców poprzedników i początku generacji. Niezależne gałęzie mogą mieć pokrywające się przedziały czasu modelowego mimo sekwencyjnego obliczania deskryptorów w jednej funkcji Pythona.

Depends_on ustanawia kolejność, ale samo nie wymaga sukcesu poprzednika. Etap obserwacyjny może następować po nieudanej weryfikacji i nadal zostać COMPLETED, jeśli ma potrzebne dane. Wymóg poprawnego produktu modeluje się przez wymagane wejście ze źródłem stage. Rozdzielenie kolejności od kontraktu sukcesu jest potrzebne do projektowania obsługi błędów: ścieżka analizy porażki nie może znikać tylko dlatego, że poprzednik nie powiódł się.

Pętla istnieje wyłącznie przez feedback między generacjami. Każde wiązanie podaje name, from_stage, output, type i opcjonalny stan initial. W pierwszej generacji używane jest initial, w kolejnych wynik wybranego wyjścia poprzedniego pokolenia. W ten sposób powrót do intencji albo planu ma jawną granicę czasową i kontrakt. Nie należy tworzyć krawędzi od ostatniego etapu do pierwszego w tym samym DAG.

Jeśli końce etapów są nieznane, koniec generacji i początek następnej mogą także pozostać UNKNOWN. Logiczne statusy nadal są analizowane, lecz nie ma podstaw do pełnego makespan. Poprawna deklaracja pętli nie gwarantuje poprawności stanu przekazywanego przez feedback. Sprawdź typ, źródło oraz wymagalność wejścia kolejnej generacji i porównaj ślad, zanim uznasz model iteracyjny za kompletny.

```
"feedback": [{
  "name":"generation_state", "from_stage":"next_generation",
  "output":"result", "type":"lion.generation/v1",
  "initial":{"generation":0,"status":"INITIAL_DESIGN"}
}]
```

Źródła: S31 _order; simulate_process

TEMAT 131

Bramki etapów i znaczenie statusów

Bramki procesu ewolucyjnego mają własną składnię. Obsługują literal, assumption, input, status, not, and, or oraz porównania eq, ne, lt, lte, gt i gte. Assumption i input oczekują znanej wartości logicznej. Status porównuje wynik wskazanego etapu z konkretnym statusem. Złożenia and i or stosują logikę trójwartościową, ale wymagają od jednego do trzydziestu dwóch argumentów. Nie używa się tu operatorów all, any ani in z bramek przepływu workspace.

Przed bramką sprawdzane są wymagane wejścia etapu. Brak znanego wejścia prowadzi do UNKNOWN lub BLOCKED, zależnie od przyczyny po stronie poprzednika. Dopiero gdy wymagane dane są dostępne, bramka rozstrzyga dalszy status. False w hipotezie daje FALSIFIED, w verify daje FAILED, w evaluate REJECTED, a w innych rolach BLOCKED. Null daje UNKNOWN. Wynik true pozwala kontynuować, lecz etap evaluate ma jeszcze reguły wyboru mutacji.

W porównaniach true i jeden nie są semantycznie tym samym. Wartości różnych niezgodnych typów prowadzą do UNKNOWN, a porządkowanie wymaga liczb. Jest to prostszy kontrakt niż bramki workspace: nie ma tu rekordów jednostek ani porządkowania tekstu przez lt. Przy przenoszeniu warunku między formatami należy przeprojektować operandy, zamiast podmienić jedynie nazwę operatora łączącego.

Status COMPLETED oznacza zakończenie analizy etapu w scenariuszu. Nie dowodzi wykonania narzędzia, uruchomienia testu czy zapisania kodu. FAILED jest modelem niespełnionego warunku weryfikacji, a FALSIFIED modelem odrzuconej hipotezy. UNKNOWN zachowuje niewiedzę. BLOCKED oznacza niemożność przejścia na zadeklarowanych wejściach lub warunkach. REJECTED opisuje niedopuszczoną ocenę lub brak wybranej mutacji.

Wynik całego procesu używa COMPLETE, PARTIAL lub INVALID, odrębnie od statusów etapów. COMPLETE wymaga także znanych zakończeń i znanych wyjść. Etap COMPLETED z brakującą wartością prymitywnego wyjścia nadal może uczynić generację PARTIAL. Dlatego w raporcie czytaj jednocześnie status, timing_status, reasons oraz stan każdego wyjścia; pojedynczy zielony etap nie zastępuje kontroli kompletności całego scenariusza.

Fałszywa bramka w roli	Status etapu
hypothesis	FALSIFIED
verify	FAILED
evaluate	REJECTED
Pozostałe role	BLOCKED

Źródła: S31 _gate; simulate_process

TEMAT 132

Mutacje, priorytety i falsyfikacja

Mutacja jest deklarowaną możliwością dalszej zmiany projektu. Rekord wymaga id, liczbowego priority i bramki when; name ułatwia interpretację. Etap evaluate ocenia wszystkie reguły i porządkuje je malejąco według priorytetu, a przy remisie rosnąco według identyfikatora. Wybór jest deterministyczny. Nie ma losowania, uczenia strategii ani automatycznego wykonywania mutacji na kodzie repozytorium.

Domyślny proces ma repair z priorytetem trzydzieści, refine z dwadzieścia i investigate z dziesięć. Repair odpowiada na FALSIFIED hipotezy lub FAILED weryfikacji. Refine wymaga COMPLETED weryfikacji. Investigate ma zawsze prawdziwą bramkę jako wariant uzupełnienia przesłanek. Nazwy opisują intencję następnej generacji. Wybranie repair nie zapisuje patcha i nie uruchamia naprawy; informacja trafia do deskryptora decyzji oraz feedback.

Nieznana bramka mutacji o wyższym priorytecie może zatrzymać wybór niższej, znanej jako dopuszczalna. Wynik decyzji jest wtedy UNKNOWN z UNRESOLVED_HIGHER_PRIORITY_MUTATION. To ważna zasada zachowania niewiedzy: nie wolno wybrać prostszego działania tylko dlatego, że przesłanki bardziej istotnego wariantu nie zostały jeszcze rozstrzygnięte. Jeśli żadna reguła nie jest dopuszczalna, wybór ma status REJECTED i brak selected_mutation.

Przykład falsyfikacji: ustaw hypothesis_viable na false. Hipoteza otrzyma FALSIFIED, a wymagające jej zwykłego produktu etapy Kontrakt, Plan i Opis kandydata zostaną zablokowane. Ocena może jednak odczytać symulowaną obserwację statusu hipotezy i wybrać repair. W ten sposób proces zachowuje wiedzę o porażce bez wytwarzania fikcyjnego poprawnego produktu. Wynik ogólny pozostanie PARTIAL, bo wcześniejsze etapy nie zakończyły się poprawnie.

Podczas projektowania mutacji rozpisz, co ma zostać odrzucone, co naprawione i co wymaga dodatkowego badania. Nadaj priorytety zgodnie z rzeczywistymi regułami, a nie kolejnością w pliku. Przetestuj remis identyfikatorów oraz UNKNOWN w regule nadrzędnej. Pełny wynik candidates zawiera ocenę każdej bramki, więc decyzję można wyjaśnić na podstawie zadeklarowanych przesłanek, zamiast uzasadniać ją po fakcie.

TEMAT 133

Deskryptory kandydatów i tożsamość wyniku

Każdy rozpatrzony etap otrzymuje deskryptor kandydata o schemacie lion.simulated-candidate/v1. Zawiera identyfikator procesu, hash procesu i scenariusza, numer generacji, etap, rolę, status, odwołania i hashe wejść, architecture_refs oraz decyzję. Pole evidence_status ma wartość SIMULATED, a effect SIMULATION_ONLY. Deskryptor jest opisem wyniku rozumowania w modelu, nie plikiem kodu powstałym przez materializację.

Silnik oblicza SHA-256 kanonicznej reprezentacji deskryptora i tworzy identyfikator simulated:sha256: zakończony tym skrótem. W events zapisywane są candidate_ref i candidate_sha256, a w artifacts rekord zawierający pełny descriptor. Umożliwia to sprawdzenie, który dokładnie opis przypisano danemu etapowi. Zmiana przesłanek, wejść albo mapowania architektury może zmienić tożsamość kandydata, nawet jeśli jego nazwa wyświetlana pozostaje podobna.

Process_sha256 identyfikuje dokument procesu, a scenario_sha256 deklaracje użyte po zastosowaniu nadpisań. Nie są hashami bieżącego drzewa Git ani całej instalacji. Architecture_refs mogą prowadzić do źródeł katalogu, ale sam hash deskryptora nie dowodzi aktualności tych plików. Jeśli potrzebujesz powiązania z kodem, zachowaj również właściwy katalog z jego źródłami, commitem, ścieżką i zakresem linii.

Opis kandydata powstaje również dla etapu nieudanego lub zablokowanego. Dzięki temu ślad zawiera zapis tego, co rozpatrzono i dlaczego nie wytworzono znanego produktu. Sam fakt obecności rekordu w artifacts nie świadczy więc o sukcesie. Należy odczytać descriptor.status, stan wyjść zdarzenia i przyczyny. Liczba kandydatów oznacza liczbę opisów etapów, a nie liczbę gotowych modułów aplikacji.

Deterministyczność oznacza, że te same dokumenty i parametry dają ten sam wynik logiczny oraz identyczne skróty. Nie jest to podpis autora, znacznik czasu kwalifikowanego ani dowód wykonania zewnętrznego. W audycie używaj skrótów do wiązania i kontroli integralności danych. Weryfikację rzeczywistego efektu, pochodzenia dowodu i zakresu uprawnień dokumentuj osobno, bez podnoszenia symulowanego opisu do rangi obserwacji runtime.

Źródła: S31 _hash; simulate_process

TEMAT 134

Ślad, limity i archiwizacja symulacji generacji

Wynik ewolucji ma schemat lion.evolution-simulation/v1 i zawiera generations, events, artifacts, diagnostics, assumptions oraz metrics. Generacje podają zakres czasu i wybraną mutację. Zdarzenia wskazują etap, wejścia, wyjścia, bramkę, poprzedników, czasy i powody. Metryki obejmują liczbę kroków i opisów kandydatów, liczniki statusów oraz makespan. Pola actual_executions, actual_test_runs i materialized_files mają zero, zgodnie z zakresem silnika.

Można zażądać od jednej do pięćdziesięciu generacji. Proces ma najwyżej dwieście etapów, a iloczyn liczby etapów i generacji nie może przekraczać dwóch tysięcy zdarzeń. Pojedyncza deklarowana wartość ma limit osiem tysięcy sto dziewięćdziesiąt dwa bajty JSON. Limity sprawdzane przed obliczeniem zapobiegają przedstawianiu nieograniczonego eksperymentu jako zwykłego wywołania interfejsu. Przekroczenie limitu wejścia daje INVALID z diagnostyką.

Pełny ślad jest ograniczony do ośmiu MiB. Osiągnięcie granicy zatrzymuje dalsze etapy i generacje z wynikiem PARTIAL oraz EVOLUTION_TRACE_LIMIT. Wartości większe niż tysiąc dwadzieścia cztery bajty mogą być pominięte w wyświetlanym śladzie i zastąpione informacją value_omitted, rozmiarem i hashem. Nie zmienia to wewnętrznego rozstrzygania kontraktów. Nie należy odczytywać takiego null jako utraty znanej wartości; trzeba sprawdzić znacznik pominięcia.

Wywołanie serwera POST /api/evolution/simulate archiwizuje wynik, jeżeli nie jest INVALID. Magazyn tworzy identyfikator simulation: z hashem dokumentu, zapisuje skompresowane dane w SQLite i zwraca archive_ref. Taki sam wynik nie wymaga drugiej identycznej kopii. Pobranie z archiwum ponownie sprawdza digest i sygnalizuje naruszenie integralności. Lista wyników udostępnia najwyżej sto ostatnich rekordów, opcjonalnie ograniczonych do katalogu.

Archiwizacja nie jest wykonaniem procesu, lecz zapisem raportu analizy. Żeby przenieść eksperyment, użyj eksportu pełnego śladu i definicji procesu. Zachowaj też nadpisania scenariusza oraz katalog, jeśli odwołania architektury są istotne. Sam archive_ref ma sens w konkretnym lokalnym magazynie; bez dokumentu wynikowego i źródeł nie stanowi samowystarczalnej dokumentacji eksperymentu.

Granica	Wartość
Generacje / etapy	50 / 200
Zdarzenia	2000
Proces + scenariusz	256 KiB UTF-8
Pełny ślad	8 MiB

Źródła: S31 simulate_process; _trace_typed; S29 CatalogStore.save_simulation; list_simulations; get_simulation

TEMAT 135

Ćwiczenie: naprawa w pierwszej generacji

Otwórz Architektura / Git i przejdź do części procesu generacji. Wczytaj domyślną definicję dla wybranego katalogu. Najpierw użyj Sprawdź i pokaż etapy, aby obejrzeć graf oraz kontrakty, a następnie wyeksportuj definicję. Zachowanie wersji bazowej ułatwi porównanie. Liczbę generacji ustaw na trzy i uruchom Symuluj pełny proces generacji bez zmiany założeń. Spodziewany wynik to COMPLETE, dwadzieścia siedem zdarzeń i refine w każdym pokoleniu.

Teraz w założeniu verification_passes pozostaw value: true, a dodaj by_generation z kluczem pierwszej generacji i wartością false. W ten sposób tylko pierwsze pokolenie ma niespełnioną przesłankę weryfikacji. Nie zmieniasz testów w repozytorium i nie opisujesz rzeczywistej awarii. Przelicz trzy generacje, a następnie odszukaj zdarzenia verify, evaluate, next_generation i intention w śladzie.

W pierwszej generacji verify otrzyma FAILED. Evaluate może nadal być COMPLETED, ponieważ odczytuje obserwację statusu weryfikacji zamiast żądać jej poprawnego produktu. Decyzja wybierze repair. W drugiej generacji intention otrzyma previous_generation z przekazaną decyzją repair, lecz sama weryfikacja korzysta już z domyślnego true i zakończy się COMPLETED. Ocena wybierze refine. Trzecia generacja również wybierze refine. Wynik całego eksperymentu pozostanie PARTIAL z powodu pierwszego niepowodzenia.

Zwróć uwagę, że czasy etapów są nadal deklaracjami scenariusza. Nieudana weryfikacja również ma przypisany modelowy czas, więc przy znanych czasach suma trzech pokoleń może pozostać dwadzieścia siedem sekund. PARTIAL nie oznacza automatycznie nieznanego makespan; może wynikać z logicznego niepowodzenia lub nieznanego produktu. Odczytaj przyczyny, liczniki statusów i znacznik kompletności każdego pokolenia.

Na końcu wyeksportuj pełny ślad, zapisz definicję procesu w katalogu, jeżeli wariant ma być ponownie używany, i nazwij scenariusz jednoznacznie. Porównaj process_sha256 i scenario_sha256 z bazą. Jeśli nadpisanie wprowadzono poza zapisaną definicją, hash procesu może pozostać ten sam, a zmieni się hash scenariusza. Taka para pozwala wyjaśnić, co dokładnie różni dwa eksperymenty.

```
"verification_passes": {
  "type":"boolean", "value":true,
  "by_generation":{"1":false}
}
```

TEMAT 136

Raportowanie procesu bez utraty znaczenia

Raport architektoniczny powinien rozdzielać deklarację, wynik obliczenia i obserwację rzeczywistego działania. W tej części Studio wynik propagacji mówi o wartościach w modelu, harmonogram o najwcześniejszych terminach zadeklarowanego DAG, a ewolucja o statusach i wyborach mutacji. Żaden z tych raportów samodzielnie nie potwierdza działania importowanego kodu, brokera ani całej federacji. To granica interpretacji, którą warto zapisać przy każdym eksperymencie.

Zachowaj tożsamość wejścia: project_id i rewizję projektu dla analiz workspace, id i hash procesu dla generacji oraz przypięty katalog architektury dla odwołań źródłowych. Dołącz użyte parametry, jednostki i nadpisania scenariusza. Wskaż zakres: pełny mały projekt, jedno wnętrze, projekt ręczny albo widok fragmentu dużego katalogu. Bez tego odbiorca może przypisać lokalny wynik całej aplikacji, mimo że stronicowanie ograniczało dokument wejściowy.

Oddziel błędy formatu od blokad kontraktu i nieznanych przesłanek. INVALID wymaga naprawy dokumentu. STALE_REVISION wymaga przeglądu wersji końców. UNKNOWN wymaga rozstrzygnięcia brakującej wartości lub niezgodnego kontekstu. CONFLICT wymaga jawnego rozwiązania konkurencyjnych zapisów. PARTIAL w czasie lub ewolucji wymaga odczytania powodów, ponieważ może oznaczać zarówno nieznany termin, jak i niepowodzenie logiczne przy znanych czasach.

W opisie wniosków używaj sformułowań odpowiadających faktycznej operacji. „Model osiągnął stan stały po trzech krokach” jest precyzyjniejsze od „system działa”. „Wybrano opis repair po założonej negatywnej weryfikacji” zachowuje różnicę między decyzją projektową a naprawą kodu. „Plan mieści się w dziewięciu deklarowanych sekundach” nie usuwa ograniczeń zasobowych, których analizator nie modeluje.

Przed zamknięciem eksperymentu zachowaj eksport definicji oraz wyniku, sprawdź obecność diagnostyki limitów i upewnij się, że odwołania źródłowe prowadzą do właściwego snapshotu. Zapisz jedną interpretację wspartą konkretnymi polami śladu i wskaż przesłanki wymagające dalszego badania. W ten sposób bogaty model mozaikowy staje się narzędziem odtwarzalnej decyzji, a jego geometria, semantyka i generacje pozostają skoordynowanymi perspektywami zamiast wzajemnie zastępujących się dowodów.

TEMAT 137

Backend jako lokalna usługa projektowa

Backend Studio stanowi lokalny adapter HTTP pomiędzy interfejsem przeglądarkowym a modelem, analizatorami i magazynami SQLite. Moduł server.py nie jest brokerem wykonującym zadania federacji. Przyjmuje dokumenty projektu, normalizuje i sprawdza ich strukturę, oblicza wyniki analiz oraz utrwala wybrane operacje. Takie rozdzielenie pozwala korzystać z tego samego modelu przez interfejs graficzny albo przez kontrolowany skrypt lokalny.

Głównym obiektem procesu jest StudioHTTPServer, dziedziczący po standardowym ThreadingHTTPServer. Instancja przechowuje Store, losowy token sesji, profil geometrii oraz leniwie inicjowany CatalogStore. Handler rozpoznaje ścieżkę żądania, sprawdza warunki wejściowe i wywołuje odpowiednią funkcję domenową. Dane domenowe pozostają zwykłymi obiektami JSON, dzięki czemu eksport nie wymaga serwera aplikacyjnego ani działającego silnika renderowania.

Odpowiedzialności są celowo rozdzielone. model.py opisuje projekt i bramki. temporal.py analizuje zależności czasowe. architecture.py operuje pełnym katalogiem. navigation.py buduje ograniczony widok wnętrza, a view_edits.py zbiera jego zmiany. repositories.py czyta Git; evolution.py symuluje proces generacyjny. store.py i catalog_store.py odpowiadają za trwałość, ale przechowują różne klasy dokumentów i mają niezależne numery rewizji.

Praktyczna konsekwencja jest istotna: samo wyświetlenie wnętrza nie zapisuje nowej rewizji katalogu, a samo obliczenie bramek nie uruchamia kodu artefaktów. Dopiero jawna operacja zapisu projektu albo scalenia nakładki zmienia właściwy magazyn. Przy analizowaniu problemu najpierw ustal, czy dotyczy on dokumentu projektu, katalogu architektury, analizy ulotnej czy archiwalnego wyniku symulacji. Pozwala to uniknąć naprawiania poprawnego magazynu z powodu błędu prezentacji.

Wersja usługi zgłaszana przez endpoint zdrowia wynosi 3.1. Jest to identyfikator aplikacji, niezależny od schematu dokumentu projektu v2 i wersji ramy. Te trzy oznaczenia należy zapisywać oddzielnie w zgłoszeniach oraz w protokołach testów.

Źródła: S37 StudioHTTPServer; Handler; VERSION

TEMAT 138

Uruchamianie i wybór przestrzeni roboczej

Punkt wejścia main przyjmuje trzy opcje operacyjne: port, katalog workspace oraz wyłączenie automatycznego otwarcia przeglądarki. Domyślny port wynosi 18765. Serwer zawsze wiąże się z adresem 127.0.0.1; nie ma argumentu umożliwiającego wystawienie go na wszystkie interfejsy. Gdy port jest zajęty, proces kończy uruchomienie z komunikatem i sugeruje wybór innego numeru.

Przy pracy ze źródeł katalog roboczy domyślnie znajduje się w podkatalogu workspace obok run_studio.py. Przy uruchomieniu pakietu bez tego pliku domyślna lokalizacja zmienia się na LION-Polyhedral-Studio w katalogu domowym użytkownika. Dlatego dwa uruchomienia z różnych instalacji mogą pokazywać różne projekty, mimo identycznego adresu aplikacji. Decydująca jest ścieżka bazy wypisana w terminalu podczas startu.

Konstruktor tworzy katalog roboczy, otwiera magazyn projektów i, gdy lista jest pusta, zapisuje startowy projekt federacji. Następnie sprawdza integralność każdego istniejącego projektu. Niespójna historia może zatem zatrzymać start; nie jest automatycznie kasowana ani zastępowana pustą bazą. Duży katalog architektury zostaje zainicjowany dopiero przy pierwszym użyciu części architektonicznej, co ogranicza koszt zwykłego uruchomienia edytora.

Przeglądarka jest otwierana po krótkim opóźnieniu przez standardowy moduł webbrowser. W PyCharm można wyłączyć ten krok i otworzyć adres ręcznie. Zamknięcie karty przeglądarki nie kończy serwera, natomiast Ctrl+C w terminalu zamyka proces i nasłuch. Po restarcie token sesji będzie inny, dlatego stara karta wymaga odświeżenia przed kolejnym żądaniem zapisującym.

Na Windows przekierowane strumienie mogą mieć kodowanie nieobsługujące polskich znaków. Funkcja main ustawia obsługę błędów kodowania na backslashreplace, aby sam komunikat diagnostyczny nie blokował startu. Nie zmienia to wymogu zapisywania dokumentów projektu jako poprawnego JSON w UTF-8. Przy zgłoszeniu problemu podaj komendę startową i ścieżkę workspace.

```
python .\run_studio.py --port 18765 --no-browser
# Oddzielna, jawnie wybrana przestrzeń robocza:
python .\run_studio.py --workspace C:\LION\StudioWorkspace
```

Źródła: S37 main; StudioHTTPServer.__init__

TEMAT 139

Obsługa HTTP i zasobów interfejsu

Interfejs aplikacji jest dostarczany jako statyczne pliki z katalogu web. Żądanie do głównego adresu wybiera index.html, a pozostałe ścieżki wskazują pliki JavaScript, CSS i inne zasoby tego katalogu. Nie ma etapu kompilacji wykonywanego przez serwer podczas każdego odświeżenia. Zmiana źródłowego pliku interfejsu wymaga jednak używania właściwej instalacji i ponownego wczytania strony.

Handler dekoduje ścieżkę URL, usuwa początkowy ukośnik i rozwiązuje ścieżkę względem WEB. Następnie sprawdza, czy końcowa lokalizacja nadal leży wewnątrz katalogu zasobów i czy jest plikiem. Próba pobrania pliku przez przejście do katalogu nadrzędnego kończy się odpowiedzią 404. Ten mechanizm nie udostępnia całego drzewa repozytorium ani dowolnej zawartości workspace przez zwykłe ścieżki HTTP.

Odpowiedzi otrzymują Content-Length, Cache-Control: no-store, X-Content-Type-Options: nosniff oraz Referrer-Policy: no-referrer. Dla skryptów rozszerzenie .js jest jawnie mapowane na text/javascript. Kod HTML, CSS i JavaScript otrzymuje również oznaczenie UTF-8. Taki kontrakt ułatwia diagnozę, ponieważ przeglądarka nie powinna interpretować pliku skryptu jako arbitralnego tekstu na podstawie własnych heurystyk.

Content-Security-Policy ogranicza domyślne źródła, skrypty i połączenia do własnego origin. Obrazy mogą pochodzić także z data:, style dopuszczają style inline, a frame-ancestors blokuje osadzanie Studio w obcej ramce. Jest to konfiguracja lokalnego interfejsu, nie opis kontroli autoryzacyjnej całej federacji. W szczególności nie należy na jej podstawie przypisywać pochodzenia lub prawdziwości metadanym importowanego artefaktu.

ThreadingHTTPServer obsługuje żądania w osobnych wątkach. To nie oznacza równoległego zapisywania tego samego projektu bez konfliktów: spójność zapisu wynika z transakcji SQLite i expected_revision. Do oceny opóźnień rozróżnij pobieranie statycznych zasobów, koszt walidacji dokumentu, dekompresję katalogu oraz czas działania zewnętrznego polecenia Git. Każda z tych czynności ma inne źródło problemów.

Źródła: S37 Handler.do_GET; Handler.send_data; StudioHTTPServer

TEMAT 140

Granica loopback oraz kontrola Host i Origin

Warstwa sieciowa ma dwa uzupełniające się ograniczenia. Konstruktor odrzuca adres inny niż 127.0.0.1, a każde żądanie HTTP przechodzi przez allowed_request. Dopuszczone wartości nagłówka Host to 127.0.0.1 z rzeczywistym portem oraz localhost z tym samym portem. Własna nazwa DNS, adres sieci lokalnej lub niezgodny numer portu spowodują odmowę nawet wtedy, gdy połączenie dotrze do procesu.

Nagłówek Origin jest sprawdzany, jeżeli występuje. Akceptowane są wyłącznie lokalne origin HTTP odpowiadające tym dwóm wartościom Host. Żądanie ze strony otwartej na innej domenie otrzyma 403. Brak Origin nie jest sam w sobie błędem, co umożliwia pracę lokalnych klientów skryptowych. W konsekwencji Origin należy traktować jako kontrolę pochodzenia żądania przeglądarkowego, a nie jako uwierzytelnienie osoby.

Istotne jest rozróżnienie dwóch nazw. Serwer nasłuchuje na 127.0.0.1, ale dopuszcza nagłówek localhost. Nie oznacza to obsługi dowolnej trasy pośredniczącej ani gotowości do publikacji przez reverse proxy. Konfiguracja nie zawiera zaufanej listy pośredników, interpretacji X-Forwarded-For ani modelu kont użytkowników. Podstawowy scenariusz to przeglądarka i proces Studio uruchomione na jednym komputerze.

W razie błędu 403 sprawdź pełny adres w pasku przeglądarki. Po zmianie portu wszystkie wywołania API muszą używać nowego portu; sam skrypt nie powinien ręcznie podszywać się pod nagłówki z poprzedniej instancji. Jeżeli pracujesz z PowerShell, wywołuj lokalny adres wprost. Nie omijaj kontroli przez modyfikowanie backendu, gdy rzeczywistą przyczyną jest stara karta lub inna instancja programu.

Test HTTP sprawdza odrzucenie obcego Host i obcego Origin, a osobny test odrzuca uruchomienie na 0.0.0.0. Te kontrole potwierdzają zachowanie implementacji na wskazanej granicy. Nie dowodzą ochrony przed dowolnym procesem działającym już z uprawnieniami lokalnego użytkownika ani przed modyfikacją plików przez taką osobę.

Warunek	Dopuszczenie
Host: 127.0.0.1:18765	Tak, dla domyślnego portu
Host: localhost:18765	Tak, dla domyślnego portu
Origin: https://inna.example	403
Brak Origin	Kontrola Host nadal obowiązuje

Źródła: S37 Handler.allowed_request; StudioHTTPServer.__init__

TEMAT 141

Token sesji i uprawnienia żądań zapisujących

Każda instancja serwera generuje token przez secrets.token_urlsafe(32). Token żyje w pamięci procesu i nie jest stałym hasłem projektu. Endpoint GET /api/state przekazuje go lokalnemu interfejsowi razem z aktualnym projektem i profilem geometrii. Przy wysyłaniu dowolnego POST klient musi umieścić tę wartość w nagłówku X-Lion-Token. Sprawdzenie wykorzystuje secrets.compare_digest.

Ochrona dotyczy wszystkich POST, również takich, które tylko analizują dane. Wywołanie walidacji, symulacji, harmonogramu, inspekcji Git lub nawigacji wymaga tokenu tak samo jak zapis. To upraszcza granicę transportową: klient wysyłający dokument lub polecenie do backendu musi najpierw uzyskać kontekst bieżącej sesji. Odczyty GET nie wymagają tego nagłówka, lecz nadal podlegają kontroli Host i Origin.

Po restarcie serwera token się zmienia. Typowy objaw to komunikat „Brak tokenu sesji. Odśwież Studio.” i status 403 w starej karcie. Prawidłowym działaniem jest ponowne pobranie stanu lub odświeżenie aplikacji. Nie zapisuj tokenu w instrukcji, repozytorium ani trwałym pliku konfiguracyjnym. Przykłady automatyzacji powinny pobierać go dynamicznie i przechowywać jedynie przez czas wykonywania skryptu.

Token nie identyfikuje człowieka, projektu ani roli organizacyjnej. Pole actor w żądaniu zapisu jest osobną etykietą deklarowaną przez klienta. Nawet poprawny token połączony z actor równym nazwie administratora nie staje się dowodem jego udziału. Podobnie statusy evidence i currentness w modelu nie są poświadczeniami uzyskanymi z systemu operacyjnego. Architektura zachowuje te rozróżnienia w metadanych dziennika.

W aplikacji graficznej funkcja api dodaje nagłówki do POST automatycznie. Workbench może sam pobrać token przez /api/state, jeżeli jeszcze go nie posiada. Przy testowaniu własnego klienta odtwarzaj ten przepływ. Dzięki temu błąd sesji będzie można odróżnić od błędu dokumentu, zanim skrypt spróbuje importować lub aktualizować większy katalog.

```
$base = 'http://127.0.0.1:18765'
$state = Invoke-RestMethod ($base + '/api/state')
$headers = @{ 'X-Lion-Token' = $state.token }
```

Źródła: S37 StudioHTTPServer.__init__; Handler.do_POST; Handler.do_GET

TEMAT 142

Kontrakt żądania JSON i kody błędów

POST musi zawierać dodatnią wartość Content-Length i typ application/json. Parametr kodowania po średniku jest dopuszczany, ponieważ serwer porównuje podstawowy typ zawartości. Ciało jest odczytywane do zadeklarowanej długości i parsowane jako JSON. Wartość najwyższego poziomu musi być obiektem; tablica, tekst albo liczba nie stanowią poprawnego żądania. Parser odrzuca stałe NaN i Infinity.

Zwykły limit ciała wynosi 8 MiB. Cztery operacje katalogowe mają limit 64 MiB: import, update, navigate i merge-view. Wyższy limit obsługuje duże katalogi i zgromadzone szkice nawigacji; nie zmienia limitu 500 artefaktów dokumentu projektu. Operacja view nadal korzysta ze zwykłego limitu, ponieważ przesyła parametry projekcji, a nie cały katalog. Limit jest sprawdzany przed odczytaniem ciała.

Najważniejsze kody odpowiedzi rozdzielają rodzaj awarii. 403 wskazuje niedopuszczony Host, Origin lub token. 413 oznacza rozmiar żądania, a 415 nieprawidłowy typ zawartości. 422 jest używany dla ValidationFailure projektu, 409 dla konfliktu rewizji. Błędy wartości, brakujących pól lub systemu plików w POST zwykle dają 400. Nieznana operacja ma 404; w GET również nieistniejący identyfikator dokumentu otrzymuje 404.

Nie należy traktować statusu 200 jako jedynego kryterium poprawności domenowej. Endpoint /api/validate zwraca 200 i pole valid, które może być fałszywe. Symulacja procesu może zwrócić dokument ze statusem INVALID. Odczyt katalogu może natomiast zakończyć się 400 przy wykryciu niespójnego digestu. Klient powinien sprawdzać jednocześnie kod transportowy, pola błędu i status wyniku konkretnej analizy.

Nieoczekiwany wyjątek jest wypisywany w terminalu serwera, a odpowiedź 500 zawiera ogólne polecenie sprawdzenia lokalnego terminala. Logger HTTP nie drukuje ciał żądań ani pełnych ścieżek źródłowych. Przy zgłoszeniu problemu zachowaj metodę, ścieżkę, kod odpowiedzi i zanonimizowany fragment komunikatu. Nie dołączaj tokenu ani całego dokumentu, jeżeli wystarczą identyfikatory i ścieżka zgłoszenia walidacji.

Kod	Znaczenie	Pierwsza kontrola
400	Nieprawidłowa wartość lub format	Treść error
403	Granica żądania lub sesji	Adres i token
409	Konflikt rewizji	expected_revision
422	Błąd struktury projektu	issues i path
413 / 415	Rozmiar lub Content-Type	Nagłówki i rozmiar
500	Nieoczekiwany wyjątek	Lokalny terminal

Źródła: S37 MAX_BODY; MAX_CATALOG_BODY; Handler.do_POST

TEMAT 143

Normalizacja i walidacja przed zapisem

Funkcja validated łączy dwa kroki: normalize_workspace i validate_workspace. Normalizacja wykonuje głęboką kopię i dodaje brakujące wartości prezentacyjne lub domyślne. Nie poprawia arbitralnie błędnych wartości już podanych przez klienta. Jeżeli parent_id wskazuje nieistniejący obiekt albo pozycja zawiera tekst zamiast liczby, problem zostanie zachowany do walidacji. Taki model postępowania ogranicza ciche zmiany znaczenia importowanego dokumentu.

Domyślna rama, kolekcje, osie, parametry relacji i pola płytek umożliwiają budowanie oszczędnych dokumentów wejściowych. Nieznane klucze rozszerzeń są zachowywane. Jest to przydatne dla eksperymentalnych metaopisów, ale nie oznacza, że każde rozszerzenie ma interpretację w interfejsie lub silniku. Pole może poprawnie przetrwać zapis i eksport, pozostając wyłącznie informacją deklaratywną. Jego semantykę należy opisać w dokumentacji projektu.

Walidacja rozpoczyna się od ograniczeń samego JSON: dopuszczonych typów, skończonych liczb, braku cyklicznych obiektów przy wywołaniu z Pythona oraz limitów głębokości i liczby odwiedzanych wartości. Następnie sprawdza schemat, tożsamość projektu, rewizję, ramę i kolekcje. Dla artefaktów analizuje między innymi unikalność identyfikatorów, rodziców, pozycję, rozmiar, parametry, płytki i źródła. Dla relacji kontroluje końce, porty, zgodność kontraktów i bramki.

Zgłoszenie ma severity, code, path i message. Ścieżka wskazuje położenie w dokumencie, a kod pozwala grupować błędy niezależnie od tłumaczenia komunikatu. W validated wystąpienie co najmniej jednego error wywołuje ValidationFailure; ostrzeżenia pozostają w odpowiedzi i nie blokują zapisu. Ostrzeżenie o nieaktualnej rewizji końca relacji nie powinno być kasowane bez zrozumienia powodu.

Pomyślna walidacja oznacza zgodność struktury z implementowanymi regułami. Nie sprawdza, czy wskazana ścieżka nadal istnieje, czy autor ma uprawnienia do wdrożenia ani czy opisany proces działa w runtime. W praktyce po walidacji struktury potrzebna jest osobna ocena źródeł i zakresu obserwacji, zanim projekt zostanie wykorzystany jako podstawa refaktoryzacji.

Źródła: S33 normalize_workspace; validate_workspace; _json_issues

TEMAT 144

Baza projektów studio.sqlite3

Magazyn Store zapisuje dokumenty projektowe w pliku studio.sqlite3 we wskazanej przestrzeni roboczej. Baza ma dwie zasadnicze tabele. projects zawiera aktualny dokument każdego projektu wraz z nazwą, numerem rewizji i czasem aktualizacji. revisions przechowuje pełny dokument każdej rewizji i odpowiadające mu zdarzenie. Widok aktualny jest więc indeksem ostatniego stanu, a historia zachowuje wszystkie zapisane stany pośrednie.

Dokument jest kodowany przez canonical: klucze są sortowane, zbędne odstępy usuwane, polskie znaki zachowywane, a NaN zabronione. Ta reprezentacja jest wspólna dla wyliczania digestów i zapisu historii. Dzięki temu różna kolejność kluczy w wejściowym JSON nie zmienia semantycznego skrótu dokumentu, natomiast zmiana wartości, tablicy lub rewizji już go zmienia. Kolejność elementów tablic pozostaje istotna.

Nowy projekt otrzymuje UUID niezależnie od identyfikatora dostarczonego w imporcie. create ustawia rewizję roboczą na zero i wywołuje save z trybem tworzenia, przez co pierwszy trwały zapis ma rewizję jeden. To pozwala wczytać kopię istniejącego dokumentu jako odrębny projekt bez nadpisywania jego źródła. Identyfikatory artefaktów wewnątrz kopii nie są automatycznie traktowane jako nowi właściciele źródeł katalogowych.

SQLite pracuje w trybie WAL, a połączenia Store ustawiają synchronous=FULL oraz czas oczekiwania 15 sekund. Każda metoda otwiera i zamyka własne połączenie w menedżerze kontekstu. Parametry SQL są wiązane jako wartości, a nazwy projektów nie są doklejane do poleceń. Lista projektów jest sortowana od najnowszej aktualizacji; dlatego domyślnie wybrany projekt może zmienić się po zapisie innego projektu.

Użytkownik powinien traktować bazę jako całość wraz z historią, nie jako edytowalny ręcznie odpowiednik project.json. Zmiana jedynie w tabeli projects spowoduje rozbieżność wobec ostatniej rewizji. Do przenoszenia pojedynczego modelu używaj eksportu JSON, a do zachowania historii pakietu asystenta lub spójnej kopii bazy. Nazwa pliku nie jest numerem wersji aplikacji.

```
projects(id, name, revision, updated_at, document)
revisions(project_id, revision, document, event)
-- revisions: klucz główny (project_id, revision)
```

Źródła: S38 Store.__init__; canonical; Store.create

TEMAT 145

Zapis transakcyjny i konflikt dwóch edycji

Każde trwałe zapisanie projektu wymaga expected_revision, czyli numeru rewizji, od której rozpoczęła się edycja. Wartość musi być liczbą całkowitą; boolean nie jest przyjmowany jako zamiennik liczby. Store otwiera transakcję BEGIN IMMEDIATE i odczytuje aktualny dokument. Jeżeli istniejąca rewizja różni się od oczekiwanej, zapis zostaje przerwany przez ConflictError, bez dodania kolejnej pozycji do historii.

Rozważ dwie karty przeglądarki otwarte na rewizji cztery. Pierwsza zapisuje poprawną zmianę i otrzymuje rewizję pięć. Druga nadal wysyła expected_revision równe cztery, więc dostaje 409. Dzięki temu jej starsza kopia nie zastąpi bez ostrzeżenia nowego stanu. Backend nie wykonuje automatycznego scalania nazw, pól, tablic lub pozycji brył. Rozwiązanie konfliktu należy do użytkownika i narzędzia obsługującego dokument.

W jednej transakcji powstają: nowy numer rewizji dokumentu, zdarzenie, pełny wpis revisions oraz aktualizacja projects. Aktualny dokument nie powinien zatem zostać przesunięty do nowej rewizji bez odpowiadającej historii. Operacja tworzenia dodatkowo odrzuca istniejącą tożsamość projektu, a zwykły zapis odrzuca tożsamość nieznaną. Import jako nowy projekt stanowi właściwą drogę dodawania niezależnej kopii dokumentu.

Przy konflikcie najpierw zabezpiecz własną niezapisaną wersję w pliku JSON, następnie odczytaj bieżący projekt i porównaj zmiany. Nie podmieniaj mechanicznie expected_revision na najnowszy numer tylko po to, aby usunąć błąd: taki zabieg deklaruje świadome zastąpienie nowszego dokumentu starszą treścią. Dla rozbudowanych zmian wykorzystaj kopię projektu i kontrolowany przegląd różnic, zachowując identyfikatory oraz przypięcie źródeł.

Transakcja chroni lokalny magazyn przed utratą aktualizacji, lecz nie jest rozproszonym konsensusem federacji. Nie rezerwuje gałęzi Git, pliku źródłowego ani zadania wykonawczego. Podobnie powodzenie zapisu nie świadczy o skutecznym wdrożeniu aplikacji. Zdarzenie otrzymuje runtime_effect równy NONE właśnie po to, by zapis projektu nie został pomylony z wykonaniem planowanego działania.

Źródła: S38 Store.save; ConflictError

TEMAT 146

Łańcuch zdarzeń i sprawdzanie integralności

Każdy zapis projektu tworzy zdarzenie zawierające identyfikator, numer rewizji, czas UTC, aktora, operację i uzasadnienie. before_digest wskazuje skrót poprzedniego dokumentu, after_digest skrót nowego, a previous_hash łączy zdarzenie z poprzednim zdarzeniem. Następnie event_hash jest obliczany z całego zdarzenia przed dopisaniem samego pola event_hash. W efekcie zmiana któregokolwiek z tych elementów zmienia oczekiwany skrót.

Funkcja changes zapisuje zwarte różnice strukturalne. Dla obiektów schodzi po kluczach i tworzy operacje add, remove lub replace ze skrótami wartości. Zmienione tablice są opisane jako zastąpienie, a ich pełna treść pozostaje w migawce rewizji. Dziennik zmian jest więc nawigacyjnym indeksem różnic, a nie samodzielnym zestawem łat odtwarzających projekt bez dokumentów historycznych. Analiza dokładnej edycji powinna zestawiać oba źródła.

Store.verify odczytuje całą historię projektu w kolejności rewizji. Sprawdza ciągłość numerów, zgodność numeru w dokumencie i zdarzeniu, powiązanie previous_hash, event_hash oraz skróty przed i po zmianie. Na końcu porównuje stan bieżący z ostatnią migawką. Pomyślny wynik zawiera valid, liczbę rewizji i head_hash. Taka kontrola wykonywana jest przy starcie dla istniejących projektów oraz przed przygotowaniem pakietu asystenta.

Pole actor jest przycinaną etykietą klienta, a actor_identity ma wartość DECLARED_LABEL. Nie ma podpisu cyfrowego ani poświadczenia konta systemowego. Osoba mająca możliwość przepisania całej bazy mogłaby przeliczyć skróty. Również wykrywanie usunięcia końcowych rewizji wymaga porównania z wcześniej zachowaną głową w niezależnym miejscu. Sam spójny łańcuch nie dowodzi kompletności całej historii świata.

W pracy audytowej zachowuj wynik integrity.json razem z pakietem i datą jego odbioru. Jeżeli potrzebujesz dowodu autorstwa, należy zaprojektować osobny proces podpisywania i przechowywania kotwicy. Obecne Studio dostarcza lokalną wykrywalność niespójności, umożliwiającą rzetelne odróżnienie uszkodzonego zapisu od zwykłej zmiany projektu; nie deklaruje zewnętrznego zaufania do wszystkich źródeł.

Źródła: S38 changes; digest; Store.save; Store.verify

TEMAT 147

Przywracanie poprzedniego projektu bez kasowania historii

Przywrócenie rewizji nie cofa wskaźnika bazy do przeszłości. Store.restore odczytuje migawkę wskazanego numeru, a następnie zapisuje jej treść przez zwykły mechanizm save z bieżącym expected_revision. Odtworzony dokument otrzymuje nowy numer, zaś zdarzenie operation opisuje restore z numerem źródłowym. Poprzednie rewizje, włącznie z tymi uznanymi za błędne, pozostają dostępne do późniejszej analizy.

Na przykład projekt w rewizji dziewięć może odtworzyć treść rewizji pięć. Wynik będzie rewizją dziesięć, a nie ponownie pięć. Tożsamość projektu pozostaje ta sama, więc nadal istnieje jedna czytelna historia decyzji. Przywrócony dokument nie zachowuje starego numeru rewizji w polu głównym, ponieważ pole to opisuje nowy zapis. Historyczne źródła i metadane wewnątrz treści są natomiast odtwarzane zgodnie z wybraną migawką.

Endpoint POST /api/restore wymaga project_id, revision i expected_revision, a opcjonalnie reason. W interfejsie użytkownik otrzymuje informację, że niezapisane edycje zostaną zastąpione. Własny klient musi sam zabezpieczyć taką kopię przed wysłaniem żądania. Backend potrafi wykryć konflikt aktualnego zapisu, ale nie zna lokalnego stanu formularzy, schowka ani osobnego pliku, który nie został jeszcze przesłany.

Jeżeli wskazanej rewizji nie ma, odczyt kończy się błędem. Jeżeli projekt zmienił się od czasu odczytu, obowiązuje zwykły konflikt optimistic concurrency. Operacja nie przywraca katalogu architektury ani jego bazy tylko dlatego, że projekt odwołuje się do starszej rewizji katalogu. Powiązanie katalogowe należy sprawdzić osobno, szczególnie przy próbie wznowienia nawigacji i scalenia wcześniej zebranego szkicu.

Powód przywrócenia powinien opisywać konkretny problem, na przykład błędne przepięcie kontraktów po imporcie. Dzięki temu historia zachowuje sens decyzji, a nie jedynie serię numerów. Po wykonaniu odtworzenia przeprowadź walidację, obejrzyj źródła i porównaj wynik z zamierzoną rewizją. Przywrócenie jest operacją projektową; nie wykonuje rollbacku zmian w repozytorium ani w działającym systemie.

```
{
  "project_id": "ID_ODCZYTANE_Z_API",
  "revision": 5,
  "expected_revision": 9,
  "reason": "Odtworzenie modelu przed błędnym importem"
}
```

Źródła: S38 Store.restore; Store.save

TEMAT 148

Kopie zapasowe i granice odzyskiwania

Studio zawiera metodę Store.backup(destination), korzystającą z mechanizmu backup SQLite. Kopiuje ona bazę projektów do wskazanego pliku przy użyciu połączenia źródłowego i docelowego. Jest to pomocniczy interfejs Python, nie endpoint HTTP ani przycisk pełnej kopii całego środowiska. Nie wolno opisywać wyeksportowanego pakietu JSON jako równoważnego kopii wszystkich baz, profili uruchomieniowych i źródłowych repozytoriów.

W workspace mogą istnieć dwie niezależne bazy: studio.sqlite3 oraz architecture.sqlite3. Pierwsza przechowuje projekty i ich dzienniki, druga katalogi architektury, rewizje katalogowe i wyniki symulacji procesu. Zabezpieczenie tylko pierwszej bazy nie zachowuje pełnego katalogu, do którego odwołuje się zapisany widok. Pakiet asystenta ogranicza tę lukę dla wybranego projektu, ale nie jest globalną kopią wszystkich katalogów i uruchomień.

Przy prostej procedurze administracyjnej zapisz edycje, zamknij Studio, upewnij się, że proces serwera zakończył działanie, a następnie skopiuj cały katalog workspace do datowanego katalogu kopii. Zatrzymanie procesu upraszcza spójność obu magazynów. Nie usuwaj ręcznie plików WAL podczas pracy bazy. Dla kopii wykonywanych bez zatrzymania zaprojektuj procedurę SQLite obejmującą obie bazy i zweryfikuj ją na oddzielnym katalogu testowym.

Odzyskiwanie sprawdzaj przez uruchomienie kopii z jawnym --workspace i innym portem. Pozwala to porównać listę projektów, historię, obecność katalogów i otwieranie wnętrz bez nadpisania bieżącego środowiska. Pomyślny start potwierdza kontrolę łańcuchów projektowych, natomiast odczyt katalogu uruchamia kontrolę jego digestu. Dodatkowo sprawdź eksport reprezentatywnego projektu i odczyt wybranego archiwalnego wyniku symulacji.

Repozytoria źródłowe pozostają poza takim backupem, jeżeli nie znajdują się w kopiowanym katalogu. Zanotowane commit i blob nadal identyfikują migawkę, lecz lokalne pliki mogą być niedostępne. W dokumentacji odtwarzania zapisz zakres kopii, datę, wersję Studio, sposób weryfikacji i informacje o brakujących repozytoriach. Dopiero odtworzona i sprawdzona kopia daje praktyczny dowód użyteczności procedury.

Źródła: S38 Store.backup; Store.verify

TEMAT 149

Baza katalogów architecture.sqlite3

CatalogStore przechowuje pełne modele architektury niezależnie od ograniczonych projektów 3D. Baza architecture.sqlite3 znajduje się w tym samym workspace co baza projektów. Tabela catalogs przechowuje bieżącą rewizję i zwięzłe podsumowanie katalogu. catalog_revisions zachowuje skompresowane dokumenty historyczne, ich SHA-256, skrót poprzednika, czas i powód zapisu. simulation_runs stanowi odrębne archiwum wyników procesu generacyjnego.

Dokument katalogu jest najpierw walidowany przez validate_catalog i kopiowany przez kanoniczne kodowanie JSON. Przy zapisie nadawany jest nowy numer rewizji. Pełna treść jest kompresowana przez gzip z mtime równym zero i trafia do kolumny BLOB. Dzięki temu duży katalog nie musi być dzielony na osobne rekordy dla każdego symbolu. Ceną jest odczyt i dekompresja pełnego dokumentu przy operacjach katalogowych.

Lista katalogów korzysta z zapisanych podsumowań, bez konieczności zwracania wszystkich węzłów. Interfejs może zatem najpierw wyświetlić nazwę, rewizję i liczby obiektów, a dopiero po wyborze wykonywać zapytania. Nie należy jednak mylić stronicowania odpowiedzi z serwerowym indeksem relacyjnym każdego symbolu: implementacja pytań działa na odczytanym dokumencie. Wydajność zależy także od rozmiaru jego rozpakowanej reprezentacji.

Magazyn włącza WAL, używa połączeń z czasem oczekiwania 20 sekund i chroni zapis blokadą RLock oraz transakcją BEGIN IMMEDIATE. Nie ustawia tego samego jawnego synchronous=FULL co Store. Przy opisywaniu gwarancji obu baz należy zachować tę różnicę implementacyjną, zamiast przenosić ustawienia jednego magazynu na drugi. Oba zamykają połączenia po zakończeniu metody.

Pierwsze utworzenie CatalogStore może dodać dołączony atlas, o ile katalog o jego tożsamości jeszcze nie istnieje. Zapisane uzasadnienie wyraźnie wskazuje, że jest to atlas źródłowy, a nie bieżąca obserwacja Git lub runtime. Późniejsze indeksowanie repozytorium tworzy osobny katalog migawki; nie aktualizuje automatycznie prawdziwości wszystkich historycznych deklaracji w atlasie federacji.

```
catalogs(id, revision, summary)
catalog_revisions(id, revision, document, sha256,
                  previous_hash, created_at, reason)
simulation_runs(run_id, catalog_id, process_id,
                created_at, sha256, document)
```

Źródła: S29 CatalogStore.__init__; CatalogStore.save

TEMAT 150

Rewizje katalogu i kontrola jego digestu

Rewizja katalogu jest niezależna od rewizji projektu. Pierwszy zapis nowej tożsamości wymaga expected_revision równego null, natomiast aktualizacja istniejącej tożsamości wymaga jej bieżącego numeru. CatalogStore.save odczytuje rzeczywisty numer pod blokadą i w transakcji, porównuje go z oczekiwaniem, a następnie zapisuje pełną nową migawkę. Konflikt nie jest rozwiązywany automatycznie przez wybór dokumentu przesłanego ostatnio.

Metoda get pozwala odczytać bieżący katalog albo konkretną rewizję. Dla obu wariantów dekompresuje BLOB, parsuje JSON i porównuje digest dokumentu ze skrótem zapisanym w tabeli. Niezgodny digest powoduje IntegrityError. Uszkodzenie skompresowanego zapisu również zatrzymuje odczyt; nie wszystkie rodzaje uszkodzenia są w tej wersji normalizowane do tej samej klasy błędu. Jest to kontrola konkretnego odczytywanego dokumentu, wykonywana za każdym takim odczytem. Lista podsumowań nie jest pełnym przebiegiem kontrolnym wszystkich historycznych migawek.

Tabela historii przechowuje również previous_hash, wskazujący SHA-256 poprzedniej rewizji. Metoda history zwraca numery, skróty, czas i powód zapisu. Aktualna implementacja get nie przechodzi jednak po całym łańcuchu poprzedników, tak jak Store.verify dla projektów. Przy opisie integralności nie należy twierdzić, że samo otwarcie jednego katalogu zweryfikowało ciągłość kompletnej historii wszystkich jego rewizji.

Przywrócenie katalogu korzysta z get starej rewizji, a potem z save. Powstaje zatem kolejna rewizja z treścią historyczną i nowym numerem. Projekty mogą nadal wskazywać własną przypiętą rewizję; nie są automatycznie masowo przepinane do odtworzonej głowy. Ta właściwość pozwala zachować odtwarzalność widoków i uniknąć niezauważalnej zmiany podstawy modelowania w otwartej sesji.

Podsumowanie odpowiedzi zapisu zawiera catalog_id, revision i sha256 obok danych opisowych katalogu. W testach i dokumentacji operacyjnej zapisuj oba numery: rewizję projektu i rewizję katalogu. Jeśli po scaleniu nakładki projekt pozostaje niezapisany, jego lokalne metadane mogą wskazywać już nową rewizję katalogową, podczas gdy trwały zapis projektu nadal reprezentuje wcześniejszy stan. Interfejs oznacza taką sytuację jako wymagającą zapisu.

Źródła: S29 CatalogStore.get; CatalogStore.history; CatalogStore.save; CatalogStore.restore

TEMAT 151

API odczytu pełnej architektury

Rodzina GET /api/architecture służy do przeglądania pełnego modelu bez ograniczania go do sceny 3D. Endpoint catalogs zwraca obiekt z tablicą catalogs. catalog zwraca sam dokument wybranego katalogu, a history obiekt z tablicą history. export również zwraca pełny JSON katalogu, ale dodaje nagłówek pobierania pliku lion-architecture.json. Wszystkie te odczyty dotyczą bieżącej rewizji wskazanego katalogu.

Parametr catalog_id powinien być podawany jawnie w skryptach, aby zmiana listy katalogów nie zmieniła przypadkowo celu operacji. Jeżeli go pominięto, pomocnicza metoda wybiera pierwszy element listy magazynu. Ten wybór jest udogodnieniem interfejsu, nie trwałym kontraktem wskazującym zawsze ai_platform. Projekt nawigacyjny musi ponadto przechowywać przypięty numer rewizji; samo odczytanie bieżącego katalogu nie zastępuje tego przypięcia.

Endpoint query przyjmuje q, kind, layer, parent_id, offset i limit. Zapytanie ogranicza wynik do potrzebnej kategorii i strony, nie usuwa pozostałych węzłów z katalogu. Parser zachowuje puste wartości parametrów, dlatego parent_id o pustej wartości może wyrażać wybór korzeni zgodnie z logiką query_catalog. Brak parametru i parametr pusty mają różne znaczenie; przy budowie adresu nie upraszczaj ich do jednego przypadku. Limit pojedynczej odpowiedzi query i node mieści się od jednego do pięciuset; offset jest nieujemną liczbą całkowitą.

Endpoint node wymaga node_id i zwraca szczegóły obiektu, jego przodków oraz stronicowaną część relacji. To właściwa ścieżka do inspektora źródeł i zależności, gdy scena pokazuje tylko agregaty gałęzi. Dokładne pole source, kwalifikacja i dane relacji pochodzą z katalogu, a nie z wnioskowania na podstawie położenia bryły. Odległość na scenie nie jest argumentem zapytania semantycznego.

Przy automatycznym odczycie zapisuj parametry filtrowania obok wyniku. Sam fragment listy nie dowodzi braku innych plików lub relacji. Jeśli potrzebny jest materiał do niezależnej analizy całego modelu, pobierz export. Jeśli wystarczy odpowiedź na konkretne pytanie, query i node ograniczą transfer oraz ułatwią wskazanie dokładnej tożsamości, na której oparto wniosek.

```
GET /api/architecture/catalogs
GET /api/architecture/catalog?catalog_id=ID
GET /api/architecture/history?catalog_id=ID
GET /api/architecture/query?catalog_id=ID&kind=file&limit=75
GET /api/architecture/node?catalog_id=ID&node_id=NODE_ID
GET /api/architecture/export?catalog_id=ID
```

Źródła: S37 Handler.workbench_get

TEMAT 152

API importu, aktualizacji i odtworzenia katalogu

Import pełnego katalogu przyjmuje obiekt catalog w POST /api/architecture/import. Backend wywołuje save z oczekiwaną rewizją null. Oznacza to utworzenie nowej tożsamości katalogowej, a nie arbitralne zastąpienie już istniejącego dokumentu. Próba importu tego samego catalog_id do magazynu, w którym już istnieje, zakończy się konfliktem. Dla świadomej edycji istniejącego katalogu przeznaczono osobny endpoint update.

Aktualizacja przyjmuje catalog, expected_revision oraz opcjonalnie catalog_id i reason. Jeżeli catalog_id podano oddzielnie, musi odpowiadać tożsamości wewnątrz dokumentu. Walidator katalogowy sprawdza strukturę węzłów, relacji, warstw i nakładek, a następnie magazyn weryfikuje oczekiwany numer. Odpowiedź jest podsumowaniem nowej rewizji, nie pełną kopią przesłanego modelu. Klient powinien zaktualizować zapamiętany numer przed następną edycją.

Przygotowanie aktualizacji wymaga zachowania pól źródłowych oraz rozróżnienia ich od design. Jeżeli zamiarem jest jedynie zmiana nazwy projektowej, roli lub rodzica modelu, właściwym miejscem jest nakładka projektowa, a nie przepisywanie historycznych faktów źródłowych. Bezpośredni endpoint pełnej aktualizacji daje dużą elastyczność, lecz nie rozpoznaje intencji autora; dlatego automatyzacja powinna wcześniej utworzyć eksport i przegląd różnic.

POST /api/architecture/restore przyjmuje catalog_id, revision i expected_revision. Odczytuje dawną migawkę i tworzy nową głowę z jej treścią. Nie usuwa rewizji pomiędzy źródłem a przywróceniem. Powód operacji jest nadawany przez magazyn i wskazuje numer odtworzonej rewizji. Otwarty projekt nadal potrzebuje świadomego dopasowania do właściwej podstawy katalogowej, jeżeli użytkownik chce kontynuować na nowym stanie.

Import i update mają limit ciała 64 MiB; restore przesyła jedynie małe parametry i korzysta ze zwykłego limitu. Wszystkie wymagają tokenu sesji. Żadna z tych operacji nie zapisuje plików w repozytorium źródłowym, nie tworzy commitu ani nie publikuje gałęzi. Zmieniają lokalny model architektury, który dopiero może stanowić podstawę oddzielnego, kontrolowanego procesu implementacyjnego.

POST	Pola obowiązkowe
/api/architecture/import	catalog
/api/architecture/update	catalog, expected_revision
/api/architecture/restore	catalog_id, revision, expected_revision

Źródła: S37 Handler.workbench_post; CatalogStore.save

TEMAT 153

Projekcja katalogu do dokumentu sceny 3D

POST /api/architecture/view tworzy dokument projektu na podstawie pełnego katalogu. Wymaga catalog_id, a opcjonalnie node_id, depth i max_nodes. Domyślne parametry to głębokość jeden oraz maksymalnie trzysta węzłów projekcji. Backend pobiera bieżący katalog, wywołuje architecture.project_view, następnie normalizuje i waliduje otrzymany projekt. Odpowiedź ma pola project i issues. Sama operacja nie tworzy trwałego projektu w bazie Store. Dopuszczalna głębokość wynosi od zera do dwudziestu, a max_nodes od jednego do czterystu pięćdziesięciu.

Projekcja jest ograniczonym dokumentem zgodnym ze schematem Studio. Zawiera artefakty i relacje potrzebne do wyświetlenia wybranego obszaru, a w metadata.catalog_view przechowuje powiązanie z pełnym katalogiem. To powiązanie obejmuje jego tożsamość, rewizję, punkt wyjścia i informacje o zakresie. Dzięki temu późniejszy zapis nakładki może odwołać się do stabilnych identyfikatorów zamiast próbować odgadywać źródło na podstawie nazwy bryły.

Pominięte węzły katalogu nie zostają usunięte. W szczególności większa liczba plików niż limit sceny nie oznacza utraty danych architektonicznych. Ograniczenie dotyczy porcji modelu przekazanej do edytora i jego walidatora. Nie należy zwiększać dokumentu sceny do kilkudziesięciu tysięcy artefaktów, ponieważ schemat roboczy ma odrębny limit. Właściwym narzędziem eksploracji kolejnych poziomów jest mechanizm navigate.

Wersja R3.1 po otwarciu takiego widoku umożliwia jego nawodnienie do nawigowalnego wnętrza. Przeglądarka rozpoznaje powiązanie katalogowe i pobiera bieżący zakres z przypiętej rewizji. Jest to istotna różnica wobec traktowania początkowej projekcji jako kompletnego świata. Dwuklik w głębszą bryłę nie powinien kończyć się pustką tylko dlatego, że jej potomkowie nie mieścili się w pierwszej odpowiedzi.

Przy własnej integracji zachowaj metadata.catalog_view oraz źródłowe identyfikatory węzłów i relacji. Usuwanie ich w celu uproszczenia JSON odcina drogę do poprawnego scalania. Jeśli dokument ma stać się całkowicie niezależnym eksperymentem, wykonaj taką decyzję jawnie i opisz utratę powiązania. Kopia projektu nie jest automatycznie kopią pełnego katalogu ani nowym dowodem istnienia źródeł.

```
{
  "catalog_id": "ID_KATALOGU",
  "node_id": "ID_REPOZYTORIUM",
  "depth": 1,
  "max_nodes": 300
}
```

Źródła: S37 Handler.workbench_post: /api/architecture/view; validated

TEMAT 154

API rzeczywistego wejścia do wnętrza

Endpoint POST /api/architecture/navigate jest podstawą rekurencyjnego wejścia do artefaktu. Wymaga catalog_id i dodatniej całkowitej revision. Odczytuje dokładnie tę migawkę katalogu, zamiast po cichu przełączać się na najnowszą głowę. node_id określa wnętrze konkretnego obiektu, a null oznacza korzeń katalogu. mode wybiera współrzędne design albo git, dzięki czemu oba drzewa mogą być przeglądane oddzielnie.

Parametry offset i limit stronicują bezpośrednie dzieci aktualnego zakresu. Domyślny limit wynosi sto dwadzieścia, dopuszczalny od jednego do trzystu. context_limit kontroluje liczbę zewnętrznych kontekstów, domyślnie dwanaście i maksymalnie sześćdziesiąt cztery. Serwerowy endpoint nie przyjmuje parametru relation_limit od klienta; używa domyślnego ograniczenia funkcji scope_view. Nie utożsamiaj możliwości funkcji Python z publicznym kontraktem HTTP.

Jeżeli przekazano project, backend najpierw sprawdza jego strukturę i zbiera bieżące edycje do szkicu. prior_draft dostarcza zmiany z wcześniejszych wnętrz. scope_view stosuje je do kopii katalogu i wyznacza nowy zakres. Odpowiedź zawiera project, draft i issues, zaś ten sam szkic zostaje umieszczony w metadata.navigation_draft projektu. Nie przybywa rewizja katalogowa ani wpis trwałego projektu tylko wskutek nawigacji.

Metadane navigation opisują zakres, ścieżkę przodków, liczbę wszystkich dzieci i załadowaną stronę. source_location zachowuje odrębne współrzędne: repozytorium, wybrany ref, commit, ścieżkę, blob i zakres linii, jeżeli są znane. Wyliczona ścieżka lokalna może być jedynie deklarowaną referencją, a nie potwierdzeniem istnienia pliku. Wartości UNKNOWN powinny pozostać widoczne zamiast zostać zastąpione pozorną precyzją.

Agregaty relacji mapują połączenia potomków na aktualnie widoczne gałęzie. Konteksty graniczne umożliwiają przejście do właściwego zewnętrznego celu bez dodawania go jako nowego dziecka katalogu. Dzięki temu nawigacja pokazuje zarówno zawieranie, jak i zależności przekraczające zakres. Użytkownik nadal może odróżnić rzeczywistą relację źródłową od jej wizualnego podsumowania.

```
{
  "catalog_id": "ID_KATALOGU",
  "revision": 4,
  "node_id": "ID_WEZLA",
  "mode": "design",
  "offset": 0,
  "limit": 120,
  "context_limit": 12
}
```

Źródła: S37 Handler.workbench_post: /api/architecture/navigate; scope_view; S34 scope_view; apply_draft; DRAFT_SCHEMA

TEMAT 155

Szkic wielopoziomowy i scalenie nakładki

Szkic nawigacji ma schemat lion.navigation-draft/v1, tożsamość katalogu, przypiętą rewizję oraz tablice nodes i relations. Dla istniejącego obiektu przechowuje zwykle id i zmiany design. Dla nowego obiektu zawiera pełny rekord katalogowy z deklarowaną kwalifikacją. apply_draft odrzuca szkic innego katalogu, innej rewizji, powtórzone identyfikatory i próbę zmiany źródłowych faktów istniejącego rekordu. Wynik jest oddzielną kopią dokumentu.

collect_draft porównuje edytowany widok z odtworzoną projekcją tej samej podstawy. Dzięki temu automatycznie wygenerowany układ brył, płytki lub prezentacja źródłowych połączeń nie stają się zmianą użytkownika tylko wskutek wejścia do wnętrza. Zmienione pola są zbierane do nakładki, a wcześniejszy szkic zostaje zachowany. Operacja sprawdza pełną topologię, także rekordy znajdujące się poza aktualną stroną sceny.

Scalenie odbywa się przez POST /api/architecture/merge-view z catalog_id, project i expected_revision. Backend pobiera wskazaną podstawę, zbiera szkic wraz z metadata.navigation_draft, stosuje go i zapisuje nową rewizję katalogu. Odpowiedź zawiera podsumowanie i changes, w tym liczbę accumulated_nodes oraz accumulated_relations. Te liczby dotyczą zgromadzonego szkicu, nie wyłącznie aktualnie widocznych brył. Konflikt nowej głowy zatrzyma zapis.

Konteksty przodków, granice i relacje agregujące nie są materializowane jako nowe źródłowe obiekty podczas scalania. Metadane techniczne nawigacji są filtrowane. Nakładka może zmieniać opis, pozycję, kontrakty i projektowe zawieranie, zachowując pierwotne informacje source. W trybie git samo położenie w drzewie nie jest komendą przepięcia projektowego rodzica; źródłowe współrzędne pozostają osobnym punktem odniesienia.

Polityka merge ma charakter upsert, bez propagowania usunięć z ograniczonej projekcji. Usunięcie bryły z widoku nie może oznaczać automatycznego skasowania niewidocznej części pełnego katalogu. Po udanym scaleniu interfejs aktualizuje przypięcie katalogowe i usuwa zużyty szkic, ale projekt wymaga jeszcze własnego zapisu. Rozróżniaj zatem trwałość katalogu, trwałość projektu i ulotny stan formularza.

Źródła: S41 collect_draft; merge_view; preserve_reindex; S34 apply_draft

TEMAT 156

Indeksowanie Git i zachowanie ręcznych modeli

POST /api/repositories/index przyjmuje lokalną path, opcjonalny ref, include_symbols oraz max_files. Domyślnie wybierany jest HEAD, analiza symboli jest włączona, a limit plików wynosi dziesięć tysięcy. Funkcja index_repository rozwiązuje referencję do commitu i czyta obiekty drzewa Git. Nie kopiuje po prostu bieżącej zawartości katalogu roboczego, dlatego niezacommitowane zmiany nie stają się automatycznie treścią przypiętej migawki.

Tożsamość katalogu jest związana z repozytorium i commitem. Kolejny indeks tego samego commitu może wzbogacić zakres, ale nie powinien wymazać ręcznego modelowania. Backend próbuje odczytać istniejący katalog tej samej tożsamości i wywołuje preserve_reindex. Scalane są węzły, relacje i warstwy, przy czym istniejące opisy mają pierwszeństwo. Zachowywane są również zapisane procesy. Powstaje nowa lokalna rewizja tej samej migawki.

Ta zasada jest bezpieczna właśnie dlatego, że obiekt Git wewnątrz przypiętego commitu nie zmienia się. Nie stanowi automatycznej migracji ręcznych nakładek pomiędzy dowolnymi commitami. Gdy ref wskazuje nowszy commit, powstaje inna tożsamość katalogowa. Porównanie i przeniesienie znaczeń wymaga wtedy świadomego modelowania, z uwzględnieniem przeniesionych plików, zmienionych symboli oraz zmiany kontraktów. Sama zgodność nazwy nie daje wystarczającego dowodu tożsamości.

Indekser ma odrębne limity: maksymalnie dwadzieścia tysięcy plików, jeden MiB odczytywanego bloba, trzydzieści dwa MiB analizowanych źródeł i sześćdziesiąt tysięcy węzłów. Pominięcia są ujawniane w coverage i diagnostics. Niektóre ścieżki chronione pozostają opisane bez odczytu zawartości. Częściowy katalog nadal może być użyteczny, ale nie uprawnia do twierdzenia, że poznano wszystkie zależności repozytorium.

Jeżeli Git odmawia odczytu z powodu właściciela repozytorium albo narzędzie jest niedostępne, stan powinien pozostać UNKNOWN z diagnostyką. Studio nie powinno samoczynnie zmieniać globalnego safe.directory ani podnosić zaufania do źródeł. Przed modelowaniem rozróżnij błąd odczytu, brak lokalnego repozytorium i poprawny odczyt ograniczony zadanym budżetem.

Źródła: S37 Handler.workbench_post: /api/repositories/index; preserve_reindex; S41 preserve_reindex; S35 index_repository; MAX_FILES; MAX_BLOB_BYTES

TEMAT 157

Odkrywanie, inspekcja i klonowanie repozytoriów

GET /api/repositories przyjmuje root wskazujący katalog poszukiwań. Domyślna wartość odpowiada katalogowi PycharmProjects w środowisku Windows autora projektu. W innym środowisku podaj własną ścieżkę; nie zakładaj, że zapis z przykładu istnieje na każdym komputerze. Odkrywanie jest ograniczoną operacją lokalną. Wynik powinien być odczytywany razem z diagnostyką, a brak pozycji nie zawsze dowodzi nieistnienia repozytorium poza przeszukanym zakresem.

POST /api/repositories/inspect wymaga path. Zwraca informacje uzyskane przez Git, między innymi gałęzie, znaczniki, commit i publiczną reprezentację adresów zdalnych. Ten krok pomaga wybrać ref przed indeksowaniem. Nie zmienia HEAD ani katalogu roboczego. W szczególności wybranie gałęzi do analizy nie jest poleceniem checkout. Inspekcja opisuje stan możliwy do odczytu przez proces Studio z jego bieżącymi uprawnieniami.

POST /api/repositories/clone wymaga url i destination, opcjonalnie branch. Implementacja dopuszcza jawny publiczny adres HTTPS repozytorium GitHub zgodny z ograniczonym wzorcem. Odrzuca dane uwierzytelniające, parametry adresu i inne protokoły. Cel musi być nowym miejscem w istniejącym katalogu nadrzędnym; operacja nie służy do nadpisywania dowolnego istniejącego projektu. Przed wysłaniem sprawdź ścieżkę, ponieważ jest interpretowana na komputerze serwera.

Klonowanie używa wariantu bez checkoutu, nie pobiera submodułów i nie uruchamia kodu repozytorium. Po zakończeniu należy osobno zlecić indeksowanie wybranej referencji. Brak widocznych plików roboczych po takim klonowaniu nie jest usterką: obiekty Git mogą już istnieć, chociaż drzewo robocze nie zostało zmaterializowane. Odpowiedź jawnie opisuje checkout_performed oraz submodules_fetched.

Ta operacja jest wyjątkiem od podstawowej pracy offline, ponieważ wymaga połączenia sieciowego z usługą repozytorium. Nie oznacza integracji z modelem AI ani automatycznego pobierania brakujących repozytoriów federacji. Użytkownik wybiera adres i cel. Błędy dostępu, sieci lub narzędzia Git należy rozpatrywać niezależnie od poprawności istniejącego atlasu i ręcznie zbudowanych warstw modelu.

Operacja	Wejście	Skutek
GET /api/repositories	root	Lista lokalnych kandydatów
POST inspect	path	Inspekcja bez checkoutu
POST index	path, ref	Nowa rewizja katalogu
POST clone	url, destination	Nowy klon bez checkoutu

Źródła: S37 Handler.workbench_get; Handler.workbench_post; clone_repository; S35 discover_repositories; inspect_repository; clone_repository

TEMAT 158

API procesu generacyjnego i archiwum wyników

GET /api/evolution/default zwraca process oraz saved_processes. Jeżeli podano catalog_id, backend może wybrać proces zapisany w katalogu; process_id pozwala wskazać konkretny. Parametr fresh=true wymusza przygotowanie świeżego procesu domyślnego zamiast użycia wybranego zapisu. Ten endpoint nie uruchamia generacji i nie zapisuje procesu automatycznie do katalogu. Jest źródłem dokumentu wejściowego dla dalszego modelowania.

POST /api/evolution/simulate przyjmuje process, opcjonalnie generations, assumptions i catalog_id. Domyślna liczba generacji wynosi trzy. Silnik sprawdza strukturę procesu, typy wejść, kolejność etapów, bramki, feedback i zadeklarowane scenariusze. Wynik jest oznaczany effect równym SIMULATION_ONLY. Materiałem wejściowym są deklaracje i jawne założenia, a nie ukryte wywołania modeli, testów repozytorium lub narzędzi wykonawczych.

Wynik o statusie innym niż INVALID jest automatycznie archiwizowany w CatalogStore. run_id ma postać simulation: z digestem wyniku. INSERT OR IGNORE sprawia, że identyczny wynik o identycznej treści nie tworzy drugiej pozycji o tej samej tożsamości. Odpowiedź otrzymuje archive_ref. Ponieważ klucz wynika z dokumentu wyniku, sama powtórka żądania nie jest dowodem wykonania nowego niezależnego eksperymentu zewnętrznego.

GET /api/evolution/runs zwraca runs, opcjonalnie filtrowane przez catalog_id; lista ma limit stu najnowszych wpisów. GET /api/evolution/run wymaga run_id i zwraca zapisany dokument po sprawdzeniu jego digestu. Tabela archiwum zachowuje powiązanie z katalogiem i procesem, lecz zapis wyniku nie zwiększa numeru rewizji katalogu. Nie należy odczytywać samej rewizji jako licznika wszystkich symulacji.

Silnik ogranicza generacje do pięćdziesięciu, etapy do dwustu i liczbę zdarzeń do dwóch tysięcy. Proces wraz ze scenariuszem ma budżet 256 KiB, pojedyncza wartość 8192 bajty, a ślad osiem MiB. Większa wartość może zostać skrócona w prezentacji śladu z zachowaniem skrótu i referencji. Limity te są domenowe i pozostają znacznie mniejsze niż transportowy limit żądania HTTP.

```
GET /api/evolution/default?catalog_id=ID&fresh=true
POST /api/evolution/simulate
GET /api/evolution/runs?catalog_id=ID
GET /api/evolution/run?run_id=simulation%3ASKROT
```

Źródła: S37 Handler.workbench_get; Handler.workbench_post; CatalogStore.save_simulation; S31 simulate_process; default_process; MAX_GENERATIONS; S29 save_simulation; list_simulations; get_simulation

TEMAT 159

API stanu projektu, listy i dziennika

GET /api/health jest najprostszą kontrolą dostępności procesu. Zwraca ok, version i mode, gdzie tryb ma wartość LOCAL_DESIGN. Wynik potwierdza odpowiedź lokalnego serwera, nie gotowość repozytoriów federacji, aktualność ich źródeł ani ukończenie indeksowania dużego katalogu. Można go użyć do rozróżnienia niedziałającego portu od problemu konkretnego projektu lub zapytania architektonicznego.

GET /api/state dostarcza kontekst startowy interfejsu: project, projects, events, token, geometry i version. Lista projects zawiera identyfikatory oraz metadane zapisanych projektów, a events jest ograniczone do ostatnich osiemdziesięciu zdarzeń wybranego projektu. Token należy traktować jako efemeryczny element bieżącej sesji. Nie zapisuj pełnej odpowiedzi state w publicznym raporcie bez usunięcia tej wartości i przeglądu pozostałych danych.

Projekt można wybrać parametrem project_id albo id. Jeśli oba występują, pierwszeństwo ma project_id. Gdy żadnego nie podano, wybierany jest pierwszy projekt na liście Store, sortowanej według czasu aktualizacji malejąco. To wygodne dla ekranu startowego, ale w skryptach administracyjnych należy zawsze wskazywać tożsamość. Inny projekt zapisany chwilę wcześniej może zmienić wynik zapytania bez jawnego identyfikatora.

GET /api/project zwraca project i ostatnich osiemdziesiąt events. GET /api/events zwraca pełną tablicę zdarzeń wybranego projektu. Pełnych dokumentów rewizji nie udostępnia osobny endpoint history projektu; znajdują się w bazie i w pakiecie asystenta. Nie należy mylić tego z /api/architecture/history, który dotyczy innego magazynu i zwraca metadane rewizji katalogowych.

Odczyty nie zapisują rewizji projektowej. Mogą jednak uruchomić inicjalizację katalogów, jeżeli wybrana operacja korzysta z workbench. Przy porównaniu odpowiedzi zachowuj project_id i revision, ponieważ nazwa projektu jest edytowalna i nie stanowi trwałego klucza. Rozmiar listy events w state także nie jest liczbą wszystkich zapisów: dla starszego projektu należy odczytać pełny dziennik lub zweryfikować pole revisions w integrity.json.

GET	Zakres odpowiedzi
/api/health	Zdrowie procesu i wersja
/api/state?project_id=ID	Stan, lista, token, geometria
/api/project?id=ID	Projekt i 80 ostatnich zdarzeń
/api/events?project_id=ID	Pełny dziennik projektu

Źródła: S37 Handler.do_GET; Handler.selected_id; Store.events

TEMAT 160

Tworzenie projektu oraz dwa etapy importu

POST /api/new tworzy odrębny trwały projekt z wybranego szablonu. Pole template wybiera punkt wyjścia, a opcjonalne name nadaje nazwę. Dostępne w interfejsie szablony obejmują ai_platform, federation, studio, demo, temporal i blank. Dla ai_platform serwer tworzy ograniczoną projekcję pierwszego dostępnego katalogu; pozostałe korzystają z seeds.new_project. Po walidacji Store.create nadaje nowy project_id i zapisuje rewizję jeden. Przykład poniżej przedstawia kompletne żądanie utworzenia samopisu; identyfikator i numer zapisanej rewizji należy odczytać z odpowiedzi.

Import projektu jest rozdzielony na przygotowanie oraz utrwalenie. POST /api/import przyjmuje document, wykonuje migrate_workspace i walidację, a następnie zwraca project i issues. Nie tworzy projektu w bazie. Dzięki temu można obejrzeć migrowany dokument, poprawić błędy i zdecydować o późniejszym zapisie. Samo poprawne przejście importu nie dowodzi, że pojawił się nowy trwały element na liście projektów.

POST /api/import/save wykonuje ten sam etap przekształcenia, lecz zapisuje wynik jako nowy projekt. Opcjonalne name jest konwertowane do tekstu i ograniczane do trzystu znaków. Odpowiedź zawiera projekt, jego events, listę projects i issues. Dotychczasowa tożsamość projektu źródłowego nie zostaje nadpisana, ponieważ Store.create zawsze nadaje nowy UUID. To właściwa ścieżka dla wariantu badawczego lub kopii modelu przesłanej z innej instalacji.

Migrator obsługuje bieżący schemat Studio oraz rozpoznawaną starszą postać mozaiki R1. Nie jest ogólnym konwerterem dowolnego JSON ani samego katalogu lion.architecture/v1. Katalog importuje się przez rodzinę architecture. Jeśli dokument nie ma obsługiwanego formatu, operacja kończy się błędem przed zapisem. Test HTTP sprawdza, że odrzucony import nie powiększa listy projektów i nie dopisuje historii.

W praktyce rozróżniaj nowy projekt z szablonu, kopię istniejącego projektu i pełny katalog źródłowy. Są to różne jednostki trwałości. Przed importem zachowaj plik wejściowy, a po przygotowaniu sprawdź tożsamości artefaktów, źródła, osie i kwalifikacje. Szczególną uwagę poświęć powiązaniu katalogowemu: nowy project_id nie oznacza, że źródłowe węzły Git również otrzymały nową tożsamość.

```
{
  "template": "studio",
  "name": "Samopis - analiza R3.1"
}
```

Źródła: S37 Handler.do_POST: /api/new; /api/import; /api/import/save

TEMAT 161

API zapisu projektu oraz kontroli struktury

POST /api/project zapisuje kompletny dokument project, a nie pojedynczą łatę zmienionego pola. Wymaga expected_revision i może otrzymać actor oraz reason. Backend normalizuje i waliduje dokument, po czym wywołuje Store.save. Odpowiedź zawiera nowy project, ostatnich osiemdziesiąt events i validation. Zapis powiększa historię nawet wtedy, gdy klient wysłał treść bardzo podobną do poprzedniej, ponieważ powstaje kolejna rewizja dokumentu.

Interfejs graficzny pamięta ostatnią zapisaną rewizję oddzielnie od bieżącego dokumentu. Do reason może dołączyć lokalne przyczyny edycji zebrane od ostatniego zapisu. Po powodzeniu aktualizuje stan zapisany, czyści lokalny stos undo i redo oraz ponownie pobiera listę projektów i token. Oczekiwanie na odpowiedź jest więc częścią operacji; nie należy zamykać procesu zaraz po kliknięciu bez potwierdzenia nowej rewizji.

POST /api/validate przyjmuje project i zwraca issues oraz valid. Nie zapisuje projektu i nie nadaje nowej rewizji. Nie wymaga, aby dokument już istniał w bazie, dlatego nadaje się do sprawdzania wersji przygotowanej w zewnętrznym edytorze. Nadal obowiązuje token sesji, typ JSON i transportowy limit żądania. Odpowiedź 200 z valid=false jest poprawnym wynikiem kontroli, a nie awarią serwera.

Istnieje ważna różnica między błędem walidacji w operacji zapisu a samą usługą walidacji. Zapis odrzucający projekt zwraca 422 wraz z issues. Usługa kontrolna zwraca listę problemów w normalnej odpowiedzi. Własny skrypt powinien najpierw sprawdzić valid, przedstawić krytyczne ścieżki użytkownikowi i dopiero potem wysłać zapis z pierwotnym expected_revision. Jeżeli w międzyczasie projekt się zmienił, konflikt pozostaje prawidłową ochroną.

Pełny dokument zawiera również metadata.navigation_draft, jeżeli istnieją niezapisane katalogowe nakładki. Zapis projektu może je utrwalić bez natychmiastowego scalania katalogu. Otworzenie projektu ma wtedy odtworzyć szkic na jego przypiętej podstawie. Zachowanie tego pola jest niezbędne, aby ręczne zmiany z wcześniejszych wnętrz nie zniknęły podczas eksportu, importu i późniejszej kontynuacji pracy.

```
{
  "issues": [],
  "valid": true
}

Powyżej: kompletna odpowiedź udanej walidacji bez zgłoszeń.
Żądanie: obiekt z polem project zawierającym pełny model.
```

Źródła: S37 Handler.do_POST: /api/project; /api/validate

TEMAT 162

API propagacji parametrów i analizy harmonogramu

POST /api/simulate przyjmuje project i opcjonalny max_steps, domyślnie dwadzieścia. Jest to symulacja propagacji parametrów przez relacje dataflow i exposes. Inne rodzaje relacji pozostają opisem, a nie instrukcją kopiowania wartości. Silnik działa na kopii znormalizowanego dokumentu, zwraca values, trace, blocked, diagnostics, steps i status. Nie zapisuje wynikowych wartości do trwałego projektu ani nie uruchamia jego kodu źródłowego.

W pojedynczym kroku wartości są odczytywane synchronicznie, a planowane zapisy stosowane dopiero po sprawdzeniu konfliktów. Dwa zapisy do tego samego parametru powodują CONFLICT i brak zastosowania danego kroku. Relacja może zostać zablokowana przez niezgodną rewizję końca, przekroczenie granicy zawierania, wynik bramki, brak powiązań lub nieznaną wartość. Powód znajduje się w śladzie; nie należy interpretować samego braku zmiany jako sukcesu całej ścieżki.

Dopuszczalny max_steps mieści się od jednego do stu. Ślad ma limit dwudziestu tysięcy wpisów. Możliwe wyniki obejmują NO_EXECUTABLE_RELATIONS, CONVERGED, CONVERGED_WITH_BLOCKS, CONFLICT i STEP_LIMIT. Zbieżność oznacza ustalenie wartości w granicach tego modelu. Nie dowodzi zgodności z zachowaniem realnych usług, które mogą mieć opóźnienia, efekty uboczne i warunki niewprowadzone do dokumentu.

POST /api/schedule przyjmuje project oraz opcjonalny duration_overrides. Analizuje relacje czasowe i zadeklarowane czasy, pozostawiając dokument bez zmian. Nadpisania scenariusza mogą wskazywać wartość wraz z jednostką dla konkretnego artefaktu. Przykładowo dłuższa praca badawcza przesuwa makespan i może ujawnić przekroczenie deadline, ale nie zmienia zapisanej duration. Wynik służy porównaniu planów oraz jawnych założeń.

Oba endpointy przechodzą przez validated. Błędny dokument zostanie odrzucony przed analizą, natomiast blokady semantyczne i niewiedza mogą pojawić się w poprawnym strukturalnie wyniku. Test HTTP porównuje projekt przed analizą i po niej oraz sprawdza niezmienioną historię. Eksport pakietu asystenta oblicza harmonogram bazowego zapisanego projektu, bez lokalnych duration_overrides z niezapisanego scenariusza przeglądarki.

```
$request = @{
  project = $s.project
  duration_overrides = @{
    ID_BADANIA = @{ value = 7000; unit = 'ms' }
  }
} | ConvertTo-Json -Depth 100
# $s: stan pobrany jak na stronie z klientem PowerShell.
# Zastąp ID_BADANIA identyfikatorem istniejącego artefaktu.
```

Źródła: S37 Handler.do_POST: /api/simulate; /api/schedule; simulate_workspace; S33 simulate_workspace

TEMAT 163

Lokalny skan źródeł i bezpieczna materializacja

POST /api/scan przyjmuje root, project i opcjonalny limit plików. Odczytuje istniejący katalog lokalny i zwraca rozszerzony projekt z kandydackimi deskryptorami oraz report. Serwer dodatkowo dołącza issues z walidacji wyniku. Skan nie zapisuje nowej rewizji; użytkownik musi później zapisać projekt. Ścieżka root jest interpretowana po stronie procesu Studio, dlatego przeglądarka na innym środowisku nie mogłaby wskazać nią własnego dysku.

To inna operacja niż indeksowanie przypiętego Git. Raport ma kwalifikację PARTIAL_LOCAL_READBACK_NOT_COMMIT_VERIFIED. Nie zapewnia atomowej migawki zmieniającego się katalogu, nie potwierdza ref i nie rozstrzyga tożsamości logicznej elementów. Analiza deklaracji Python wykorzystuje składnię bez importowania modułów. Nawet znaleziony lokalny kandydat celu importu pozostaje UNRESOLVED_RUNTIME. Statyczna obserwacja nazwy nie stanowi dowodu faktycznego wywołania.

Skan przyjmuje od jednego do pięciuset plików, domyślnie sto pięćdziesiąt. Dodatkowe budżety obejmują dziesięć MiB na plik, pięćdziesiąt MiB łącznie, dwadzieścia tysięcy wpisów katalogowych i sześćdziesiąt cztery symbole na plik. Nadal obowiązują granice pięciuset artefaktów i dwóch tysięcy relacji projektu. Raport gaps wyjaśnia pominięcia. Korzeń będący dowiązaniem lub junction jest odrzucany, zamiast stać się niekontrolowanym rozszerzeniem zakresu.

Materializacja działa przeciwnie do skanu: GET /api/export z format=materialize tworzy ZIP kandydatów z zapisanego projektu. Zawiera project.snapshot.json, manifest, deskryptory i notatki. Niektóre rodzaje artefaktów otrzymują inertny candidate.py z metadanymi. Nie ma w nim generowanych importów, wywołań ani treści implementacji wynikających z opisu użytkownika. Nazwy katalogów eksportu zależą od skrótów stabilnych identyfikatorów, nie od swobodnych ścieżek podanych w modelu.

Manifest jawnie stwierdza SOURCE_CANDIDATE_NOT_DEPLOYED oraz brak efektu runtime i authority. Taki pakiet jest materiałem dla późniejszej implementacji i przeglądu zależności. Nie zastępuje uruchomienia testów, utworzenia commitu ani decyzji wdrożeniowej. Skan i materializacja razem tworzą czytelny przepływ od obserwacji do kandydata, zachowując rozdzielenie między źródłem, projektem i działającym systemem.

Źródła: S32 scan_sources; materialize; MAX_FILE_BYTES; MAX_ARTIFACTS

TEMAT 164

Trzy eksporty i kontrola zawartości ZIP

GET /api/export wybiera projekt przez project_id lub id i przyjmuje format. Dostępne są json, assistant i materialize. Domyślny json zwraca czytelny dokument z wcięciami jako lion-project.json. Ten plik zawiera pełny stan pojedynczego zapisanego projektu, włącznie z metadanymi, lecz bez całej historii rewizji i bez niezależnej bazy katalogów. Jest najprostszym formatem wymiany modelu do ponownego importu.

Format assistant zwraca lion-assistant-context.zip z historią, źródłami i kontekstem analitycznym. Format materialize zwraca lion-artifact-candidates.zip, przeznaczony do dalszej pracy implementacyjnej. Oba są budowane w pamięci i wysyłane jako application/zip. Samo pobranie nie instaluje zawartości i nie zapisuje plików źródłowych na dysku serwera. Wybór miejsca zapisania pobranego pliku należy do przeglądarki lub lokalnego klienta.

Serwerowa funkcja zip_bytes sprawdza nazwy wszystkich wpisów. Odrzuca ścieżki bezwzględne, elementy nadrzędne, odwrotne ukośniki i dwukropki. Ograniczenie dotyczy nazw wewnątrz tworzonego ZIP i zapobiega przemyceniu niebezpiecznej ścieżki eksportowej. Nie jest ogólnym skanerem semantyki wszystkich tekstów zapisanych jako dane. Zawartość deskryptora nadal może zawierać deklarowane nazwy, hipotezy i źródła wymagające interpretacji.

Interfejs downloadPackage przed pobraniem pakietu zapisuje brudny albo importowany projekt. Dzięki temu archiwum odpowiada trwałej rewizji, a nie przypadkowej kombinacji zapisanych i niezapisanych formularzy. Własny klient wywołujący GET bezpośrednio otrzyma stan z bazy, niezależnie od edycji otwartej karty. Jeżeli porównujesz zawartość z ekranem, sprawdź revision i zakończenie poprzedniego POST zapisu.

Nieznany format powoduje błąd wartości. W przypadku projektu powiązanego z katalogiem pakiet asystenta próbuje dołączyć przypiętą migawkę; zwykły JSON nie robi tego automatycznie. Przy przekazywaniu materiału innemu analitykowi dobierz format do zadania: projekt do edycji, pakiet do rekonstrukcji historii albo kandydaci do planowania implementacji. Nazwa archiwum sama nie opisuje kompletności federacji, dlatego dołącz zakres źródeł i świadome luki.

```
GET /api/export?project_id=ID&format=json
GET /api/export?project_id=ID&format=assistant
GET /api/export?project_id=ID&format=materialize
```

Źródła: S37 Handler.do_GET: /api/export; zip_bytes; assistant_package

TEMAT 165

Pakiet asystenta jako kontekst odtwarzalnej pracy

assistant_package najpierw sprawdza integralność historii wybranego projektu. Następnie pobiera bieżący dokument, wszystkie rewizje, pełny dziennik, źródła i wynik walidacji. project.json opisuje stan końcowy, events.jsonl pozwala czytać zdarzenia po jednym wierszu, a revisions przechowuje pełne migawki z numerowanymi nazwami. integrity.json zawiera wynik kontroli i głowę lokalnego łańcucha. Tak przygotowany pakiet umożliwia analizę bez dostępu do pierwotnej bazy.

assistant_context.md opisuje cel dalszej pracy, tożsamość, rewizję i digest projektu. Wyraźnie oddziela deklaracje aktora oraz historyczne źródła od potwierdzenia deploymentu. validation.json zachowuje zgłoszenia, a sources.json zbiera źródła artefaktów wraz z artifact_id. Pakiet nie dołącza automatycznie pełnej zawartości wszystkich wskazanych plików źródłowych. Odnośnik do pliku pozostaje referencją, której dostępność analityk musi sprawdzić w swoim środowisku.

impact.json porównuje artefakty ostatniej i przedostatniej rewizji, a następnie wyznacza konserwatywne, nieskierowane sąsiedztwo po relacjach bieżącego projektu. To potencjalny obszar wpływu, nie dowód zależności wykonawczej. Usunięty artefakt albo zmiana samej relacji wymaga dodatkowego przeglądu dokumentów historycznych. Nie należy utożsamiać tej prostej metody z pełnym analizatorem zależności kodu, śledzeniem wywołań ani oceną skutków deploymentu.

schedule.json zawiera harmonogram zapisanego projektu. Lokalne nadpisania scenariusza czasowego nie są automatycznie przenoszone do tego pliku. Jeżeli projekt ma powiązanie katalogowe i dostępna jest wskazana rewizja, dołączane są architecture/catalog.json oraz history.json. Pakiet zawiera również metadane dostępnych symulacji i najwyżej pięć najnowszych dokumentów wynikowych. Brak przypiętej migawki jest ujawniany przez architecture/gap.json zamiast zastępowania jej inną głową.

Odbiorca powinien zacząć od integrity i validation, następnie porównać projekt ze źródłami i historią. Zachowanie identyfikatorów i przypiętych SHA umożliwia kontynuację pracy bez mylenia obiektów o podobnych nazwach. Pakiet jest szczególnie użyteczny przy zgłaszaniu błędu nawigacji lub kontraktu, ponieważ utrwala model i powody zmian. Nadal nie stanowi podpisanego poświadczenia autorstwa ani kompletnego zapisu działania całej federacji.

Źródła: S37 assistant_package; zip_bytes; S38 Store.verify; Store.history; Store.events

TEMAT 166

Praktyczny klient PowerShell: kontrola i walidacja

Najprostszy klient administracyjny powinien zacząć od żądania zdrowia, potem pobrać state i użyć otrzymanego tokenu dla POST. Poniższy skrypt nie tworzy projektu ani nie zapisuje rewizji. Wysyła do walidacji dokładnie ten dokument, który właśnie odczytał. Jest dobrym testem rozdzielającym dostępność HTTP, stan sesji, kodowanie JSON i reguły modelu. Uruchamiaj go przy działającym lokalnie Studio na wskazanym porcie.

ConvertTo-Json wymaga odpowiednio dużego parametru Depth, ponieważ dokument zawiera płytki, porty, bramki, metadane i ewentualny szkic nawigacji. Zbyt płytkie kodowanie potrafi zamienić zagnieżdżone obiekty na reprezentacje tekstowe, tworząc błędy, których nie było w oryginale. Wysyłanie jawnych bajtów UTF-8 ogranicza problemy z polskimi nazwami między różnymi wersjami PowerShell. W razie problemu najpierw sprawdź rezultat walidacji, nie wykonuj zapisu naprawczego.

W przykładzie nagłówki są przechowywane w pamięci skryptu. Nie wypisuj ich ani pełnej odpowiedzi state do publicznego pliku. Do raportu wystarczą version, project_id, revision oraz code i path zgłoszeń. Jeśli serwer został ponownie uruchomiony pomiędzy GET a POST, pobierz nowy stan. Nie zakładaj, że poprzedni token jest związany z plikiem bazy i pozostanie poprawny po restarcie.

Po udanej kontroli można zaprojektować osobny krok zapisu z pełnym project i expected_revision równym odczytanej rewizji. Nie dodawaj go do skryptu diagnostycznego bez konkretnej potrzeby. Analizę stanu i jego zmianę warto rozdzielać, ponieważ powtarzanie diagnostyki powinno pozostawiać historię niezmienioną. Przy skryptach zmieniających katalog architektury obowiązuje oddzielny numer rewizji i inny dokument wejściowy.

Polecenie curl służy tu tylko do odczytu zdrowia. curl.exe jawnie wybiera program systemowy zamiast ewentualnego aliasu powłoki. Jeżeli używasz innego portu, zmień bazowy adres w jednym miejscu i ponownie pobierz token. Test nie wymaga klucza API modelu, połączenia z AI ani dostępu do zdalnego repozytorium.

```
$base = 'http://127.0.0.1:18765'
curl.exe ($base + '/api/health')
$s = Invoke-RestMethod ($base + '/api/state')
$h = @{ 'X-Lion-Token' = $s.token }
$j = @{ project = $s.project } | ConvertTo-Json -Depth 100
$bytes = [Text.Encoding]::UTF8.GetBytes($j)
$r = Invoke-RestMethod -Method Post `
  -Uri ($base + '/api/validate') -Headers $h `
  -ContentType 'application/json; charset=utf-8' -Body $bytes
$r.valid
$r.issues | Select-Object severity, code, path, message
```

Źródła: S37 Handler.do_GET; Handler.do_POST; validated

TEMAT 167

Diagnostyka błędów lokalnej usługi

Jeżeli strona nie otwiera się, najpierw sprawdź terminal procesu i adres wypisany podczas startu. Port 18765 może być zajęty przez inną instancję. Użycie --port 18766 rozwiązuje konflikt nasłuchu, ale wymaga otwarcia właściwego adresu w przeglądarce. Nie diagnozuj problemu źródeł, zanim /api/health nie odpowiada z oczekiwaną wersją. Ta kolejność ogranicza mieszanie usterek transportowych z modelowaniem architektury.

Jeżeli strona działa, ale POST ma 403, zwykle należy sprawdzić starą kartę, zmieniony token, Host i Origin. Przy 409 problemem jest rozbieżność rewizji, nie uszkodzenie bazy. Przy 422 odczytaj issues i przejdź do wskazanego path. Przy 413 sprawdź, czy wysyłasz projekt zamiast katalogu lub czy szkic nie urósł ponad budżet. Przy 415 popraw Content-Type zamiast zmieniać sam dokument.

Puste wnętrze wymaga porównania node_id, trybu nawigacji i liczby total_children. Prawdziwy liść może nie mieć dzieci. Strona offset poza zakresem może nie pokazywać żadnego elementu, chociaż zakres ma potomków. Niedostępna przypięta rewizja powinna zwrócić błąd, a nie zostać zastąpiona przypadkową najnowszą. Liczba widocznych relacji zależy również od agregacji, stronicowania i limitu kontekstów; nie jest automatycznie pełną liczbą relacji katalogu.

Jeżeli serwer nie startuje z powodu integralności, zabezpiecz workspace przed dalszymi eksperymentami. Nie wycinaj ręcznie problematycznej rewizji i nie przeliczaj zdarzeń tylko po to, by uzyskać zielony wynik. Odtwarzaj sprawdzoną kopię w oddzielnym katalogu. W przypadku Git sprawdź dostępność narzędzia, uprawnienia procesu i wybraną referencję. Stan UNKNOWN jest poprawną informacją o ograniczeniu obserwacji, nie zachętą do automatycznego zwiększania zaufania.

Zgłoszenie powinno podawać wersję usługi, komendę uruchomienia, workspace, identyfikator projektu, obie rewizje i kroki odtwarzające. Dołącz odpowiedź API bez tokenu, zanonimizowany komunikat terminala i pakiet asystenta, jeśli dotyczy konkretnego modelu. Sam zrzut sceny pokazuje objaw, ale nie przechowuje pełnego szkicu, historii ani przypięcia źródłowego potrzebnego do odtworzenia błędu.

Objaw	Sprawdź najpierw
Brak odpowiedzi	Proces, adres i port
POST 403	Nowy token i lokalny origin
Zapis 409	Oczekiwaną rewizję
Walidacja 422	issues.code i issues.path
Puste wnętrze	node_id, tryb, stronę i liczniki
Git UNKNOWN	Diagnostykę narzędzia i dostępu

Źródła: S37 Handler.do_GET; Handler.do_POST; main

TEMAT 168

Wersje, tryb offline i granica odpowiedzialności

W paczce występuje kilka niezależnych wersji. Dystrybucja Python ma numer 0.3.1, usługa HTTP zgłasza 3.1, a server_version nagłówka identyfikuje rodzinę LION-Studio/3. Dokument projektu nadal używa lion.polyhedral.studio/v2. Katalog ma lion.architecture/v1, szkic lion.navigation-draft/v1, a proces lion.evolution-process/v1. Numeru wydania aplikacji nie należy mechanicznie wstawiać w pole schema dokumentu, ponieważ zniszczyłoby to jego rozpoznawalność.

pyproject.toml deklaruje Python co najmniej 3.11 i pustą listę zależności uruchomieniowych. Backend, zapis, analiza i serwowanie interfejsu korzystają ze standardowej biblioteki. Git jest osobnym zewnętrznym narzędziem potrzebnym dla operacji repozytoryjnych. Uruchomienie run_studio.py dodaje lokalny katalog src do ścieżki importu, natomiast instalacja pakietu udostępnia polecenie lion-polyhedral-studio. To dwa sposoby uruchomienia tej samej usługi, z możliwie różnymi domyślnymi lokalizacjami workspace.

Podstawowa praca modelera jest lokalna i nie wymaga API modelu językowego. Serwer nie zawiera ukrytego klienta generacyjnego wywołującego AI na podstawie opisu artefaktu. Symulacja generacji oblicza zadeklarowany proces, a materializacja tworzy kandydackie pliki. Jawne klonowanie publicznego GitHub wymaga sieci; kliknięcie odnośnika źródłowego może również otworzyć usługę zewnętrzną. Te działania należy odróżniać od samego rysowania i analizowania dokumentu.

Zapis lokalny nie rejestruje automatycznie dowodu wdrożenia, autorstwa ani uprawnienia do działania. Etykiety osi, actor, owner_ref i opis kontraktu pozostają deklaracjami, dopóki odrębny proces nie zweryfikuje ich podstawy. Granice tej implementacji są częścią prawidłowego użycia Studio: pozwalają modelować ambitną architekturę ewolucyjną bez przedstawiania każdego kandydackiego elementu jako już działającego komponentu federacji.

Przy rozwijaniu backendu zachowuj te rozdzielenia także w testach. Zmiana schematu wymaga jawnej migracji; zmiana usługi nie musi zmieniać formatu danych. Nowy endpoint wykonawczy wymagałby osobnego projektu uprawnień, obserwowalności i śladów, ponieważ obecny token lokalny nie jest systemem tożsamości agentów. Dokumentuj rzeczywisty kod, zakres obserwacji i ograniczenia, aby samoopis aplikacji pozostał użytecznym modelem technicznym.

Warstwa	Identyfikator
Pakiet Python	0.3.1
Usługa health	3.1
Projekt	lion.polyhedral.studio/v2
Katalog	lion.architecture/v1
Szkic	lion.navigation-draft/v1
Proces	lion.evolution-process/v1

Źródła: S09 project.version; project.requires-python; project.scripts; S37 VERSION; Handler.server_version; S16 main launcher

TEMAT 169

Dwie warstwy implementacji: mechanizm R1 i Studio R3.1

Repozytorium zawiera dwie powiązane, lecz odrębne implementacje. Pakiet lion_polyhedral jest referencyjnym demonstratorem fundamentu kandydującego R1: niezmiennych deskryptorów artefaktów, granic kompozycji, przyjętych połączeń, reduktora zmian i prostego archiwum dowodów. Pakiet lion_studio realizuje aplikację edycyjną R3.1: projekty, scenę 3D, katalog architektury, nawigację, symulacje, harmonogramy i historię. Wspólna geometria nie oznacza identycznej serializacji ani jednakowych reguł wszystkich pól.

Obszar	Format lub mechanizm	Przeznaczenie
R1: artefakt	lion.polyhedral.artifact.candidate.v1	Niezmienna specyfikacja
R1: kompozycja	lion.polyhedral.mosaic.candidate.v1	Przenośny snapshot
R1: wiadomość	lion.polyhedral.message.candidate.v1	Propozycja delty offline
Studio	lion.polyhedral.studio/v2	Edytowalny dokument projektu

Nazwy ArtifactSpec i FaceRole należy w tym kontekście traktować jako określenia pojęciowe. W referencyjnym kodzie R1 istnieją klasa Artifact i stałe nazw ścian, a nie klasy o takich nazwach. Natomiast pola kafli, bramek i parametrów dokumentu Studio są implementowane przez model edytora. Rozszerzenie jednej warstwy trzeba sprawdzić w jej walidatorze; skopiowanie pola do drugiego formatu nie tworzy automatycznie poprawnej konwersji.

Przed zmianą programista ustala, czy modyfikuje semantykę specyfikacji, dokument projektu, projekcję katalogu, czy wyłącznie prezentację. Przesunięcie bryły może należeć do layoutu. Zmiana schematu portu dotyczy kontraktu. Wczytanie następnej strony dzieci jest nawigacją. Każdy z tych przypadków ma inny zakres testowania i inną historię. Dzięki temu poprawka renderingu nie staje się przypadkową migracją danych.

1. Uruchom run_checks.py z katalogu rozpakowanego projektu.
2. Dla zachowania R1 odczytaj model.py i test_candidate.py.
3. Dla zachowania edytora sprawdź model.py pakietu lion_studio oraz odpowiedni test_studio_*.py.
4. Opisz wynik jako lokalny prototyp lub funkcję Studio; integrację produkcyjną LION potwierdzaj oddzielnymi źródłami.

Kontrakty fundamentu mają status propozycji i wykonywalnej referencji offline. Nie uruchamiają istniejących brokerów federacji i nie nadają uprawnień przez sam fakt utworzenia poprawnego deskryptora. Dokumentacja powinna zachowywać tę granicę także wtedy, gdy aplikacja przedstawia samą siebie jako artefakt.

Podczas zgłaszania problemu podaj wersję pakietu z pyproject.toml, format otwartego dokumentu i nazwę operacji. Błąd demonstratora R1 odtwarza się innym wejściem niż błąd nawigacji katalogu. Dołączenie minimalnego pliku oraz pełnego komunikatu walidacji pozwala wskazać właściwy moduł i uniknąć naprawy wykonywanej w niewłaściwej warstwie.

Źródła: S26 Artifact, PROFILE, FACES; S33 SCHEMA, migrate_workspace; S18 Status i granice wniosku

TEMAT 170

Tożsamość logiczna, rewizja i digest semantyczny

Artifact przechowuje semantykę oraz wygląd jako kanoniczne ciągi JSON. Odczyt właściwości semantic zwraca nowy słownik, więc zmiana zagnieżdżonej struktury przez wywołującego nie podmienia stanu istniejącego obiektu. Sama klasa jest zamrożona. To prosta metoda utrzymania niezmienności specyfikacji bez udostępniania modyfikowalnego słownika jako źródła prawdy.

artifact_id określa ciągłość logiczną. revision nazywa konkretną wersję specyfikacji, a semantic_digest wiąże jej kanoniczną treść. Funkcja canonical sortuje klucze słowników, zachowuje znaki Unicode, usuwa zbędne odstępy i odrzuca NaN oraz Infinity. Digest jest SHA-256 bajtów UTF-8 tej reprezentacji. Kolejność elementów list pozostaje znacząca: kod nie normalizuje automatycznie każdej listy do zbioru.

```
from lion_polyhedral.model import Artifact

base = Artifact.create('example:document', 'r1', 'document')
view = base.with_display(position=[4, 0, 2], display_scale=2)
assert base.semantic_digest == view.semantic_digest
assert base.ref == view.ref
```

W przykładzie zmienia się położenie i skala prezentacji, lecz referencja semantyczna pozostaje identyczna. Przy uruchamianiu takiego fragmentu z katalogu repozytorium ustaw katalog src na ścieżce importów lub zainstaluj projekt w wybranym środowisku. Procedura nie wymaga uruchamiania serwera ani otwierania przeglądarki.

Registry odrzuca próbę dodania innej treści pod istniejącą parą artifact_id i revision. Zmiana pojemności portu, kontraktu albo dzieci wymaga zatem nowej rewizji. Pole payload ma osobne ref i bytes_sha256: hash deskryptora nie zastępuje kontroli bajtów wskazanego pliku. Samo utworzenie deskryptora nie odczytuje payloadu i nie dowodzi, że adresowany plik rzeczywiście istnieje.

Przy diagnozie rozbieżności najpierw porównaj tożsamość logiczną, następnie rewizję, następnie digest specyfikacji. Dopiero potem porównuj materialne bajty i wygląd. Rejestr nie rozwiązuje nazwy do najnowszej dostępnej wersji. Takie rozdzielenie pozwala zachować historyczne odniesienia po przeniesieniu pliku, ale wymaga świadomego aktualizowania jego lokalizatora oraz sprawdzenia konsumentów starej ścieżki.

W serializacji to_dict jawnie występują semantic, semantic_digest oraz display. Konstruktor from_dict przelicza digest i porównuje go z zadeklarowanym. To właściwa droga importu obcego deskryptora; tworzenie obiektu z pominięciem sprawdzenia nie powinno zastępować kontrolowanego odczytu danych dostarczonych przez inny proces.

Źródła: S26 canonical, digest, Artifact, Registry.add; S46 ArtifactTests

TEMAT 171

Profil czternastu ścian jako stabilny słownik ról

Profil R1 ma identyfikator truncated-octahedron-14. Sześć ścian kwadratowych tworzy stały rdzeń, a osiem sześciokątnych grupuje funkcje. Adres ściany wynika z jej nazwy, a nie z kolejności rysowania, koloru ani orientacji kamery. Każdy deskryptor zawiera dokładnie czternaście wpisów faces. Nieznana nazwa, brak wymaganej ściany albo niezgodny kształt powodują odrzucenie specyfikacji.

Ściana rdzenia	Odpowiedzialność
identity	Tożsamość logiczna i rewizja
contract	Schematy i reguły zgodności
boundary	Publiczna granica kompozycji
provenance	Pochodzenie i odnośniki dowodowe
lifecycle	Etapy istnienia
observability	Obserwacje i ślady

Ściana funkcjonalna	Rodzina interfejsów
input / output	Przyjmowanie i publikowanie danych
context / knowledge	Kontekst odbiorcy i dostęp do wiedzy
cognition / evolution	Hipotezy oraz propozycje zmian
composition / routing	Składanie i odkrywanie połączeń

Artifact.create aktywuje rdzeń, a ściany funkcjonalne uaktywnia wtedy, gdy otrzymują porty. Nieaktywna ściana nie może zawierać portów. Walidator sprawdza te reguły strukturalne; nie implementuje pełnego systemu pamięci, cyklu życia czy rozumowania tylko dlatego, że istnieje odpowiednia nazwa. Funkcję konkretnej realizacji określa jej kontrakt i payload.

Profil pozwala opisać dokument bez wymyślania mu wejść wykonawczych, a brokera z wieloma portami bez ograniczania go do czternastu odbiorców. Jedna ściana może grupować wiele portów, a każdy port może mieć określoną pojemność. Liczba ścian jest konwencją porządkującą metadane, nie liczbą równoległych procesów ani dowodem kompletności taksonomii.

Przy projektowaniu nowej rodziny artefaktów wybierz najpierw istniejące role i określ znaczenie portów. Jeżeli dwie role stale się pokrywają, zapisz problem jako ograniczenie profilu. Zmiana nazw stabilnych ścian wymaga nowej wersji profilu oraz mapy migracji adresów. Samo przemalowanie figury nie zmienia nazwy ściany ani semantycznego kontraktu połączenia. Warto sprawdzić osobno artefakt dokumentu, kontraktu, brokera i całej aplikacji; różnorodność tych przykładów ujawnia luki, których pojedynczy model demonstracyjny nie pokaże.

Dla czytelności opisuj aktywność niezależnie od obecności bryły na scenie. Niewykorzystywana funkcja może pozostać nieaktywną fasetą deskryptora. Ukrycie jej etykiety w interfejsie użytkownika nie powinno usuwać wymaganego wpisu z formatu. Test poprawnego pustego artefaktu jest równie ważny jak test rozbudowanego brokera.

Źródła: S26 PROFILE, CORE_FACES, FUNCTION_FACES, SHAPES, Artifact.create; S18 Profil faset 6 + 8

TEMAT 172

Port i endpoint: dokładny adres końca relacji

Port opisuje miejsce przyłączenia do konkretnej ściany. Funkcja port tworzy rekord z polami id, direction, schema, contract_ref, capacity oraz role. Identyfikator jest unikatowy w obrębie danej ściany. Kierunek przyjmuje in, out lub duplex. Pojemność jest dodatnią liczbą całkowitą i oznacza liczbę równocześnie zadeklarowanych połączeń w jednym Registry. Nie jest przepustowością w wiadomościach na sekundę.

```
from lion_polyhedral.model import Artifact, Endpoint, port

worker = Artifact.create(
    'worker', 'r1', 'model',
    face_ports={'input': [
        port('job', 'in', 'schema:job:v1', 'contract:flow:v1')
    ]}
)
target = Endpoint('worker', 'r1', 'input', 'job')
```

Endpoint zawiera artifact_id, revision, face i port. Jego konstruktor sprawdza poprawność podstawowych wartości i nazwę ściany. Pełne rozstrzygnięcie następuje w Registry: rejestr musi znaleźć wskazaną rewizję artefaktu, aktywną ścianę oraz istniejący port. Poprawnie sformatowany adres może zatem nadal wskazywać obiekt nieobecny w aktualnym snapshotcie.

Pole	Co ustala	Czego nie ustala
schema	Nazwę schematu danych	Walidacji dowolnego payloadu
contract_ref	Wspólny kontrakt połączenia	Podpisu lub zgody operatora
role	Rodzinę data/context/evidence	Wiarygodności przesłanej treści
capacity	Limit deklarowanych wiązań	Fizycznej wydajności procesu

W modelu R1 zgodność schematów opiera się na równości identyfikatorów, a nie na automatycznym dowodzeniu podtypowania. Dwa odmiennie nazwane schematy nie są uznawane za zgodne nawet wtedy, gdy człowiek dostrzega podobieństwo. Konwersję należy opisać osobnym adapterem i sprawdzić jej zachowanie, zamiast obchodzić porównanie nazw.

Podczas przeglądu błędnego połączenia odczytaj oba pełne adresy. Upewnij się, że źródło emituje, cel przyjmuje, schematy oraz kontrakty są takie same, a role odpowiadają typowi relacji. Następnie zbadaj granicę kompozycji i użycie pojemności. Przeniesienie bryły bliżej drugiej nie naprawi żadnego z tych warunków; potrzebna jest zmiana deskryptora lub jawnie nowa rewizja.

Pole role domyślnie przyjmuje data. Dla relacji kontekstowej lub dowodowej trzeba zadeklarować odpowiednią rolę obu końców; sama nazwa ściany context albo provenance tego nie robi. Walidacja odnosi się do rekordów portów, dzięki czemu interfejs nie wywodzi znaczenia wyłącznie z wyglądu lub miejsca na wielościanie.

Źródła: S26 port, _validate_port, Endpoint, Registry.endpoint_port; S46 ConnectionTests

TEMAT 173

Rejestr połączeń i strukturalny handshake

Link nadaje połączeniu własne link_id oraz wskazuje source, target, accepted_by, relation_type i contract_ref. Rejestr traktuje zgodę obu końców jako wymaganą deklarację strukturalną. Lista accepted_by musi zawierać dokładnie dwa różne endpointy odpowiadające źródłu i celowi. Jednostronna akceptacja nie wystarcza, nawet jeżeli pozostałe pola są zgodne.

Registry.connect najpierw waliduje istniejącą strukturę, a następnie próbuje dołączyć nowe wiązanie. Ten porządek wykrywa sytuację, w której wcześniejsza zmiana kontenera naruszyła warunki już zaakceptowanych tras. Nowe połączenie jest zapisywane dopiero po przejściu wszystkich kontroli; nie ma częściowo zaakceptowanego linku pozostawionego w rejestrze po zwykłym błędzie walidacji.

1. Rozstrzygnij oba endpointy do dokładnych rewizji i portów.

2. Sprawdź role portów względem dataflow, context albo evidence.

3. Porównaj kontrakt linku z kontraktami obu portów.

4. Potwierdź wspólny bezpośredni kontener i właściwy kierunek.

5. Porównaj schematy, sprawdź budżety pojemności i zapisz link.

Ponowne dodanie identycznego Link o tym samym ID jest idempotentne. Próba wykorzystania tego ID do innego wiązania kończy się kolizją. Inne ID nie pozwala jednak obejść limitu portu: pojemność jest liczona według rzeczywistych końców i ich aliasów, a nie liczby unikatowych nazw w formularzu.

Akceptacja w tym prototypie nie stanowi uwierzytelnionego podpisu. accepted_by jest danymi przekazanymi do procesu, nie wynikiem rozmowy z dwoma niezależnymi usługami. Registry działa w jednym lokalnym snapshotcie i nie zapewnia rozproszonej rezerwacji, odnawialnych dzierżaw ani zgodności między kilkoma brokerami. Docelowy runtime musiałby połączyć sprawdzenie strukturalne z istniejącym mechanizmem tożsamości i dopuszczenia efektu.

Najbardziej użyteczny test ręczny polega na utworzeniu poprawnej pary, a następnie zmianie jednego warunku naraz: usuń zgodę celu, odwróć kierunek, zmień schemat lub spróbuj drugiego linku ponad capacity. Każda odmowa powinna pozostawić liczbę połączeń bez zmian. Takie próby wyjaśniają kontrakt skuteczniej niż samo obejrzenie poprawnie narysowanej linii.

W przypadku uszkodzonej topologii Registry.validate odtwarza dotychczasowe linki w izolowanym rejestrze. Powtórne sprawdzenie obejmuje również współdzielone aliasy. Programista rozszerzający warunki zgodności powinien zachować tę własność: walidacja nowego linku i walidacja istniejącego snapshotu muszą stosować te same reguły, a nie dwie rozbieżne implementacje.

Źródła: S26 Link, Registry.connect, Registry._connect_validated_structure; S46 ConnectionTests.test_one_sided_handshake_rejected

TEMAT 174

Granica kompozycji, eksport i wspólna pojemność aliasów

Złożony artefakt udostępnia wnętrze przez exposed_ports. Każde mapowanie wskazuje publiczną parę face i port oraz child_endpoint. Dziecko musi należeć bezpośrednio do children danego kontenera. Przekierowanie do dowolnego odległego obiektu nie spełnia kontraktu eksportu. Zagnieżdżenie wielopoziomowe uzyskuje się przez kolejne poprawne granice, a nie przez pominięcie pośrednich kontenerów.

Publiczny port i port dziecka muszą mieć identyczny kierunek, schemat, contract_ref i role. Pojemność publiczna nie może przekraczać pojemności wewnętrznej. Pozwala to celowo udostępnić mniejszy budżet, lecz blokuje deklarowanie większej dostępności niż posiada realizacja. Mapowanie nie może też przemianować zwykłych danych na dowód; równość role jest sprawdzana niezależnie od schematu.

Przypadek	Wynik walidacji	Przyczyna
Port rodzica → własne dziecko	Dopuszczalny przy zgodności	Jawna granica
Port rodzica → wnuk bez pośrednika	Odrzucony	Cel nie jest bezpośrednim dzieckiem
Dwa eksporty tego samego portu	Wspólny budżet końca	Alias nie mnoży zasobu
Broker → ukryty port dziecka	Odrzucony bez wspólnego scope	Ominięcie granicy

Registry._alias_path rozwija endpoint publiczny przez wszystkie mapowania aż do końcowego portu. Przy ocenie capacity zlicza wykorzystanie każdego adresu po drodze. Dwa publiczne aliasy tego samego wewnętrznego wejścia dzielą zatem jego limit. Identycznie działa wielopoziomowy eksport: ograniczenie może pochodzić z portu pośredniego, nawet gdy końcowy port ma większą pojemność.

Zawieranie jest sprawdzane pod kątem cyklu. Każda referencja dziecka wiąże ID, rewizję i semantic_digest, więc zmiana specyfikacji dziecka bez aktualizacji granicy powoduje niezgodność. Rejestr ponownie sprawdza trasy po dodaniu kontenera, gdyż taki dodatek może zmienić wspólne zakresy istniejących endpointów.

Przy rozbudowie aplikacji modeluj granicę najpierw dla jednej pary: zewnętrzny broker, kontener i wewnętrzny worker. Dopiero potem dodaj drugi alias i drugi poziom zagnieżdżenia. Oczekiwanym wynikiem jest zachowanie poprawnej pierwszej trasy oraz odmowa połączenia omijającego granicę lub przekraczającego wspólną pojemność. Poprawny układ graficzny powinien odzwierciedlać ten wynik, a nie go zastępować.

Jedno publiczne gniazdo nie może wskazywać dwóch różnych dzieci. Jeśli potrzebny jest rozdzielacz, opisz go jako osobny artefakt z własnym kontraktem i portami. W ten sposób wybór odbiorcy oraz reguły podziału przepływu stają się widocznym elementem architektury, zamiast niejawnej właściwości aliasu.

Źródła: S26 Registry._validate_structure, Registry._alias_path, Registry._scopes; S46 test_nested_boundary_is_used_and_external_bypass_rejected, test_two_public_aliases_share_child_capacity

TEMAT 175

Mosaic: przenośny snapshot i osobna tożsamość instancji

Mosaic jest zamkniętą serializacją referencyjnej kompozycji R1. Zawiera schema, root, artifacts i links. Referencja root wskazuje konkretną specyfikację korzenia, a każdy artefakt przenosi własną semantykę, digest i display. Konstruktor from_dict tworzy rejestr, rozstrzyga korzeń, waliduje zawieranie i kolejno odtwarza połączenia. Nie wystarcza więc samo posiadanie poprawnego JSON: jego odnośniki muszą tworzyć spójny snapshot.

```
from lion_polyhedral.demo import fixture
from lion_polyhedral.model import Mosaic

mosaic, artifacts = fixture()
replayed = Mosaic.from_dict(mosaic.decompose())
assert replayed.decompose() == mosaic.decompose()
assert replayed.digest == mosaic.digest
```

compose przyjmuje korzeń, kolekcję artefaktów oraz linki i prowadzi dane przez ten sam walidator. Nie tworzy dzieci wyłącznie na podstawie listy dostarczonych obiektów: relacje children pozostają jawnie zapisane w deskryptorach. Konstruktor odrzuca powtórzony artefakt lub link w serializacji. decompose zwraca odtworzoną strukturę danych, umożliwiając eksport i porównanie bez renderowania.

Digest Mosaic obejmuje cały kanoniczny dokument kompozycji, w tym serializowany wygląd artefaktów. Dlatego przesunięcie bryły może zachować semantic_digest pojedynczego Artifact, a jednocześnie zmienić digest snapshotu Mosaic. Przy porównaniu wyników zawsze nazwij sprawdzany poziom integralności: specyfikację, payload albo pełny dokument.

RuntimeBinding służy do wskazania instancji niezależnej od specyfikacji. Łączy instance_id, artifact_id, revision, semantic_digest i observation_ref. Utworzenie tego rekordu niczego nie uruchamia. Pole observation_ref jest odnośnikiem, który dopiero wymaga właściwego dowodu obserwacji. Demonstrator wykorzystuje nazwę offline-instance-001, opisując lokalny przebieg, a nie produkcyjną instancję brokera.

Próba roundtrip potwierdza zachowanie serializacji oraz więzów referencyjnych. Nie dowodzi równoważności zachowania programu po podziale modułu ani poprawności operacyjnej każdego kontraktu. Aby przenieść ten wzorzec do refaktoryzacji LION, dołącz testy konsumentów zmienianych interfejsów i dokładne powiązanie wyników z rewizją kodu. Snapshot jest wtedy przenośnym nośnikiem modelu, a nie zastępczym dowodem wykonania całej aplikacji.

Przed zapisaniem kompozycji sprawdź także, czy wszystkie potrzebne obiekty zostały rzeczywiście dostarczone do compose. Referencja do brakującego dziecka zostanie odrzucona. Korzeń może opisywać całą aplikację, ale obecność takiej powłoki nie oznacza automatycznie, że zawarte obiekty mają wspólnego właściciela operacyjnego albo dziedziczą jego uprawnienia.

Źródła: S26 Mosaic, compose, RuntimeBinding; S46 test_roundtrip_preserves_nested_structure_links_and_digests

TEMAT 176

Wiadomość zmiany i ograniczony reduktor offline

delta_message tworzy wiadomość lion.polyhedral.message.candidate.v1 z operacją propose_delta. Wiadomość wskazuje message_id, artifact_id, base_revision, next_revision, context, changes i payload_bytes_sha256. Funkcja kopiuje changes, więc późniejsza modyfikacja słownika przekazanego przez wywołującego nie podmienia przygotowanej treści. Hash payloadu dotyczy kanonicznych bajtów UTF-8 samych changes.

OfflineReducer przechowuje current, historię specyfikacji, znane konteksty oraz pamięć wiadomości. Dopuszcza wyłącznie zmiany contract_ref, faces i payload. Nie służy do dowolnego edytowania tożsamości albo struktury children. Topologia wymaga pełnej walidacji Mosaic, dlatego operacja delty ma świadomie węższy zakres niż całość deskryptora.

Warunek	Odpowiedź lub błąd	Zmiana current
Zgodna baza i kontekst	candidate_applied_offline	Nowa specyfikacja
Inna baza lub artefakt	conflict	Brak
Brak dokładnego kontekstu	needs_context	Brak
Ten sam ID i treść po wyniku	duplicate	Brak
Ten sam ID, inna treść	Invalid: kolizja	Brak

Nowa rewizja musi być niepusta, różna od bazowej i wcześniej nieużyta w historii reduktora. Po zaakceptowaniu zmiany konstruowany jest nowy Artifact, więc obowiązują wszystkie niezmienniki ścian i portów. Wynik zawiera effect równe local_candidate_only. To wynik przetworzenia lokalnego kandydata, nie potwierdzenie materializacji pliku ani wysłania wiadomości przez broker.

Zmiany zawierające nazwę klucza obejmującą authority są odrzucane także w zagnieżdżonych słownikach i listach. Ten strażnik rozdziela propozycję semantyczną od uprawnień. Nie należy interpretować go jako pełnego systemu autoryzacji; jest ograniczoną regułą demonstratora, sprawdzaną przed utworzeniem kolejnej specyfikacji.

W rozszerzeniu protokołu każda nowa operacja powinna otrzymać zdefiniowaną bazę, wynik terminalny i warunki ponowienia. Dodanie nieznanych pól do istniejącego kontraktu CommunicationEnvelope nie jest przewidzianą metodą integracji. Specyfikacja proponuje typowany payload lub adapter, zachowujący dotychczasowy kontrakt koperty. R1 nie realizuje takiego podłączenia do działającego LION; implementuje jedynie kontrolowaną logikę lokalnego przejścia.

W czasie debugowania zachowaj poprzedni Artifact i sprawdź, że jego digest nie uległ zmianie po apply. Reduktor tworzy następcę, więc historyczna specyfikacja pozostaje dostępna. Jeśli zmiana faces jest niekompletna albo zawiera niepoprawny port, walidacja konstruktora przerwie operację; nie należy traktować błędu jak poprawnie zmaterializowanej nowej rewizji.

Źródła: S27 delta_message, OfflineReducer.apply, _has_authority_key; S18 Protokół

TEMAT 177

Kontekst, ponowienia i rozwiązywanie konfliktów wiadomości

Wiadomość R1 wiąże dwa odrębne hashe kontekstu: shared_context_digest oraz projection_digest. Pierwszy określa współdzielony materiał odniesienia, drugi jego projekcję dla odbiorcy. context.ref jest kluczem do dokładnie takiej pary zapisanej w reduktorze. Sama obecność podobnej nazwy kontekstu albo znajomość tylko jednego digestu nie wystarcza do zastosowania delty.

Po needs_context reduktor zapamiętuje fingerprint wiadomości, ale nie zapisuje terminalnego rezultatu. Można uzupełnić contexts i ponowić dokładnie tę samą wiadomość. Zmiana jej zawartości pod starym message_id powoduje kolizję, nawet jeśli pierwsza próba nie wykonała zmiany. Korekta treści propozycji wymaga nowego ID i ponownego uzgodnienia bazy.

1. Odbierz needs_context i zachowaj oryginalną wiadomość.

2. Uzyskaj wskazaną parę digestów z właściwego źródła.

3. Uzupełnij kontekst pod dokładnym ref; nie podstawiaj innej projekcji.

4. Ponów tę samą treść oraz message_id.

5. Po rezultacie terminalnym kolejne identyczne próby traktuj jako duplicate.

Konflikt stale_base_or_wrong_artifact oznacza, że propozycja nie dotyczy aktualnego artefaktu lub jego rewizji. Reduktor zwraca bieżącą referencję current. Nie przepisuje zmian automatycznie na nową bazę, gdyż mogłoby to odtworzyć nieaktualne założenia. Operator lub nadrzędny proces musi przeanalizować różnicę i przygotować nową propozycję.

Wyniki terminalne są buforowane. Po poprawnym zastosowaniu wiadomości duplikat nie tworzy drugiej rewizji i zwraca kopię pierwotnego wyniku. Także zapamiętany konflikt pozostaje terminalnym wynikiem tego ID. Mechanizm jest lokalny i działa w pamięci jednego obiektu OfflineReducer; restart procesu usuwa tę pamięć, jeżeli zewnętrzna aplikacja nie zapewni własnego trwałego zapisu.

Do diagnostyki zapisz pełne ID wiadomości, bazę, docelową rewizję, oba hashe kontekstu i status odpowiedzi. Zachowaj też dokładne wejście, ponieważ porównanie samych opisów tekstowych nie wykryje każdej kolizji. Testy rozdzielają niezgodny payload digest, brak kontekstu, powtórzenie i nieaktualną bazę. Takie rozróżnienie jest konieczne, aby mechanizm ponowień nie uznał prośby o kontekst za wykonanie operacji ani konfliktu za błąd transportu.

Włączenie trwałej deduplikacji byłoby oddzielnym rozszerzeniem: trzeba związać zapis wiadomości, wynik i zmianę stanu w spójną transakcję. Samo skopiowanie słownika seen do pliku po wykonaniu operacji pozostawia okno awarii. Obecny prototyp nie deklaruje takiej gwarancji, dlatego test powtórzenia dotyczy jednego działającego reduktora.

Źródła: S27 OfflineReducer.apply; S46 ReducerTests

TEMAT 178

EvidenceArchive: integralność łańcucha i granice dowodu

EvidenceArchive zapisuje kolejne rekordy JSONL. Każdy zawiera index, previous, payload i record_sha256. Pierwszy rekord wskazuje głowę GENESIS złożoną z sześćdziesięciu czterech zer. Hash obejmuje indeks, poprzednią głowę i payload. verify odczytuje cały plik, sprawdza zamknięty zestaw pól, kolejność indeksów, połączenie ogniw i zgodność hashy.

```
from lion_polyhedral.evidence import EvidenceArchive

archive = EvidenceArchive('run-output/manual_evidence.jsonl')
record = archive.append({'event': 'manual_check', 'result': 'PASS'})
head = record['record_sha256']
state = archive.verify(expected_head=head)
assert state['externally_anchored'] is True
```

Zmienna head w przykładzie musi zostać zachowana poza kontrolowanym plikiem, jeżeli ma wykrywać jego skrócenie lub całkowite przepisanie. Sam argument expected_head oznacza porównanie z oczekiwaną wartością; kod nie zapewnia niezależnego magazynu kotwicy ani zaufania do osoby podającej ten argument. Bez kotwicy spójny, ponownie przeliczony łańcuch może przejść walidację.

append najpierw weryfikuje istniejący stan, dopisuje rekord w UTF-8 z pojedynczym znakiem końca linii i ponownie sprawdza nową głowę. Uszkodzony JSON, brak pola, zła kolejność czy zmiana payloadu prowadzą do Invalid. API jest dopisujące, ale użytkownik posiadający dostęp do pliku nadal może zmienić jego zawartość poza tym API.

Implementacja nie ma blokad wieloprocesowych, fsync ani rozproszonego konsensusu. Wynik verify jawnie informuje, że autentyczność nie została ustalona oraz brak gwarancji trwałości zapisu. Z tego powodu archiwum demonstratora nadaje się do wyjaśnienia integralności lokalnej sekwencji, a nie do zastąpienia produkcyjnego systemu dowodów i retencji.

Pliki results dostarczone w paczce są wygenerowanymi przykładami użytkownika. Ich zawartość może różnić się od wcześniejszego manifestu po ponownym uruchomieniu demonstratora. Nie należy na tej podstawie twierdzić, że bieżący kod został właśnie przetestowany albo że wykryto ingerencję. Do nowej próby użyj osobnego katalogu wyjściowego, zapisz komendę, środowisko i głowę łańcucha, a następnie powiąż raport z dokładną rewizją programu.

Nie zapisuj ręcznie nowej wartości record_sha256, aby ukryć błąd poprzedniego ogniwa. Najpierw zabezpiecz oryginalny plik i ustal, który rekord przestał odpowiadać zachowanej głowie. Korekta analizy powinna powstać jako nowa informacja z przyczyną i odniesieniem do wcześniejszego wyniku, nie jako cicha podmiana historycznego payloadu.

TEMAT 179

Graf wpływu, cykle wiedzy i falsyfikacja mechanizmu

DependencyGraph w referencyjnym R1 przechowuje jawnie zadeklarowane węzły i krawędzie. Nie analizuje automatycznie całego repozytorium. Węzeł ma kind, ref i status, a relacja typ knowledge albo execution. Dodawanie nieistniejących końców oraz nieznanego typu kończy się błędem. Wszystkie powiązania demonstracji są zatem modelem podanym przez autora przykładu.

Krawędzie knowledge mogą tworzyć cykle. Algorytm osiągalności wykorzystuje kolejkę i zbiór odwiedzonych, dlatego sprzężenie zwrotne nie prowadzi do nieskończonej rekursji. Dla execution sprawdzana jest osiągalność w tej samej projekcji: dodanie krawędzi zamykającej cykl zostaje odrzucone przed zmianą grafu. To rozróżnienie oddziela pętlę wiedzy od konkretnego porządku wykonania.

```
from lion_polyhedral.evolution import DependencyGraph

graph = DependencyGraph()
for name in ('contract', 'code', 'test'):
    graph.add_node(name, name, 'example:' + name)
graph.add_edge('contract', 'code')
graph.add_edge('code', 'test')
impact = graph.impact('contract', 'changed input schema')
```

impact oznacza zmieniony węzeł i wszystkie osiągalne cele jako needs_revalidation. Zapisuje numer sekwencji, przyczynę, uporządkowany zbiór affected oraz kopię poprzedniego stanu. Nie uruchamia testów, nie przepisuje kodu i nie stwierdza fałszu wszystkich twierdzeń. Wynik jest sygnałem zakresu ponownej oceny zgodnym z zadeklarowanym kierunkiem krawędzi.

1. Zmień kontrakt i sprawdź, czy model wskazuje oczekiwanych konsumentów.
2. Dodaj cykl knowledge i upewnij się, że analiza kończy pracę.
3. Spróbuj cyklu execution; stan krawędzi powinien pozostać niezmieniony.
4. Zachowaj historyczny wynik testu starej rewizji, oznaczając oddzielnie jego zastosowanie do nowej.

W specyfikacji migracji opisano bogatszy docelowy graf: wymagania, kontrakty, symbole, definicje testów, przebiegi i dowody. Takie typy oraz reguły wrażliwości nie są w całości implementowane przez prostą klasę DependencyGraph. Przy rozwoju dodawaj je jawnie, zachowując przyczynę każdej propagacji. Powtórne zacytowanie tego samego root_ref nie powinno tworzyć nowego niezależnego dowodu, a zmiana samego układu 3D nie powinna unieważniać kontraktów wykonawczych.

Obecny impact używa wszystkich osiągalnych krawędzi obu dopuszczonych typów. Nie rozróżnia jeszcze zmiany komentarza od zmiany publicznego interfejsu. Jeżeli wynik wydaje się zbyt szeroki, najpierw oceń jakość zadanego grafu. Dopiero później wprowadzaj predykaty wrażliwości, obejmując testem przypadek zmiany oraz przypadku pozostającego poza wpływem.

TEMAT 180

Demonstrator R1: pełny przebieg lokalnego eksperymentu

run_demo.py jest przenośnym wejściem do lion_polyhedral.demo. Dodaje katalog src do ścieżki importów i uruchamia main. Najbezpieczniej wykonać demonstrację w osobnym katalogu wyników, aby nie mieszać nowego przebiegu z historycznymi przykładami results. Nie jest potrzebny serwer Studio, konto zewnętrzne ani połączenie z brokerem.

```
.\.venv\Scripts\python.exe run_demo.py --output run-output
Get-Content run-output\demo_result.json -Encoding UTF8
```

fixture tworzy siedem artefaktów: assumption, contract, evidence, worker, component, broker i application. Worker znajduje się wewnątrz component. Jego wejście job jest eksportowane przez publiczną granicę komponentu. Broker łączy output.dispatch z component.input.job. Korzeń application zawiera elementy najwyższego poziomu, a pełna kompozycja zachowuje jedno zaakceptowane połączenie.

Następnie demonstrator tworzy reduktor dla assumption, przygotowuje rewizję r1 do r2 oraz zmianę contract_ref z contract:base:v1 na contract:base:v2 i stosuje ją dwukrotnie. Pierwszy wynik powinien być candidate_applied_offline, drugi duplicate. Graf przykładowy propaguje wpływ od założenia przez kontrakt, kod, testy, dowód i projekcję. Sprzężenie projekcji do założenia jest poprawnym cyklem knowledge.

Plik wyjściowy	Treść kontrolna
mosaic_fixture.json	Artefakty, granice i połączenie
message_candidate.json	Wiadomość delty z kontekstem
impact_graph.json	Węzły, relacje i historia wpływu
demo_evidence.jsonl	Dwa zdarzenia dopisane do archiwum
demo_result.json	Podsumowanie i wynik roundtrip

W podsumowaniu oczekuj PASS_OFFLINE_DEMONSTRATOR, artifact_count równego 7, accepted_routes równego 1 i roundtrip_equal równego true. Pole live_lion pozostaje NOT_TESTED. Zawarty RuntimeBinding jest opisem instancji demonstracyjnej. Zgoda portów ma charakter strukturalny; kod nie uwierzytelnia usług i nie nadaje uprawnień.

Ponowne uruchomienie w tym samym katalogu dopisze kolejne rekordy do istniejącego archiwum JSONL, zamiast zaczynać dowód od pustego pliku. Dlatego liczba rekordów może rosnąć mimo identycznego modelu. Dla porównania dwóch niezależnych prób wybierz dwa nowe katalogi, zachowaj oba zestawy plików i porównuj odpowiednie digests. Sama liczba udanych przebiegów lokalnego przykładu nie rozszerza jego zakresu na produkcyjną federację.

W razie błędu przeczytaj wyjątek przed ponownym uruchomieniem, ponieważ częściowo istniejący katalog wyjściowy może zawierać pliki poprzedniej próby. Zachowaj je osobno. Powtarzalny scenariusz kontrolny powinien wskazywać ścieżkę interpretera, wersję Pythona i nowy katalog, tak aby inny programista mógł odtworzyć ten sam ograniczony eksperyment bez dostępu do Twojego workspace.

Źródła: S15 __main__; S23 fixture, impact_fixture, run, main

TEMAT 181

Układ repozytorium i punkty uruchomienia programu

Katalog projektu łączy źródła aplikacji, narzędzia utrzymaniowe, specyfikacje i raporty. Nie każdy plik jest częścią uruchamianego serwera. Podczas przeglądu zależności warto oddzielić kod importowany przez proces Studio, samodzielne narzędzia CLI oraz historyczne rezultaty. Taki podział pozwala wskazać rzeczywiste miejsce poprawki bez automatycznego przenoszenia całej dokumentacji do katalogu implementacji.

Ścieżka	Odpowiedzialność
src/lion_studio	Serwer, model, katalogi, nawigacja i interfejs WWW
src/lion_polyhedral	Referencyjny fundament R1
tools	Indeksowanie, geometria i budowanie pakietu
tests	Regresja zachowania aplikacji
specs oraz docs	Formaty, kontrakty i objaśnienia
reports oraz results	Raporty wydania i wygenerowane przykłady
workspace	Lokalny stan użytkownika

pyproject.toml określa nazwę lion-polyhedral-studio, wersję 0.3.1 i wymaganie Python co najmniej 3.11. Lista zależności wykonawczych jest pusta. Pakowanie korzysta z setuptools, a dane pakietu lion_studio obejmują pliki web oraz katalogi JSON i JSON.GZ. Pusty zestaw zależności nie oznacza braku wymagań środowiskowych: Git jest potrzebny do funkcji repozytoryjnych, a przeglądarka do edycji graficznej.

Dwa skrypty konsolowe instalowanego pakietu to lion-polyhedral-studio i lion-polyhedral-demo. W rozpakowanym projekcie odpowiednikami są run_studio.py oraz run_demo.py. run_checks.py zbiera testy i sprawdza geometrię. Uruchamianie tych plików przez wybrany interpreter w PyCharm ułatwia utrzymanie jednego środowiska zamiast przypadkowego mieszania systemowego Pythona z .venv.

```
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe run_studio.py --no-browser
```

W konfiguracji PyCharm wybierz run_studio.py jako plik skryptu, a katalog repozytorium jako katalog roboczy. Parametry dodawaj w polu argumentów skryptu. Jeżeli projekt jest zainstalowany jako pakiet bez lokalnego launchera, serwer wybiera domyślny workspace w katalogu domowym użytkownika. Dlatego w procedurach odtwarzania problemu zapisuj także argument --workspace: ta sama wersja programu może otworzyć inny zestaw projektów zależnie od sposobu uruchomienia.

Nie oceniaj roli pliku wyłącznie po położeniu w drzewie. specs zawiera normatywne propozycje i formaty, a reports odwołuje się do konkretnych prób historycznych. Kod narzędzia może opisywać większy zakres operacji niż aktualnie dostępne przyciski interfejsu. W zgłoszeniu rozwojowym zaznacz, czy oczekiwane zachowanie ma być dostępne przez CLI, serwer, czy graficzny modeler, oraz podaj plik odpowiedzialny za ten kanał.

Źródła: S09 project, project.scripts, tool.setuptools.package-data; S16 __main__; S37 main

TEMAT 182

Read-only index_sources: indeks lokalnych bajtów

tools/index_sources.py jest samodzielnym narzędziem odczytu lokalnych plików. Przyjmuje istniejący --root, co najmniej jeden względny --include i jawny --output. Wynik ma schemat lion.polyhedral.source-binding-candidates/v1. Narzędzie nie importuje ani nie wykonuje indeksowanego kodu. Jego zadaniem jest zarejestrowanie odczytanych bajtów i ostrożnych kandydatów odniesień.

```
$repo = 'C:\Users\d2j3\PycharmProjects\ai_platform'
python tools\index_sources.py `
  --root $repo `
  --include cyber_lion/contracts `
  --include LION/evidence `
  --output ..\lion-source-index.json `
  --source-ref 'operator-selected-local-copy'
```

Względne zakresy zapisuj z separatorem /. Niedopuszczalne są ścieżki absolutne, cofnięcie przez .. oraz separator odwrotny wewnątrz include. Powtarzane zakresy są normalizowane i deduplikowane. W zwykłych katalogach narzędzie wybiera rozszerzenia .py, .md, .json, .yaml, .yml i .toml. W jawnie objętej części LION/evidence może indeksować także inne rozszerzenia, w tym dane binarne.

Domyślny limit odczytu wynosi 16 MiB na plik. Symlinki są pomijane. Sprawdzane są metadane przed otwarciem, po odczycie i po ponownym odczycie stanu ścieżki, aby wykryć część zmian równoległych. Jest to odczyt sekwencyjny z detekcją wybranych wyścigów, a nie atomowy snapshot aktywnego repozytorium.

Raport zawsze rozdziela wpisy entries od gaps. Brakujący plik, nadmierny rozmiar, odmowa odczytu czy błąd składni nie są usuwane z obrazu pokrycia. Status PARTIAL_LOCAL_READBACK_NOT_COMMIT_VERIFIED wyraźnie mówi, że lokalne bajty nie dowodzą określonego commitu ani wdrożenia. --source-ref jest niezweryfikowaną etykietą operatora, niezależnie od tego, czy wpisano tam nazwę gałęzi, czy hash.

Umieszczaj output poza root. Istniejący plik wewnątrz źródłowego drzewa jest chroniony przed nadpisaniem, a symlink jako cel wyniku zostaje odrzucony. Zapis wykorzystuje chwilowy plik w katalogu wyniku i os.replace. Po zakończeniu przejrzyj indexed_files, gap_count oraz repository_coverage; sukces wykonania komendy nie oznacza pełnego pokrycia repozytorium ani potwierdzenia semantycznych zależności.

Zmniejszając --max-bytes, ograniczasz koszt odczytu, lecz zwiększasz prawdopodobieństwo luk OVERSIZED. Zwiększenie limitu nie usuwa błędów składni ani odmowy dostępu. Najpierw określ rodzinę potrzebnych plików, potem dobierz budżet. Indeksowanie ma wspierać konkretny model zależności; odczyt coraz większego drzewa bez przeglądu gaps nie gwarantuje, że objęto właściwy zakres analizy.

Źródła: S52 build_index, normalize_include, write_output, main; S17 Indeks źródeł i inventory_digest

TEMAT 183

AST, Markdown i przenośny digest inwentarza

PythonInventory odwiedza drzewa AST, odczytując definicje klas, funkcji i funkcji asynchronicznych. Stos rodziców daje nazwy kwalifikowane, takie jak Cell.run. Zapisywane są początek i koniec zakresu linii. Taki wynik ma status SYMBOL_LOCATOR_NOT_LOGICAL_ID: nazwa symbolu jest lokalizatorem w pliku, a nie dowodem trwałej tożsamości artefaktu pomiędzy refaktoryzacjami.

Importy otrzymują moduł, nazwę, alias, poziom względny i linię deklaracji. Pozostają UNRESOLVED z przyczyną AST_DECLARATION_ONLY. Deklaracja importu nie dowodzi wykonania importu w danym przebiegu. Analiza nie wywołuje funkcji, nie importuje badanego modułu i nie rozstrzyga wszystkich dynamicznych sposobów pobierania kodu.

markdown_candidates jest konserwatywną heurystyką lokalnych odnośników, a nie pełnym parserem CommonMark. Rozpoznaje część linków inline i definicji referencyjnych, oblicza kandydującą ścieżkę względną i zachowuje fragment. Nie otwiera celu. Próba wyjścia poza root pozostaje rekordem OUTSIDE_ROOT; odnośniki sieciowe nie są pobierane.

Wynik	Co potwierdza	Co pozostaje otwarte
sha256 pliku	Tożsamość odczytanych bajtów	Commit i zachowanie
Definicja AST	Deklarację symbolu w zakresie	Rzeczywiste wywołania
Import candidate	Tekstową deklarację zależności	Dostępność i wykonanie
Link Markdown	Kandydaturę odnośnika	Istnienie i znaczenie celu

inventory_digest wiąże uporządkowane ścieżki względne, hashe, rozmiary, zakres i jawne luki. Podstawa obliczenia jest dostępna jako inventory_digest_basis. Hash celowo pomija bezwzględny root, datę odczytu, stat i treść source_ref. Przeniesienie niezmienionego zestawu bajtów do innego katalogu nie powinno więc zmienić tożsamości inwentarza; zmiana jednego bajtu powinna ją zmienić.

```
python -m unittest discover -s tools `
  -p test_index_sources.py -v
```

Sześć testów narzędzia obejmuje między innymi brak wykonania kodu, binarne evidence, linki, przenośność digestu, błędy składni i odmowę odczytu. Przy porównaniu dwóch raportów najpierw porównuj inventory_digest_basis i gaps. Różnica czasu observed_at nie jest zmianą treści, a identyczna etykieta source_ref nie przesądza zgodności bajtów. Taki porządek chroni przed utożsamieniem opisu operatora z obserwacją narzędzia.

Błąd parsowania Pythona nie musi usuwać wpisu bajtowego: odczytany plik zachowuje hash, a gap wskazuje nieudaną analizę AST. To ważne przy audycie, ponieważ brak deklaracji symboli może wynikać z uszkodzonej składni, a nie z pustego modułu. Przy ręcznej korekcie modelu zachowaj oba fakty, nie zastępując częściowego wyniku fikcyjną listą kompletnych zależności.

Źródła: S52 PythonInventory, markdown_candidates, canonical_bytes, build_index; S54 SourceIndexTests

TEMAT 184

Narzędzie geometrii: konstrukcja i niezmienniki bryły

tools/geometry_profile.py buduje referencyjny ośmiościan ścięty z unikatowych permutacji współrzędnych (0, ±1, ±2). Ściany kwadratowe leżą na płaszczyznach osiowych o odległości 2, a sześciokątne na płaszczyznach z normalnymi o znakach ±1 i stałą 3. Te dane określają profil projekcji, nie materialną odległość między usługami federacji.

Dla każdej ściany skrypt wybiera leżące na niej wierzchołki, oblicza centroid i porządkuje indeksy kątowo w jej płaszczyźnie. Z kolejnych par tworzy krawędzie, zliczając ich przynależność do ścian. Na tej podstawie wyprowadza również sąsiedztwo faset. Nie korzysta z zewnętrznej biblioteki modelowania ani z ręcznie narysowanego obrazu.

Niezmiennik	Wartość oczekiwana
Wierzchołki	24
Krawędzie	36
Ściany	14: 6 kwadratów i 8 sześciokątów
Incydencja krawędzi	Dokładnie 2 ściany
Kwadrat długości krawędzi	2
Charakterystyka Eulera	V − E + F = 2
Warunki półprzestrzeni	Spełnione przez wszystkie wierzchołki

```
python tools\geometry_profile.py
```

Samodzielne uruchomienie zapisuje specs/geometry_profile.json i wypisuje obiekt verification. Podczas run_checks.py funkcja build jest także wywoływana jako sprawdzenie geometrii, bez konieczności renderowania sceny. Jeśli zmieniasz generator profilu, kontroluj zarówno liczby elementów, jak i przypisanie nazw ścian do płaszczyzn; poprawna liczba wielokątów nie gwarantuje poprawnego adresowania ról.

Sąsiedztwo dwóch ścian określa wspólna krawędź geometryczna. Nie tworzy ono automatycznie Link między artefaktami. Podobnie skala prezentacji nie zwiększa capacity portu. W polu units wynik określa jednostki jako dowolne jednostki widoku, a status przypomina, że profil nie nadaje semantycznego autorytetu.

Deformowanie pojedynczych ścian wymagałoby nowego kontraktu: kontroli płaszczyzn, wypukłości, orientacji i niezanikania powierzchni. Obecne narzędzie potwierdza stałą bryłę referencyjną. Przy rozwoju renderera utrzymuj osobno test matematycznej konstrukcji oraz test widoczności etykiet, wyboru ścian i interakcji myszy. Jeden z tych zestawów nie zastępuje drugiego.

Gdy asercja generatora zawiedzie, zachowaj zmienione wejście i pełny traceback. Nie aktualizuj automatycznie oczekiwanych liczb, aby test przeszedł. Najpierw ustal, czy zamierzona była zmiana profilu, czy wystąpił błąd porządkowania wierzchołków. Nowy profil potrzebuje własnego identyfikatora i planu zgodności ze stabilnymi adresami portów, które wcześniejsze modele już przechowują.

Źródła: S51 build, dot, cross; S21 verification, adjacency; S14 build()['verification']

TEMAT 185

Budowa katalogu atlasu i kontrola zakresu metadanych

tools/build_atlas_catalog.py przekształca dostarczony dataset atlasu JSON w przenośny katalog metadanych. Wejściem nie jest dowolny projekt Studio ani bezpośrednio strona HTML; trzeba dostarczyć wyodrębniony dataset o oczekiwanej strukturze. Skrypt nie wykonuje analizowanego kodu. Jego funkcja build_catalog tworzy węzły, źródła, warstwy i relacje, a przed zapisem stosuje walidację katalogu.

```
python tools\build_atlas_catalog.py atlas-dataset.json `
  --output ..\rebuilt-ai-platform.json.gz
```

Jawny --output poza źródłami jest wygodny podczas eksperymentu. Domyślny cel wskazuje src/lion_studio/data/ai_platform_architecture.json.gz, czyli dane dołączane do aplikacji. Nadpisanie tego pliku jest zmianą pakietu i powinno być świadomie poddane przeglądowi. Sam skrypt nie podmienia istniejących rewizji w bazie workspace/architecture.sqlite3.

Warstwy są wyprowadzane z typu i ścieżki pliku. Obejmują implementację, kontrakty, wiedzę, evidence, operacje, testy oraz governance. Jest to klasyfikacja źródłowa, a nie automatycznie przyznana rola wykonawcza. Metadane źródła mogą wiązać repozytorium, ścieżkę, commit, blob i linie. Dane lokalne bez wiarygodnego przypięcia nie otrzymują fikcyjnego commitu.

signature_contract i _safe_type zachowują strukturę typów i nazwy parametrów, pomijając literały oraz wartości domyślne. Do katalogu nie kopiuje się pełnych ciał plików, instrukcji ani operacyjnych wartości. Umożliwia to modelowanie interfejsów bez przenoszenia przypadkowych sekretów z argumentów domyślnych. Zachowany typ jest nadal kandydatem kontraktu, nie dowodem kompletności rzeczywistego interfejsu.

Wynik jest serializowany z sortowaniem kluczy i kompresowany gzip z mtime równym zero. Dla identycznego wejścia oznacza to powtarzalne bajty kompresji. Wypisywane są rozmiar, liczba węzłów, relacji i coverage. Porównując przebudowę z paczką R3.1, uwzględnij historyczny zakres dostarczonego atlasu: 20 760 węzłów oraz 15 470 relacji. To pokrycie konkretnego datasetu, a nie zapewnienie aktualności całej działającej federacji.

Gzip jest tylko kontenerem danych, nie certyfikatem ich pochodzenia. Zachowaj hash wejściowego datasetu, wersję generatora i raport coverage obok wyniku. Jeśli nowy dataset pomija rodzinę plików, identyczna liczba warstw nie czyni go równoważnym staremu. Sprawdzaj również rozstrzyganie końców relacji i brak wymyślonych danych dla repozytoriów, których zawartości nie dostarczono.

Źródła: S50 build_catalog, signature_contract, _safe_type, main; S11 coverage i statystyki budowy

TEMAT 186

Budowanie wydania: manifesty i przejrzane poprzednie wersje

tools/package_studio.py buduje dystrybucję ZIP, tworząc MANIFEST.json oraz UPGRADE_MANIFEST.json. Pierwszy opisuje hashe payloadu wydania, drugi określa pliki zarządzane przez instalator i dopuszczalne stany poprzedników. Oba mają różne funkcje: kompletność archiwum nie oznacza, że każdy zawarty plik zostanie nadpisany przy aktualizacji istniejącego projektu.

```
python tools\package_studio.py `
  --baseline ..\reviewed-r1\LION_POLYHEDRAL_ARTIFACT_R1 `
  --previous ..\reviewed-r3\LION_POLYHEDRAL_ARTIFACT_R1 `
  --output ..\LION_POLYHEDRAL_STUDIO_R3_1.zip
```

Parametr --baseline jest wymagany. --previous można powtarzać dla dodatkowych przejrzanych wydań. Wskazane katalogi muszą zawierać rzeczywiście zweryfikowane wersje, ponieważ ich hashe stają się podstawą automatycznej zgody na zastąpienie plików. Narzędzie nie potrafi odróżnić zatwierdzonego wydania od przypadkowego katalogu nazwanego reviewed-r3.

Ze zbierania payloadu wyłączone są między innymi .venv, .idea, .git, workspace, kopie instalatora, build, dist, cache i katalogi egg-info. results oraz uv.lock mogą należeć do zawartości archiwum, ale są chronione przed zarządzaną aktualizacją. Własne, niewymienione pliki użytkownika nie stają się automatycznie składnikami do usunięcia.

Dla bazowego R1 skrypt oblicza z góry dopuszczalne warianty końców linii LF i CRLF wybranych tekstów UTF-8. Nie normalizuje arbitralnych lokalnych modyfikacji podczas instalacji. Każdy zaakceptowany wariant ma konkretny hash, a oryginalne bajty celu zostają zachowane w kopii. Takie podejście pozwala obsłużyć znane różnice platform bez przyzwolenia na dowolne zmiany kodu.

Output ZIP musi znajdować się poza repozytorium, co zapobiega włączeniu tworzonego archiwum do kolejnego payloadu. Pakiet zachowuje wspólny katalog główny projektu. Po budowie przejrzyj liczbę plików, rozmiar i SHA-256, a instalator przetestuj na kopii poprzednika. Ponieważ skrypt zapisuje manifesty w źródłowym katalogu, uruchamiaj go w kontrolowanym drzewie wydania, którego zmiany można porównać przed dystrybucją.

Hash manifestu potwierdza zgodność bajtów z zadeklarowanym zestawem, lecz manifest sam nie jest podpisem wydawcy. Przechowuj opublikowany hash archiwum w niezależnym kanale uzgodnionym dla wydania. Podczas odtwarzania akceptacji rozróżniaj archiwum kandydujące od finalnego: zmiana raportu w paczce zmieni hash ZIP, nawet jeżeli testowane pliki kodu pozostały identyczne.

TEMAT 187

Aktualizacja istniejącego projektu: najpierw plan, potem apply

W przesłanej kopii repozytorium nie ma UPGRADE_MANIFEST.json. Poniższe polecenia dotyczą kompletnej dystrybucji aktualizacyjnej zawierającej ten manifest. Sama obecność install_upgrade.py nie wystarcza do wykonania planu. Użyj właściwej opublikowanej paczki albo zbuduj kontrolowane wydanie narzędziem package_studio; nie twórz ręcznie listy hashy na podstawie nieprzejrzanych lokalnych plików.

install_upgrade.py należy uruchamiać z osobno rozpakowanej paczki, kierując --target na istniejący projekt. Skrypt odrzuca instalację do katalogu, z którego sam działa. Chroni to przed pomyleniem źródła aktualizacji z celem. Wbudowany domyślny target wskazuje użytkownikowy katalog LION_POLYHEDRAL_ARTIFACT_R1; w procedurze administracyjnej lepiej podać cel jawnie i przeczytać wypisany plan.

```
$target = 'C:\LION\Studio'
python .\install_upgrade.py --target $target
python .\install_upgrade.py --target $target --apply
```

C:\LION\Studio jest tu przykładową ścieżką do zastąpienia. Pierwsza komenda działa w trybie PLAN. Wypisuje target, files_to_write, conflicts oraz preserved i nie zapisuje plików docelowych. Druga stosuje zmianę. Przed apply zapisz projekty i zatrzymaj serwer uruchomiony z aktualizowanego drzewa, aby proces nie korzystał jednocześnie ze starych i nowych modułów.

Dla każdego zarządzanego pliku instalator weryfikuje hash źródła według UPGRADE_MANIFEST. Następnie porównuje bajty celu. Identyczny aktualny plik pomija. Istniejący różny plik może zastąpić tylko wtedy, gdy jego hash należy do przejrzanej bazy, zatwierdzonego wariantu końców linii albo reviewed_previous_hashes. Nieznana lokalna zmiana powoduje konflikt przed pierwszym zapisem.

Po czystym planie apply tworzy katalog .lion-upgrade-backups z czasowym identyfikatorem, kopiuje stare bajty i zapisuje restore.json. Każdy docelowy plik jest sprawdzany ponownie tuż przed zmianą. Zapis następuje przez chwilowy plik oraz os.replace, po czym sprawdzany jest wynikowy hash. Zwykły wyjątek w tej pętli uruchamia cofnięcie już zmienionych plików.

Aktualizacja nie zarządza .idea, .venv, .git, workspace, results ani uv.lock. Własne pliki niewymienione w manifeście pozostają na miejscu. Nie oznacza to migracji każdej możliwej bazy z przyszłych wersji: instalator aktualizuje wskazany kod, a zgodność danych zależy od kontraktów aplikacji.

Zachowaj pełny wynik planu oraz ścieżkę kopii. Liczba zmienianych plików zależy od rzeczywistych bajtów poprzednika i nie musi równać się liczbie z historycznej akceptacji wydania. Po aktualizacji uruchom ten sam interpreter PyCharm, sprawdź wersję i odśwież przeglądarkę, aby pobrać aktualne zasoby interfejsu.

TEMAT 188

Kopia, rollback i rozwiązywanie konfliktów lokalnych

Każde zastosowane wydanie zapisuje dziennik restore.json we własnym katalogu .lion-upgrade-backups. Dziennik zawiera target, czas, listę plików oraz hashe before i after. Pliki istniejące przed aktualizacją mają w kopii zachowane dokładne bajty. Plik nowo dodany nie wymaga poprzedniej kopii; podczas cofania zostaje usunięty.

```
$backup = 'C:\LION\Studio\.lion-upgrade-backups\WYBRANY_ZNACZNIK'
python .\install_upgrade.py --rollback $backup
python .\install_upgrade.py --rollback $backup --apply
```

Pierwsza komenda pokazuje zamiar cofnięcia bez zapisu. Podstaw rzeczywisty katalog kopii i nie używaj dosłownie WYBRANY_ZNACZNIK. Przed wykonaniem rollback instalator sprawdza, czy kopia należy do wskazanego projektu, czy obecne pliki nadal odpowiadają hashom after oraz czy stare kopie odpowiadają before. Dopiero po pełnej kontroli przywraca pliki.

Jeżeli po aktualizacji programista zmienił zarządzany plik, automatyczny rollback odmawia działania. To zamierzona ochrona nowej pracy. Najpierw skopiuj różnicę do osobnego miejsca i przeanalizuj ją względem wersji po aktualizacji. Nie usuwaj wpisu manifestu ani nie wpisuj nowego hasha tylko po to, aby wymusić cofnięcie; utraciłbyś podstawę kontroli integralności.

Podobnie konflikt podczas apply oznacza, że lokalny plik nie odpowiada żadnemu przejrzanemu poprzednikowi. Wykonaj ręczne scalenie na kopii roboczej, zachowując dotychczasowy plik i nowy odpowiednik. Sprawdź zachowanie po scaleniu. Instalator nie oferuje opcji force, która automatycznie uznaje każdą lokalną zmianę za bezpieczną do zastąpienia.

safe_path odrzuca ścieżki absolutne w manifeście, segmenty .., nieprawidłowe separatory, chronione prefiksy i dowiązania w sprawdzanej drodze. Ochrona obejmuje także junctions tam, gdzie udostępnia je użyta wersja Pythona. Nie należy przenosić katalogu kopii do innego projektu ani zmieniać w dzienniku target, ponieważ weryfikacja przynależności jest częścią mechanizmu.

Automatyczne cofnięcie przy zwykłym wyjątku instalacji i jawny rollback to dwie różne ścieżki. Nagła utrata zasilania nie jest normalnym wyjątkiem obsługiwanym przez proces. W takiej sytuacji zachowaj dziennik i sprawdź stan każdego pliku przed dalszą operacją. Kopia jest podstawą rekonstrukcji, a nie powodem do założenia, że każda awaria już została bezwarunkowo cofnięta.

TEMAT 189

Testy regresji i prawidłowe czytanie raportu R3.1

run_checks.py zbiera testy z tests/test_*.py oraz sześć testów narzędzia index_sources. Następnie wywołuje generator geometrii i wypisuje podsumowanie tests_run, failures, errors oraz geometry. Kod wyjścia wynosi zero przy pomyślnym wyniku zestawu unittest. Sprawdzenie profilu jest dodatkowym krokiem programu, a nie osobną deklaracją przetestowania interakcji w przeglądarce.

```
.\.venv\Scripts\python.exe run_checks.py
$LASTEXITCODE
```

Rodzina plików testowych	Liczba metod testowych
Referencyjny kandydat R1	32
Model Studio i szablony	27
Topologia i czas	30
Architektura, nawigacja i szkice	59
Repozytoria i importery	31
Ewolucja	25
Magazyny, HTTP i instalacja	36
Narzędzie indeksowania	6

Powyższy podział sumuje się do 246 metod testowych w dostarczonym źródle. W grupie magazynów, HTTP i instalacji mieszczą się testy Store, CatalogStore, zwykłego HTTP, Workbench HTTP oraz instalatora. Liczba jest inwentarzem testów tej paczki; nie stanowi świeżego wyniku ich wykonania w aktualnym środowisku czytelnika.

reports/STUDIO_R31_ACCEPTANCE.json dokumentuje historyczne 246 testów bez błędów i niepowodzeń na Linux, na Windows z interpreterem Python 3.13.13 oraz w zainstalowanym pakiecie. Raport wskazuje ponadto 27 kontroli kontrolera interfejsu i 27 sprawdzeń Windows Chrome. To wyniki procesu wydania z 15 października 2026, przechowywane wraz z logami. Nie należy przedstawiać ich jako ponownego uruchomienia podczas lektury instrukcji.

Do diagnozy własnej zmiany uruchom najpierw odpowiedni test behawioralny, a potem wymagany zestaw kontrolny. Zachowaj pełny log, wersję Pythona i dokładną rewizję plików. Przykładowo poprawka aliasów potrzebuje testów wspólnej pojemności, a poprawka wejścia do wnętrza - odczytu głębszych zakresów oraz zachowania edycji. Test liczący tylko elementy DOM nie potwierdzi semantyki katalogu.

Historyczny raport instalatora obejmuje apply, rollback, zachowanie chronionych bajtów i konflikt przed zapisem. Raport przeglądarki sprawdzał również szerokości 1440 × 1000 i 1200 × 800. Gdy nowe środowisko ma inną rozdzielczość, wersję przeglądarki albo odmienne dane, dodaj odpowiednią próbę zamiast rozszerzać wcześniejszy PASS poza jego rzeczywisty zakres.

Źródła: S14 TestSuite i wynik końcowy; S10 verification; S13 Wynik historycznej regresji; S12 checks

TEMAT 190

Scenariusz ręcznej akceptacji po aktualizacji

Poniższa procedura jest wzorcem do własnego wykonania, a nie informacją o nowo przeprowadzonym teście. Utwórz oddzielny workspace próbny lub zachowaj eksport projektu przed eksperymentem. Zapisuj wersję programu, źródło katalogu, jego rewizję oraz wybrany tryb nawigacji. Dzięki temu ewentualna różnica wyniku będzie miała określony kontekst.

1. Uruchom Studio i otwórz katalog ai_platform w widoku 3D.

2. Wejdź kolejno do Źródeł repozytorium, cyber_lion, katalogu i pliku obecnego w atlasie.

3. Otwórz klasę lub funkcję posiadającą dzieci; wróć okruszkami do przodka.

4. Przełącz Model projektu i Drzewo Git, kontrolując ścieżkę źródłową.

5. Dodaj ręczny podobiekt, zmień nazwę, odwiedź drugi zakres i wróć.

6. Zapisz projekt, przeładuj go, następnie jawnie scal widok z katalogiem.

Przy każdym wejściu powinny zostać pobrane rzeczywiste bezpośrednie dzieci pełnego katalogu, niezależnie od głębokości początkowej projekcji. Domyślna strona mieści 36 dzieci. Jeżeli licznik pokazuje większą całość, sprawdź Następne i Poprzednie. Liść bez dzieci jest poprawnym wynikiem; nie należy mylić go z zakresem, którego kolejna strona nie została jeszcze załadowana.

Wybierz zakres z zależnościami przechodzącymi między gałęziami. Sprawdź dokładną relację, agregację i odnośnik kontekstu zewnętrznego. Dwuklik odnośnika powinien prowadzić do rzeczywistego celu. Zmieniaj limit kontekstu 12, 32 i 64, obserwując jawne pominięcia. Ukrycie linii nie powinno usuwać relacji z danych projektu.

Po nawigacji ręczne zmiany z obu wnętrz muszą pozostać w szkicu. Po scaleniu sprawdź je w katalogu i potwierdź zachowanie oryginalnego źródłowego commitu, ścieżki oraz rodzica. Samo przechodzenie między wnętrzami nie powinno generować kolejnych rewizji katalogu ani dopisywać granicznych odnośników jako prawdziwych dzieci.

Na zakończenie wyeksportuj JSON projektu i pełny katalog. Zapisz, które kroki przeszły, jakie miały oczekiwane wartości oraz jakie ograniczenia pozostały. Jeżeli dane są historycznym atlasem, nie oceniaj braku nowego pliku jako błędu nawigacji: najpierw sprawdź, czy dany plik rzeczywiście znajduje się w przypiętym katalogu. Akceptacja ma rozróżniać błąd interfejsu, brak pokrycia wejścia i poprawnie oznaczoną niepewność.

TEMAT 191

Rozwiązywanie problemów: Unicode, port i częściowy model

Jeżeli serwer nie startuje, zacznij od pełnego komunikatu w konsoli PyCharm. run_studio.py domyślnie wybiera 18765, aby oddzielić Studio od usług LION używających innych portów. Gdy ten port jest zajęty, wybierz kolejny przez --port. Nie kończ przypadkowych procesów tylko dlatego, że odpowiadają na localhost; na komputerze mogą działać równolegle inne usługi.

```
.\.venv\Scripts\python.exe run_studio.py `
  --port 18766 --no-browser
# Otwórz adres wypisany przez serwer w konsoli.
```

Serwer wiąże się z 127.0.0.1. Parametr --no-browser blokuje automatyczne otwarcie karty, co ułatwia ręczne porównanie adresu. Po restarcie serwera stara karta może posiadać nieaktualny token sesji; przy odmowie zapisu najpierw odśwież aplikację i sprawdź konsolę, zamiast wyłączać kontrolę tokenu lub Origin.

W Windows przekierowane wyjście może używać kodowania bez polskich liter. main ustawia errors na backslashreplace, dzięki czemu sam tekst logowania nie powinien blokować startu. Dla własnych skryptów diagnostycznych można jawnie użyć Python UTF-8 i odczytywać pliki przez -Encoding UTF8. Nie zmieniaj bajtów kodu źródłowego tylko w celu dopasowania ich do starej strony kodowej konsoli.

```
$env:PYTHONUTF8 = '1'
$env:PYTHONIOENCODING = 'utf-8'
.\.venv\Scripts\python.exe run_checks.py
```

Gdy scena wygląda na pustą, sprawdź scope, liczbę wszystkich dzieci, bieżącą stronę, tryb topologii i limit kontekstu. R3.1 pobiera wnętrza z przypiętej rewizji katalogu. Nieobecność w tej rewizji nie zostanie naprawiona samym zwiększeniem skali kamery. Jeżeli linie znikają, sprawdź ustawienie relacji zaznaczonego obiektu oraz licznik pominięć.

Dla Git odróżniaj brak lokalnego repozytorium, nierozstrzygnięty ref, nieodczytany zakres oraz błąd ownership. Status UNKNOWN nie jest potwierdzeniem nieistnienia projektu. Przy konflikcie zapisu projektu albo katalogu zachowaj eksport i porównaj expected_revision z aktualną historią. Automatyczne przepisanie szkicu na nowszą bazę mogłoby utracić znaczenie ręcznej zmiany; świadoma rekonsyliacja jest częścią poprawnej obsługi konfliktu.

W zgłoszeniu dołącz minimalny eksport, opis ostatniej operacji i pełny komunikat błędu. Zrzut ekranu pokazuje objaw, lecz nie zastępuje identyfikatora katalogu, rewizji i danych szkicu. Te informacje pozwalają odróżnić problem renderowania od niezgodnej bazy albo brakującego źródła.

TEMAT 192

Rozszerzenia, migracja R1 i przyrostowa integracja z LION

migrate_workspace przyjmuje dokument Studio-v2 albo kompozycję lion.polyhedral.mosaic.candidate.v1. W drugim przypadku najpierw waliduje cały Mosaic. Nieznany schemat i błędny digest są odrzucane. Wynik otrzymuje nowy project_id oraz efekt DESIGN_ONLY. Oryginalny dokument, korzeń i rekordy R1 pozostają w metadanych migracji.

Porty R1 są odwzorowane na kafle r1-face-* i parametry o typie any z nieznaną wartością. duplex staje się inout. Zachowane są schematy, pojemności, oryginalne rewizje i deklaracje akceptacji. Jeśli obiekt R1 ma więcej niż jednego semantycznego rodzica, Studio wybiera jednego parent_id do drzewa, zachowując pełną listę i ostrzeżenie w metadanych. Nie jest to bezwarunkowe zrównanie obu modeli.

Rozszerzenie własnego formatu wymaga uzgodnienia schematu, walidacji, serializacji, migracji i interfejsu. Nowa relacja wykonawcza potrzebuje testów kierunku, granicy, rewizji oraz skutku symulacji. Nowe pole opisowe powinno przetrwać normalizację, zapis i eksport. Zmiana nawigacji musi zachować źródłowe fakty oraz nie dopisywać agregacji jako rzeczywistych połączeń.

1. Wybierz jedną rodzinę: założenie, kontrakt, symbol, test i dowód.

2. Dodaj opis sidecar bez przenoszenia źródeł i historycznych evidence.

3. Zwiąż źródło z dokładnym commitem, bajtami i istniejącym ownerem.

4. Sprawdź zmianę kontraktu, zerwaną referencję, cykl i zmianę samego widoku.

5. Dopiero po akceptacji wydziel moduł za kompatybilną fasadą i przygotuj rollback.

specs/DEPENDENCIES_AND_MIGRATION.md opisuje taki przyrostowy kierunek. Istniejące kontrakty BeanSpec, BeanInstance, CompositionContract i CommunicationEnvelope zachowują swoje role; powłoka wielościenna ma je referencjonować, a nie ustanawiać drugie źródło prawdy. Modeler i demonstrator nie stanowią dowodu, że integracja tych właścicieli już działa w aktualnej federacji.

Dokumenty w paczce mają różne daty i generacje. CHECKPOINT.json nadal opisuje R3 i wersję 0.3.0, podczas gdy pyproject.toml oraz akceptacja R3.1 wskazują nowsze wydanie. Traktuj taki checkpoint jako historyczny stan procesu. O faktycznie analizowanym zachowaniu rozstrzyga odczyt odpowiedniej implementacji, jej formatów i właściwych raportów, z zachowaniem źródłowego zakresu. Kolejny upgrade powinien aktualizować także te odnośniki, aby samoopis aplikacji pozostawał spójny z jej kodem.

TEMAT 193

Słownik pojęć modelowania

Poniższe pojęcia odnoszą się do modelu używanego przez Studio. Ich znaczenie należy odczytywać razem z kwalifikacją źródła i nazwą formatu. Podobna nazwa w rdzeniu kandydującym R1 nie zawsze oznacza ten sam dokument JSON co w projekcie Studio.

Pojęcie	Znaczenie w tej instrukcji
Artefakt	Wersjonowany element opisu z tożsamością, parametrami, źródłami i kaflami
Kafel	Rola umieszczona na nazwanej ścianie artefaktu
Port	Adresowany punkt interfejsu o typie, kierunku i ewentualnym powiązaniu z parametrem
Relacja	Jawne połączenie końców, wraz z rodzajem, metadanymi i opcjonalną bramką
Wiązanie	Reguła przekazania lub transformacji wartości parametru
Zakres	Bieżące wnętrze i strona widoczna w scenie

Kompozycja określa organizację podobiektów. Zawieranie w dokumencie Studio zapisuje parent_id. W modelu katalogu istnieje źródłowy parent_id i opcjonalny design.parent_id. Odpowiedzialność semantyczna, owner_ref i jawny poziom abstrakcji są osobnymi informacjami; nie wynikają bezpośrednio z drzewa.

Ściana jest nazwą grupującą role w profilu widoku. Jej geometryczna normalna służy renderowaniu, a nie obliczaniu prawdopodobieństwa. Przestrzeń XZ w domyślnym układzie ułatwia wybór wielu obiektów. Ręczna pozycja zachowuje zamiar użytkownika i nie jest pomiarem sprzężenia między komponentami.

Kontekst zewnętrzny reprezentuje koniec relacji poza bieżącą stroną lub wnętrzem. Agregacja podsumowuje wiele źródłowych relacji między gałęziami. Oba rodzaje rekordów są pomocnicze dla nawigacji; nie powinny zostać zapisane do pełnego katalogu jako nowy komponent albo kontrakt.

Źródła: S33 normalize_workspace; S34 scope_view; S21 face roles

TEMAT 194

Słownik źródeł i wykonania

Pojęcie	Znaczenie praktyczne
Commit	Niezmienny identyfikator odczytanego stanu repozytorium Git
Tree i blob	Obiekty Git opisujące drzewo oraz zawartość konkretnego pliku
Ref	Nazwa wybranej gałęzi, taga lub innego wskazania, rozstrzygana do commita
Nakładka design	Opis proponowanej architektury zachowujący źródłowe fakty
Szkic	Skumulowane, jeszcze niescalone edycje z wielu zakresów
Rewizja projektu	Kolejny utrwalony dokument w bazie Studio
Rewizja katalogu	Osobny, utrwalony stan pełnego modelu architektury

STATIC_CANDIDATE oznacza wynik ograniczonej analizy źródłowej, na przykład odczytu AST lub referencji do ścieżki. Nie stanowi dowodu, że dana gałąź kodu została wykonana. SOURCE_SNAPSHOT i podobne kwalifikacje identyfikują materiał wejściowy, a nie żywą obserwację federacji.

STALE informuje o niespełnieniu warunku aktualnego przypięcia. W relacji przepływu może chodzić o różnicę między zapisaną rewizją końca a obecną rewizją artefaktu. Powiązanie bieżących rewizji aktualizuje deklarację, lecz nie zastępuje sprawdzenia typu portu, jednostki, reguły przejścia przez granicę i znaczenia kontraktu.

SIMULATION_ONLY określa obliczenie na opisie procesu. Wygenerowany deskryptor kandydata nie oznacza utworzenia pracującej usługi. Lokalny dziennik i łańcuch hashy pomagają wykryć zmianę zapisanych danych; bez niezależnego punktu odniesienia i podpisów nie poświadczają tożsamości autora ani całej historii świata zewnętrznego.

UNKNOWN zachowuje brak rozstrzygnięcia. W instrukcji należy go odróżniać od błędu walidacji, jawnego FALSE, braku dzieci w odczytanym zakresie oraz pozycji pominiętej przez limit projekcji. Każdy z tych stanów wymaga innej czynności diagnostycznej.

Źródła: S35 index_repository; S41 collect_draft; S31 simulate_process

TEMAT 195

Indeks źródeł aplikacji część pierwsza

Identyfikatory S odsyłają do plików odczytanych z przekazanego archiwum. Skrót SHA-256 poniżej obejmuje pierwsze 12 znaków hasha konkretnego pliku. Pełny manifest znajduje się w aplikacji; SHA archiwum wejściowego podano na stronie 2.

Źródło	Ścieżka w katalogu aplikacji	SHA-256 skrót
S01	CHECKPOINT.json Tematy na stronach 2, 192	64d5c6c3a987
S02	MANIFEST.json Tematy na stronach 2	2873d000c102
S03	README.md Tematy na stronach 1, 4, 9	d7d6c732a8f3
S04	STUDIO_CONTRACT.md Tematy na stronach 5	84a7b31e10c8
S05	docs/ARCHITECTURE_WORKBENCH_R3.md Tematy na stronach 104	6fa0b0c2205e
S06	docs/FILE_INTERFACE_AND_TOPOLOGY.md Tematy na stronach 71	d4f7a02f9cee
S07	docs/NAVIGATION_R31_PL.md Tematy na stronach 3, 22, 86, 89, 190-191, 199	152da0d136f3
S08	install_upgrade.py Tematy na stronach 186-188	ea62bc0d1d1c
S09	pyproject.toml Tematy na stronach 2, 168, 181	fc95448ad66f
S10	reports/STUDIO_R31_ACCEPTANCE.json Tematy na stronach 2, 189	e6408a8392db
S11	reports/r3/catalog_build.json Tematy na stronach 81-82, 185	673727032d83
S12	reports/r31/browser-acceptance.json Tematy na stronach 189-190	b64a055b70ea
S13	reports/r31/windows-tests.txt Tematy na stronach 189	571cbe625d92
S14	run_checks.py Tematy na stronach 184, 189	5f244f6a5b94

Symbol w stopce wskazuje klasę, funkcję lub sekcję tematyczną, którą należy odszukać w danym pliku. Ścieżki nie oznaczają automatycznego wdrożenia tego kodu do federacji. Pliki testowe opisują konkretne przypadki weryfikacyjne; instrukcje i schematy uzupełniają kontrakt implementacji.

TEMAT 196

Indeks źródeł aplikacji część druga

Identyfikatory S odsyłają do plików odczytanych z przekazanego archiwum. Skrót SHA-256 poniżej obejmuje pierwsze 12 znaków hasha konkretnego pliku. Pełny manifest znajduje się w aplikacji; SHA archiwum wejściowego podano na stronie 2.

Źródło	Ścieżka w katalogu aplikacji	SHA-256 skrót
S15	run_demo.py Tematy na stronach 180	1e8d797090ca
S16	run_studio.py Tematy na stronach 6, 10, 168, 181, 191	a77c81b68101
S17	specs/DEPENDENCIES_AND_MIGRATION.md Tematy na stronach 179, 182, 192	afa78d9de801
S18	specs/FOUNDATION.md Tematy na stronach 8, 45, 52-65, 72, 169, 171, 176	ff43d3b2eb04
S19	specs/architecture.schema.json Tematy na stronach 74	ebac5b94b09e
S20	specs/file_interface.schema.json Tematy na stronach 71	387c74de3702
S21	specs/geometry_profile.json Tematy na stronach 1, 184, 193	4dcf1a9b0734
S22	specs/topology.schema.json Tematy na stronach 47	835cb9551a66
S23	src/lion_polyhedral/demo.py Tematy na stronach 180	b2afb4151b9f
S24	src/lion_polyhedral/evidence.py Tematy na stronach 178	9136821a2cda
S25	src/lion_polyhedral/evolution.py Tematy na stronach 179	06c7b7edb02c
S26	src/lion_polyhedral/model.py Tematy na stronach 42, 52, 169-175	f5c631836156
S27	src/lion_polyhedral/protocol.py Tematy na stronach 176-177	54325f2d213b
S28	src/lion_studio/architecture.py Tematy na stronach 73-76, 83, 97, 102	d41d14d7db5d

Symbol w stopce wskazuje klasę, funkcję lub sekcję tematyczną, którą należy odszukać w danym pliku. Ścieżki nie oznaczają automatycznego wdrożenia tego kodu do federacji. Pliki testowe opisują konkretne przypadki weryfikacyjne; instrukcje i schematy uzupełniają kontrakt implementacji.

TEMAT 197

Indeks źródeł aplikacji część trzecia

Identyfikatory S odsyłają do plików odczytanych z przekazanego archiwum. Skrót SHA-256 poniżej obejmuje pierwsze 12 znaków hasha konkretnego pliku. Pełny manifest znajduje się w aplikacji; SHA archiwum wejściowego podano na stronie 2.

Źródło	Ścieżka w katalogu aplikacji	SHA-256 skrót
S29	src/lion_studio/catalog_store.py Tematy na stronach 7, 92-93, 134, 149-150, 158	4656caf4390c
S30	src/lion_studio/data/ai_platform_architecture.json.gz Tematy na stronach 77, 81-82, 94, 104	bb8044af840b
S31	src/lion_studio/evolution.py Tematy na stronach 62, 65, 127-134, 136, 158, 194	4e79064f9984
S32	src/lion_studio/importers.py Tematy na stronach 38, 71, 101-102, 163	0ecd05abfc5e
S33	src/lion_studio/model.py Tematy na stronach 41-46, 48-51, 53-54, 56-60, 63-64, 68-70, 72, 105-112, 114, 143, 162, 169, 192-193, 200	c995612493a0
S34	src/lion_studio/navigation.py Tematy na stronach 8, 44, 78-79, 85-90, 154-155, 193	8e57b48158aa
S35	src/lion_studio/repositories.py Tematy na stronach 79-80, 94-101, 156-157, 194	6f4ea108b586
S36	src/lion_studio/seeds.py Tematy na stronach 6, 41, 43, 46, 61, 68, 115, 123	6bd86709f6ee
S37	src/lion_studio/server.py Tematy na stronach 2, 6-7, 11, 36, 40, 91, 103, 137-142, 151-154, 156-162, 164-168, 181, 191, 199-200	97750d2a2fe7
S38	src/lion_studio/store.py Tematy na stronach 7, 12, 144-148, 165	b95635c1245a
S39	src/lion_studio/temporal.py Tematy na stronach 119-122, 124-125	858a4cfcb7d0
S40	src/lion_studio/topology.py Tematy na stronach 8, 47, 116-118	f085ba5ca585
S41	src/lion_studio/view_edits.py Tematy na stronach 66-67, 75, 78, 85, 88, 90-91, 103, 155-156, 194, 200	7496cb96dcd1
S42	src/lion_studio/web/app.js Tematy na stronach 3, 6, 14-16, 19-21, 23-35, 37, 39, 42-44, 46-53, 55-72, 113, 126, 199	c08b8e219fec

Symbol w stopce wskazuje klasę, funkcję lub sekcję tematyczną, którą należy odszukać w danym pliku. Ścieżki nie oznaczają automatycznego wdrożenia tego kodu do federacji. Pliki testowe opisują konkretne przypadki weryfikacyjne; instrukcje i schematy uzupełniają kontrakt implementacji.

TEMAT 198

Indeks źródeł aplikacji część czwarta

Identyfikatory S odsyłają do plików odczytanych z przekazanego archiwum. Skrót SHA-256 poniżej obejmuje pierwsze 12 znaków hasha konkretnego pliku. Pełny manifest znajduje się w aplikacji; SHA archiwum wejściowego podano na stronie 2.

Źródło	Ścieżka w katalogu aplikacji	SHA-256 skrót
S43	src/lion_studio/web/index.html Tematy na stronach 13	6a95d705c987
S44	src/lion_studio/web/renderer.js Tematy na stronach 7, 17-18	af5a3ba80c19
S45	src/lion_studio/web/workbench.js Tematy na stronach 39, 73, 76-77, 83-84, 93, 96, 100, 135	bb28545a0c11
S46	tests/test_candidate.py Tematy na stronach 170, 172-175, 177-178	27e6791768db
S47	tests/test_studio_install.py Tematy na stronach 187-188	7faa63130415
S48	tests/test_studio_model.py Tematy na stronach 192	ea7748a36ac7
S49	tests/test_studio_temporal.py Tematy na stronach 120	6f194d327db8
S50	tools/build_atlas_catalog.py Tematy na stronach 185	44da6feff543
S51	tools/geometry_profile.py Tematy na stronach 45, 184	c6c03d082a7f
S52	tools/index_sources.py Tematy na stronach 182-183	b6fe45af68cb
S53	tools/package_studio.py Tematy na stronach 186	b71d5d0b55c5
S54	tools/test_index_sources.py Tematy na stronach 183	596ea48c8796

Symbol w stopce wskazuje klasę, funkcję lub sekcję tematyczną, którą należy odszukać w danym pliku. Ścieżki nie oznaczają automatycznego wdrożenia tego kodu do federacji. Pliki testowe opisują konkretne przypadki weryfikacyjne; instrukcje i schematy uzupełniają kontrakt implementacji.

TEMAT 199

Szybka mapa diagnostyki

Diagnozę rozpocznij od ustalenia warstwy, w której wystąpił objaw. Zmiana ustawienia sceny nie naprawi błędnego kontraktu, a ponowne indeksowanie repozytorium nie przywróci niezapisanej edycji z zamkniętej karty. Przed próbą naprawy wyeksportuj potrzebny model i zanotuj jego współrzędne.

Objaw	Pierwsze sprawdzenie	Następny krok
Strona się nie otwiera	Adres i terminal serwera	Uruchom właściwy port i workspace
Nie widać wnętrza	Bieżący zakres, licznik dzieci i filtry	Wejdź przez katalog lub sprawdź liść
Brakuje linii	Tryb linii, kontekst i pominięcia	Zmień limit albo otwórz pełny katalog
Nie działa zapis	Token, walidacja i oczekiwana rewizja	Zachowaj kopię i rozstrzygnij konflikt
Nieznany wynik czasu	Czas trwania, jednostka, bramka i piny	Uzupełnij brakującą deklarację
Git odmawia odczytu	Raport inspekcji i właściciel repozytorium	Napraw środowisko poza modelem
Brak nowego kodu po materializacji	Rodzaj eksportu i opis kandydata	Przeprowadź osobną implementację
Instalator nie tworzy planu	Obecność UPGRADE_MANIFEST.json	Użyj kompletnej dystrybucji aktualizacji

Jeżeli model pochodzi z katalogu, rozróżnij zapis projektu od scalenia nakładki. Projekt może zawierać szkic, którego katalog jeszcze nie ma. Po udanym scaleniu aplikacja aktualizuje przypiętą rewizję i usuwa skumulowany szkic z kopii roboczej. Zapisz także tę kopię projektu, jeżeli chcesz zachować nowe przypięcie przy kolejnym otwarciu.

Przy zgłoszeniu problemu dołącz zapisany pakiet asystenta, dokładną czynność, wynik i komunikat z terminala. Usuń dane niepotrzebne do odtworzenia. Raport powinien określać, czy problem dotyczy lokalnego dokumentu, źródłowego snapshotu, ograniczenia widoku czy oczekiwanego wykonania poza zakresem Studio.

Źródła: S37 Handler; S42 renderNavigation; S07 limity

TEMAT 200

Odbiór samodzielnie zbudowanego modelu

Model można uznać za przygotowany do dalszej pracy wtedy, gdy inna osoba potrafi odtworzyć jego zakres, źródła i przyczyny powiązań. Sam estetyczny układ brył nie wystarcza. Końcowy odbiór powinien sprawdzić zarówno dokument, jak i czynności wykonywane przez użytkownika.

1. Zapisz nazwę, identyfikator i rewizję projektu. Dla modelu katalogowego dopisz identyfikator i przypiętą rewizję katalogu.

2. Sprawdź zawieranie oraz właściciela odpowiedzialności jako osobne informacje. Ustal, czy poziomy abstrakcji mają wspólną ramę.

3. Przejdź co najmniej przez katalog, plik i deklarację. Potwierdź, że powrót zachowuje zmiany z poprzedniego wnętrza.

4. Otwórz relację i odczytaj dokładne końce, rodzaj, źródło oraz ewentualny kontrakt. Rozpoznaj agregacje.

5. Dla przepływu wartości sprawdź typy, jednostki, kierunek portów, rewizje i bramki. Dla czasu sprawdź odrębny kontrakt zależności.

6. Uruchom walidację i właściwą analizę. Zapisz rozstrzygnięcia UNKNOWN i powód ich pozostawienia.

7. Zapisz projekt, otwórz go ponownie i porównaj stan. Jeżeli potrzebujesz aktualizacji katalogu, wykonaj jawne scalenie.

8. Wyeksportuj pakiet asystenta i zachowaj kopię danych potrzebną do odtworzenia. Sprawdź, czy materiał nie zawiera zbędnych danych wrażliwych.

Wynikiem odbioru jest określony, odtwarzalny model projektowy wraz z zakresem dowodów i ograniczeń. Wdrożenie zmian w repozytoriach, przyjęcie kontraktu przez usługę oraz obserwacja działania systemu są kolejnymi zadaniami. Należy je dokumentować oddzielnie od stanu sceny i wyniku symulacji.

Taki zapis pozwala rozwijać 0!n etapami: zachować źródłową tożsamość elementów, opisać propozycję nowego układu, sprawdzić jej lokalne reguły i przekazać konkretny materiał do implementacji. Kolejna wersja aplikacji może rozszerzyć te mechanizmy, zachowując rozróżnienie między opisem, decyzją i wykonaniem.

Źródła: S33 validate_workspace; S37 assistant_package; S41 collect_draft